

Data Structures and Algorithms

Bachelor's Degree in Informatics Engineering (UPC)

Fall 2019/2020

Instructors



Gabriel Valiente

T10



Gabriel Valiente

P11, L11



Albert Oliveras

P12, L12



Enric Rodríguez

L13

Outline

1. Analysis of Algorithms I
2. Analysis of Algorithms II
3. Divide and Conquer I
4. Divide and Conquer II
5. Dictionaries I
6. Dictionaries II
7. Priority Queues
8. Graphs I
9. Graphs II
10. Exhaustive Search and Generation I
11. Exhaustive Search and Generation II
12. Notions of Intractability I
13. Notions of Intractability II
14. Review

Lecture 1

Analysis of Algorithms I

Analysis of Algorithms I

Contents

- Asymptotic notation CLRS 3.1
- Standard notations and common functions CLRS 3.2

Reading

- T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein.
Introduction to Algorithms. The MIT Press, 3rd edition, 2009

Analysis of Algorithms I

Overview

- A way to describe behavior of functions in the limit. We are studying **asymptotic** efficiency.
- Describe **growth** of functions.
- Focus on what's important by abstracting away low-order terms and constant factors.
- How we indicate running times of algorithms.
- A way to compare **sizes** of functions:

$$f \in O(g)$$

$$f \leq g$$

$$f \in o(g)$$

$$f < g$$

$$f \in \Omega(g)$$

$$f \geq g$$

$$f \in \omega(g)$$

$$f > g$$

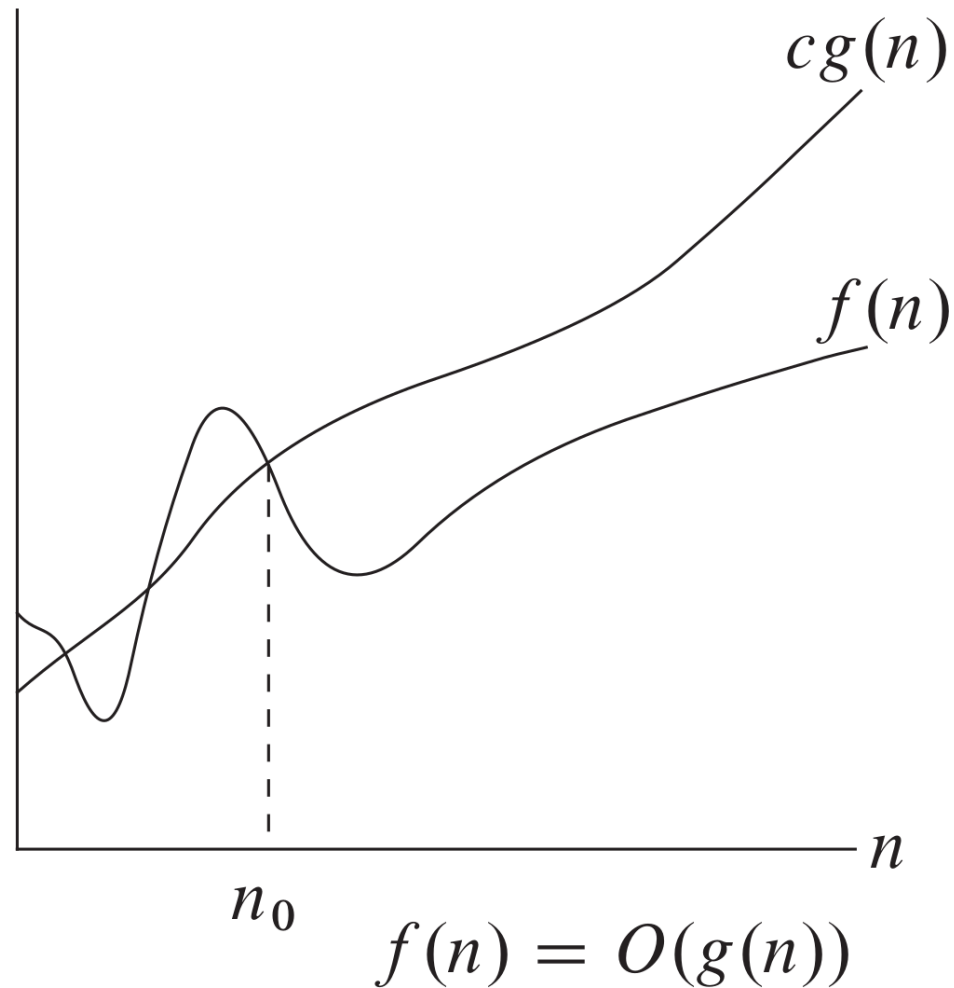
$$f \in \Theta(g)$$

$$f = g$$

Analysis of Algorithms I

3.1 Asymptotic notation

O -notation



Analysis of Algorithms I

3.1 Asymptotic notation

O -notation

$$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \\ \text{such that } 0 \leq f(n) \leq c g(n) \text{ for all } n \geq n_0\}$$

Another view, probably easier to use:

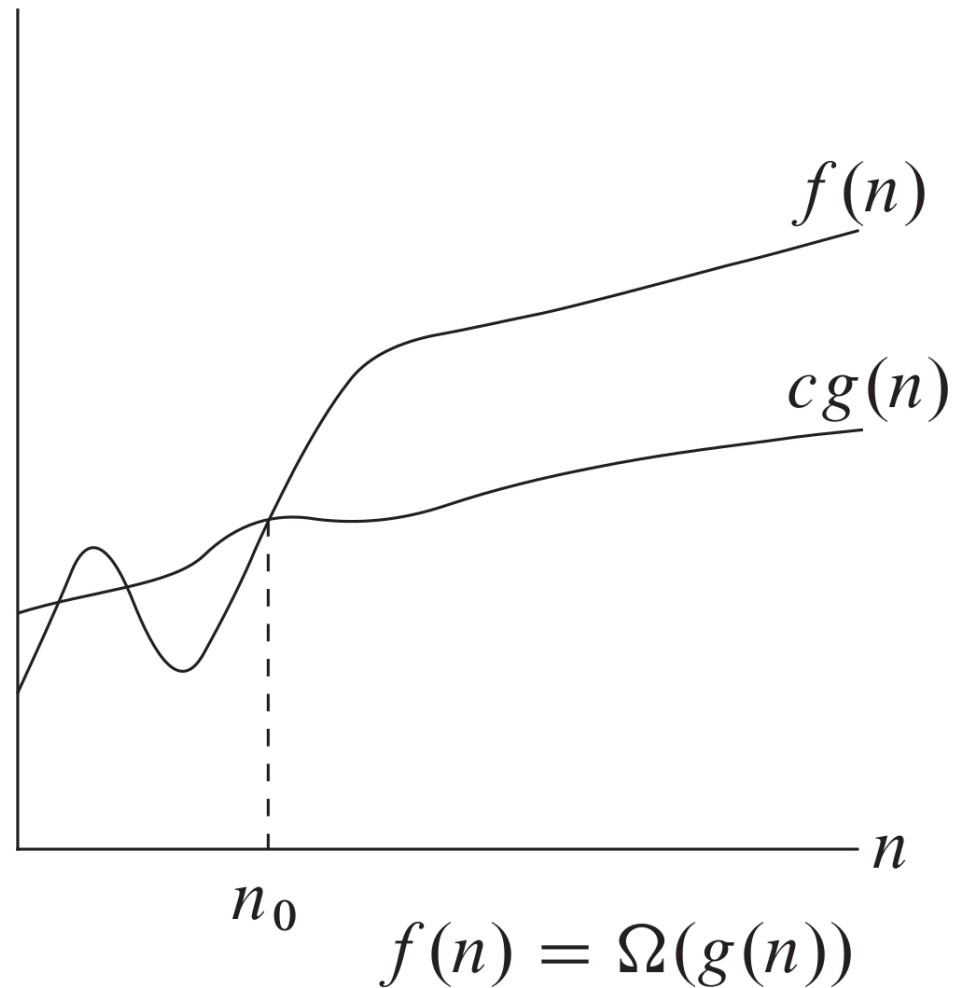
$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} < \infty$$

- $g(n)$ is an **asymptotic upper bound** for $f(n)$.
- If $f(n) \in O(g(n))$, we write $f(n) = O(g(n))$.
- $2n^2 = O(n^3)$, with $c = 1$ and $n_0 = 2$.

Analysis of Algorithms I

3.1 Asymptotic notation

Ω -notation



Analysis of Algorithms I

3.1 Asymptotic notation

Ω -notation

$$\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \\ \text{such that } 0 \leq c g(n) \leq f(n) \text{ for all } n \geq n_0\}$$

Another view, probably easier to use:

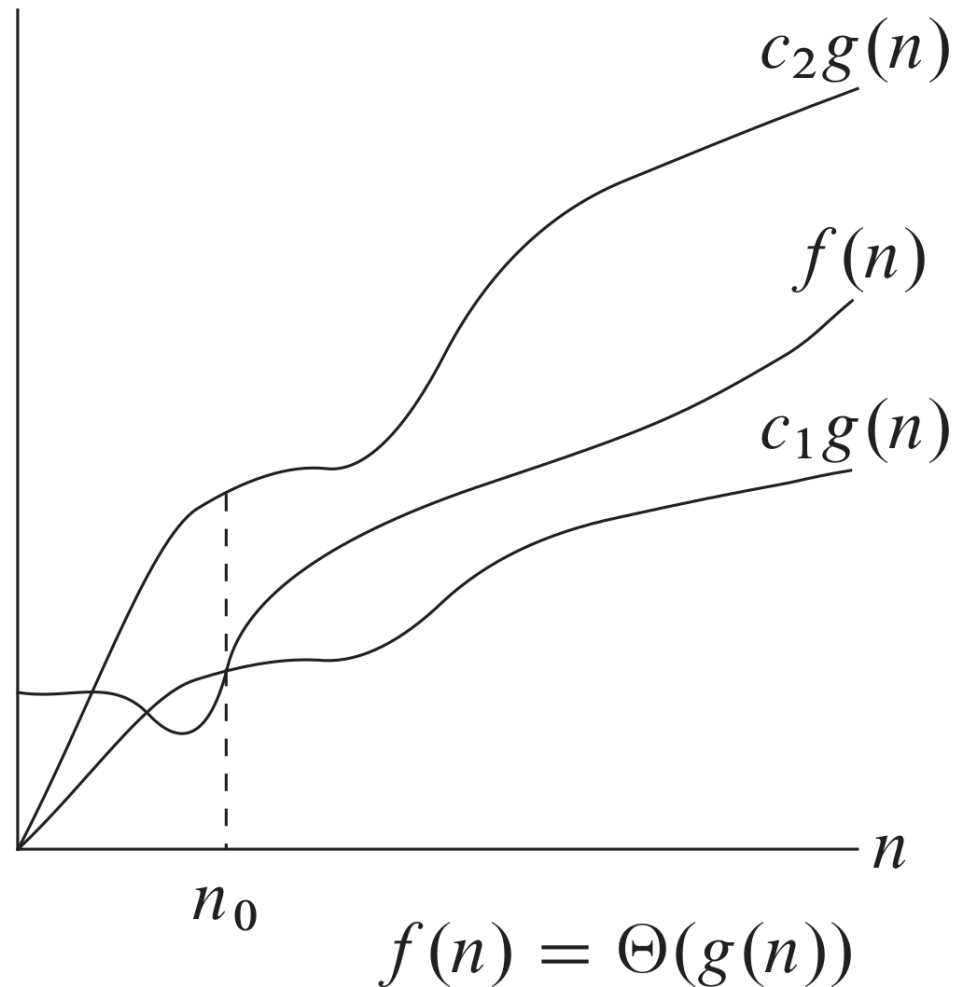
$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} > 0$$

- $g(n)$ is an **asymptotic lower bound** for $f(n)$.
- If $f(n) \in \Omega(g(n))$, we write $f(n) = \Omega(g(n))$.
- $\sqrt{n} = \Omega(\lg n)$, with $c = 1$ and $n_0 = 16$.

Analysis of Algorithms I

3.1 Asymptotic notation

Θ -notation



Analysis of Algorithms I

3.1 Asymptotic notation

Θ -notation

$\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0\}$

Another view, probably easier to use:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \in \mathbb{R}^+$$

- $g(n)$ is an **asymptotically tight bound** for $f(n)$.
- If $f(n) \in \Theta(g(n))$, we write $f(n) = \Theta(g(n))$.
- $n^2/2 - 2n = \Theta(n^2)$, with $c_1 = 1/4$, $c_2 = 1/2$, and $n_0 = 8$.

Analysis of Algorithms I

3.1 Asymptotic notation

o -notation

$o(g(n)) = \{f(n) : \text{for all constants } c > 0, \text{ there exists a constant } n_0 > 0 \text{ such that } 0 \leq f(n) < c g(n) \text{ for all } n \geq n_0\}$

Another view, probably easier to use: $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$.

- $n^{1.9999} = o(n^2)$.
- $n^2 / \lg n = o(n^2)$.
- $n^2 \neq o(n^2)$ (just like $2 \not\leq 2$).
- $n^2 / 1000 \neq o(n^2)$.

Analysis of Algorithms I

3.1 Asymptotic notation

ω -notation

$\omega(g(n)) = \{f(n) : \text{for all constants } c > 0, \text{ there exists a constant } n_0 > 0 \text{ such that } 0 \leq c g(n) < f(n) \text{ for all } n \geq n_0\}$

Another view, probably easier to use: $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$.

- $n^{2.0001} = \omega(n^2)$.
- $n^2 \lg n = \omega(n^2)$.
- $n^2 \neq \omega(n^2)$.

Analysis of Algorithms I

3.1 Asymptotic notation

Comparisons of functions

Transitivity:

$f(n) = \Theta(g(n))$ and $g(n) = \Theta(h(n))$ implies $f(n) = \Theta(h(n))$.

Same for O , Ω , o , and ω .

Reflexivity:

$f(n) = \Theta(f(n))$.

Same for O and Ω .

Symmetry:

$f(n) = \Theta(g(n))$ if and only if $g(n) = \Theta(f(n))$.

Transpose symmetry:

$f(n) = O(g(n))$ if and only if $g(n) = \Omega(f(n))$.

$f(n) = o(g(n))$ if and only if $g(n) = \omega(f(n))$.

Analysis of Algorithms I

3.1 Asymptotic notation

Comparisons of functions

- $f(n)$ is **asymptotically smaller** than $g(n)$ if $f(n) = o(g(n))$.
- $f(n)$ is **asymptotically larger** than $g(n)$ if $f(n) = \omega(g(n))$.

No trichotomy

Although intuitively, we can liken O to $\leq$, Ω to $\geq$, and so on, unlike real numbers, where $a < b$, $a = b$, or $a > b$, we might not be able to compare functions.

Example

$n^{1+\sin n}$ and n , since $1 + \sin n$ oscillates between 0 and 2.

Analysis of Algorithms I

3.2 Standard notations and common functions

Monotonicity

- $f(n)$ is **monotonically increasing** if $m \leq n \implies f(m) \leq f(n)$.
- $f(n)$ is **monotonically decreasing** if $m \geq n \implies f(m) \geq f(n)$.
- $f(n)$ is **strictly increasing** if $m < n \implies f(m) < f(n)$.
- $f(n)$ is **strictly decreasing** if $m > n \implies f(m) > f(n)$.

Analysis of Algorithms I

3.2 Standard notations and common functions

Exponentials

Useful identities:

- $a^{-1} = 1/a$,
- $(a^m)^n = a^{mn}$,
- $a^m a^n = a^{m+n}$.

Can relate rates of growth of polynomials and exponentials: for all real constants a and b such that $a > 1$,

$$\lim_{n \rightarrow \infty} \frac{n^b}{a^n} = 0,$$

which implies that $n^b = o(a^n)$.

Analysis of Algorithms I

3.2 Standard notations and common functions

Logarithms

Notations:

- $\lg n = \log_2 n$ (binary logarithm),
- $\ln n = \log_e n$ (natural logarithm),
- $\lg^k n = (\lg n)^k$ (exponentiation),
- $\lg \lg n = \lg(\lg n)$ (composition).

Useful identities for all real $a > 0$, $b > 0$, $c > 0$, and n , and where logarithm bases are not 1:

- $a = b^{\log_b a}$,
- $\log_c(ab) = \log_c a + \log_c b$,
- $\log_b a^n = n \log_b a$,
- $\log_b a = (\log_c a) / (\log_c b)$,
- $\log_b(1/a) = -\log_b a$,
- $\log_b a = 1 / \log_a b$,
- $a^{\log_b c} = c^{\log_b a}$.

Analysis of Algorithms I

3.2 Standard notations and common functions

Logarithms

Just as polynomials grow more slowly than exponentials, logarithms grow more slowly than polynomials. In

$$\lim_{n \rightarrow \infty} \frac{n^b}{a^n} = 0,$$

substitute $\lg n$ for n and 2^a for a :

$$\lim_{n \rightarrow \infty} \frac{\lg^b n}{(2^a)^{\lg n}} = \lim_{n \rightarrow \infty} \frac{\lg^b n}{n^a} = 0,$$

implying that $\lg^b n = o(n^a)$.

Analysis of Algorithms I

3.2 Standard notations and common functions

Factorials

$n! = 1 \cdot 2 \cdot 3 \cdots n$. Special case: $0! = 1$. Can use **Stirling's approximation**,

$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \left(1 + \Theta\left(\frac{1}{n}\right)\right),$$

to derive that $\lg(n!) = \Theta(n \lg n)$.

Analysis of Algorithms I

Order of growth

Order of growth	Name	
$O(1) = O(n^0)$	Constant	Polynomial
$O(\log n)$	Logarithmic	Polynomial
$O(\sqrt{n}) = O(n^{1/2})$		Polynomial
$O(n)$	Linear	Polynomial
$O(n \log n)$	Quasi-linear	Polynomial
$O(n^2)$	Quadratic	Polynomial
$O(n^3)$	Cubic	Polynomial
$O(2^n)$		Exponential
$O(3^n)$		Exponential
$O(n!)$		Exponential

Analysis of Algorithms I

Order of growth

Linear

$$f = O(n)$$

$$f(2n) \approx 2f(n)$$

$$f(10n) \approx 10f(n)$$

Quadratic

$$f = O(n^2)$$

$$f(2n) \approx 4f(n)$$

$$f(10n) \approx 100f(n)$$

Exponential

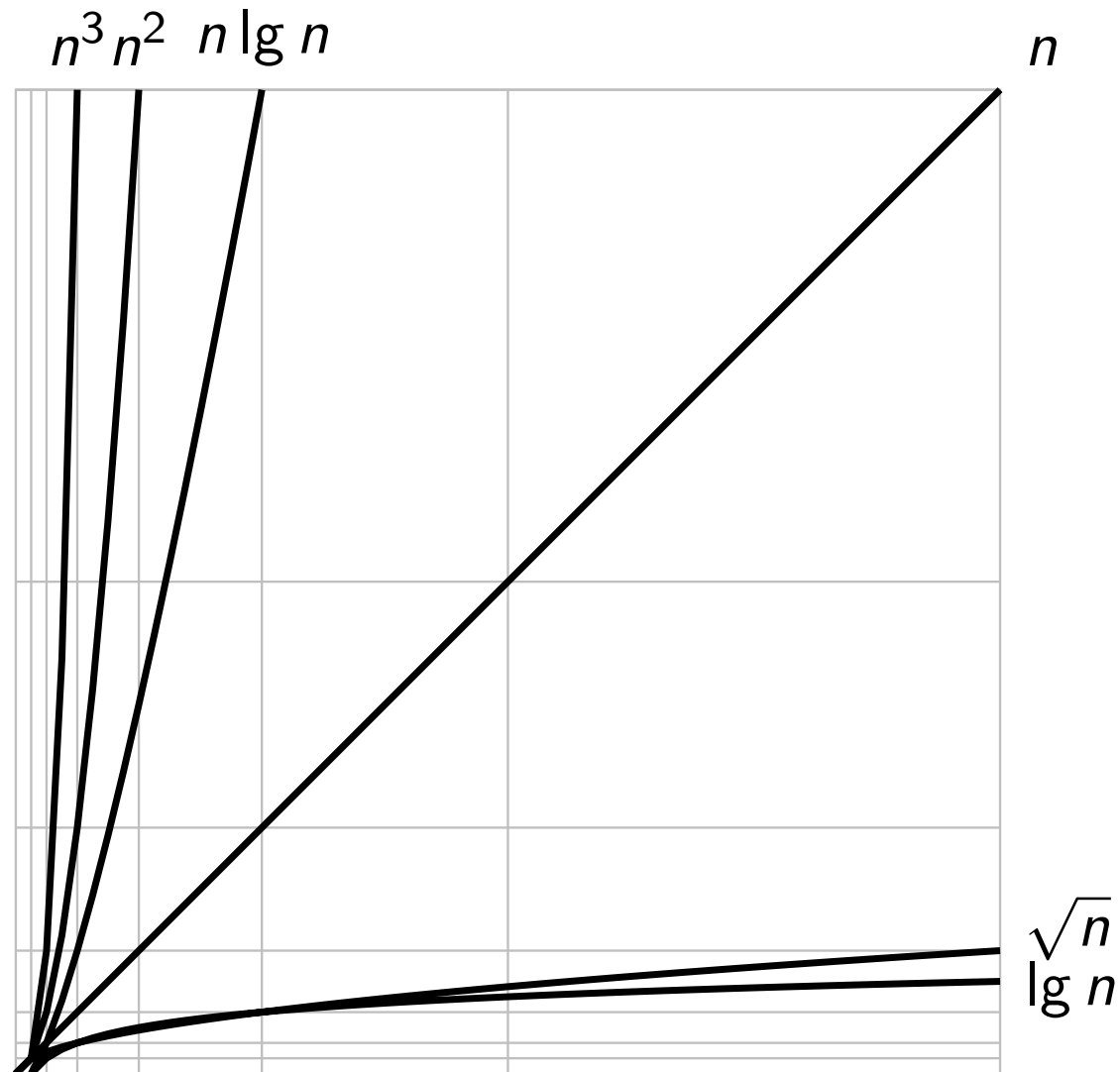
$$f = O(2^n)$$

$$f(n+1) \approx 2f(n)$$

$$f(n+10) \approx 1000f(n)$$

Analysis of Algorithms I

Order of growth



Analysis of Algorithms I

Epilogue

You are facing a high wall that stretches infinitely in both directions. There is a door in the wall, but you don't know how far away or in which direction. It is pitch dark, but you have a very dim lighted candle that will enable you to see the door when you are right next to it. Show that there is an algorithm that enables you to find the door by walking at most $O(n)$ steps, where n is the number of steps that you would have taken if you knew where the door is and walked directly to it. What is the constant multiple in the big- O bound for your algorithm?

Analysis of Algorithms I

Epilogue



Lecture 2

Analysis of Algorithms II

Analysis of Algorithms II

Contents

- The substitution method for solving recurrences CLRS 4.3
- The master method for solving recurrences CLRS 4.5

Reading

- T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein.
Introduction to Algorithms. The MIT Press, 3rd edition, 2009

Analysis of Algorithms II

Overview

- Recurrences give us a natural way to characterize the running times of recursive algorithms.
- A **recurrence** is a function defined in terms of
 - one or more base cases, and
 - itself, with smaller arguments.

Example (Solution: $T(n) = n$.)

- $$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ T(n-1) + 1 & \text{if } n > 1 \end{cases}$$

Example (Solution: $T(n) = n \lg n + n$.)

- $$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2T(n/2) + n & \text{if } n > 1 \end{cases}$$

Analysis of Algorithms II

Overview

Example (Solution: $T(n) = \lg \lg n$.)

- $$T(n) = \begin{cases} 0 & \text{if } n = 2 \\ T(\sqrt{n}) + 1 & \text{if } n > 2 \end{cases}$$

Example (Solution: $\Theta(n \lg n)$.)

- $$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ T(n/3) + T(2n/3) + n & \text{if } n > 1 \end{cases}$$

Analysis of Algorithms II

Overview

Analyzing recursive algorithms

In algorithm analysis, we usually express both the recurrence and its solution using asymptotic notation.

- Example: $T(n) = 2T(n/2) + \Theta(n)$, with solution $T(n) = \Theta(n \lg n)$.
- The boundary conditions are usually expressed as $T(n) = O(1)$ for sufficiently small n .
- When we desire an exact, rather than asymptotic, solution, we need to deal with boundary conditions.
- In practice, we just use asymptotics most of the time, and we ignore boundary conditions.

Analysis of Algorithms II

4.3 The substitution method for solving recurrences

Substitution method

1. Guess the solution.
2. Use induction to find the constants and show that the solution works.

Analysis of Algorithms II

4.3 The substitution method for solving recurrences

Example

- $$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2T(n/2) + n & \text{if } n > 1 \end{cases}$$

1. Guess: $T(n) = n \lg n + n$.

2. Induction:

Basis: $n = 1$ implies $n \lg n + n = 1 = T(n)$.

Inductive step: Inductive hypothesis is that $T(k) = k \lg k + k$ for all $k < n$.

We will use this inductive hypothesis for $T(n/2)$.

Analysis of Algorithms II

4.3 The substitution method for solving recurrences

Example (Continued)

$$\begin{aligned}T(n) &= 2T\left(\frac{n}{2}\right) + n \\&= 2\left(\frac{n}{2} \lg \frac{n}{2} + \frac{n}{2}\right) + n && \text{(by inductive hypothesis)} \\&= n \lg \frac{n}{2} + n + n \\&= n(\lg n - \lg 2) + n + n \\&= n \lg n - n + n + n \\&= n \lg n + n\end{aligned}$$

Analysis of Algorithms II

4.5 The master method for solving recurrences

Master method

- Used for many recurrences of the form

$$T(n) = aT(n/b) + f(n)$$

where $a \geq 1$, $b > 1$, and $f(n) > 0$.

- Describe the running time of an algorithm that divides a problem of size n into a subproblems, each of size n/b , where a and b are positive constants.
- The a subproblems are solved recursively, each in time $T(n/b)$.
- The function $f(n)$ encompasses the cost of dividing the problem and combining the results of the subproblems.

Analysis of Algorithms II

4.5 The master method for solving recurrences

Theorem (Master theorem)

Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be a function, and let $T(n)$ be defined on the nonnegative integers by the recurrence

$$T(n) = aT(n/b) + f(n)$$

where we interpret n/b to mean either $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$. Then $T(n)$ has the following asymptotic bounds:

- 1. If $f(n) = O(n^{\log_b a} / n^\epsilon)$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.*
- 2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$.*
- 3. If $f(n) = \Omega(n^{\log_b a} n^\epsilon)$ for some constant $\epsilon > 0$, **and if $a f(n/b) \leq c f(n)$ for some constant $c < 1$ and all sufficiently large n** , then $T(n) = \Theta(f(n))$.*

Analysis of Algorithms II

4.5 The master method for solving recurrences

Master theorem

- In each of the three cases, we compare $f(n)$ with $n^{\log_b a}$, and the larger of the two functions determines the solution to the recurrence.
- Case 1: If $f(n)$ is polynomially smaller than $n^{\log_b a}$, the solution is $T(n) = \Theta(n^{\log_b a})$.
- Case 2: If the two functions are the same size, we multiply by a logarithmic factor, and the solution is $T(n) = \Theta(n^{\log_b a} \lg n) = \Theta(f(n) \lg n)$.
- Case 3: If $f(n)$ is polynomially larger than $n^{\log_b a}$, the solution is $T(n) = \Theta(f(n))$.

Analysis of Algorithms II

4.5 The master method for solving recurrences

Master theorem

- The three cases do not cover all the possibilities for $f(n)$.
- There is a gap between cases 1 and 2 when $f(n)$ is smaller than $n^{\log_b a}$ but not polynomially smaller.
- There is a gap between cases 2 and 3 when $f(n)$ is larger than $n^{\log_b a}$ but not polynomially larger.
- If the function $f(n)$ falls into one of these gaps, or if the **regularity condition** in case 3 fails to hold, you cannot use the master method to solve the recurrence.
- However, the regularity condition is satisfied by most of the polynomially bounded functions that we shall encounter.

Analysis of Algorithms II

4.5 The master method for solving recurrences

Example ($T(n) = 8T(n/2) + \Theta(n^2)$)

For this recurrence, we have $a = 8$, $b = 2$, $f(n) = \Theta(n^2)$, and $n^{\log_b a} = n^{\lg 8} = n^3$. Since $f(n)$ is polynomially smaller than n^3 , because $n^2 = O(n^{3-\epsilon})$ for $\epsilon = 1$, case 1 applies, and $T(n) = \Theta(n^3)$.

Analysis of Algorithms II

4.5 The master method for solving recurrences

Example ($T(n) = 2T(n/2) + \Theta(n)$)

For this recurrence, we have $a = 2$, $b = 2$, $f(n) = \Theta(n)$, and $n^{\log_b a} = n^{\lg 2} = n$. Since $f(n)$ and n are the same size, case 2 applies, and $T(n) = \Theta(n \lg n)$.

Analysis of Algorithms II

4.5 The master method for solving recurrences

Example ($T(n) = 5T(n/2) + \Theta(n^3)$)

For this recurrence, we have $a = 5$, $b = 2$, $f(n) = \Theta(n^3)$, and $n^{\log_b a} = n^{\lg 5}$. Since $f(n)$ is polynomially larger than $n^{\lg 5}$, because $n^3 = \Omega(n^{\lg 5 + \epsilon})$ for $\epsilon = 3 - \lg 5 > 0$, and the regularity condition holds, because $a f(n/b) = 5(n/2)^3 = 5n^3/8 \leq c n^3$ for $c = 5/8 < 1$, case 3 applies, and $T(n) = \Theta(n^3)$.

Analysis of Algorithms II

4.5 The master method for solving recurrences

Example ($T(n) = 3T(n/4) + n \lg n$)

For this recurrence, we have $a = 3$, $b = 4$, $f(n) = n \lg n$, and $n^{\log_b a} = n^{\log_4 3} = O(n^{0.793})$. Since $n^{\log_4 3}$ is polynomially smaller than $f(n)$, because $n \lg n = \Omega(n^{\log_4 3 + \epsilon})$ for $\epsilon = 1 - \log_4 3 > 0$, and the regularity condition holds, because $a f(n/b) = 3(n/4) \lg(n/4) \leq (3/4)n \lg n = c f(n)$ for $c = 3/4 < 1$ and sufficiently large n , case 3 applies, and $T(n) = \Theta(n \lg n)$.

Analysis of Algorithms II

4.5 The master method for solving recurrences

Example ($T(n) = 2T(n/2) + n \lg n$)

For this recurrence, we have $a = 2$, $b = 2$, $f(n) = n \lg n$, and $n^{\log_b a} = n^{\lg 2} = n$.

- $f(n) = n \lg n$ is asymptotically larger than $n^{\log_b a} = n$.
- $f(n) = n \lg n$ is not **polynomially** larger than $n^{\log_b a} = n$.
- The ratio $f(n)/n^{\log_b a} = (n \lg n)/n = \lg n$ is asymptotically less than n^ϵ for any constant $\epsilon > 0$.
- The recurrence falls into the gap between case 2 and case 3.

Analysis of Algorithms II

4.5 The master method for solving recurrences

Theorem (Master theorem, simplified)

Let $a \geq 1$ and $b > 1$ be constants, let $k \geq 0$ be a constant, and let $T(n)$ be defined on the nonnegative integers by the recurrence

$$T(n) = aT(n/b) + n^k$$

where we interpret n/b to mean either $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$. Then $T(n)$ has the following asymptotic bounds:

1. *If $k < \log_b a$, then $T(n) = \Theta(n^{\log_b a})$.*
2. *If $k = \log_b a$, then $T(n) = \Theta(n^k \lg n)$.*
3. *If $k > \log_b a$, then $T(n) = \Theta(n^k)$.*

Analysis of Algorithms II

4.5 The master method for solving recurrences

Theorem (Master theorem, simplified)

Let $a \geq 1$ and $b > 1$ be constants, let $k \geq 0$ be a constant, and let $T(n)$ be defined on the nonnegative integers by the recurrence

$$T(n) = aT(n/b) + n^k$$

where we interpret n/b to mean either $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$. Then $T(n)$ has the following asymptotic bounds:

- 1. If $b^k < a$, then $T(n) = \Theta(n^{\log_b a})$.*
- 2. If $b^k = a$, then $T(n) = \Theta(n^k \lg n)$.*
- 3. If $b^k > a$, then $T(n) = \Theta(n^k)$.*

Analysis of Algorithms II

4.5 The master method for solving recurrences

Example ($T(n) = 8T(n/2) + \Theta(n^2)$)

For this recurrence, we have $a = 8$, $b = 2$, $f(n) = \Theta(n^2)$, and $n^{\log_b a} = n^{\lg 8} = n^3$. Since $f(n)$ is polynomially smaller than n^3 , because $n^2 = O(n^{3-\epsilon})$ for $\epsilon = 1$, case 1 applies, and $T(n) = \Theta(n^3)$.

Since $k = 2 < 3 = \lg 8 = \log_b a$

Since $b^k = 2^2 < 8 = a$

Analysis of Algorithms II

4.5 The master method for solving recurrences

Example ($T(n) = 2T(n/2) + \Theta(n)$)

For this recurrence, we have $a = 2$, $b = 2$, $f(n) = \Theta(n)$, and $n^{\log_b a} = n^{\lg 2} = n$. Since $f(n)$ and n are the same size, case 2 applies, and $T(n) = \Theta(n \lg n)$.

Since $k = 1 = \lg 2 = \log_b a$

Since $b^k = 2^1 = 2 = a$

Analysis of Algorithms II

4.5 The master method for solving recurrences

Example ($T(n) = 5T(n/2) + \Theta(n^3)$)

For this recurrence, we have $a = 5$, $b = 2$, $f(n) = \Theta(n^3)$, and $n^{\log_b a} = n^{\lg 5}$. Since $f(n)$ is polynomially larger than $n^{\lg 5}$, because $n^3 = \Omega(n^{\lg 5 + \epsilon})$ for $\epsilon = 3 - \lg 5 > 0$, and the regularity condition holds, because $a f(n/b) = 5(n/2)^3 = 5n^3/8 \leq c n^3$ for $c = 5/8 < 1$, case 3 applies, and $T(n) = \Theta(n^3)$.

Since $k = 3 > \lg 5 = \log_b a$

Since $b^k = 2^3 > 5 = a$

Lecture 3

Divide and Conquer I

Divide and Conquer I

Contents

- The divide-and-conquer approach CLRS 2.3.1
- Analyzing divide-and-conquer algorithms CLRS 2.3.2
- Description of quicksort CLRS 7.1
- Performance of quicksort CLRS 7.2
- Lower bounds for sorting CLRS 8.1

Reading

- T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein.
Introduction to Algorithms. The MIT Press, 3rd edition, 2009

Divide and Conquer I

2.3.1 The divide-and-conquer approach

Divide and conquer

A common approach to design algorithms.

Divide the problem into a number of subproblems that are smaller instances of the same problem.

Conquer the subproblems by solving them recursively.

Base case: If the problems are small enough, just solve them by brute force.

Combine the subproblem solutions to give a solution to the original problem.

Divide and Conquer I

2.3.1 The divide-and-conquer approach

Merge sort

- A sorting algorithm based on divide and conquer.
- We state each subproblem as sorting a subarray $A[p..r]$.
- Initially, $p = 1$ and $r = n$, but these values change as we recurse through subproblems.

To sort $A[p..r]$:

Divide by splitting into two subarrays $A[p..q]$ and $A[q + 1..r]$, where q is the halfway point of $A[p..r]$.

Conquer by recursively sorting the two subarrays $A[p..q]$ and $A[q + 1..r]$.

Combine by merging the two sorted subarrays $A[p..q]$ and $A[q + 1..r]$ to produce a single sorted subarray $A[p..r]$.

Divide and Conquer I

2.3.1 The divide-and-conquer approach

Merge sort

```
MERGE-SORT( $A, p, r$ )
```

```
  if  $p < r$  then
```

```
     $q = \lfloor (p + r) / 2 \rfloor$ 
```

```
    MERGE-SORT( $A, p, q$ )
```

```
    MERGE-SORT( $A, q + 1, r$ )
```

```
    MERGE( $A, p, q, r$ )
```

check for base case

divide

conquer

conquer

combine

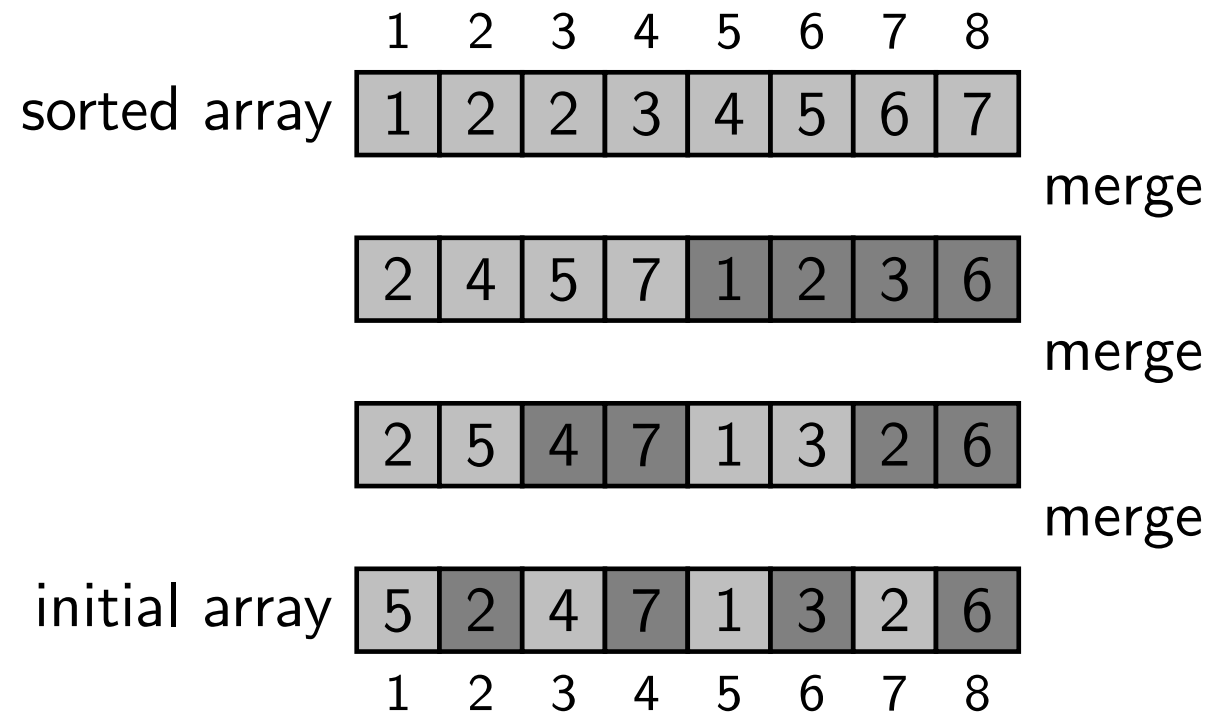
The initial call is MERGE-SORT($A, 1, n$).

Divide and Conquer I

2.3.1 The divide-and-conquer approach

Merge sort

Example (Bottom-up view for $n = 8$)



Divide and Conquer I

2.3.1 The divide-and-conquer approach

Merge sort

MERGE(A, p, q, r)

$n_1 = q - p + 1$

$n_2 = r - q$

$L[1..n_1 + 1] = \text{new array}$

$R[1..n_2 + 1] = \text{new array}$

for $i = 1$ **to** n_1 **do**

$L[i] = A[p + i - 1]$

for $j = 1$ **to** n_2 **do**

$R[j] = A[q + j]$

$L[n_1 + 1] = \infty$

$R[n_2 + 1] = \infty$

$i = 1$

$j = 1$

for $k = p$ **to** r **do**

if $L[i] \leq R[j]$ **then**

$A[k] = L[i]$

$i = i + 1$

else

$A[k] = R[j]$

$j = j + 1$

Divide and Conquer I

2.3.1 The divide-and-conquer approach

Merge sort

- Merging takes $\Theta(n)$ time, where $n = r - p + 1$ is the number of elements being merged.
 - The first two **for** loops take $\Theta(n_1 + n_2) = \Theta(n)$ time.
 - The last **for** loop makes n iterations, each taking constant time, for $\Theta(n)$ time.
- We use a special **sentinel** value, ∞ , to simplify the code.
- We know in advance that there are exactly $r - p + 1$ nonsentinel elements. We can stop once we have performed $r - p + 1$ basic steps. Never a need to check for sentinels, since they are always greater than any element.
- Rather than even counting basic steps, just fill up the output array from index p up through and including index r .

Divide and Conquer I

2.3.2 Analyzing divide-and-conquer algorithms

Overview

We use a **recurrence** to describe the running time of a divide-and-conquer algorithm. Let $T(n)$ be the running time on a problem of size n .

- If the problem size is small enough (say, $n \leq c$ for some constant c), we have a base case. The brute-force solution takes constant time.
- Otherwise, suppose that we divide into a subproblems, each $1/b$ the size of the original. (In merge sort, $a = b = 2$.)
- Let the time to divide a problem of size n be $D(n)$.
- We have a subproblems to solve, each of size n/b . Each subproblem takes $T(n/b)$ time to solve. We spend $aT(n/b)$ time solving subproblems.
- Let the time to combine solutions be $C(n)$.

Divide and Conquer I

2.3.2 Analyzing divide-and-conquer algorithms

Overview

- We get the recurrence

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq c \\ aT(n/b) + D(n) + C(n) & \text{otherwise} \end{cases}$$

Divide and Conquer I

2.3.2 Analyzing divide-and-conquer algorithms

Analyzing merge sort

- For simplicity, assume that n is a power of 2. Each divide step yields two subproblems, both of size exactly $n/2$.
- The base case occurs when $n = 1$. When $n \geq 2$, we have
 - Divide** Just compute q as the average of p and r .
 - Conquer** Recursively solve 2 subproblems, each of size $n/2$.
 - Combine** Merging n elements takes $\Theta(n)$ time.
- The recurrence for merge sort running time is

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases}$$

- By the master theorem, we can show that this recurrence has the solution $T(n) = \Theta(n \lg n)$.

Divide and Conquer I

2.3.1 The divide-and-conquer approach

Exercise 2.3-5

- Consider the **searching problem**: Given a sequence A of n numbers and a value v , output an index i such that $v = A[i]$ or the special value NIL if v does not appear in A .
- Observe that if the sequence A is sorted, we can check the midpoint of the sequence against v and eliminate half of the sequence from further consideration.
- The **binary search** algorithm repeats this procedure, halving the size of the remaining portion of the sequence each time.
- Write pseudocode, either iterative or recursive, for binary search.
- Argue that the worst-case running time of binary search is $\Theta(\lg n)$.

Divide and Conquer I

2.3.1 The divide-and-conquer approach

Solution to Exercise 2.3-5

- BINARY-SEARCH takes a sorted array A , a value v , and a range $[low..high]$ of the array, in which we search for the value v .
- The procedure compares v to the array entry at the midpoint of the range and decides to eliminate half the range from further consideration.
- We give both iterative and recursive versions, each of which returns either an index i such that $A[i] = v$, or NIL, if no entry of $A[low..high]$ contains the value v .
- An initial call to either version should have the parameters A , v , 1, n .

Divide and Conquer I

2.3.1 The divide-and-conquer approach

Solution to Exercise 2.3-5

```
BINARY-SEARCH(A, v, low, high)  
  while low ≤ high do  
    mid = ⌊(low + high)/2⌋  
    if v = A[mid] then  
      return mid  
    else if v > A[mid] then  
      low = mid + 1  
    else  
      high = mid - 1  
  return NIL
```

Divide and Conquer I

2.3.1 The divide-and-conquer approach

Solution to Exercise 2.3-5

```
BINARY-SEARCH(A, v, low, high)  
  if low > high then  
    return NIL  
  mid =  $\lfloor (\textit{low} + \textit{high}) / 2 \rfloor$   
  if v = A[mid] then  
    return mid  
  else if v > A[mid] then  
    return BINARY-SEARCH(A, v, mid + 1, high)  
  else  
    return BINARY-SEARCH(A, v, low, mid - 1)
```

Divide and Conquer I

2.3.1 The divide-and-conquer approach

Solution to Exercise 2.3-5

- An initial call to either version should have the parameters A , v , 1 , n .
- Both procedures terminate the search unsuccessfully when the range is empty (when $low > high$) and terminate it successfully if the value v has been found.
- Based on the comparison of v to the middle element in the searched range, the search continues with the range halved.
- The recurrence for these procedures is therefore $T(n) = T(n/2) + \Theta(1)$, whose solution is $T(n) = \Theta(\lg n)$.

Divide and Conquer I

7.1 Description of quicksort

- Quicksort is based on the three-step process of divide-and-conquer. To sort the subarray $A[p..r]$:

Divide: Partition $A[p..r]$ into two (possibly empty) subarrays $A[p..q - 1]$ and $A[q + 1..r]$, such that each element in the first subarray $A[p..q - 1]$ is $\leq A[q]$ and $A[q]$ is $\leq$ each element in the second subarray $A[q + 1..r]$.

Conquer: Sort the two subarrays by recursive calls to QUICKSORT.

Combine: No work is needed to combine the subarrays, because they are sorted in place.

- Perform the divide step by a procedure PARTITION, which returns the index q that marks the position separating the subarrays.

Divide and Conquer I

7.1 Description of quicksort

```
QUICKSORT( $A, p, r$ )
```

```
  if  $p < r$  then
```

```
     $q = \text{PARTITION}(A, p, r)$ 
```

```
    QUICKSORT( $A, p, q - 1$ )
```

```
    QUICKSORT( $A, q + 1, r$ )
```

- The initial call is QUICKSORT($A, 1, n$).

Divide and Conquer I

7.1 Description of quicksort

Hoare partitioning (Problem 7-1)

PARTITION(A, p, r)

$x = A[p]$

$i = p - 1$

$j = r + 1$

while TRUE **do**

repeat

$j = j - 1$

until $A[j] \leq x$

repeat

$i = i + 1$

until $A[i] \geq x$

if $i < j$ **then**

 exchange $A[i]$ with $A[j]$

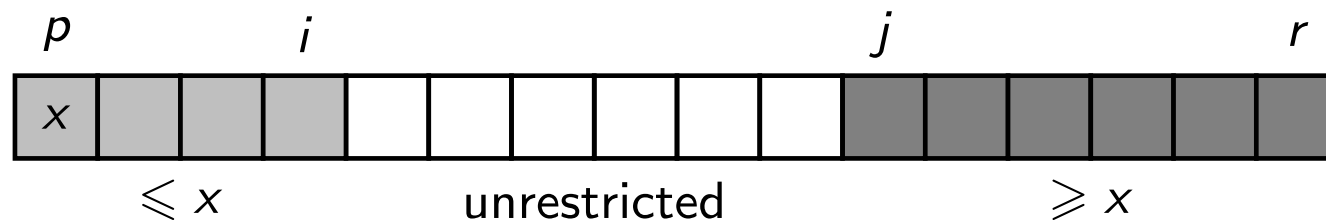
else

return j

Divide and Conquer I

7.1 Description of quicksort

- Hoare PARTITION always selects the first element $A[p]$ in the subarray $A[p..r]$ as the **pivot**, the element around which to partition.
- As the procedure executes, the array is partitioned into three regions, some of which may be empty:
 - All entries in $A[p..i]$ are $\leq$ pivot.
 - All entries in $A[j..r]$ are $\geq$ pivot.
 - No entries in $A[i + 1..j - 1]$ have yet been examined, and so we do not know how they compare to the pivot.



Divide and Conquer I

7.1 Description of quicksort

Correctness of Hoare partitioning

Use the loop invariant to prove correctness of PARTITION:

- Before the loop starts, all the conditions of the loop invariant are satisfied, because $A[p]$ is the pivot and the subarrays $A[p..i]$ and $A[j..r]$ are empty.
- While the loop is running, index i is incremented and index j is decremented until $A[i] \geq \text{pivot} \geq A[j]$. If these inequalities are strict, $A[i]$ is too large to belong to the first region and $A[j]$ is too small to belong to the second region. Thus, by swapping $A[i]$ and $A[j]$, we can extend the two regions.
- When the **while** loop terminates, all elements in A are partitioned into two subarrays $A[p..q]$ and $A[q + 1..r]$, where $p \leq q < r$, such that $A[p..q] \leq A[q + 1..r]$, and the value $q = j$ is returned.

Divide and Conquer I

7.1 Description of quicksort

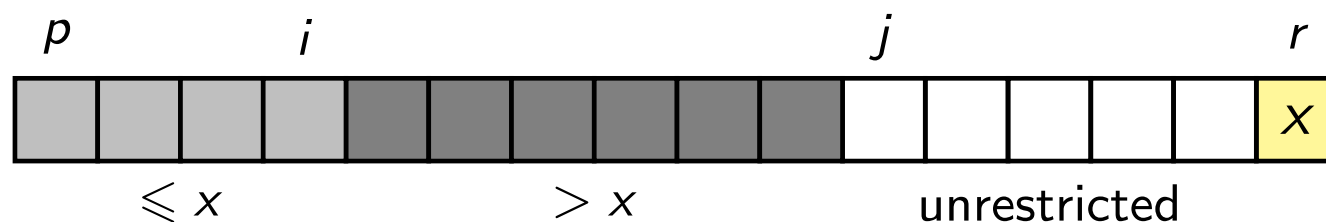
Lomuto partitioning

```
PARTITION( $A, p, r$ )  
   $x = A[r]$   
   $i = p - 1$   
  for  $j = p$  to  $r - 1$  do  
    if  $A[j] \leq x$  then  
       $i = i + 1$   
      exchange  $A[i]$  with  $A[j]$   
  exchange  $A[i + 1]$  with  $A[r]$   
  return  $i + 1$ 
```

Divide and Conquer I

7.1 Description of quicksort

- Lomuto PARTITION always selects the last element $A[r]$ in the subarray $A[p..r]$ as the **pivot**, the element around which to partition.
- As the procedure executes, the array is partitioned into four regions, some of which may be empty:
 - All entries in $A[p..i]$ are $\leq$ pivot.
 - All entries in $A[i+1..j-1]$ are $>$ pivot.
 - No entries in $A[j..r-1]$ have yet been examined, and so we do not know how they compare to the pivot.
 - $A[r] = \text{pivot}$.



Divide and Conquer I

7.1 Description of quicksort

Correctness of Lomuto partitioning

Use the loop invariant to prove correctness of PARTITION:

- Before the loop starts, all the conditions of the loop invariant are satisfied, because $A[r]$ is the pivot and the subarrays $A[p..i]$ and $A[i + 1..j - 1]$ are empty.
- While the loop is running, if $A[j] \leq \text{pivot}$, then $A[j]$ and $A[i + 1]$ are swapped and then i and j are incremented. If $A[j] > \text{pivot}$, then only j is incremented.
- When the loop terminates, $j = r$, so all elements in A are partitioned into one of the three cases: $A[p..i] \leq \text{pivot}$, $A[i + 1..r - 1] > \text{pivot}$, and $A[r] = \text{pivot}$.

The last two lines of PARTITION move the pivot element from the end of the array to between the two subarrays.

Divide and Conquer I

7.2 Performance of quicksort

Worst case

- Occurs when the subarrays are completely unbalanced every time.
- Have 0 elements in one subarray and $n - 1$ elements in the other subarray.
- Since the recursive call on an array of size 0 just returns, $T(0) = \Theta(1)$, and the recurrence for the running time is

$$\begin{aligned} T(n) &= T(n-1) + T(0) + \Theta(n) \\ &= T(n-1) + \Theta(n) \end{aligned}$$

- Using the substitution method, this recurrence has the solution $T(n) = \Theta(n^2)$.

Divide and Conquer I

7.2 Performance of quicksort

Best case

- Occurs when the subarrays are completely balanced every time.
- Each subarray has at most $n/2$ elements, since one is of size $\lfloor n/2 \rfloor$ and the other is of size $\lceil n/2 \rceil - 1$.
- The recurrence for the running time is then

$$T(n) = 2T(n/2) + \Theta(n)$$

- By case 2 of the master theorem, this recurrence has the solution $T(n) = \Theta(n \lg n)$.

Divide and Conquer I

8.1 Lower bounds for sorting

Lower bounds

- $\Omega(n)$ to examine all the input.
- All sorting algorithms seen so far (bubble sort, selection sort, insertion sort, merge sort, quicksort) are comparison sort algorithms and are $\Omega(n \lg n)$.
- We will show that $\Omega(n \lg n)$ is a lower bound for comparison sort algorithms.

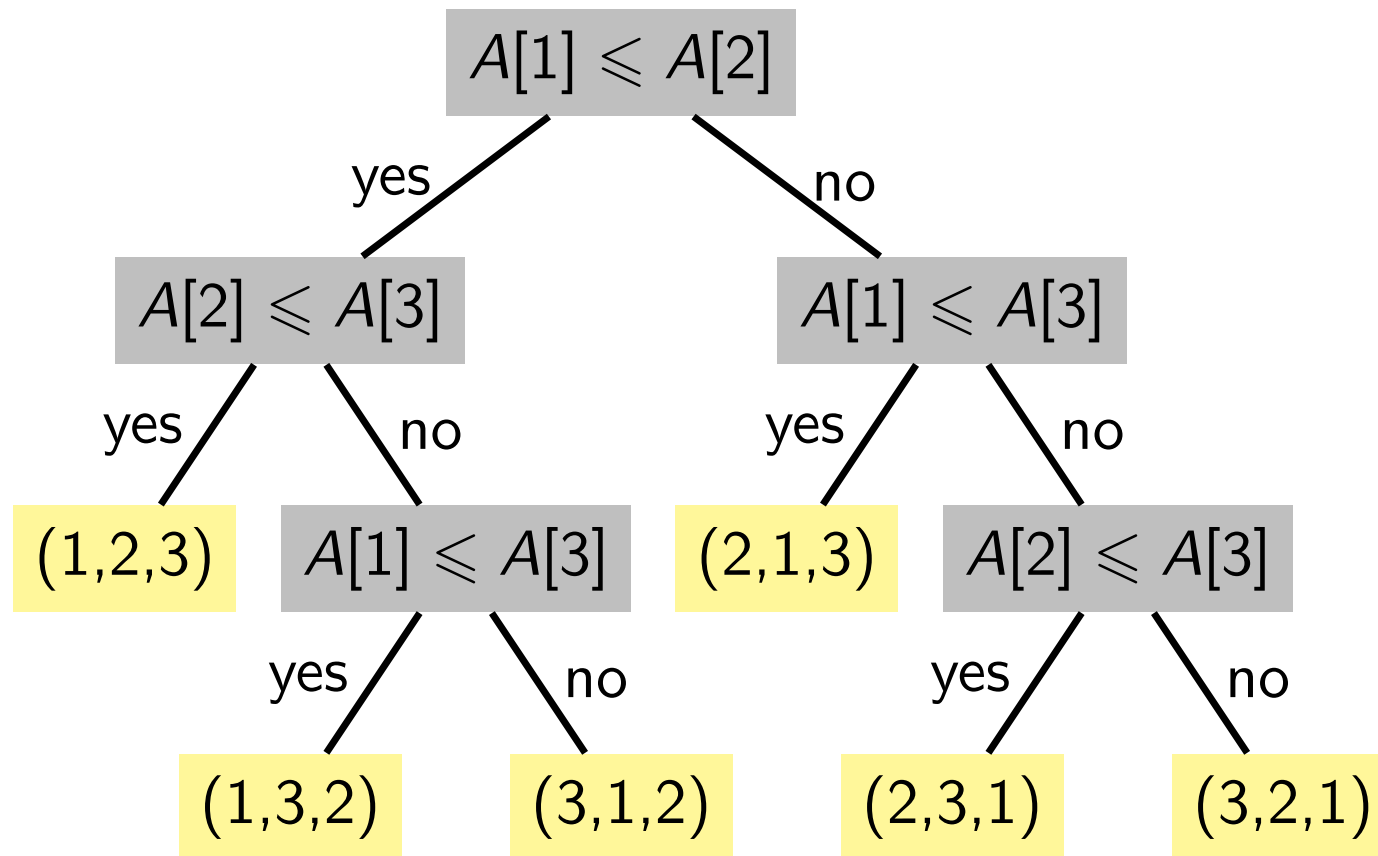
Decision tree

- Abstraction of any comparison sort.
- Represents comparisons made by a specific sorting algorithm on inputs of a given size. We are counting **only** comparisons.
- Abstracts away everything else: control and data movement.

Divide and Conquer I

8.1 Lower bounds for sorting

Example (Decision tree for insertion sort on 3 elements)



Divide and Conquer I

8.1 Lower bounds for sorting

Decision tree

- How many leaves on the decision tree? There are at least $n!$ leaves, because every permutation must appear at least once.
- For any comparison sort, there is one tree for each n . View the tree as if the algorithm splits in two at each node, based on the information it has determined up to that point.
- The tree models all possible execution traces.
- What is the length of the longest path from root to leaf? Depends on the algorithm. $\Theta(n^2)$ for insertion sort. $\Theta(n \lg n)$ for merge sort.

Divide and Conquer I

8.1 Lower bounds for sorting

Lemma

Any binary tree of height h has at most 2^h leaves.

Theorem

Any decision tree that sorts n elements has height $\Omega(n \lg n)$.

Corollary

Merge sort is an asymptotically optimal comparison sort.

Divide and Conquer I

8.1 Lower bounds for sorting

Lemma

Any binary tree of height h has at most 2^h leaves.

Proof.

By induction on h .

Basis: $h = 0$. The tree is just a node, which is a leaf. $2^h = 1$.

Inductive step: Assume true for height $h - 1$. Extend tree of height $h - 1$ by making as many new leaves as possible. Each leaf becomes parent to two new leaves. Let $L(h)$ be the maximum number of leaves in a binary tree of height h . Then:

$$L(h) = 2 \cdot L(h - 1) = 2 \cdot 2^{h-1} = 2^h$$

Divide and Conquer I

8.1 Lower bounds for sorting

Theorem

Any decision tree that sorts n elements has height $\Omega(n \lg n)$.

Proof.

Consider a decision tree of height h with ℓ leaves corresponding to a comparison sort on n elements.

- Because each of the $n!$ permutations of the input appears as some leaf, we have $n! \leq \ell$.
- By the lemma, we have $n! \leq \ell \leq 2^h$.
- By taking logarithms, $h \geq \lg(n!)$
- Using Stirling's approximation, $n! > (n/e)^n$. Then:

$$h > \lg(n/e)^n = n \lg(n/e) = n \lg n - n \lg e = \Omega(n \lg n)$$

Divide and Conquer I

8.1 Lower bounds for sorting

Corollary

Merge sort is an asymptotically optimal comparison sort.

Proof.

- The length of the longest simple path from the root of a decision tree to any of its leaves represents the worst-case number of comparisons that the corresponding sorting algorithm performs.
- The worst-case number of comparisons for a given comparison sort algorithm equals the height of its decision tree.
- The $O(n \lg n)$ upper bound on the running time for merge sort matches the $\Omega(n \lg n)$ worst-case lower bound from the theorem.

Lecture 4

Divide and Conquer II

Divide and Conquer II

Contents

- Karatsuba's algorithm
- Strassen's algorithm for matrix multiplication CLRS 4.2
- Minimum and maximum CLRS 9.1
- Selection in worst-case linear time CLRS 9.3

Reading

- T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein.
Introduction to Algorithms. The MIT Press, 3rd edition, 2009

Divide and Conquer II

Karatsuba's algorithm

Integer multiplication

- If $x = (x_{n-1}, \dots, x_1, x_0)$ and $y = (y_{n-1}, \dots, y_1, y_0)$ are **long** n -bit integers, then

$$x = \sum_{i=0}^{n-1} x_i 2^i, \quad y = \sum_{j=0}^{n-1} y_j 2^j, \quad x \cdot y = \sum_{i=0}^{n-1} x_i 2^i \sum_{j=0}^{n-1} y_j 2^j$$

Analysis

- Two nested loops, each iterate n times, and innermost loop body takes constant time: $\Theta(n^2)$ time.

Divide and Conquer II

Karatsuba's algorithm

Simple divide-and-conquer method

```
MULT( $x, y, n$ )  
  if  $n = 1$  then  
    return  $x y$   
  else  
    partition  $x$  into  $a, b$  such that  $x = a 2^{n/2} + b$   
    partition  $y$  into  $c, d$  such that  $y = c 2^{n/2} + d$   
     $ac = \text{MULT}(a, c, n/2)$   
     $ad = \text{MULT}(a, d, n/2)$   
     $bc = \text{MULT}(b, c, n/2)$   
     $bd = \text{MULT}(b, d, n/2)$   
    return  $ac 2^n + (ad + bc) 2^{n/2} + bd$ 
```

Divide and Conquer II

Karatsuba's algorithm

Analysis

- Because multiplying by 2^n and $2^{n/2}$ is equivalent to shifting,

$$x y = (a 2^{n/2} + b) (c 2^{n/2} + d) = a c 2^n + (a d + b c) 2^{n/2} + b d$$

- Recurrence is

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq 1 \\ 4 T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases}$$

- By the master method, solution is $T(n) = \Theta(n^2)$.

Divide and Conquer II

Karatsuba's algorithm

Karatsuba's method

```
MULT( $x, y, n$ )  
  if  $n = 1$  then  
    return  $x y$   
  else  
    partition  $x$  into  $a, b$  such that  $x = a 2^{n/2} + b$   
    partition  $y$  into  $c, d$  such that  $y = c 2^{n/2} + d$   
     $ac = \text{MULT}(a, c, n/2)$   
     $bd = \text{MULT}(b, d, n/2)$   
     $abcd = \text{MULT}(a + b, c + d, n/2)$   
    return  $(abcd - ac - bd) 2^{n/2} + ac 2^n + bd$ 
```

Divide and Conquer II

Karatsuba's algorithm

Analysis

- Because multiplying by 2^n and $2^{n/2}$ is equivalent to shifting,

$$x y = ((a + b)(c + d) - a c - b d) 2^{n/2} + a c 2^n + b d$$

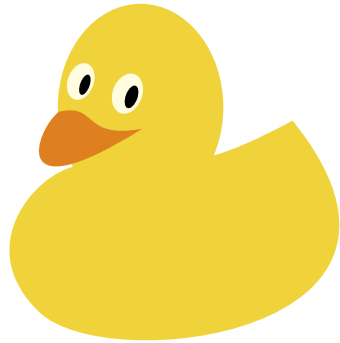
- Recurrence is

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq 1 \\ 3T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases}$$

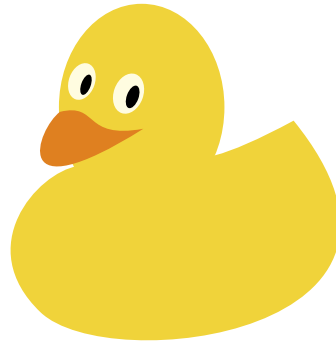
- By the master method, solution is $T(n) = \Theta(n^{\lg 3})$.

Divide and Conquer II

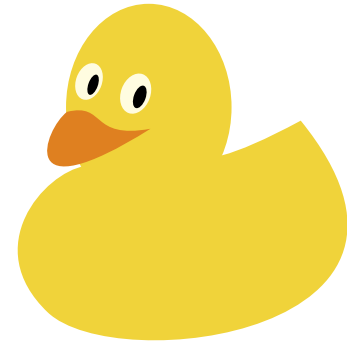
Karatsuba's algorithm



Anatoly
Karatsuba
(1937–2008)



Yuri Ofman
(1939–)



Andrey
Kolmogorov
(1903–1987)

Divide and Conquer II

4.2 Strassen's algorithm for matrix multiplication

Matrix multiplication

- If $A = (a_{ij})$ and $B = (b_{ij})$ are **square** $n \times n$ matrices, then in the product $C = A \cdot B$, we define the entry c_{ij} by

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj}$$

for $i, j = 1, \dots, n$.

- Need to compute n^2 entries of C .
- Each entry is the sum of n values.

Divide and Conquer II

4.2 Strassen's algorithm for matrix multiplication

Obvious method

```
MULT( $A, B, n$ )  
  let  $C$  be a new  $n \times n$  matrix  
  for  $i = 1$  to  $n$  do  
    for  $j = 1$  to  $n$  do  
       $c_{ij} = 0$   
      for  $k = 1$  to  $n$  do  
         $c_{ij} = c_{ij} + a_{ik} \cdot b_{kj}$   
  return  $C$ 
```

Analysis

- Three nested loops, each iterates n times, and innermost loop body takes constant time: $\Theta(n^3)$ time.

Divide and Conquer II

4.2 Strassen's algorithm for matrix multiplication

Is that the best we can do?

- Seems like any algorithm to multiply matrices must take $\Omega(n^3)$ time:
 - Must compute n^2 entries.
 - Each entry is the sum of n terms.
- But with Strassen's method, we can multiply matrices in $o(n^3)$ time.
 - Strassen's algorithm runs in $\Theta(n^{\lg 7})$ time.
 - $2.80 \leq \lg 7 \leq 2.81$
 - Hence, runs in $O(n^{2.81})$ time.

Divide and Conquer II

4.2 Strassen's algorithm for matrix multiplication

Simple divide-and-conquer method

- Assume that n is a power of 2.
- Partition each of A , B , and C into four $n/2 \times n/2$ matrices:

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

- Rewrite $C = A \cdot B$ as

$$\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \cdot \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

Divide and Conquer II

4.2 Strassen's algorithm for matrix multiplication

Simple divide-and-conquer method

- This corresponds to the four equations
 - $C_{11} = A_{11} \cdot B_{11} + A_{12} \cdot B_{21}$
 - $C_{12} = A_{11} \cdot B_{12} + A_{12} \cdot B_{22}$
 - $C_{21} = A_{21} \cdot B_{11} + A_{22} \cdot B_{21}$
 - $C_{22} = A_{21} \cdot B_{12} + A_{22} \cdot B_{22}$
- Each of these four equations multiplies two $n/2 \times n/2$ matrices and then adds their $n/2 \times n/2$ products.
- Use these equations to get a divide-and-conquer algorithm.

Divide and Conquer II

4.2 Strassen's algorithm for matrix multiplication

Simple divide-and-conquer method

```
MULT( $A, B, n$ )  
  let  $C$  be a new  $n \times n$  matrix  
  if  $n = 1$  then  
     $c_{11} = a_{11} \cdot b_{11}$   
  else  
    Partition  $A$ ,  $B$ , and  $C$  into four  $n/2 \times n/2$  matrices  
     $C_{11} = \text{MULT}(A_{11}, B_{11}, n/2) + \text{MULT}(A_{12}, B_{21}, n/2)$   
     $C_{12} = \text{MULT}(A_{11}, B_{12}, n/2) + \text{MULT}(A_{12}, B_{22}, n/2)$   
     $C_{21} = \text{MULT}(A_{21}, B_{11}, n/2) + \text{MULT}(A_{22}, B_{21}, n/2)$   
     $C_{22} = \text{MULT}(A_{21}, B_{12}, n/2) + \text{MULT}(A_{22}, B_{22}, n/2)$   
  return  $C$ 
```

Divide and Conquer II

4.2 Strassen's algorithm for matrix multiplication

Analysis

- Let $T(n)$ be the time to multiply two $n \times n$ matrices.

Base case: When $n = 1$, perform one scalar multiplication,
 $T(1) = \Theta(1)$.

Recursive case: When $n > 1$,

- Dividing takes $\Theta(1)$ time, using index calculations.
- Conquering makes 8 recursive calls, each multiplying $n/2 \times n/2$ matrices, contributing $8T(n/2)$ time.
- Combining takes $\Theta(n^2)$ time to add $n/2 \times n/2$ matrices four times.

Divide and Conquer II

4.2 Strassen's algorithm for matrix multiplication

Analysis

- Recurrence is

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 8T(n/2) + \Theta(n^2) & \text{if } n > 1 \end{cases}$$

- By the master method, solution is $T(n) = \Theta(n^3)$.
- Asymptotically, no better than the obvious method.

Divide and Conquer II

4.2 Strassen's algorithm for matrix multiplication

Strassen's method

- Perform only 7 recursive multiplications of $n/2 \times n/2$ matrices, rather than 8.
- Will cost several additions of $n/2 \times n/2$ matrices, but just a constant number more.

Analysis

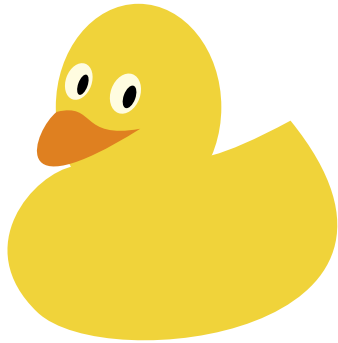
- Recurrence is

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 7T(n/2) + \Theta(n^2) & \text{if } n > 1 \end{cases}$$

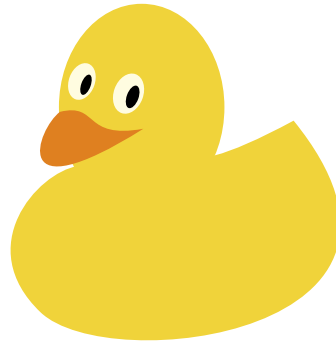
- By the master method, solution is $T(n) = \Theta(n^{\lg 7})$.

Divide and Conquer II

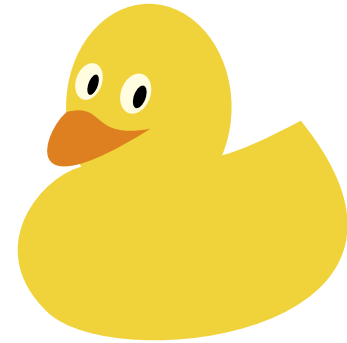
4.2 Strassen's algorithm for matrix multiplication



Volker Strassen
(1936–)



Don
Coppersmith
(1950–)



Shmuel
Winograd
(1936–)

Divide and Conquer II

9.1 Minimum and maximum

Selection problem

- Given a set A of n (distinct) numbers and an integer i , with $1 \leq i \leq n$, find the element $x \in A$ that is larger than exactly $i - 1$ other elements in A (the i th smallest element of A).
- We can solve the selection problem in $O(n \lg n)$ time
 - Sort the numbers using an $O(n \lg n)$ time algorithm, such as merge sort.
 - Return the i th element in the sorted array.
- The **minimum** is the i th smallest element for $i = 1$.
- The **maximum** is the i th smallest element for $i = n$.
- The (lower) **median** is the i th smallest element for $i = n/2$.

Divide and Conquer II

9.1 Minimum and maximum

- We can easily obtain an upper bound of $n - 1$ comparisons for finding the minimum of a set of n elements.
 - Examine each element in turn and keep track of the smallest one.
 - This is the best we can do, because each element, except the minimum, must be compared to a smaller element at least once.

```
MINIMUM( $A, n$ )  
   $min = A[1]$   
  for  $i = 2$  to  $n$  do  
    if  $min > A[i]$  then  
       $min = A[i]$   
  return  $min$ 
```

- The maximum can be found in exactly the same way.

Divide and Conquer II

9.1 Minimum and maximum

Simultaneous minimum and maximum

- A simple algorithm to find both the minimum and the maximum is to find each one independently.
 - There will be $n - 1$ comparisons for the minimum and $n - 2$ comparisons for the maximum, for a total of $2n - 2$ comparisons.
 - This will result in $\Theta(n)$ time, which is asymptotically optimal.

Divide and Conquer II

9.1 Minimum and maximum

Simultaneous minimum and maximum

- In fact, at most $3\lfloor n/2 \rfloor$ comparisons suffice to find both the minimum and the maximum.
 - Maintain both the minimum and maximum elements seen thus far.
 - Do not compare each element to the minimum and maximum separately, but process elements in pairs.
 - Compare the elements of a pair first **to each other**, then compare the smaller to the minimum so far and the larger to the maximum so far.
- This leads to only 3 comparisons for every 2 elements.

Divide and Conquer II

9.1 Minimum and maximum

Simultaneous minimum and maximum

- Setting up the initial values for the minimum and maximum depends on whether n is odd or even.
 - If n is odd, set both the minimum and maximum to the value of the first element, then process the rest of the elements in pairs.
 - If n is even, compare the first two elements to determine the initial values of the minimum and maximum, then process the rest of the elements in pairs.

Divide and Conquer II

9.1 Minimum and maximum

Simultaneous minimum and maximum

- Analysis of the total number of comparisons:
 - If n is odd, we perform $3(n-1)/2 = 3\lfloor n/2 \rfloor$ comparisons.
 - If n is even, we perform 1 initial comparison followed by $3(n-2)/2$ comparisons, for a total of $3n/2 - 2$.

$$\frac{3(n-2)}{2} + 1 = \frac{3n-6}{2} + 1 = \frac{3n}{2} - 3 + 1 = \frac{3n}{2} - 2$$

- In either case, the total number of comparisons is at most $3\lfloor n/2 \rfloor$.

Divide and Conquer II

9.3 Selection in worst-case linear time

- We can find the i th smallest element in $O(n)$ time **in the worst case**.
- The algorithm SELECT finds the desired element by recursively partitioning the input array.
- We **guarantee** a good split upon partitioning the array.
- We use the deterministic partitioning algorithm from quicksort, but modified to take the element to partition around as an input parameter.

Divide and Conquer II

9.3 Selection in worst-case linear time

Lomuto partitioning

```
PARTITION( $A, p, r, x$ )  
   $i = 1$   
  while  $A[i] \neq x$  do  
     $i = i + 1$   
  exchange  $A[i]$  with  $A[r]$   
   $i = p - 1$   
  for  $j = p$  to  $r - 1$  do  
    if  $A[j] \leq x$  then  
       $i = i + 1$   
      exchange  $A[i]$  with  $A[j]$   
  exchange  $A[i + 1]$  with  $A[r]$   
  return  $i + 1$ 
```

Divide and Conquer II

9.3 Selection in worst-case linear time

SELECT works on an array of $n > 1$ elements. It executes the following steps:

1. Divide the n elements into groups of 5. Get $\lceil n/5 \rceil$ groups: $\lfloor n/5 \rfloor$ groups with exactly 5 elements and, if 5 does not divide n , one group with the remaining $n \bmod 5$ elements.
2. Find the median of each of the $\lceil n/5 \rceil$ groups:
 - Run insertion sort on each group. Takes $O(1)$ time per group since each group has at most 5 elements.
 - Then just pick the median from each group, in $O(1)$ time.
3. Find the median x of the $\lceil n/5 \rceil$ medians by a recursive call to SELECT. (If $\lceil n/5 \rceil$ is even, then by our convention, x is the lower median.)

Divide and Conquer II

9.3 Selection in worst-case linear time

4. Using the modified version of PARTITION that takes the pivot element as input, partition the input array around x . Let x be the k th smallest element of the array after partitioning, so that there are $k - 1$ elements on the low side of the partition and $n - k$ elements on the high side.
5. Now there are three possibilities:
 - If $i = k$, just return x .
 - If $i < k$, return the i th smallest element on the low side of the partition by making a recursive call to SELECT.
 - If $i > k$, return the $(i - k)$ th smallest element on the high side of the partition by making a recursive call to SELECT.

Divide and Conquer II

9.3 Selection in worst-case linear time

```
SELECT( $A, p, r, i$ )
```

```
    divide  $A$  into  $n/5$  groups of 5 elements each
```

```
     $M$  = median of each of the  $n/5$  groups
```

```
     $x$  = SELECT( $M, 1, n/5, n/10$ )
```

```
     $k$  = PARTITION( $A, p, r, x$ )
```

```
    if  $i = k$  then
```

```
        return  $x$ 
```

```
    else
```

```
        if  $i < k$  then
```

```
            return SELECT( $A, p, k - 1, i$ )
```

```
        else
```

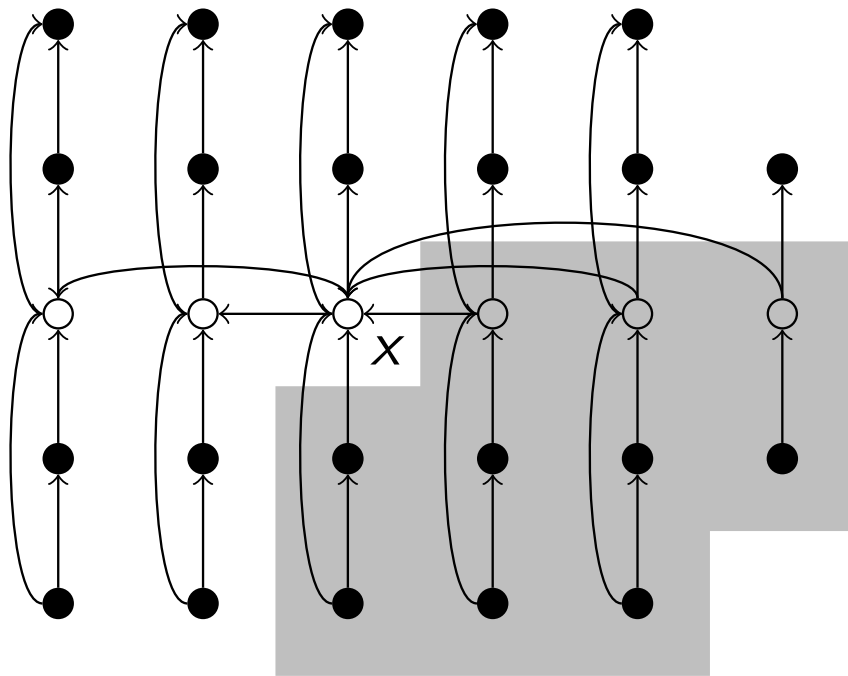
```
            return SELECT( $A, k + 1, r, i - k$ )
```

Divide and Conquer II

9.3 Selection in worst-case linear time

Analysis

- Start by getting a lower bound on the number of elements that are greater than the partitioning element x .



- Each group is a column.
- Each white circle is the median of a group.
- Arrows go from larger elements to smaller elements.
- Elements in the region on the lower right are known to be greater than x .

Divide and Conquer II

9.3 Selection in worst-case linear time

Analysis

- At least half of the medians found in step 2 are $\geq x$.
- Look at the groups containing these medians that are $\geq x$. All of them contribute 3 elements that are $> x$ (the median of the group and the 2 elements in the group greater than the group's median), except for 2 of the groups:
 - The group containing x , which has only 2 elements $> x$.
 - The group with < 5 elements.
- Forget about these 2 groups. That leaves at least

$$\left\lceil \frac{1}{2} \left\lceil \frac{n}{5} \right\rceil \right\rceil - 2$$

groups with 3 elements known to be $> x$.

Divide and Conquer II

9.3 Selection in worst-case linear time

Analysis

- Thus, we know that at least

$$3 \left(\left\lceil \frac{1}{2} \left\lceil \frac{n}{5} \right\rceil \right\rceil - 2 \right) \geq \frac{3n}{10} - 6$$

elements are $> x$.

- Symmetrically, the number of elements that are $< x$ is at least $3n/10 - 6$.
- Therefore, when we call SELECT recursively in step 5, it is on at most $7n/10 + 6$ elements.

Divide and Conquer II

9.3 Selection in worst-case linear time

Analysis

- Develop a recurrence for the worst-case running time $T(n)$ of SELECT:
 - Steps 1, 2, and 4 each take $O(n)$ time.
 - Making groups of 5 elements.
 - Sorting $\lceil n/5 \rceil$ groups in $O(1)$ time each.
 - Partitioning the n -element array around x .
 - Step 3 takes time $T(\lceil n/5 \rceil)$.
 - Step 5 takes time at most $T(7n/10 + 6)$, assuming that T is monotonically increasing.
- Assume that $T(n) = O(1)$ for small enough n . We will use $n < 140$ as **small enough**. Why 140? We will see why later.

Divide and Conquer II

9.3 Selection in worst-case linear time

Analysis

- We get the recurrence

$$T(n) \leq \begin{cases} O(1) & \text{if } n < 140 \\ T(\lceil n/5 \rceil) + T(7n/10 + 6) + O(n) & \text{if } n \geq 140 \end{cases}$$

- We solve this recurrence by substitution. Inductive hypothesis is that $T(n) \leq cn$ for some constant c and all $n > 0$.
- Assume that c is large enough that $T(n) \leq cn$ for all $n < 140$. So we are concerned only with the case $n \geq 140$.
- Pick a constant a such that the function described by the $O(n)$ term in the recurrence is $\leq an$ for all $n > 0$.

Divide and Conquer II

9.3 Selection in worst-case linear time

Analysis

- Substitute the inductive hypothesis in the right-hand side of the recurrence:

$$\begin{aligned}T(n) &\leq c\lceil n/5 \rceil + c(7n/10 + 6) + an \\&\leq cn/5 + c + 7cn/10 + 6c + an \\&= 9cn/10 + 7c + an \\&= cn + (-cn/10 + 7c + an)\end{aligned}$$

- This last quantity is $\leq cn$ if

$$\begin{aligned}-cn/10 + 7c + an &\leq 0 \\cn/10 - 7c &\geq an \\c(n - 70) &\geq 10an \\c &\geq 10a(n/(n - 70))\end{aligned}$$

Divide and Conquer II

9.3 Selection in worst-case linear time

Analysis

- Because we assumed $n \geq 140$, we have $n/(n - 70) \leq 2$.
- Thus, $20a \geq 10a(n/(n - 70))$, so choosing $c \geq 20a$ gives $c \geq 10a(n/(n - 70))$, which in turn gives us the condition that we need to show that $T(n) \leq cn$.
- We conclude that $T(n) = O(n)$, so that SELECT runs in linear time in all cases.
- Why 140? We could have used any integer > 70 .
 - For $n > 70$, the fraction $n/(n - 70)$ decreases as n increases.
 - We picked $n \geq 140$ so that the fraction would be ≤ 2 , which is an easy constant to work with.

Divide and Conquer II

9.3 Selection in worst-case linear time

Exercise 9.3-3

Show how quicksort can be made to run in $O(n \lg n)$ time in the worst case, assuming that all elements are distinct.

Solution to Exercise 9.3-3

- A modification to quicksort that allows it to run in $O(n \lg n)$ time in the worst case uses the deterministic PARTITION algorithm that was modified to take an element to partition around as an input parameter.
- SELECT takes an array A , the bounds p and r of the subarray in A , and the rank i , and in time linear in the size of the subarray $A[p..r]$ it returns the i th smallest element in $A[p..r]$.

Divide and Conquer II

9.3 Selection in worst-case linear time

Solution to Exercise 9.3-3 (Cont'd)

```
BEST-CASE-QUICKSORT( $A, p, r$ )  
  if  $p < r$  then  
     $i = \lfloor (r - p + 1)/2 \rfloor$   
     $x = \text{SELECT}(A, p, r, i)$   
     $q = \text{PARTITION}(A, p, r, x)$   
    BEST-CASE-QUICKSORT( $A, p, q - 1$ )  
    BEST-CASE-QUICKSORT( $A, q + 1, r$ )
```

Divide and Conquer II

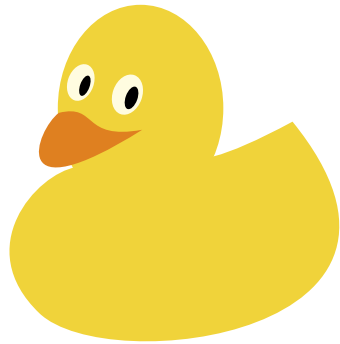
9.3 Selection in worst-case linear time

Solution to Exercise 9.3-3 (Cont'd)

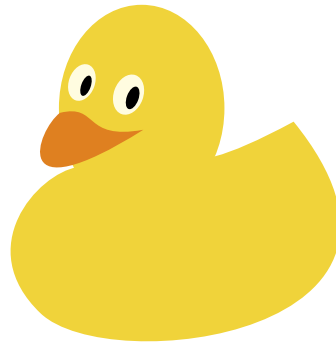
- For an n -element array, the largest subarray that BEST-CASE-QUICKSORT recurses on has $n/2$ elements. This situation occurs when $n = r - p + 1$ is even: then the subarray $A[q + 1..r]$ has $n/2$ elements, and the subarray $A[p..q - 1]$ has $n/2 - 1$ elements.
- Because BEST-CASE-QUICKSORT always recurses on subarrays that are at most half the size of the original array, the recurrence for the worst-case running time is $T(n) \leq 2T(n/2) + \Theta(n) = O(n \lg n)$.

Divide and Conquer II

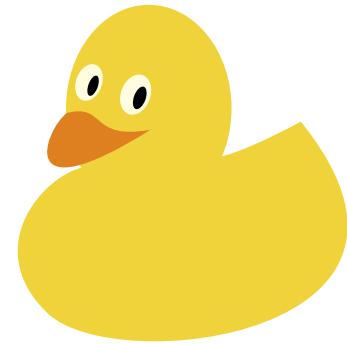
9.3 Selection in worst-case linear time



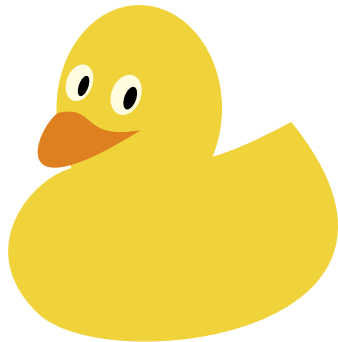
Manuel Blum
(1938–)



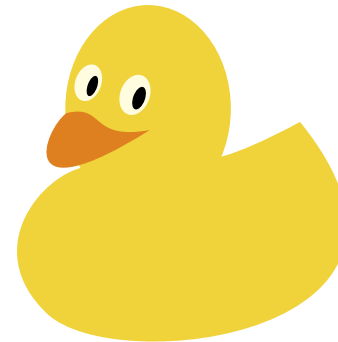
Robert W. Floyd
(1936–2001)



Vaughan Pratt
(1944–)



Ronald L. Rivest
(1947–)



Robert E. Tarjan
(1948–)

Lecture 5

Dictionaries I

Dictionaries I

Contents

- Counting and Probability CLRS C
- Indicator random variables.....CLRS 5.2
- Direct-address tables.....CLRS 11.1
- Hash tables.....CLRS 11.2
- Hash functions CLRS 11.3

Reading

- T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein.
Introduction to Algorithms. The MIT Press, 3rd edition, 2009

Dictionaries I

Overview

- Many applications require a dynamic that support only the **dictionary operations** search, insert, and delete.
- A hash table is effective for implementing a dictionary.
 - The average time to search for an element in a hash table is $\Theta(1)$, under reasonable assumptions.
 - Searching for an element in a hash table takes $\Theta(n)$ time in the worst case, however.
- A hash table is a generalization of an ordinary array.
 - With an ordinary array, we store the element whose key is k in position k of the array.
 - Given a key k , we find the element whose key is k by just looking in the k -th position in the array. This is called **direct addressing**.

Dictionaries I

Overview

- A hash table is a generalization of an ordinary array.
 - Direct addressing is applicable when we can afford to allocate an array with one position for every possible key.
- We use a hash table when we do not want to (or cannot) allocate an array with one position per possible key.
 - We use a hash table when the number of keys actually stored is small relative to the number of possible keys.
 - A hash table is an array, but it typically uses a size proportional to the number of keys to be stored (rather than the number of possible keys).
 - Given a key k , do not use k as the index into the array.
 - Instead, compute a function of k , and use that value to index the array. We call this function a **hash function**.

Dictionaries I

5.2 Indicator random variables

- A simple yet powerful technique for computing the expected value of a random variable.
- Given a sample space S and an event A , the **indicator random variable**

$$I\{A\} = \begin{cases} 1 & \text{if } A \text{ occurs} \\ 0 & \text{if } A \text{ does not occur} \end{cases}$$

- The expected value of an indicator random variable associated with an event A is equal to the probability that A occurs.

Lemma

For an event A in a sample space S , let $X_A = I\{A\}$. Then $E[X_A] = Pr\{A\}$.

Proof.

$$E[X_A] = E[I\{A\}] = 1 \cdot Pr\{A\} + 0 \cdot Pr\{\bar{A}\} = Pr\{A\}.$$

Dictionaries I

5.2 Indicator random variables

Example (Expected number of heads in one flip of a fair coin.)

- Sample space is $S = \{H, T\}$.
- $Pr\{H\} = Pr\{T\} = 1/2$.
- $X_H = I\{H\}$ (number of heads in one flip).
- $E[X_H] = E[I\{H\}] = 1 \cdot Pr\{H\} + 0 \cdot Pr\{T\} = 1/2$.

Dictionaries I

5.2 Indicator random variables

Example (Expected number of heads in n flips of a fair coin.)

- Sample space is $S = \{H, T\}$.
- $Pr\{H\} = Pr\{T\} = 1/2$.
- $X_i = I\{\text{the } i\text{th flip results in event } H\}$, for $i = 1, \dots, n$.
- $X_H = \sum_{i=1}^n X_i$ (number of heads in n flips).
- $E[X_i] = Pr\{H\} = 1/2$, for $i = 1, \dots, n$.
- $E[X_H] = E[\sum_{i=1}^n X_i] = \sum_{i=1}^n E[X_i] = \sum_{i=1}^n 1/2 = n/2$.

Dictionaries I

5.2 Indicator random variables

Exercise 5.2-5

Let $A[1..n]$ be an array of n distinct numbers. If $i < j$ and $A[i] > A[j]$, then the pair (i, j) is called an **inversion**. Suppose that the elements of A form a uniform random permutation of $\{1, 2, \dots, n\}$. Use indicator random variables to compute the expected number of inversions.

Solution to Exercise 5.2-5

- $X_{ij} = I\{A[i] > A[j]\}$ for $1 \leq i < j \leq n$ (indicator random variable for one inversion in the array).
- $Pr\{X_{ij} = 1\} = 1/2$ because given two distinct random numbers, the probability that the first is greater than the second is $1/2$.
- $E[X_{ij}] = 1/2$.

Dictionaries I

5.2 Indicator random variables

Solution to Exercise 5.2-5 (Cont'd)

- $X = \sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}$ (indicator random variable for the number of inversions in the array).
- $E[X] = E[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}]$.

$$\begin{aligned} E[X] &= E \left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij} \right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n 1/2 \\ &= \binom{n}{2} \frac{1}{2} = \frac{n(n-1)}{2} \cdot \frac{1}{2} = \frac{n(n-1)}{4} \end{aligned}$$

- The expected number of inversions is $n(n-1)/4$.

Dictionaries I

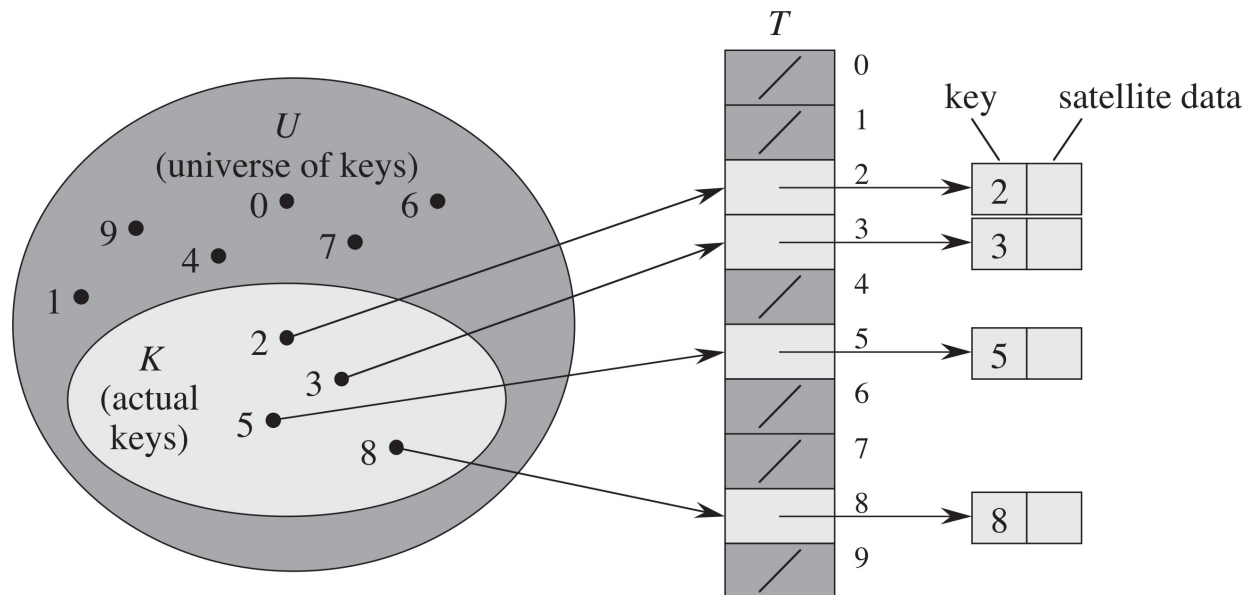
11.1 Direct-address tables

- How to compute hash functions? We will look at the multiplication and division methods.
- What to do when the hash function maps multiple keys to the same table entry? We will look at chaining and open addressing.
- Maintain a dynamic set in which each element has a key drawn from the universe $U = \{0, 1, \dots, m - 1\}$, where m is not too large.
- Assume that no two elements have the same key.

Dictionaries I

11.1 Direct-address tables

- Represent the dynamic set by an array, or **direct-address table**, denoted by $T[0..m-1]$.
 - Each position, or **slot**, corresponds to a key in U .
 - If there is an element x with key k , then $T[k]$ contains a pointer to x .
 - Otherwise, $T[k]$ is empty, represented by NIL.



Dictionaries I

11.1 Direct-address tables

- Dictionary operations are trivial and take $O(1)$ time each.

```
SEARCH( $T, k$ )  
  return  $T[k]$ 
```

```
INSERT( $T, x$ )  
   $T[x.key] = x$ 
```

```
DELETE( $T, x$ )  
   $T[x.key] = NIL$ 
```

Dictionaries I

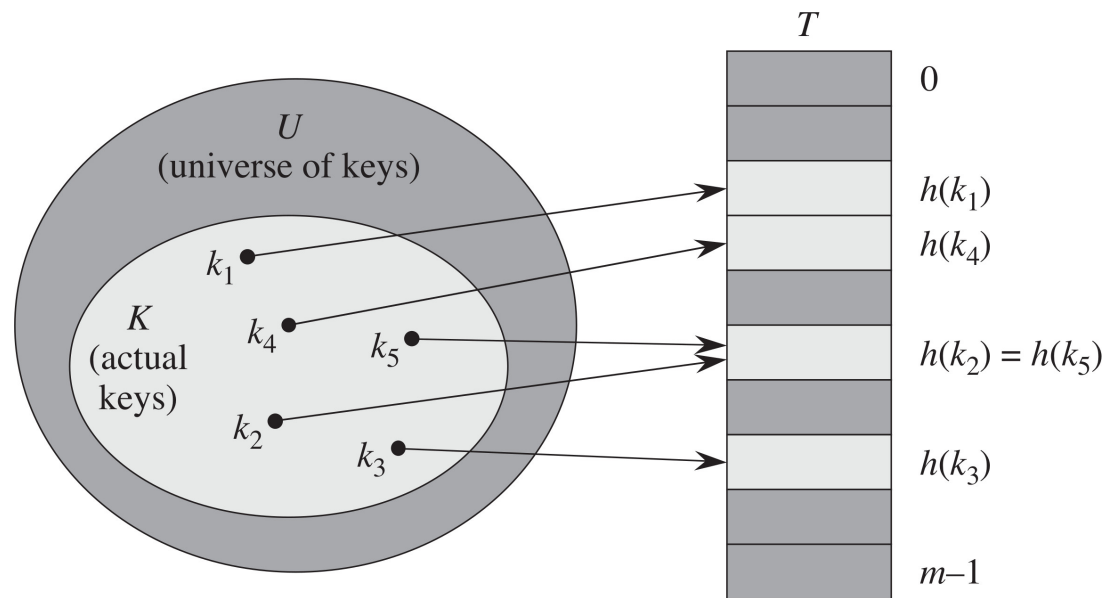
11.2 Hash tables

- The problem with direct addressing is, if the universe U is large, storing a table T of size $|U|$ may be impractical, or even impossible.
- Often, the set K of keys **actually stored** is small, compared to U , and most of the space allocated for T is wasted.
- When K is much smaller than U , a hash table requires much less storage than a direct-address table.
- Can reduce the storage requirement to $\Theta(|K|)$.
- Can still get $O(1)$ search time, but in the **average case**, not the **worst case**.
- Instead of storing an element with key k in slot k , use a **hash function** h and store the element in slot $h(k)$.

Dictionaries I

11.2 Hash tables

- $h : U \rightarrow \{0, 1, \dots, m - 1\}$ maps the universe U of keys into the slots of a **hash table** $T[0..m - 1]$.
- An element with key k **hashes** to slot $h(k)$.

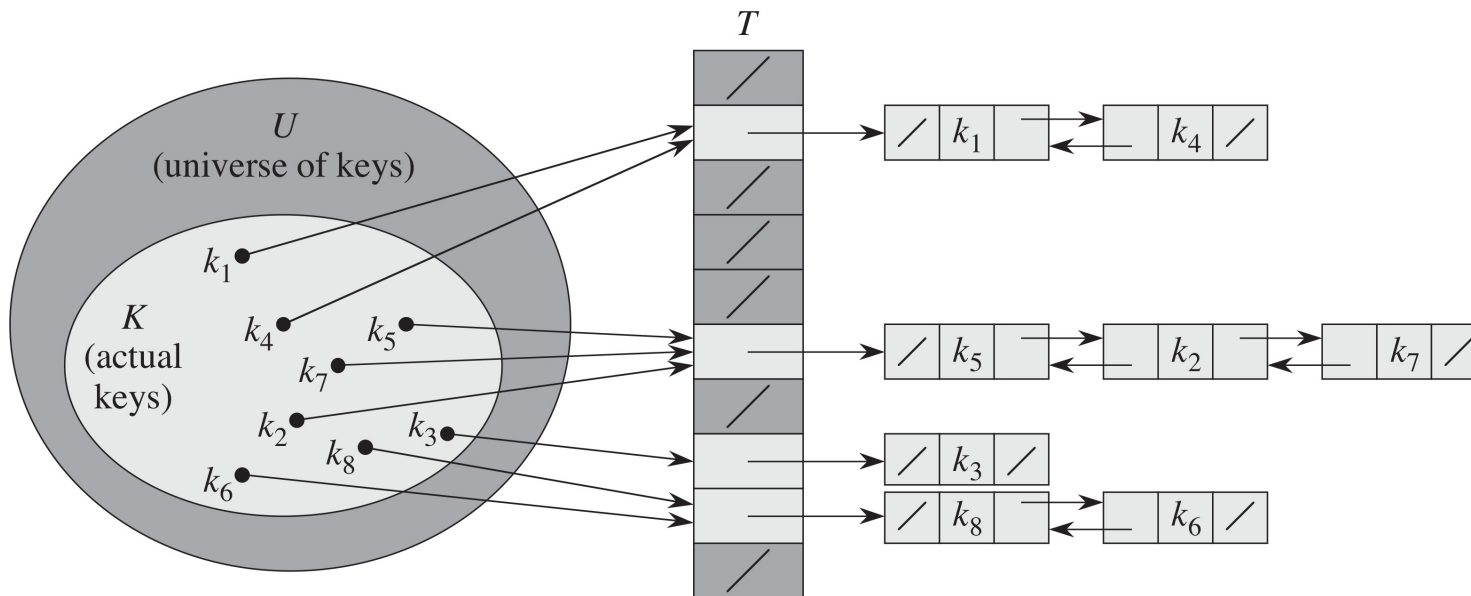


- When two or more keys hash to the same slot, there is a **collision**.

Dictionaries I

11.2 Hash tables

- In the simplest collision resolution, **chaining**, we put all elements that hash to the same slot into a linked list.



- Slot j contains a pointer to the head of the list of all stored elements that hash to j .
- If there are no such elements, slot j contains NIL.

Dictionaries I

11.2 Hash tables

- Dictionary operations are easy to implement when collisions are resolved by chaining.
- The worst-case running time for search is proportional to the length of the list of elements in slot $h(k)$.

SEARCH(T, k)

search for an element with key k in list $T[h(k)]$

- The worst-case running time for insertion is $O(1)$, assuming that the element x being inserted is not already in the list.

INSERT(T, x)

insert x at the head of list $T[h(x.key)]$

- The worst-case running time for deletion is $O(1)$ if the lists are doubly linked.

DELETE(T, x)

delete x from the list $T[h(x.key)]$

Dictionaries I

11.2 Hash tables

- How long does it take to search for an element with a given key in a hash table with chaining?
- Given a hash table T with m slots that stores n elements, define the **load factor** α for T as n/m , the average number of elements stored in a chain.
- The analysis will be in terms of α , which can be less than, equal to, or greater than 1.
- The worst case is when all n keys hash to the same slot, creating a list of length n .
- The worst-case time for searching is $\Theta(n)$ plus the time to compute the hash function.
- The average-case performance of hashing depends on how well the hash function h distributes the set of keys to be stored among the m slots.

Dictionaries I

11.2 Hash tables

- Assume **simple uniform hashing**, where any given element is equally likely to hash into any of the m slots.
- For $j = 0, 1, \dots, m - 1$, denote the length of list $T[j]$ by n_j , so that $n = n_0 + n_1 + \dots + n_{m-1}$.
- Average value of n_j is $E[n_j] = \alpha = n/m$.
- Assume that we can compute the hash function in $O(1)$ time, so that the time required to search for an element with key k depends on the length $n_{h(k)}$ of the list $T[h(k)]$.
- If the hash table contains no element with key k , then the search is **unsuccessful**.
- If the hash table does contain an element with key k , then the search is **successful**.

Dictionaries I

11.2 Hash tables

Theorem

*In a hash table in which collisions are resolved by chaining, under the assumption of simple uniform hashing, an **unsuccessful** search takes average-case time $\Theta(1 + \alpha)$.*

Proof.

- Any key k not already in the table is equally likely to hash to any of the m slots.
- To search unsuccessfully for a key k we need to search to the end of list $T[h(k)]$, which has expected length $E[n_{h(k)}] = \alpha$.
- The expected number of elements examined in an unsuccessful search is α .
- Adding the time for computing $h(k)$, the total time required is $\Theta(1 + \alpha)$.

Dictionaries I

11.2 Hash tables

In a successful search, the probability that a list is searched is proportional to the number of elements it contains.

Theorem

*In a hash table in which collisions are resolved by chaining, under the assumption of simple uniform hashing, a **successful** search takes average-case time $\Theta(1 + \alpha)$.*

Proof.

- Assume that the element being searched for is equally likely to be any of the n elements in the table.
- The number of elements examined during a successful search for x is 1 plus the number of elements that appear before x in the list.
- These elements were all inserted **after** x was inserted, because we insert at the head of the list.

Dictionaries I

11.2 Hash tables

Proof.

- To find the expected number of elements examined, we take the average (over the n elements x in the table) of 1 plus the expected number of elements added to the list after x was added to the list.
- Let x_i denote the i th element inserted into the table, for $i = 1, \dots, n$, and let $k_i = \text{key}[x_i]$.
- $X_{ij} = I\{h(k_i) = h(k_j)\}$, for all i, j .
- $\Pr\{h(k_i) = h(k_j)\} = 1/m$ (simple uniform hashing).
- $E[X_{ij}] = 1/m$.

Dictionaries I

11.2 Hash tables

Proof.

- The expected number of elements examined in a successful search is

$$\begin{aligned} & E \left[\frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n X_{ij} \right) \right] \\ &= \frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n E[X_{ij}] \right) = \frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n \frac{1}{m} \right) \\ &= 1 + \frac{1}{nm} \sum_{i=1}^n (n - i) = 1 + \frac{1}{nm} \left(n^2 - \frac{n(n+1)}{2} \right) \\ &= 1 + \frac{n-1}{2m} = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n} \end{aligned}$$

Dictionaries I

11.2 Hash tables

Proof.

- Adding the time for computing $h(k)$, the total time required for a successful search is $\Theta(2 + \alpha/2 - \alpha/2n) = \Theta(1 + \alpha)$.

Interpretation

- If the number of slots is proportional to the number of elements in the table, $n = O(m)$, and then $\alpha = n/m = O(m)/m = O(1)$, and searching takes constant time on average.
- Since insertion and deletion take $O(1)$ worst-case time, we can support all dictionary operations in $O(1)$ time on average.

Dictionaries I

11.3 Hash functions

What makes a good hash function?

- Ideally, the hash function satisfies the assumption of simple uniform hashing.
- In practice, it is not possible to satisfy this assumption, since we do not know in advance the probability distribution that keys are drawn from, and the keys may not be drawn independently.
- Often use heuristics, based on the domain of the keys, to create a hash function that performs well.

Dictionaries I

11.3 Hash functions

Keys as natural numbers

- Hash functions assume that the keys are natural numbers.
- When they are not, have to interpret them as natural numbers.

Example (Interpret a character string as an integer expressed in some radix notation.)

Suppose the string is CLRS.

- ASCII values: $C = 67$, $L = 76$, $R = 82$, $S = 83$.
- There are 128 basic ASCII values.
- Interpret CLRS as
$$(67 \cdot 128^3) + (76 \cdot 128^2) + (82 \cdot 128^1) + (83 \cdot 128^0) = 141,764,947.$$

Dictionaries I

11.3 Hash functions

Division method

- $h(k) = k \bmod m$.
- Example: $m = 20$ and $k = 91$. $h(k) = 11$.
- Advantage: Fast, since it requires just one division operation.
- Disadvantage: Have to avoid certain values of m .
 - Powers of 2 are bad. If $m = 2^p$ for integer p , then $h(k)$ is just the least significant p bits of k .
 - If k is a character string interpreted in radix 2^p , then $m = 2^p - 1$ is bad. Permuting characters in a string does not change its hash value (Exercise 11.3-3).
- Good choice for m : A prime not too close to an exact power of 2. Example: $n = 2000$, average of 3 elements in an unsuccessful search, $m = 701$, $h(k) = k \bmod 701$.

Dictionaries I

11.3 Hash functions

Division method

- Good choice for m : A prime not too close to an exact power of 2. Example: $n = 2000$, average of 3 elements in an unsuccessful search, $m = 701$, $h(k) = k \bmod 701$.
 - $2000/3 = 666.\bar{6}$
 - $2^9 = 512 < 666.\bar{6} < 1,024 = 2^{10}$
 - $(2^9 + 2^{10})/2 = 768$
 - $761 \leq 768 \leq 769$, 761 prime, 769 prime
 - Both $m = 761$ and $m = 769$ are good choices.

Lecture 6

Dictionaries II

Dictionaries II

Contents

- What is a binary search tree CLRS 12.1
- Querying a binary search tree CLRS 12.2
- Insertion and deletion CLRS 12.3
- AVL trees CLRS 13-3

Reading

- T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein.
Introduction to Algorithms. The MIT Press, 3rd edition, 2009

Dictionaries II

Overview

Search trees

- Data structures that support many dynamic-set operations.
- Can be used as both a dictionary and as a priority queue.
- Basic operations take time proportional to the height of the tree.
 - For complete binary tree with n nodes: worst case $\Theta(\log n)$.
 - For linear chain of n nodes: worst case $\Theta(n)$.
- Different types of search trees include binary search trees, red-black trees, AVL trees, and B-trees.
- We will cover binary search trees, tree walks, operations on binary search trees, and AVL trees.

Dictionaries II

12.1 What is a binary search tree

An important data structure for dynamic sets.

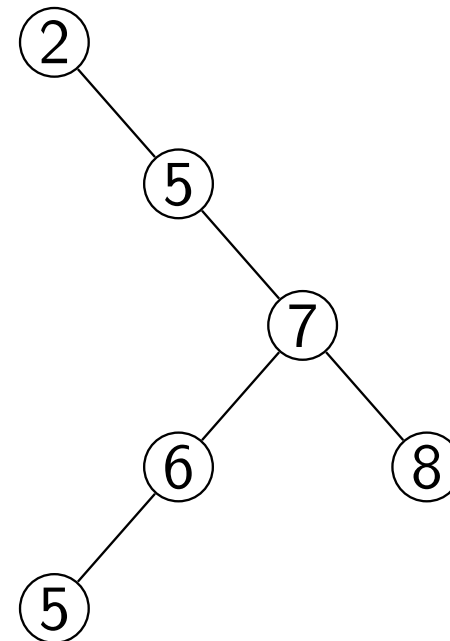
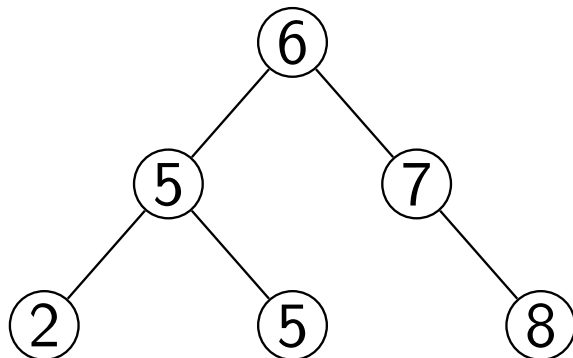
- Accomplish many dynamic-set operations in $O(h)$ time, where h is the height of the tree.
- We represent a binary tree by a linked data structure in which each node is an object.
- $T.root$ points to the root of tree T .
- Each node contains the attributes
 - *key* (and possibly other satellite data)
 - *left* (points to left child)
 - *right* (points to right child)
 - *p* (points to parent), with $T.root.parent = \text{NIL}$
- Stored keys must satisfy the **binary-search-tree property**.
 - If y is in left subtree of x , then $y.key \leq x.key$.
 - If y is in right subtree of x , then $y.key \geq x.key$.

Dictionaries II

12.1 What is a binary search tree

- For any node x , the keys in the left subtree of x are at most $x.key$, and the keys in the right subtree of x are at least $x.key$.
- The worst-case running time for most search-tree operations is proportional to the height of the tree.

Example (Binary search trees)



Dictionaries II

12.1 What is a binary search tree

- The binary-search-tree property allows us to print keys in a binary search tree in order, recursively, using an algorithm called an **inorder tree walk**.
- Elements are printed in monotonically increasing order.

```
INORDER-TREE-WALK( $x$ )  
  if  $x \neq \text{NIL}$  then  
    INORDER-TREE-WALK( $x.\text{left}$ )  
    print  $x.\text{key}$   
    INORDER-TREE-WALK( $x.\text{right}$ )
```

- Correctness of inorder tree walk follows by induction directly from the binary-search-tree property.
- Intuitively, the walk takes $\Theta(n)$ time for a tree with n nodes, because we visit and print each node once.

Dictionaries II

12.2 Querying a binary search tree

Searching

- Initial call is $\text{TREE-SEARCH}(T.\text{root}, k)$.

```
TREE-SEARCH( $x, k$ )  
  if  $x = \text{NIL}$  or  $k = x.\text{key}$  then  
    return  $x$   
  else if  $k < x.\text{key}$  then  
    return  $\text{TREE-SEARCH}(x.\text{left}, k)$   
  else  
    return  $\text{TREE-SEARCH}(x.\text{right}, k)$ 
```

- The algorithm recurses, visiting nodes on a downward path from the root.
- Running time is $O(h)$, where h is the height of the tree.

Dictionaries II

12.2 Querying a binary search tree

Minimum and maximum

- The binary-search-tree property guarantees that
 - the minimum key of a binary search tree is located at the leftmost node
 - the maximum key of a binary search tree is located at the rightmost node
- Traverse the appropriate pointers (*left* or *right*) until NIL is reached.

```
TREE-MINIMUM(x)  
  while x.left  $\neq$  NIL do  
    x = x.left  
  return x
```

Dictionaries II

12.2 Querying a binary search tree

Minimum and maximum

- Traverse the appropriate pointers (*left* or *right*) until NIL is reached.

```
TREE-MAXIMUM(x)  
  while x.right  $\neq$  NIL do  
    x = x.right  
  return x
```

- Both procedures visit nodes that form a downward path from the root to a leaf.
- Running time is $O(h)$, where h is the height of the tree.

Dictionaries II

12.2 Querying a binary search tree

Successor and predecessor

- Assuming that all keys are distinct, the successor of a node x is the node y with the smallest key greater than $x.key$.
- If node x has the largest key in the binary search tree, then we say that the successor of x is NIL.
- There are two cases:
 - If the right subtree of node x is non-empty, then the successor of x is just the leftmost node in the right subtree of x .
 - If the right subtree of node x is empty and x has a successor y , then y is the lowest ancestor of x whose left child is also an ancestor of x .

Dictionaries II

12.2 Querying a binary search tree

Exercise 12.2-6

Consider a binary search tree T whose keys are distinct. Show that if the right subtree of a node x in T is empty and x has a successor y , then y is the lowest ancestor of x whose left child is also an ancestor of x . (Recall that every node is its own ancestor.)

Dictionaries II

12.2 Querying a binary search tree

Solution to Exercise 12.2-6

- First, we establish that y must be an ancestor of x . If y were not an ancestor of x , then let z denote the lowest common ancestor of x and y . By the binary-search-tree property, $x < z < y$ and then, y cannot be the successor of x .
- Next, observe that $y.left$ must be an ancestor of x because if it were not, then $y.right$ would be an ancestor of x , which implies either $y < x \leq y.right$ or $y < y.right \leq x$ and then, y cannot be the successor of x .
- Finally, suppose that y is not the lowest ancestor of x whose left child is also an ancestor of x . Let z denote this lowest ancestor. Then z must be in the left subtree of y , which implies $x < z < y$ and then, y cannot be the successor of x .

Dictionaries II

12.2 Querying a binary search tree

Successor and predecessor

```
TREE-SUCCESSOR(x)  
    if x.right  $\neq$  NIL then  
        return TREE-MINIMUM(x.right)  
  
    y = x.p  
    while y  $\neq$  NIL and x = y.right do  
        x = y  
        y = y.p  
    return y
```

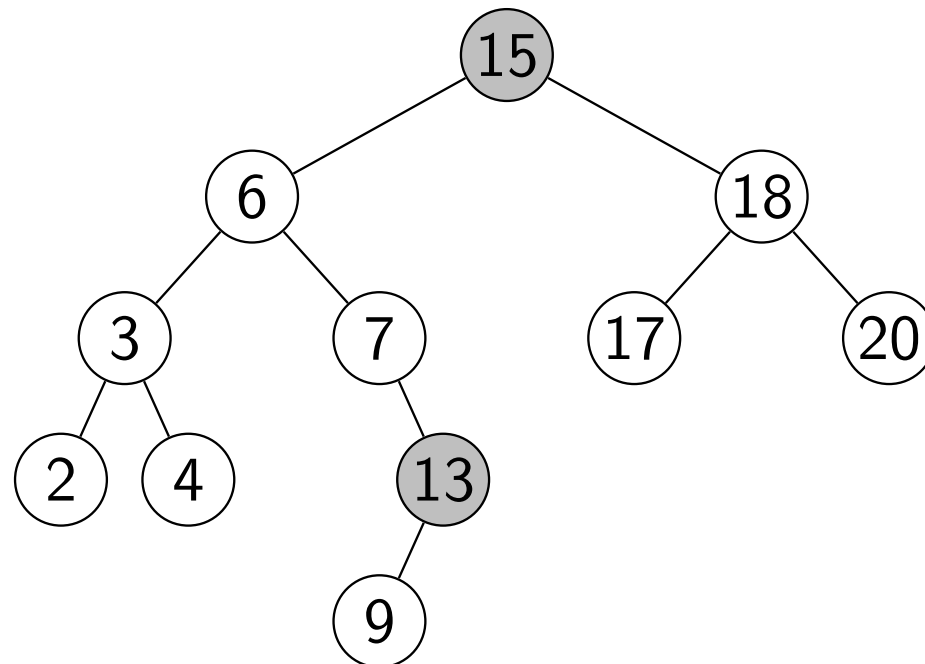
- To find *y*, we simply go up the tree from *x* until we encounter a node that is the left child of its parent.
- TREE-PREDECESSOR is symmetric to TREE-SUCCESSOR.

Dictionaries II

12.2 Querying a binary search tree

Successor and predecessor

- Both procedures visit nodes either on a path down the tree or on a path up the tree.
- Running time is $O(h)$, where h is the height of the tree.



Dictionaries II

12.2 Querying a binary search tree

Exercise 12.2-3

Write the TREE-PREDECESSOR procedure.

Solution to Exercise 12.2-3

```
TREE-PREDECESSOR( $x$ )  
  if  $x.left \neq \text{NIL}$  then  
    return TREE-MAXIMUM( $x.left$ )  
  
   $y = x.p$   
  while  $y \neq \text{NIL}$  and  $x = y.left$  do  
     $x = y$   
     $y = y.p$   
  return  $y$ 
```

- To find y , we simply go up the tree from x until we encounter a node that is the right child of its parent.

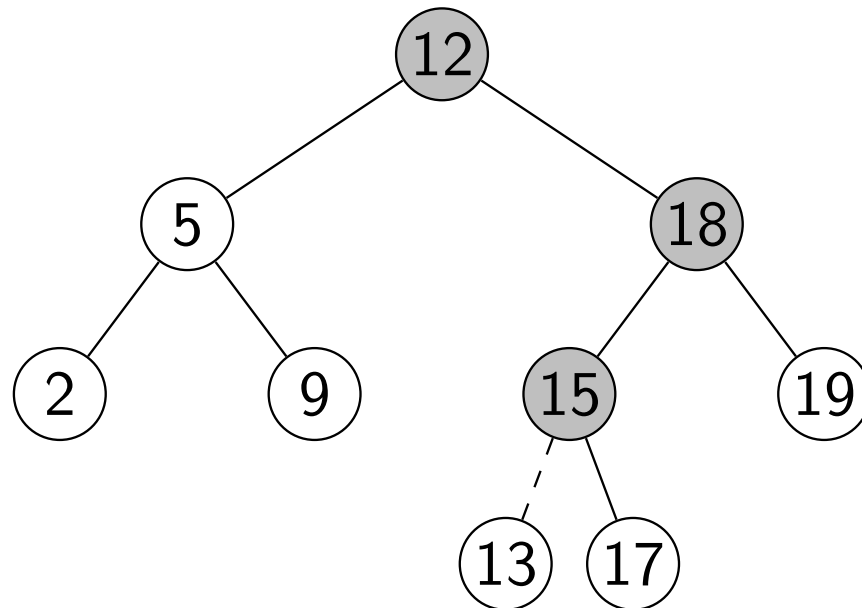
Dictionaries II

12.3 Insertion and deletion

Insertion

- To insert a value v into a binary search tree T , the procedure takes a node z with $z.key = v$, $z.left = \text{NIL}$, $z.right = \text{NIL}$.

Example (Inserting an item with key 13 into a binary search tree)



Dictionaries II

12.3 Insertion and deletion

Exercise 12.3-1

Give a recursive version of the TREE-INSERT procedure.

Solution to Exercise 12.3-1

- Beginning at the root of the tree, trace a downward path, maintaining two pointers.
 - Pointer x to trace the downward path, looking for a NIL to replace with item z .
 - **Trailing pointer** y to keep track of the parent of x .
- Running time is $O(h)$, where h is the height of the tree.

```
TREE-INSERT( $T, z$ )  
  TREE-INSERT(NIL,  $T.root, z$ )
```

Dictionaries II

12.3 Insertion and deletion

Solution to Exercise 12.3-1

```
TREE-INSERT( $y, x, z$ )  
  if  $x \neq \text{NIL}$  then  
    if  $z.\text{key} < x.\text{key}$  then  
      TREE-INSERT( $x, x.\text{left}, z$ )  
    else  
      TREE-INSERT( $x, x.\text{right}, z$ )  
  
   $z.p = y$   
  if  $y = \text{NIL}$  then  
     $T.\text{root} = z$   
  else if  $z.\text{key} < y.\text{key}$  then  
     $y.\text{left} = z$   
  else  
     $y.\text{right} = z$ 
```

Dictionaries II

12.3 Insertion and deletion

Solution to Exercise 12.3-1

- To insert a value v into a binary search tree T , take a node z with $z.key = v$, $z.left = \text{NIL}$, $z.right = \text{NIL}$ and $x = T.root$.

```
TREE-INSERT( $x, z$ )
  if  $x = \text{NIL}$  then
    return  $z$ 
  else
    if  $z.key \leq x.key$  then
       $x.left = \text{TREE-INSERT}(x.left, z)$ 
       $x.left.p = x$ 
    else
       $x.right = \text{TREE-INSERT}(x.right, z)$ 
       $x.right.p = x$ 
    return  $x$ 
```

Dictionaries II

12.3 Insertion and deletion

Deletion

- Deleting node z from binary search tree T has three cases.
 - If z has no children, just remove it (modifying its parent to replace z with NIL as a child).
 - If z has just one child, then elevate that child to take the position of z in the tree (by modifying the parent of z to replace z by its child).
 - If z has two children, then find the successor y of z and replace z by y , splicing y out of its current location.
- Running time is $O(h)$, where h is the height of the tree.

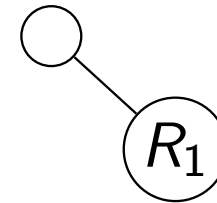
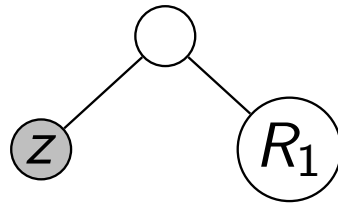
Dictionaries II

12.3 Insertion and deletion

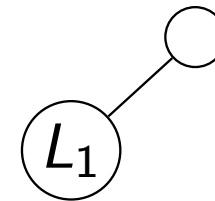
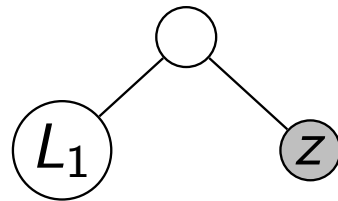
Deletion

- If z has no children, just remove it (modifying its parent to replace z with NIL as a child).

(Node z is the left child of its parent, if any.)



(Node z is the right child of its parent, if any.)



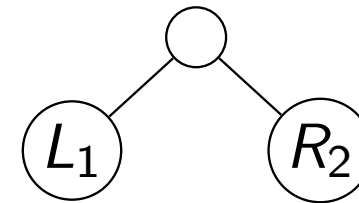
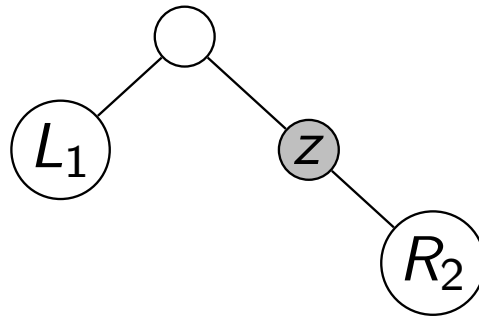
Dictionaries II

12.3 Insertion and deletion

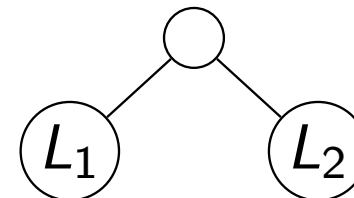
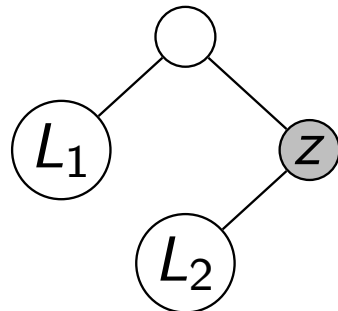
Deletion

- If z has just one child, then elevate that child to take the position of z in the tree (by modifying the parent of z to replace z by its child).

(Node z has no left child.)



(Node z has no right child.)

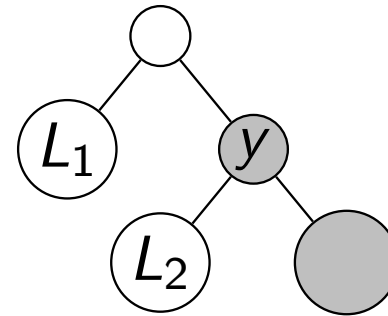
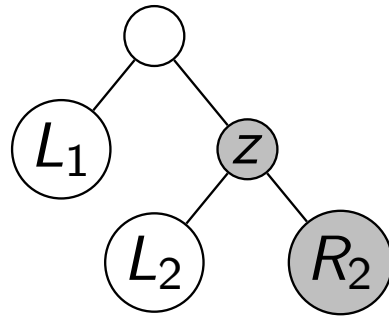


Dictionaries II

12.3 Insertion and deletion

Deletion

- If z has two children, then find the successor y of z and replace z by y , splicing y out of its current location.



Dictionaries II

12.3 Insertion and deletion

Deletion

- Deleting node z from binary search tree T

```
TREE-DELETE( $T, z$ )  
  if  $z \neq \text{NIL}$  then  
    if  $z.\text{left} = \text{NIL}$  then  
       $z = z.\text{right}$   
    else if  $z.\text{right} = \text{NIL}$  then  
       $z = z.\text{left}$   
    else  
       $y = \text{TREE-DELETE-MINIMUM}(T, z.\text{right})$   
       $y.\text{left} = z.\text{left}$   
       $y.\text{right} = z.\text{right}$   
       $z = y$ 
```

- Running time is $O(h)$, where h is the height of the tree.

Dictionaries II

12.3 Insertion and deletion

Deletion

```
TREE-DELETE-MINIMUM(z)  
  if z.left  $\neq$  NIL then  
    return TREE-DELETE-MINIMUM(z.left)  
  else  
    y = z  
    z = z.right  
    return y
```

- Running time is $O(h)$, where h is the height of the tree.

Dictionaries II

12.3 Insertion and deletion

Exercise 12.3-3

We can sort a given set of n numbers by first building a binary search tree containing these numbers (using TREE-INSERT repeatedly to insert the numbers one by one) and then printing the numbers by an inorder tree walk. What are the worst-case and best-case running times for this sorting algorithm?

Dictionaries II

12.3 Insertion and deletion

Solution to Exercise 12.3-3

```
TREE-SORT(A)
```

```
    let T be an empty binary search tree
```

```
    for i = 1 to n do
```

```
        TREE-INSERT(T.root, A[i])
```

```
    INORDER-TREE-WALK(T.root)
```

- The worst case occurs when a linear chain of nodes results from the repeated TREE-INSERT operations, with $\Theta(n^2)$ running time.
- The best case occurs when a binary tree of height $\Theta(\log n)$ results from the repeated TREE-INSERT operations, with $\Theta(n \log n)$ running time.

Dictionaries II

13-3 AVL trees

Problem 13-3

- An **AVL tree** is a binary search tree that is **height balanced**: for each node x , the heights of the left and right subtrees of x differ by at most 1.
- To implement an AVL tree, we maintain an extra attribute in each node: $x.h$ is the height of node x .
- As for any other binary search tree T , we assume that $T.root$ points to the root node.

Problem 13-3-a

- Prove that an AVL tree with n nodes has height $O(\log n)$.
- Hint: Prove that an AVL tree of height h has at least F_h nodes, where F_h is the h th Fibonacci number.

Dictionaries II

13-3 AVL trees

Solution to Problem 13-3-a

- Let $T(h)$ be the minimum number of nodes in an AVL tree of height h . We want to show that $T(h) \geq F_h$.
- We proceed by induction. For the base case, note that $T(1) \geq T(0) \geq 1$. Thus, $T(1) \geq F_1$ and $T(0) \geq F_0$.
- Now, assume that $T(h') \geq F_{h'}$ for all $h' < h$.
- The root of an AVL tree of height h has a child of height at least $h - 1$ and another child of height at least $h - 2$.
- Thus, $T(h) \geq T(h - 1) + T(h - 2)$ and, by induction hypothesis, $T(h) \geq F_{h-1} + F_{h-2} = F_h$.
- Since $F_h \geq \phi^h / \sqrt{5}$, an AVL tree with n nodes and height h must satisfy $n \geq T(h) \geq F_h \geq \phi^h / \sqrt{5}$. Thus, $h = O(\log n)$.

Dictionaries II

13-3 AVL trees

Exercise 3.2-6

Show that the golden ratio $\phi = (1 + \sqrt{5})/2$ and its conjugate $\hat{\phi} = (1 - \sqrt{5})/2$ both satisfy the equation $x^2 = x + 1$.

Solution to Exercise 3.2-6

$$\phi^2 = \left(\frac{1 + \sqrt{5}}{2} \right)^2 = \frac{6 + 2\sqrt{5}}{4} = \frac{3 + \sqrt{5}}{2} = \frac{1 + \sqrt{5}}{2} + 1 = \phi + 1$$

$$\hat{\phi}^2 = \left(\frac{1 - \sqrt{5}}{2} \right)^2 = \frac{6 - 2\sqrt{5}}{4} = \frac{3 - \sqrt{5}}{2} = \frac{1 - \sqrt{5}}{2} + 1 = \hat{\phi} + 1$$

Dictionaries II

13-3 AVL trees

Exercise 3.2-7

Prove by induction that the i th Fibonacci number satisfies the equality $F_i = (\phi^i - \hat{\phi}^i)/\sqrt{5}$, where ϕ is the golden ratio and $\hat{\phi}$ is its conjugate.

Solution to Exercise 3.2-6

- $F_0 = (\phi^0 - \hat{\phi}^0)/\sqrt{5} = (1 - 1)/\sqrt{5} = 0$
- $F_1 = (\phi^1 - \hat{\phi}^1)/\sqrt{5} = \sqrt{5}/\sqrt{5} = 1$

$$\begin{aligned} F_{i+1} &= F_i + F_{i-1} = (\phi^i - \hat{\phi}^i)/\sqrt{5} + (\phi^{i-1} - \hat{\phi}^{i-1})/\sqrt{5} \\ &= ((\phi^i - \hat{\phi}^i) + (\phi^{i-1} - \hat{\phi}^{i-1}))/\sqrt{5} \\ &= ((\phi^i + \phi^{i-1}) - (\hat{\phi}^i + \hat{\phi}^{i-1}))/\sqrt{5} \\ &= (\phi^{i-1}(\phi + 1) - \hat{\phi}^{i-1}(\hat{\phi} + 1))/\sqrt{5} \\ &= (\phi^{i-1}\phi^2 - \hat{\phi}^{i-1}\hat{\phi}^2)/\sqrt{5} = (\phi^{i+1} - \hat{\phi}^{i+1})/\sqrt{5} \end{aligned}$$

Dictionaries II

13-3 AVL trees

Solution to Problem 13-3-a

- Now, $|\hat{\phi}| < 1$ implies $|\hat{\phi}^i|/\sqrt{5} < 1/\sqrt{5} < 1/2$.
- Thus, $F_i = (\phi^i - \hat{\phi}^i)/\sqrt{5} = \lfloor \phi^i/\sqrt{5} + 1/2 \rfloor \geq \phi^i/\sqrt{5}$.
- Therefore, $h \leq \log_{\phi}(\sqrt{5} n) = \log_{\phi} \sqrt{5} + \log_{\phi} n$, and then $h = O(\log n)$.

Dictionaries II

13-3 AVL trees

Problem 13-3-b

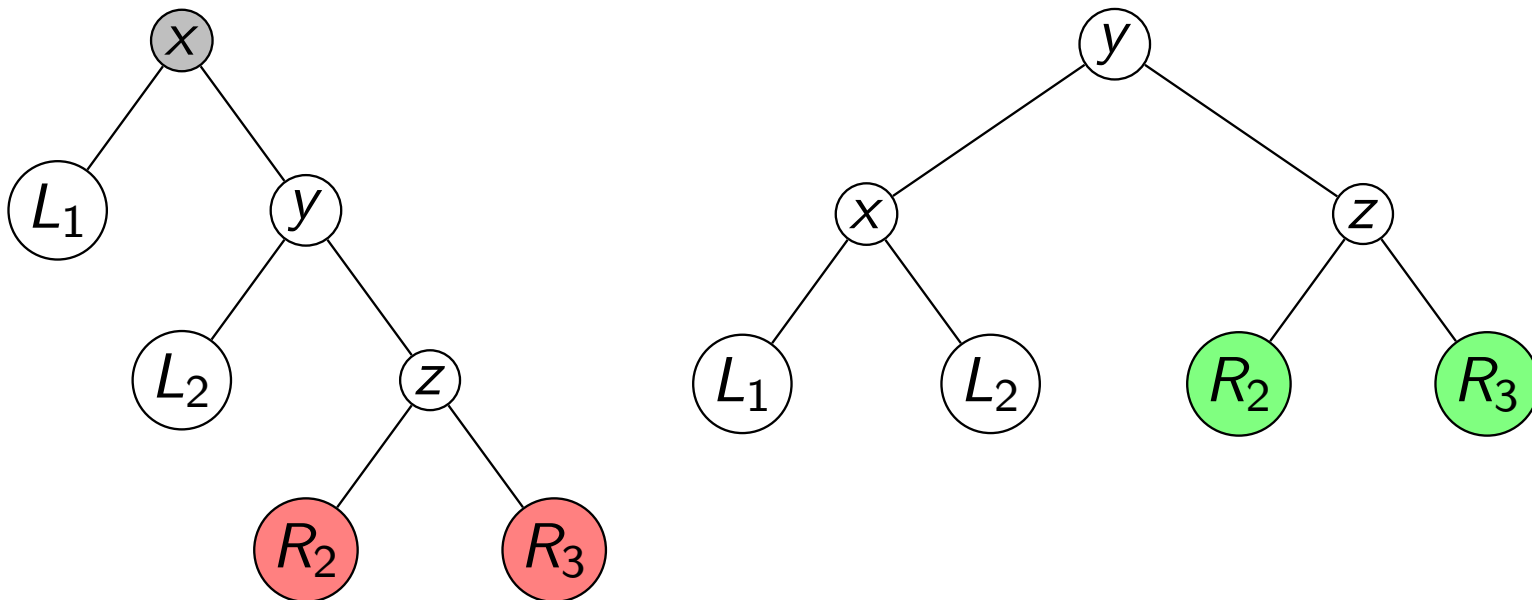
- To insert into an AVL tree, we first place a node into the appropriate place in binary search tree order.
- Afterward, the tree might no longer be height balanced.
- Specifically, the heights of the left and right children of some node might differ by 2. Describe a procedure $\text{BALANCE}(x)$, which takes a subtree rooted at x whose left and right children are height balanced and have heights that differ by at most 2, that is, $|x.\text{right}.h - x.\text{left}.h| \leq 2$, and alters the subtree rooted at x to be height balanced.
- Hint: Use rotations.

Dictionarys II

13-3 AVL trees

Solution to Problem 13-3-b

- After an insertion into an AVL tree, four generic situations may arise that lead to an unbalanced tree, which can be balanced using one or two rotations.
- Left rotation.

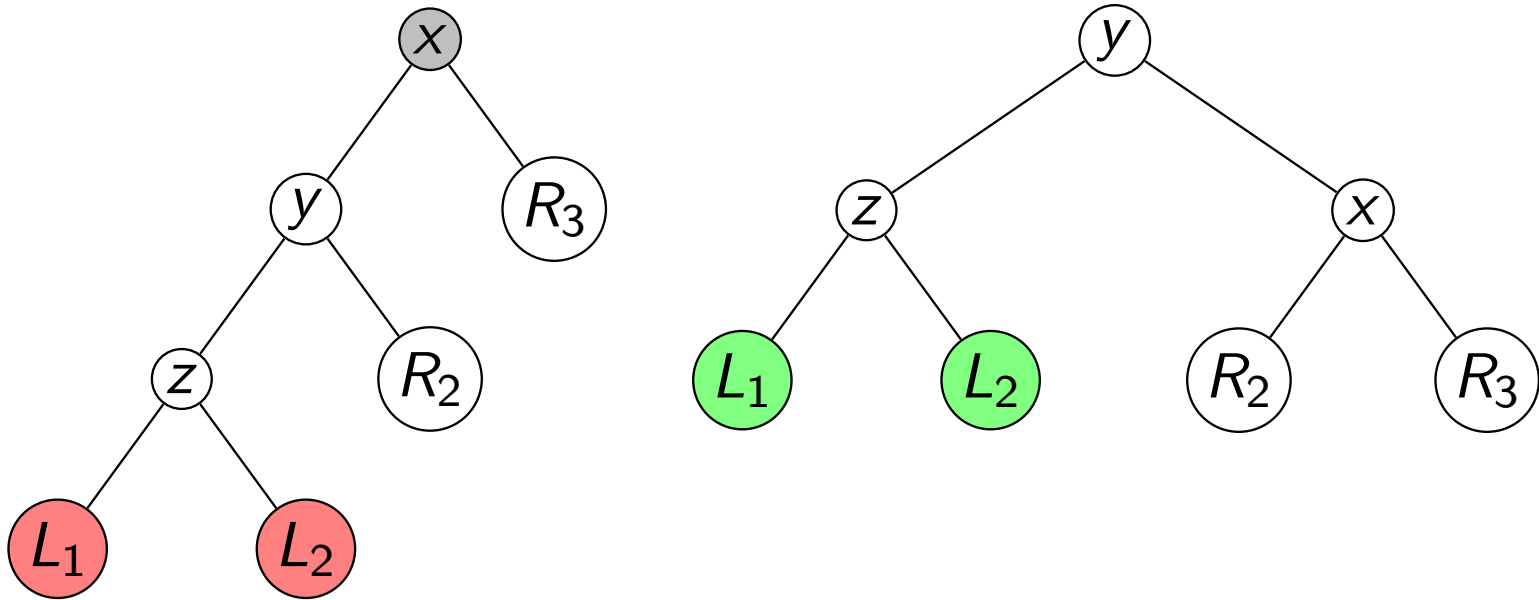


Dictionarys II

13-3 AVL trees

Solution to Problem 13-3-b

- Right rotation.

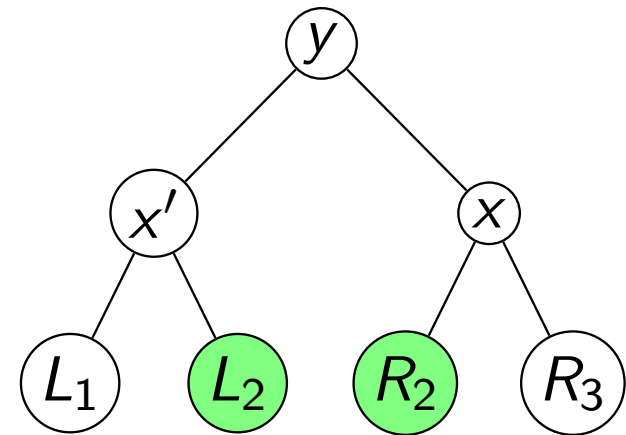
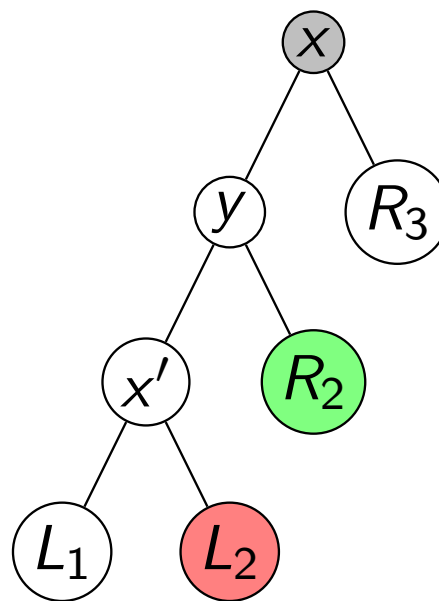
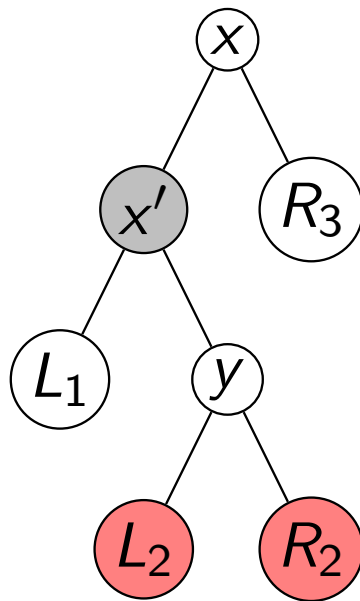


Dictionarys II

13-3 AVL trees

Solution to Problem 13-3-b

- Left-right rotation.

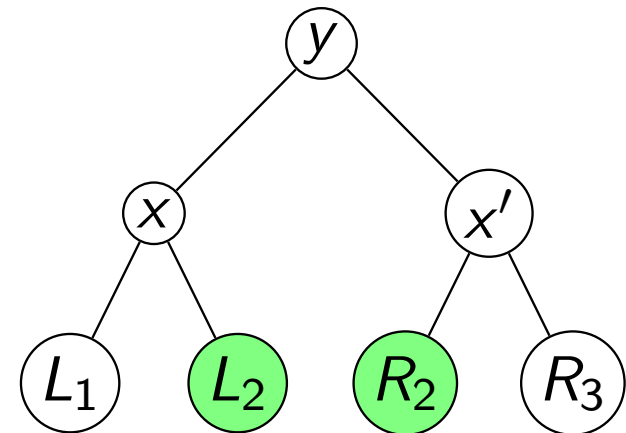
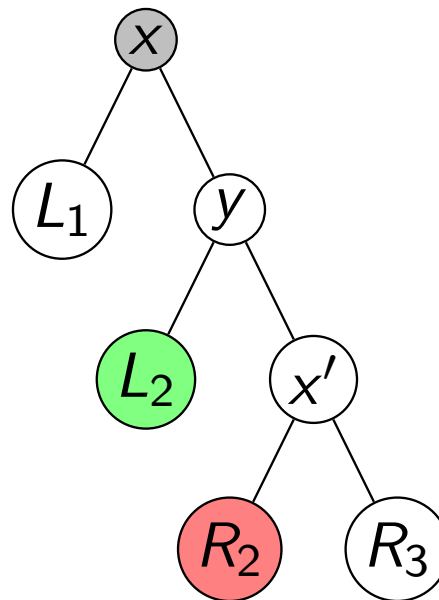
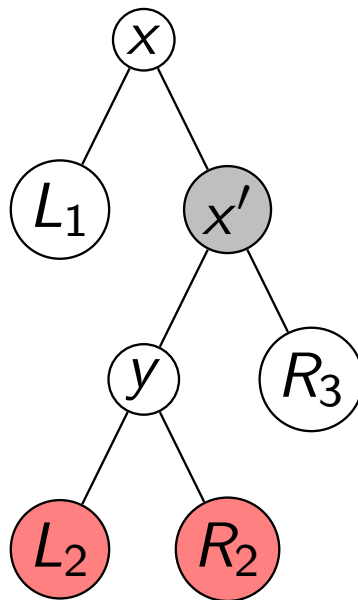


Dictionarys II

13-3 AVL trees

Solution to Problem 13-3-b

- Right-left rotation.



Dictionaries II

13-3 AVL trees

Solution to Problem 13-3-b

```
BALANCE(x)  
  if  $|x.right.h - x.left.h| \leq 1$  then  
    return x  
  else if  $x.left.h > x.right.h$  then  
    y = x.left  
    if  $y.left.h > y.right.h$  then  
      LEFT-ROTATE(y)  
    return RIGHT-ROTATE(x)  
  else  
    y = x.right  
    if  $y.left.h > y.right.h$  then  
      RIGHT-ROTATE(y)  
    return LEFT-ROTATE(x)
```

Dictionaries II

13-3 AVL trees

Solution to Problem 13-3-b

- LEFT-ROTATE performs the corresponding tree rotation and returns the root of the subtree after the rotation.

```
LEFT-ROTATE(x)  
    y = x.right  
    x.right = y.left  
    y.left = x  
    x.h = 1 + max(x.left.h, x.right.h)  
    y.h = 1 + max(x.h, y.right.h)  
    x = y  
    return x
```

Dictionaries II

13-3 AVL trees

Solution to Problem 13-3-b

- RIGHT-ROTATE performs the corresponding tree rotation and returns the root of the subtree after the rotation.

```
RIGHT-ROTATE(x)  
  y = x.left  
  x.left = y.right  
  y.right = x  
  x.h = 1 + max(x.left.h, x.right.h)  
  y.h ← 1 + max(y.left.h, x.h)  
  x = y  
  return x
```

Dictionaries II

13-3 AVL trees

Problem 13-3-c

- Using part (b), describe a recursive procedure $\text{AVL-INSERT}(x, z)$ that takes a node x within an AVL tree and a newly created node z (whose key has already been filled in), and adds z to the subtree rooted at x , maintaining the property that x is the root of an AVL tree.
- As in TREE-INSERT , assume that $z.\text{key}$ has already been filled in and that $z.\text{left} = \text{NIL}$ and $z.\text{right} = \text{NIL}$; also assume that $z.h = 0$.
- Thus, to insert the node z into the AVL tree T , we call $\text{AVL-INSERT}(T.\text{root}, z)$.

Dictionaries II

13-3 AVL trees

Solution to Problem 13-3-c

```
AVL-INSERT( $x, z$ )  
  if  $x = \text{NIL}$  then  
    return  $z$   
  if  $z.\text{key} \leq x.\text{key}$  then  
     $x.\text{left} = \text{AVL-INSERT}(x.\text{left}, z)$   
     $x.\text{left}.p = x$   
     $x.h = x.\text{left}.h + 1$   
  else  
     $x.\text{right} = \text{AVL-INSERT}(x.\text{right}, z)$   
     $x.\text{right}.p = x$   
     $x.h = x.\text{right}.h + 1$   
   $x = \text{BALANCE}(x)$   
  return  $x$ 
```

Dictionaries II

13-3 AVL trees

Problem 13-3-d

- Show that AVL-INSERT, run on an n -node AVL tree, takes $O(\lg n)$ time and performs $O(1)$ rotations.

Solution to Problem 13-3-d

- The height of the AVL tree is $O(\log n)$.
- Therefore, the insertion and update of the height attribute take $O(\log n)$ time.
- In addition, the BALANCE operation decreases the height of the originally unbalanced tree by 1 after the rotation and thus, it will not cause any propagation of rotations to the rest of the tree.
- Therefore, AVL-INSERT takes $O(\log n)$ time to insert the node, and performs $O(1)$ rotations.

Lecture 7

Priority Queues

Priority Queues

Contents

- Heaps CLRS 6.1
- Maintaining the heap property CLRS 6.2
- Building a heap CLRS 6.3
- The heapsort algorithm CLRS 6.4
- Priority queues CLRS 6.5

Reading

- T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein.
Introduction to Algorithms. The MIT Press, 3rd edition, 2009

Priority Queues

Overview

- Like merge sort, but unlike insertion sort, heapsort runs in $O(n \log n)$ time.
- Like insertion sort, but unlike merge sort, heapsort sorts in place (only a constant number of array elements are stored outside the input array at any time).
- Heapsort combines the best of the two sorting algorithms.
- The heap data structure is useful for heapsort and it also makes an efficient priority queue.

Priority Queues

6.1 Heaps

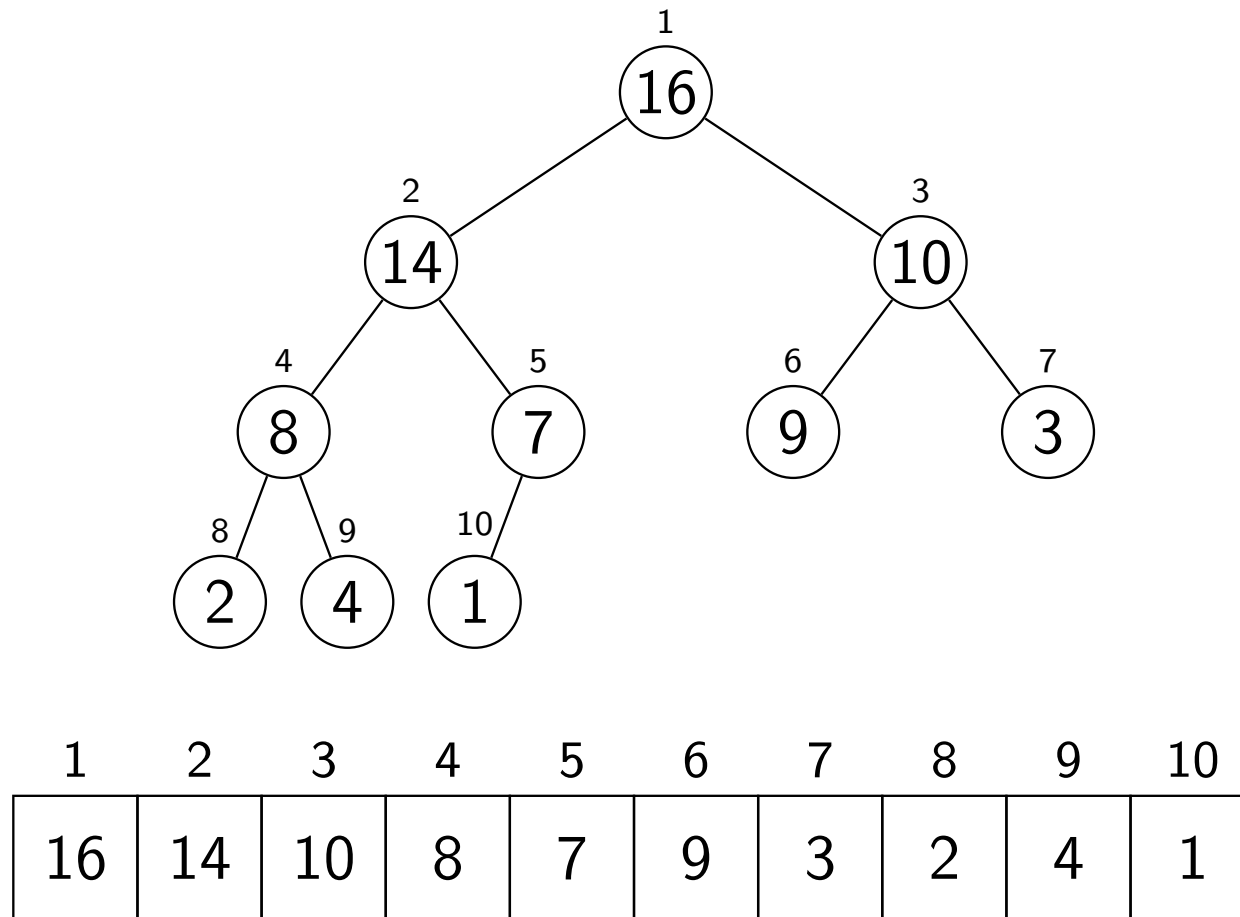
Heap data structure

- A **heap** data structure is an array object that we can view as a nearly complete binary tree.
 - Each node of the tree corresponds to an element of the array.
 - The tree is completely filled on all levels except possibly the lowest, which is filled from the left up to a point.
- An array A that represents a heap is an object with two attributes:
 - $A.length$ (the number of elements in the array)
 - $A.heap-size$ (how many elements in the heap are stored within the array)
- Only the elements in $A[1..A.heap-size]$, where $0 \leq A.heap-size \leq A.length$, are valid elements of the heap.

Priority Queues

6.1 Heaps

Example (A max-heap viewed as a binary tree and an array.)



Priority Queues

6.1 Heaps

Heap data structure

- The root of the tree is $A[1]$.
- Given the index i of a node, we can easily compute the indices of its parent, left child, and right child.

```
PARENT( $i$ )  
    return  $\lfloor i/2 \rfloor$ 
```

```
LEFT( $i$ )  
    return  $2i$ 
```

```
RIGHT( $i$ )  
    return  $2i + 1$ 
```

- Computing indices is fast with the binary representation.
 - Shift right by one bit position to divide by 2.
 - Shift left by one bit position to multiply by 2.

Priority Queues

6.1 Heaps

Heap property

- There are two kinds of heaps: max-heaps and min-heaps.
- In a max-heap, the **max-heap property** is that for every node i other than the root, $A[\text{PARENT}(i)] \geq A[i]$.
 - The largest element is at the root.
 - The subtree rooted at a node contains values no larger than that contained at the node itself.
- In a min-heap, the **min-heap property** is that for every node i other than the root, $A[\text{PARENT}(i)] \leq A[i]$.
 - The smallest element is at the root.
 - The subtree rooted at a node contains values no smaller than that contained at the node itself.
- The heapsort algorithm uses max-heaps.

Priority Queues

6.2 Maintaining the heap property

- MAX-HEAPIFY is used to maintain the max-heap property.

MAX-HEAPIFY(A, i)

$\ell = \text{LEFT}(i)$

$r = \text{RIGHT}(i)$

$largest = i$

if $\ell \leq A.heap\text{-}size$ **and** $A[\ell] > A[largest]$ **then**

$largest = \ell$

if $r \leq A.heap\text{-}size$ **and** $A[r] > A[largest]$ **then**

$largest = r$

if $largest \neq i$ **then**

exchange $A[i]$ with $A[largest]$

MAX-HEAPIFY($A, largest$)

Priority Queues

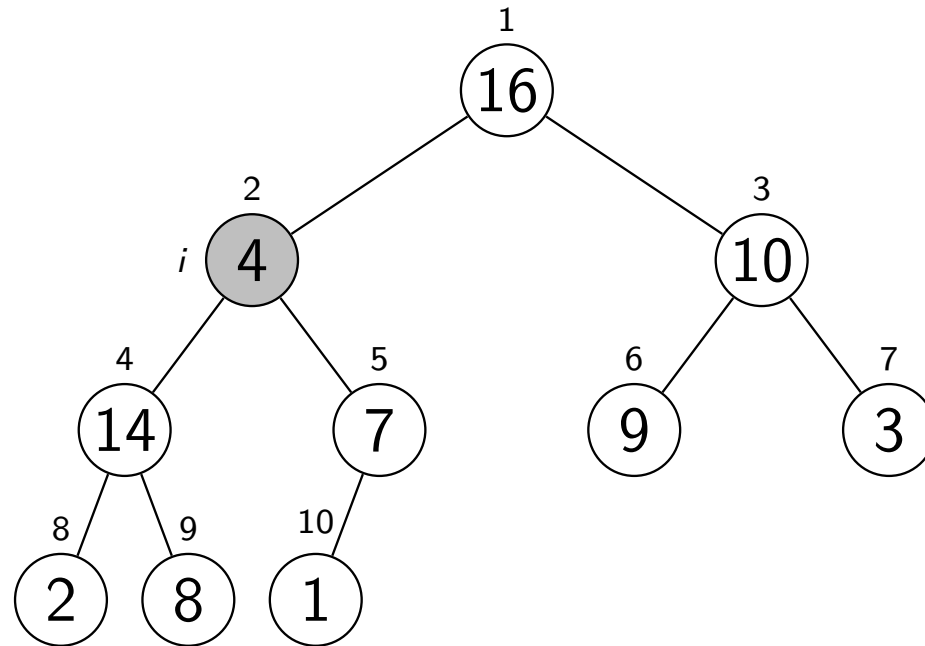
6.2 Maintaining the heap property

- MAX-HEAPIFY is used to maintain the max-heap property.
 - Assume that the trees rooted at $\text{LEFT}(i)$ and $\text{RIGHT}(i)$ are max-heaps, but that $A[i]$ might be smaller than its children (thus violating the max-heap property).
 - After MAX-HEAPIFY, the tree rooted at i is a max-heap.
- At each step,
 - Compare $A[i]$, $A[\text{LEFT}(i)]$, and $A[\text{RIGHT}(i)]$.
 - If necessary, swap $A[i]$ with the larger of the two children to preserve the max-heap property.
 - Continue this process of comparing and swapping down the heap, until the tree rooted at i is a max-heap.
- Since a heap is a nearly complete binary tree, the running time of MAX-HEAPIFY on a node of height h is $O(h) = O(\log n)$.

Priority Queues

6.2 Maintaining the heap property

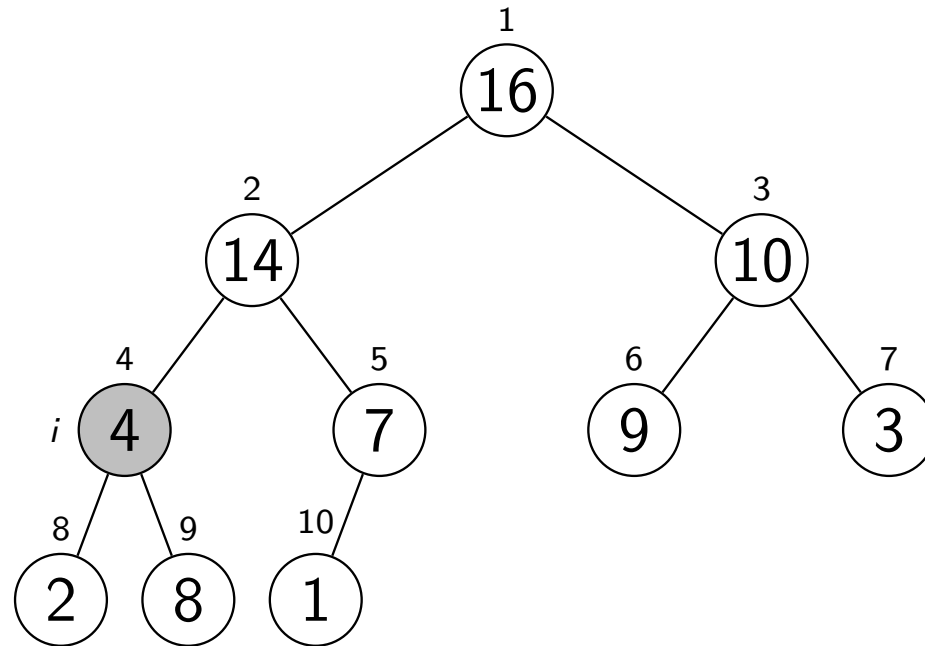
Example (MAX-HEAPIFY)



Priority Queues

6.2 Maintaining the heap property

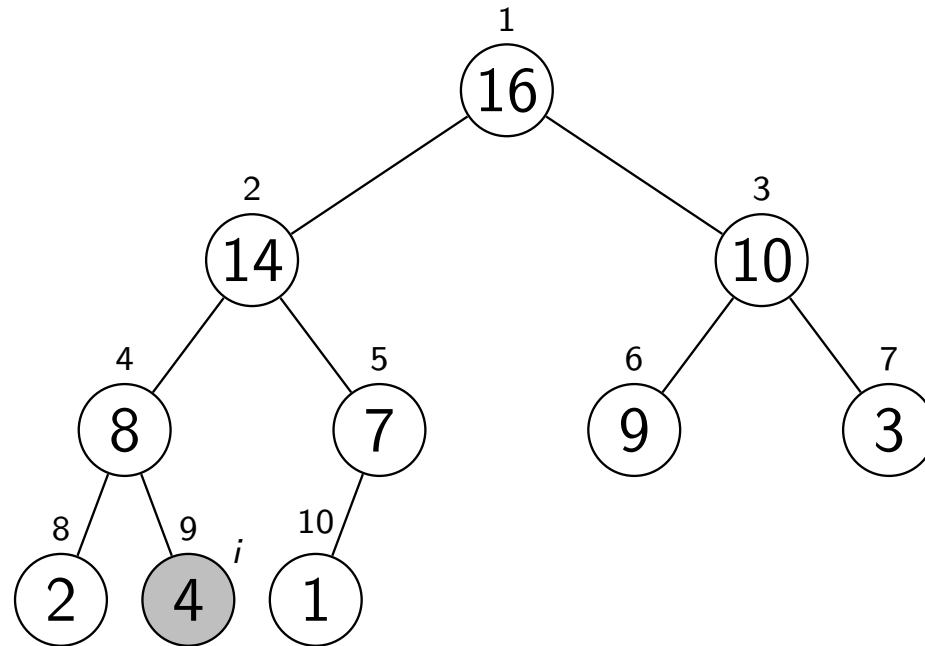
Example (MAX-HEAPIFY)



Priority Queues

6.2 Maintaining the heap property

Example (MAX-HEAPIFY)



Priority Queues

6.3 Building a heap

- We can use MAX-HEAPIFY bottom-up to convert an array $A[1..n]$, where $n = A.length$, into a max-heap.
- The elements in the subarray $A[\lfloor n/2 \rfloor + 1..n]$ are all leaves of the tree, that is, 1-element heaps to begin with.

```
BUILD-MAX-HEAP( $A$ )  
     $A.heap-size = A.length$   
    for  $i = \lfloor A.length/2 \rfloor$  downto 1 do  
        MAX-HEAPIFY( $A, i$ )
```

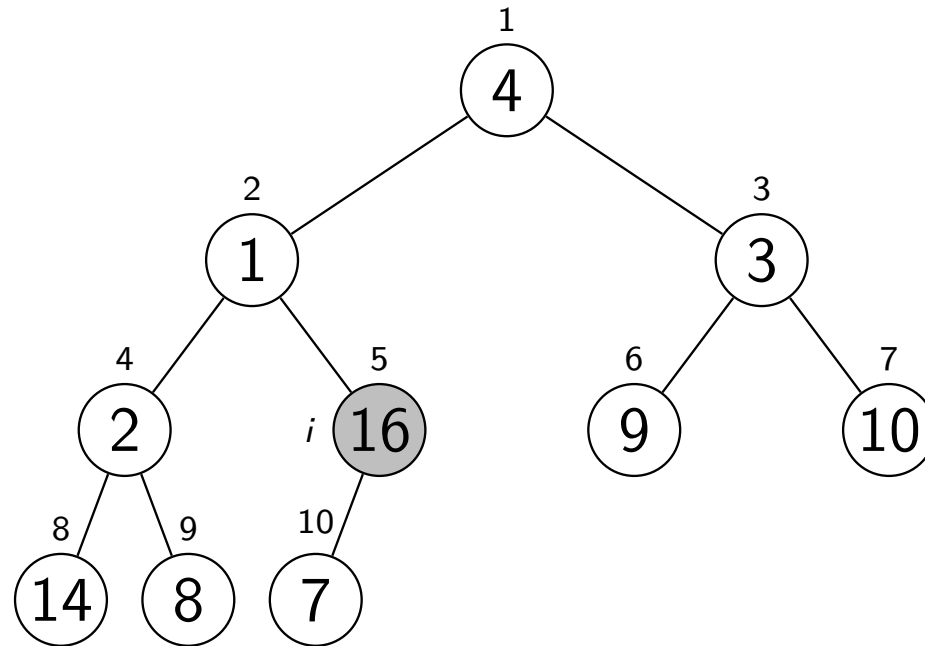
Exercise 6.1-7

Show that, with the array representation for storing an n -element heap, the leaves are the nodes indexed by $\lfloor n/2 \rfloor + 1, \lfloor n/2 \rfloor + 2, \dots, n$.

Priority Queues

6.3 Building a heap

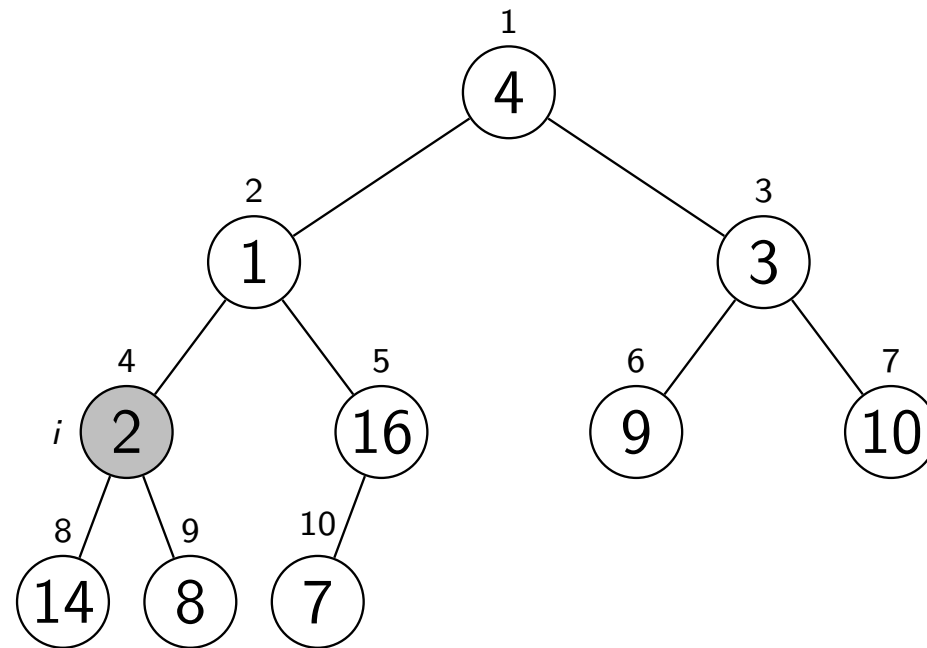
Example (BUILD-MAX-HEAP)



Priority Queues

6.3 Building a heap

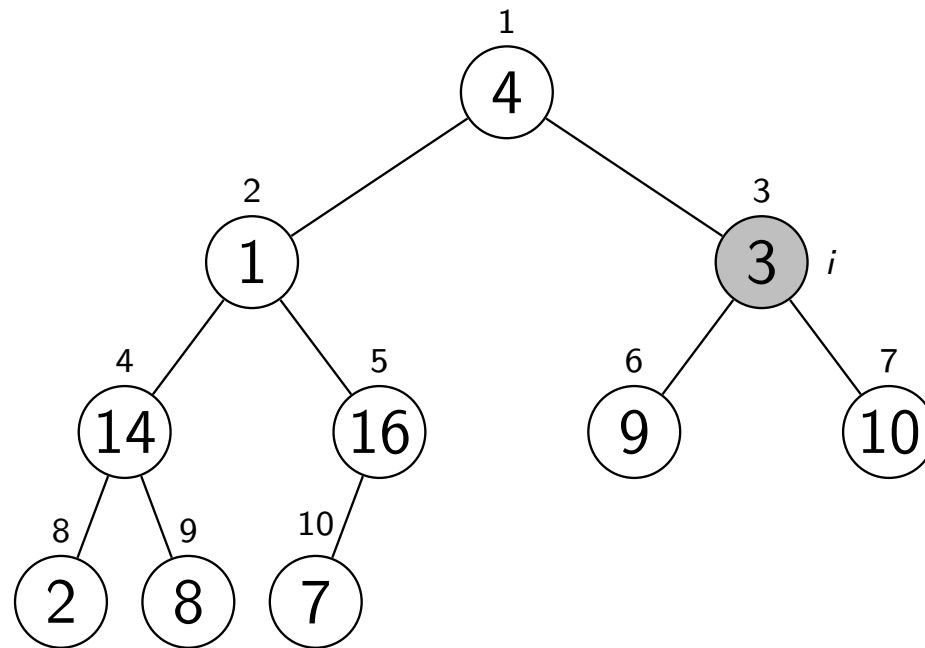
Example (BUILD-MAX-HEAP)



Priority Queues

6.3 Building a heap

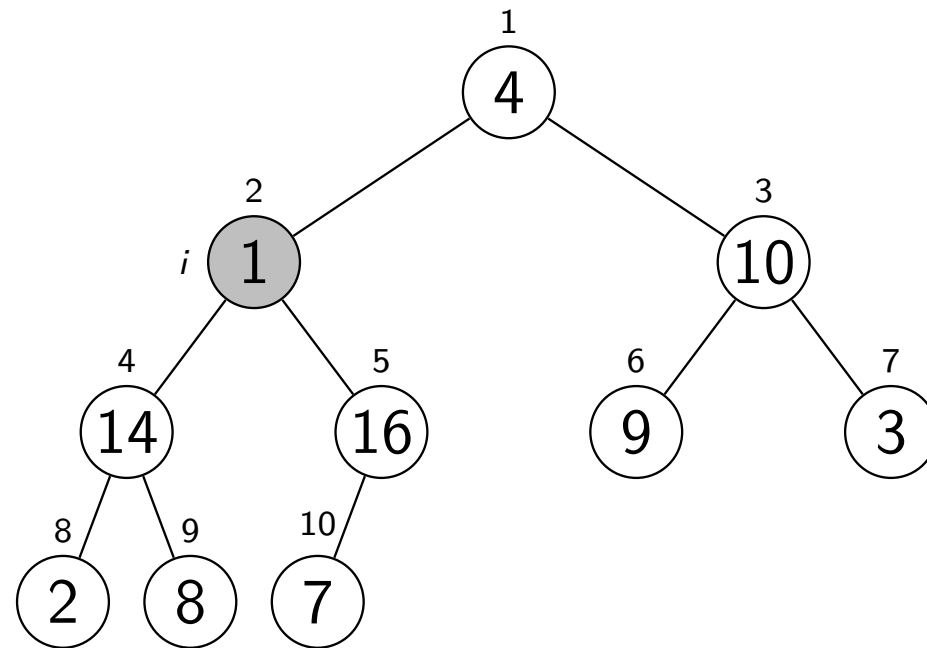
Example (BUILD-MAX-HEAP)



Priority Queues

6.3 Building a heap

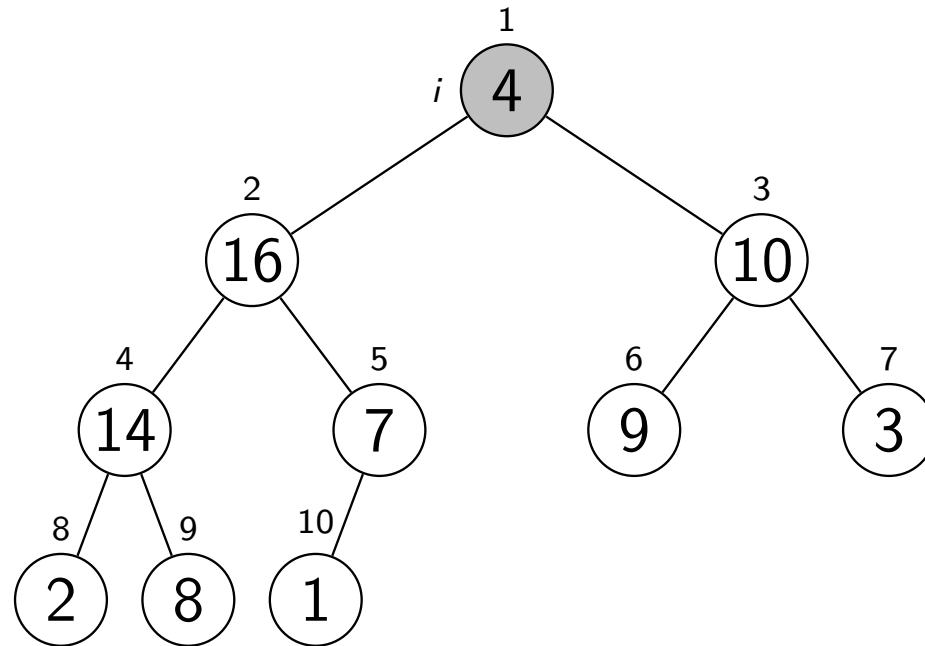
Example (BUILD-MAX-HEAP)



Priority Queues

6.3 Building a heap

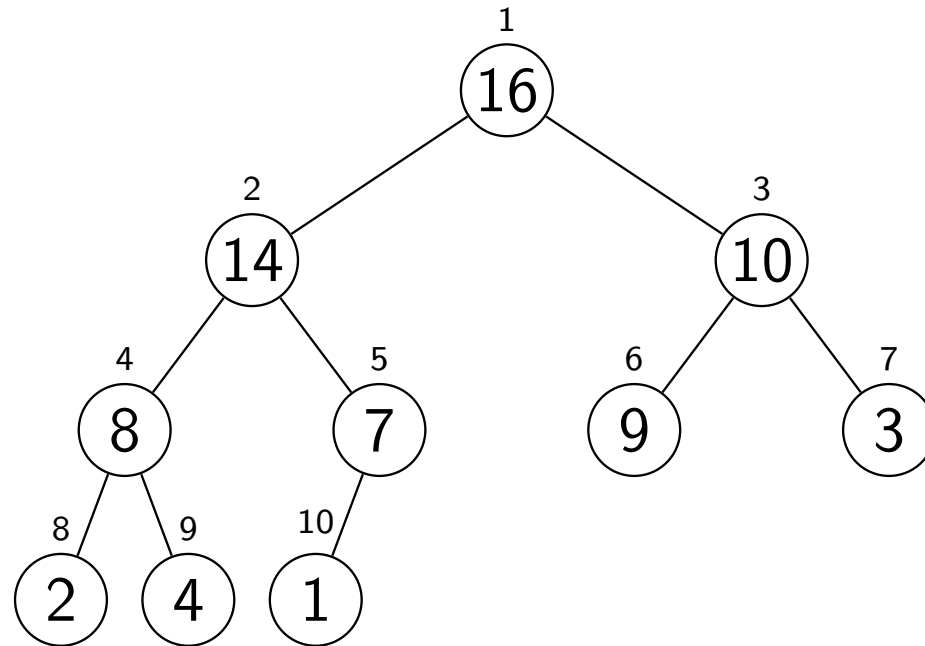
Example (BUILD-MAX-HEAP)



Priority Queues

6.3 Building a heap

Example (BUILD-MAX-HEAP)



Priority Queues

6.3 Building a heap

- The running time of BUILD-MAX-HEAP is $O(n \log n)$, because it makes $O(n)$ calls to MAX-HEAPIFY, each of which takes $O(\log n)$ time.
- However, MAX-HEAPIFY takes time linear in the height of the node in the tree, and most nodes have a small height.

Exercise 6.1-2

Show that an n -element heap has height $\lfloor \lg n \rfloor$.

Exercise 6.3-3

Show that there are at most $\lceil n/2^{h+1} \rceil$ nodes of height h in any n -element heap.

Priority Queues

6.3 Building a heap

Exercise 6.1-1

What are the minimum and maximum numbers of elements in a heap of height h ?

Solution to Exercise 6.1-1

Since a heap is an almost-complete binary tree (complete at all levels except possibly the lowest), it has at most $2^{h+1} - 1$ elements (if it is complete) and at least $2^h - 1 + 1 = 2^h$ elements (if the lowest level has just 1 element and the other levels are complete).

Solution to Exercise 6.1-2

- Given an n -element heap of height h , we know that $2^h \leq n \leq 2^{h+1} - 1 < 2^{h+1}$.
- Thus, $h \leq \lg n < h + 1$. Since h is an integer, $h = \lfloor \lg n \rfloor$.

Priority Queues

6.3 Building a heap

Solution to Exercise 6.3-3

- For a complete binary tree, it is easy to show that there are $\lceil n/2^{h+1} \rceil$ nodes of height h . For an incomplete tree, it can be proved by induction on h that there are at most $\lceil n/2^{h+1} \rceil$ nodes of height h .
- Base case: The nodes of height 0 are the leaves and there are $\lceil n/2 \rceil$ leaves in an n -element heap, by Exercise 6.1-7.
- Inductive step: Assume there are at most $\lceil n/2^{k+1} \rceil$ nodes of height k in any n -element heap for any $k < h$. Removing all the leaves from a tree of height h yields a new tree of height $h - 1$ with $n - \lceil n/2 \rceil = \lfloor n/2 \rfloor$ nodes. The number of nodes of height $h - 1$ in the new tree is the same as the number of nodes of height h in the original tree, and it is given by $\lceil \lfloor n/2 \rfloor / 2^{h-1+1} \rceil \leq \lceil (n/2) / 2^h \rceil = \lceil n/2^{h+1} \rceil$.

Priority Queues

6.3 Building a heap

- MAX-HEAPIFY takes $O(h)$ time on a node of height h .
- The total cost of BUILD-MAX-HEAP is

$$\sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) = O \left(n \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h} \right)$$

- Evaluate the last summation by substituting $x = 1/2$ in the formula $\sum_{k=0}^{\infty} k x^k = x/(1-x)^2$ for $|x| < 1$, yielding

$$\sum_{h=0}^{\infty} \frac{h}{2^h} = \frac{1/2}{(1 - 1/2)^2} = 2$$

- Thus, the running time of BUILD-MAX-HEAP is $O(n)$.

Priority Queues

6.4 The heapsort algorithm

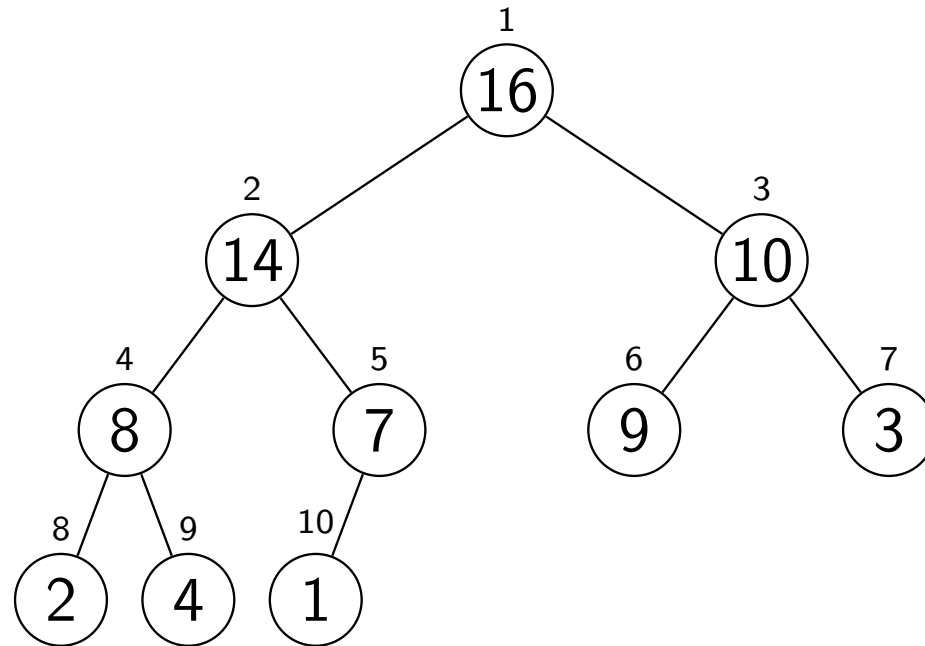
- Build a max-heap from the input array.
- Starting with the root (the largest element), place the largest element into the correct place in the array by swapping it with the element in the last position of the array.
- Discard this last node from the heap by decreasing the heap size, and restore the max-heap property.
- Repeat this process until only one node (the smallest element) remains, and therefore it is in the correct place in the array.

```
HEAPSORT(A)
  BUILD-MAX-HEAP(A)
  for  $i = A.length$  downto 2 do
    exchange  $A[1]$  with  $A[i]$ 
     $A.heap-size = A.heap-size - 1$ 
    MAX-HEAPIFY(A, 1)
```

Priority Queues

6.4 The heapsort algorithm

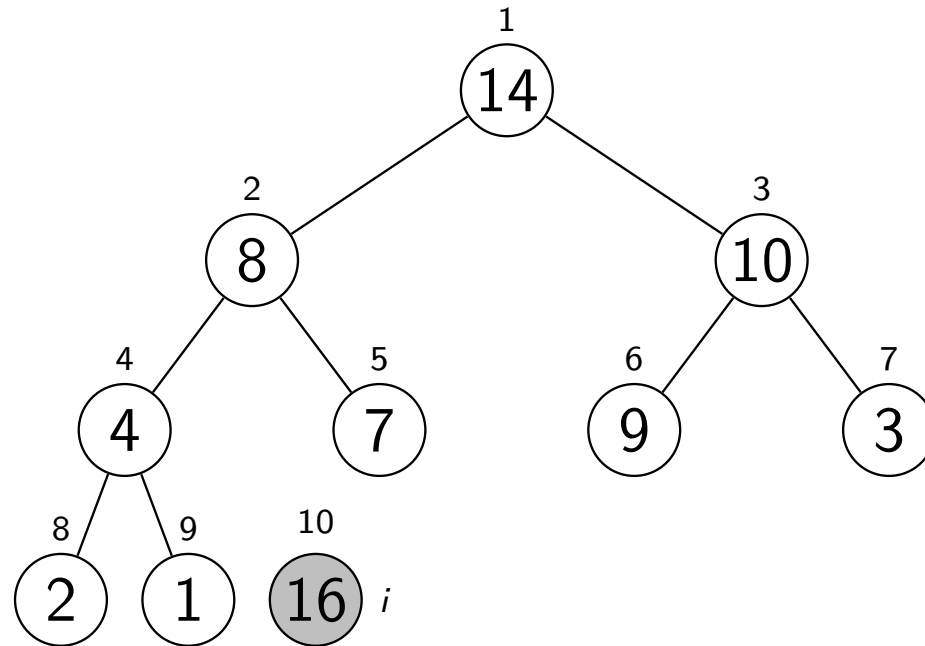
Example (HEAPSORT)



Priority Queues

6.4 The heapsort algorithm

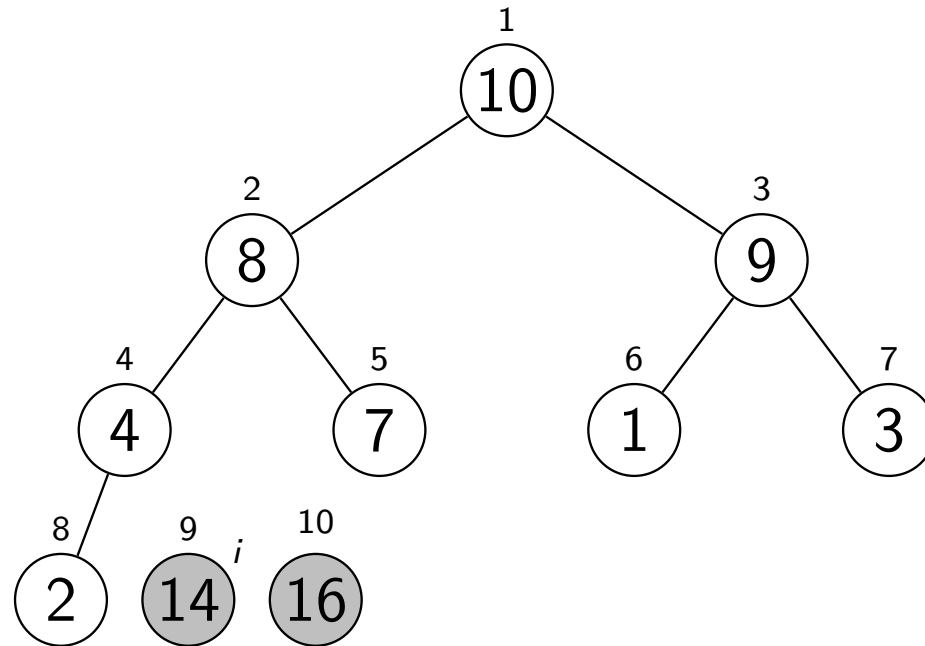
Example (HEAPSORT)



Priority Queues

6.4 The heapsort algorithm

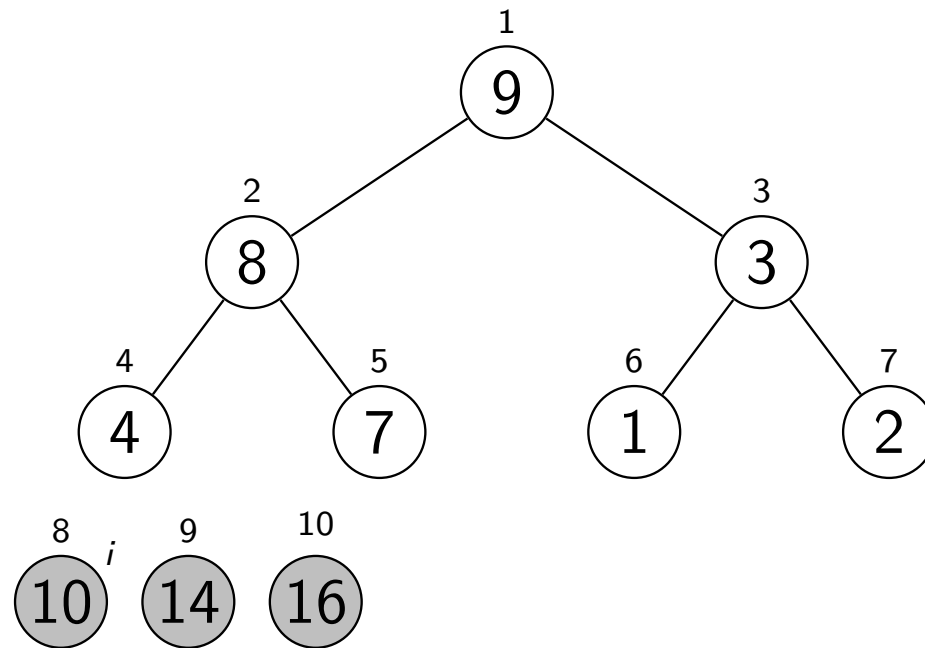
Example (HEAPSORT)



Priority Queues

6.4 The heapsort algorithm

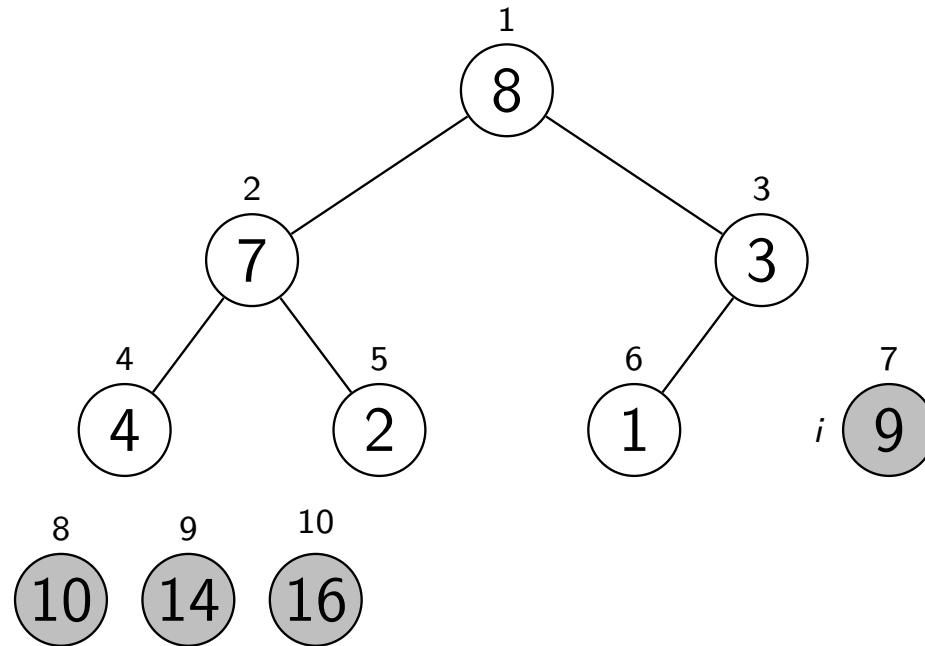
Example (HEAPSORT)



Priority Queues

6.4 The heapsort algorithm

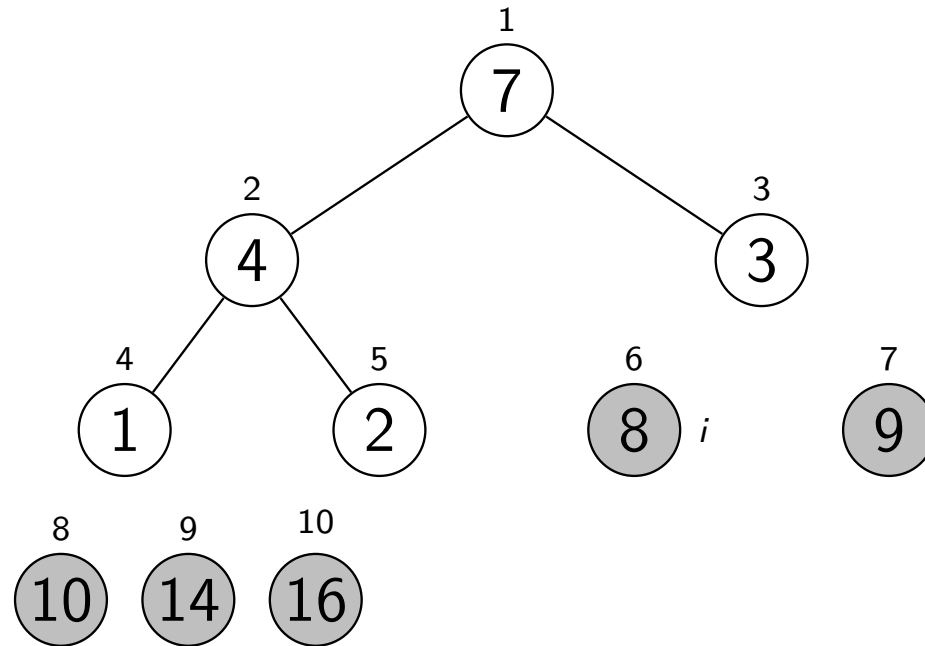
Example (HEAPSORT)



Priority Queues

6.4 The heapsort algorithm

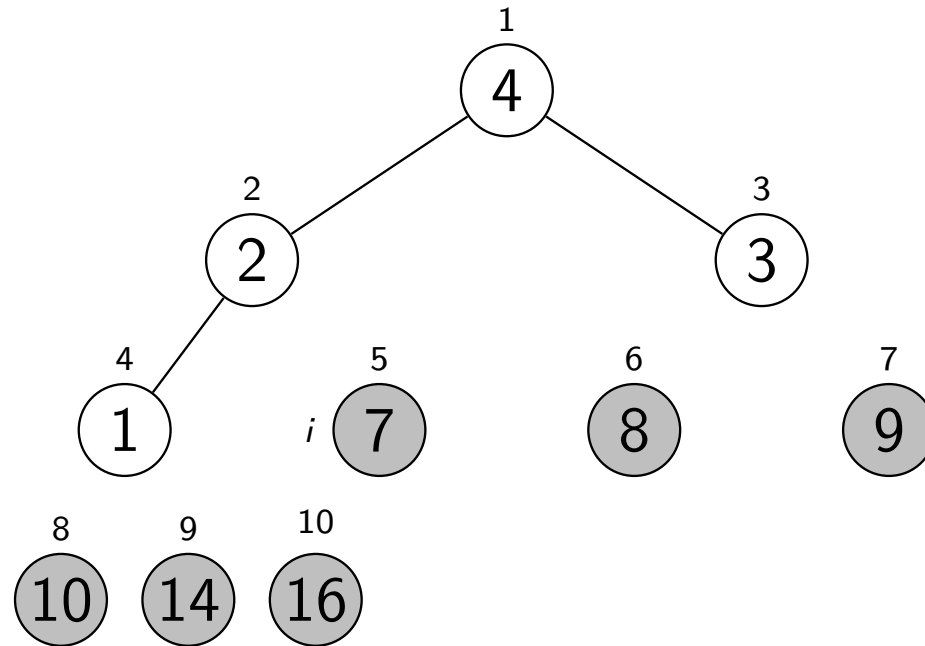
Example (HEAPSORT)



Priority Queues

6.4 The heapsort algorithm

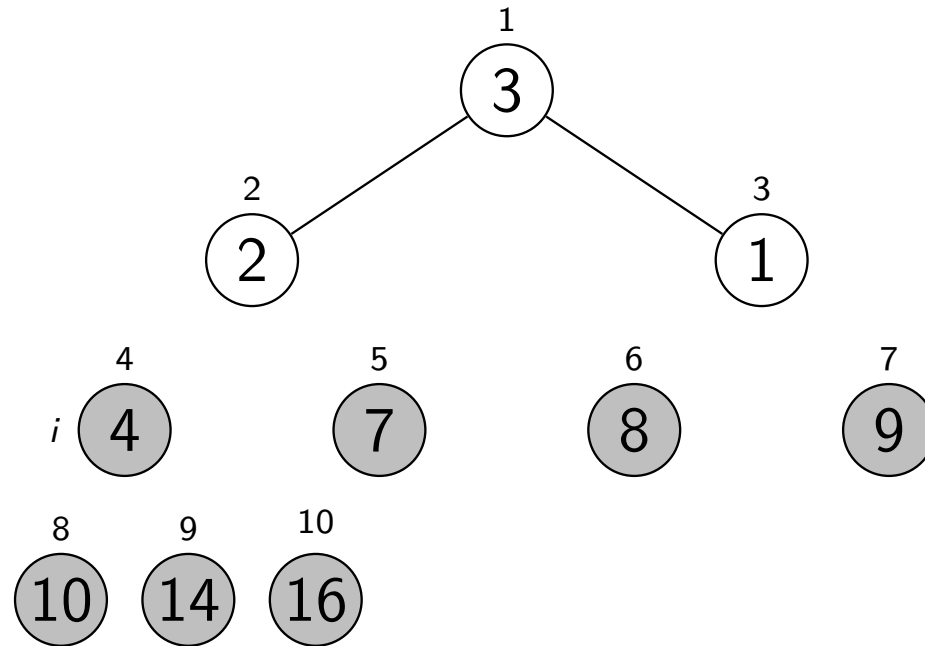
Example (HEAPSORT)



Priority Queues

6.4 The heapsort algorithm

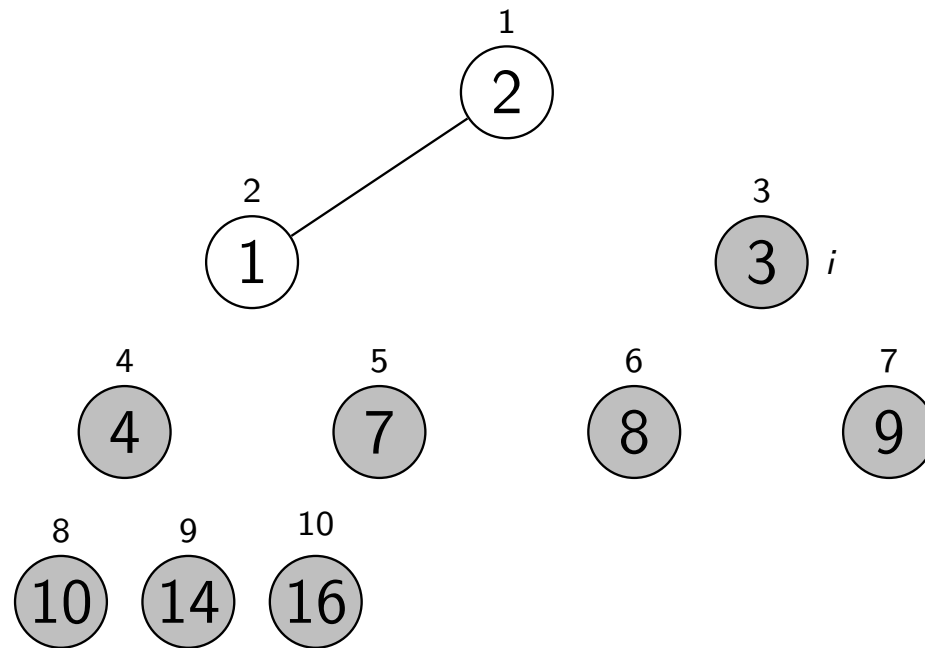
Example (HEAPSORT)



Priority Queues

6.4 The heapsort algorithm

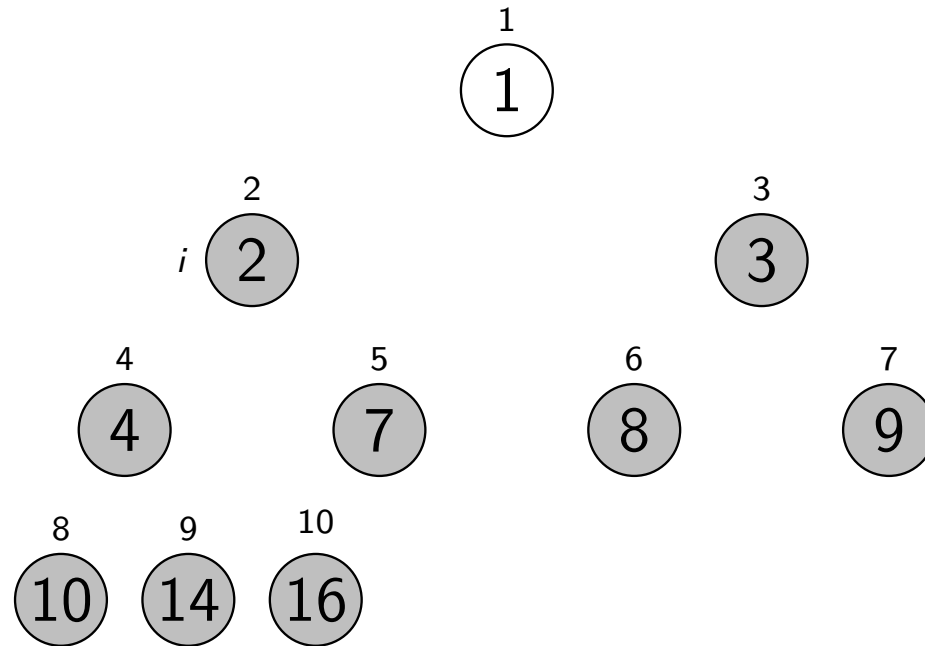
Example (HEAPSORT)



Priority Queues

6.4 The heapsort algorithm

Example (HEAPSORT)



Priority Queues

6.4 The heapsort algorithm

- HEAPSORT takes $O(n \log n)$ time, since BUILD-MAX-HEAP takes $O(n)$ time and each of the $n - 1$ calls to MAX-HEAPIFY takes $O(\log n)$ time.

Exercise 6.4-3

What is the running time of HEAPSORT on an array A of length n that is already sorted in increasing order? What about decreasing order?

Exercise 6.4-4

Show that the worst-case running time of HEAPSORT is $\Omega(n \log n)$.

Priority Queues

6.5 Priority queues

Heap implementation of priority queue

- A heap gives a good compromise between fast insertion but slow extraction and vice versa. Both take $O(\log n)$ time.
- Focus on max-priority queues, which are based on max-heaps.
- A **priority queue** is a data structure for maintaining a set S of elements, each with an associated value called a **key**, supporting the dynamic set operations:
 - $\text{INSERT}(S, x)$ to insert the element x into the set S .
 - $\text{MAXIMUM}(S)$ to return the element of S with the largest key.
 - $\text{EXTRACT-MAX}(S)$ to remove and return the element of S with the largest key.
 - $\text{INCREASE-KEY}(S, x, k)$ to increase the (at least as large as k) value of the key of element x to the new value k .

Priority Queues

6.5 Priority queues

Finding the largest element

```
HEAP-MAXIMUM(A)  
    return A[1]
```

- Running time is $O(1)$.

Priority Queues

6.5 Priority queues

Extracting the largest element

```
HEAP-EXTRACT-MAX(A)  
  if A.heap-size < 1 then  
    error “heap underflow”  
  max = A[1]  
  A[1] = A[A.heap-size − 1]  
  A.heap-size = A.heap-size − 1  
  MAX-HEAPIFY(A, 1)  
  return max
```

- Running time on an n -element heap is $O(\log n)$.

Priority Queues

6.5 Priority queues

Increasing key value

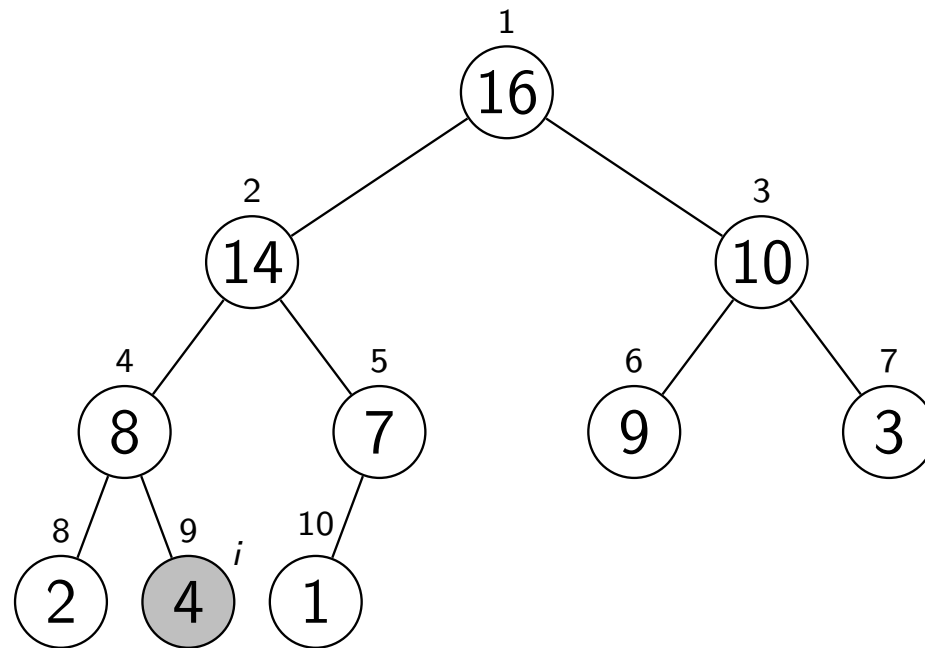
```
HEAP-INCREASE-KEY( $A, i, k$ )  
  if  $k < A[i]$  then  
    error “new key is smaller than current key”  
   $A[i] = k$   
  while  $i > 1$  and  $A[\text{PARENT}(i)] < A[i]$  do  
    exchange  $A[i]$  with  $A[\text{PARENT}(i)]$   
     $i = \text{PARENT}(i)$ 
```

- Running time on an n -element heap is $O(\log n)$.

Priority Queues

6.5 Priority queues

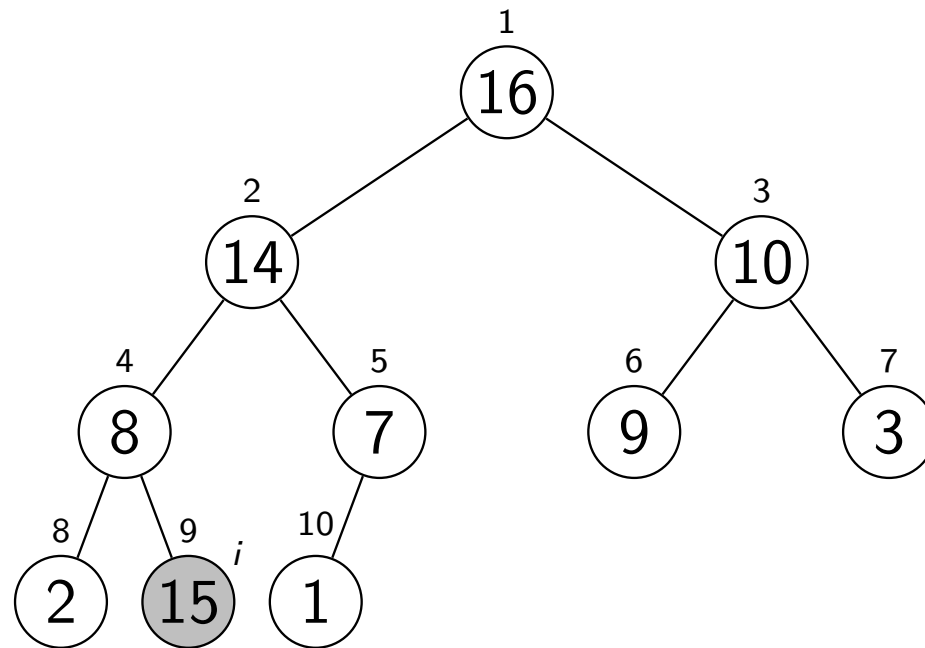
Example (HEAP-INCREASE-KEY)



Priority Queues

6.5 Priority queues

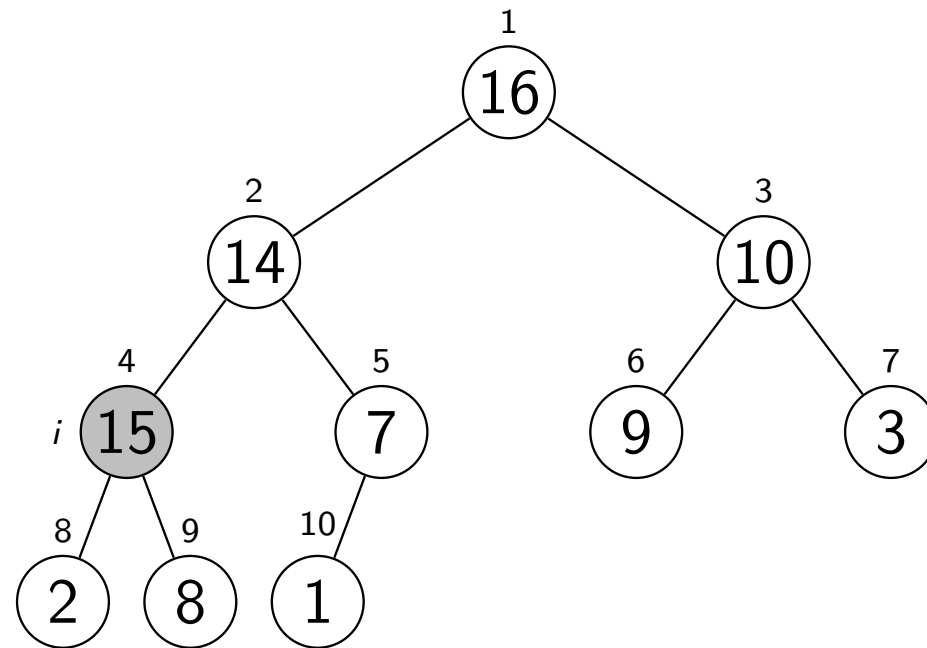
Example (HEAP-INCREASE-KEY)



Priority Queues

6.5 Priority queues

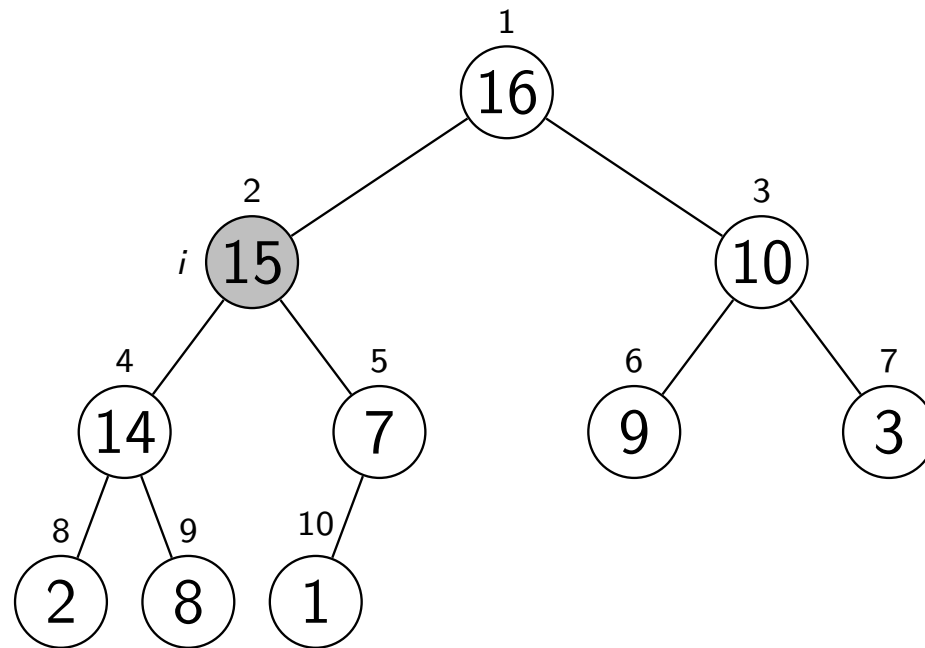
Example (HEAP-INCREASE-KEY)



Priority Queues

6.5 Priority queues

Example (HEAP-INCREASE-KEY)



Priority Queues

6.5 Priority queues

Inserting into the heap

```
HEAP-MAX-INSERT( $A, k$ )
```

```
     $A.\text{heap-size} = A.\text{heap-size} + 1$ 
```

```
     $A[A.\text{heap-size}] = -\infty$ 
```

```
    HEAP-INCREASE-KEY( $A, A.\text{heap-size}, k$ )
```

- Running time on an n -element heap is $O(\log n)$.

Summary

- Priority-queue operations take $O(\log n)$ time.
 - MAX-HEAP-INSERT(A, k)
 - HEAP-MAXIMUM(A)
 - HEAP-EXTRACT-MAX(A)
 - HEAP-INCREASE-KEY(A, i, k)

Lecture 8

Graphs I

Graphs I

Contents

- Representation of graphs CLRS 22.1
- Depth-first search CLRS 22.3
- Topological sort CLRS 22.4

Reading

- T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein.
Introduction to Algorithms. The MIT Press, 3rd edition, 2009

Graphs I

22.1 Representation of graphs

Overview

- A graph $G = (V, E)$ can be **directed** or **undirected**.
- Two standard ways to represent a graph for algorithms:
 - A collection of adjacency lists.
 - An adjacency matrix.
- A graph $G = (V, E)$ is **sparse** if $|E|$ is much less than $|V|^2$.
- A graph $G = (V, E)$ is **dense** if $|E|$ is close to $|V|^2$.
- When expressing the running time of an algorithm, it is often in terms of both $|V|$ and $|E|$.

Graphs I

22.1 Representation of graphs

Adjacency lists

- The **adjacency-list representation** of a graph $G = (V, E)$ is an array Adj of $|V|$ lists, one for each vertex in V .
- For each $u \in V$, the adjacency list $Adj[u]$ has all the vertices v such that there is an edge $(u, v) \in E$.
- In pseudocode, we treat the array as an attribute of the graph, leading to notation such as $G.Adj[u]$ for all the vertices adjacent to u in G .
- If $G = (V, E)$ is a directed graph, the sum of the lengths of all the adjacency lists is $|E|$, since an edge of the form (u, v) is represented by having v appear in $G.Adj[u]$.

Graphs I

22.1 Representation of graphs

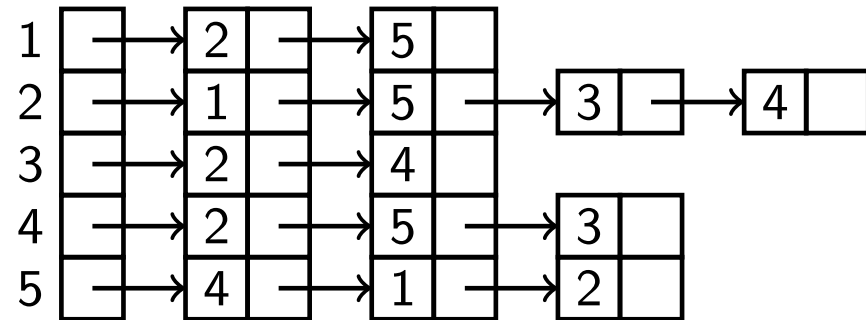
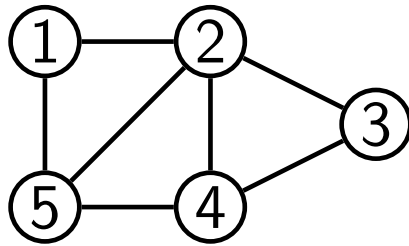
Adjacency lists

- If $G = (V, E)$ is an undirected graph, the sum of the lengths of all the adjacency lists is $2|E|$, since an undirected edge of the form (u, v) is represented by having v appear in $G.Adj[u]$ and vice versa.
- For both directed and undirected graphs, the adjacency-list representation uses $\Theta(|V| + |E|)$ space.
- Listing all vertices adjacent to vertex u takes $\Theta(\deg(u))$ time.
- Determining whether $(u, v) \in E$ takes $O(\deg(u))$ time.
- Adjacency lists can be adapted to represent a **weighted graph** $G = (V, E)$, with edge-weight function $w : E \rightarrow \mathbb{R}$.

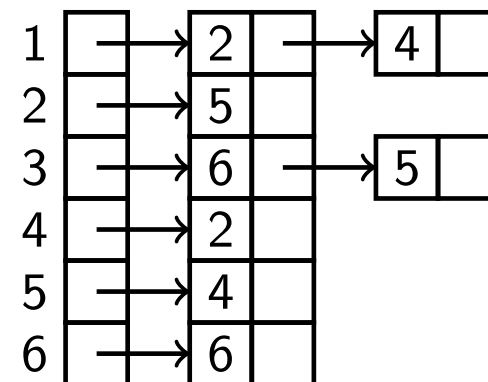
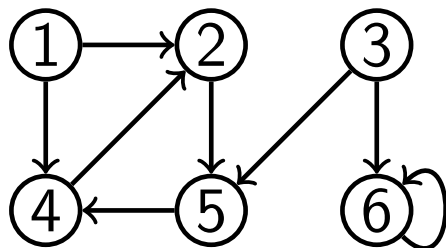
Graphs I

22.1 Representation of graphs

Example (Undirected graph)



Example (Directed graph)



Graphs I

22.1 Representation of graphs

Adjacency matrix

- The **adjacency matrix** representation of a graph $G = (V, E)$ with the vertices numbered $1, 2, \dots, |V|$ (in some arbitrary manner) is a $|V| \times |V|$ matrix $A = (a_{ij})$ such that

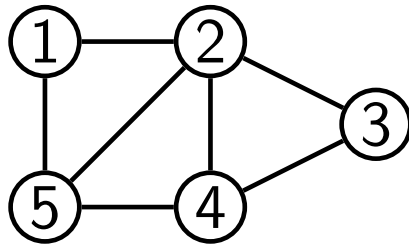
$$a_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E \\ 0 & \text{otherwise} \end{cases}$$

- For both directed and undirected graphs, the adjacency-matrix representation uses $\Theta(|V|^2)$ space, independent of $|E|$.
- Listing all vertices adjacent to vertex u takes $\Theta(|V|)$ time.
- Determining whether $(u, v) \in E$ takes $\Theta(1)$ time.
- Adjacency matrices can be adapted to represent a **weighted graph** $G = (V, E)$, with edge-weight function $w : E \rightarrow \mathbb{R}$.

Graphs I

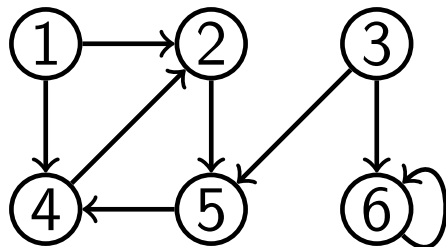
22.1 Representation of graphs

Example (Undirected graph)



	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

Example (Directed graph)



	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

Graphs I

22.1 Representation of graphs

Representing attributes

- Most graph algorithms need to maintain attributes for vertices and/or edges.
- We indicate these attributes using dot notation.
 - $v.d$ for attribute d of vertex v .
 - $(u, v).f$ for attribute f of edge (u, v) .
- When representing a graph $G = (V, E)$ using adjacency lists, vertex attributes can be represented in additional arrays, such as an array $d[1, \dots, |V|]$ to store $v.d$ in array entry $d[v]$.

Graphs I

22.3 Depth-first search

Overview

- The strategy used by **depth-first search** is to search deeper in the graph whenever possible.
- Depth-first search **colors** vertices during the search to indicate their state.
 - white** undiscovered.
 - gray** discovered, but not finished (not done exploring from it).
 - black** finished (have found everything reachable from it).
- Depth-first search **timestamps** each vertex.
 - $v.d$ when v is first discovered (and grayed).
 - $v.f$ when v is finished (and blackened).
- Depth-first search can also compute the predecessor $v.\pi$ of each vertex v .

Graphs I

22.3 Depth-first search

- The **discovery time** and **finishing time** timestamps are integers such that $1 \leq u.d < u.f \leq 2|V|$ for every vertex u .
- The input graph G may be directed or undirected.
- The variable **time** is a global variable used for timestamping.

DFS(G)

for each vertex $u \in G.V$ **do**

$u.color = \text{WHITE}$

$time = 0$

for each vertex $u \in G.V$ **do**

if $u.color = \text{WHITE}$ **then**

 DFS-VISIT(G, u)

Graphs I

22.3 Depth-first search

DFS-VISIT(G, u)

$time = time + 1$

$u.d = time$

$u.color = \text{GRAY}$

discover u

for each vertex $v \in G.Adj[u]$ **do**

explore (u, v)

if $v.color = \text{WHITE}$ **then**

DFS-VISIT(G, v)

$time = time + 1$

$u.f = time$

$u.color = \text{BLACK}$

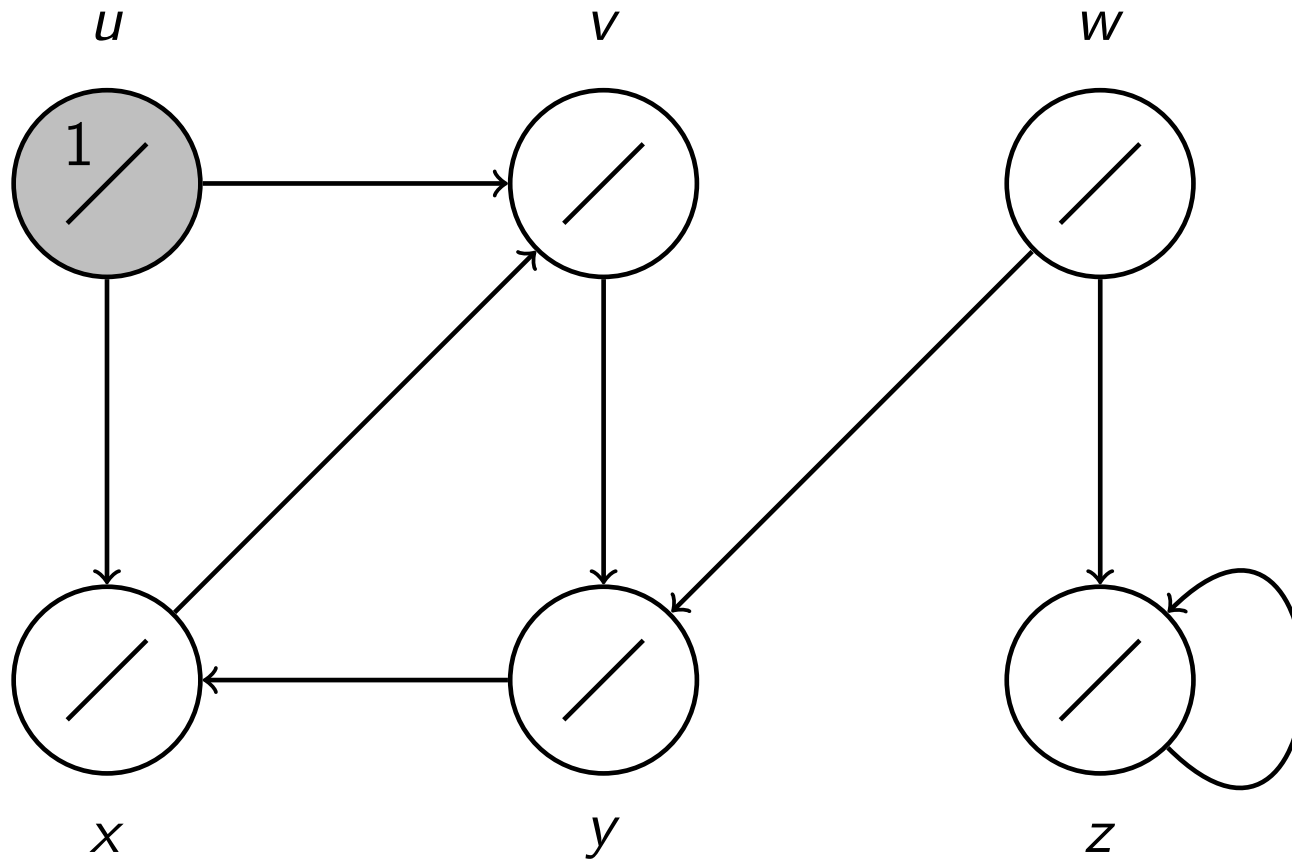
finish u

- Depth-first search takes $\Theta(|V| + |E|)$ time, because the loops over all the vertices take $\Theta(|V|)$ time, exclusive of DFS-VISIT, which is called once for each (white) vertex, the loop in DFS-VISIT executes once for each adjacent vertex, and the sum of the lengths of all the adjacency lists is $\Theta(|E|)$.

Graphs I

22.3 Depth-first search

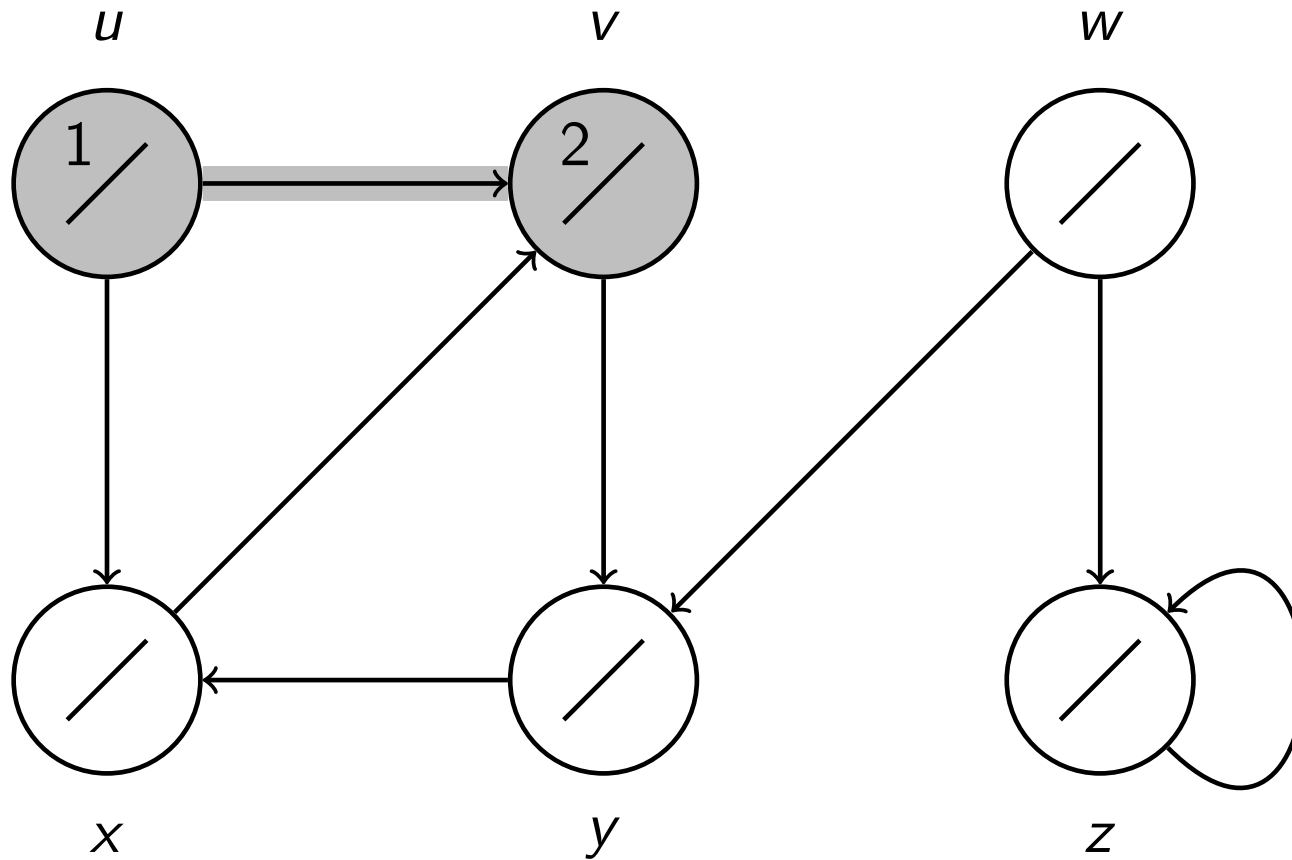
Example



Graphs I

22.3 Depth-first search

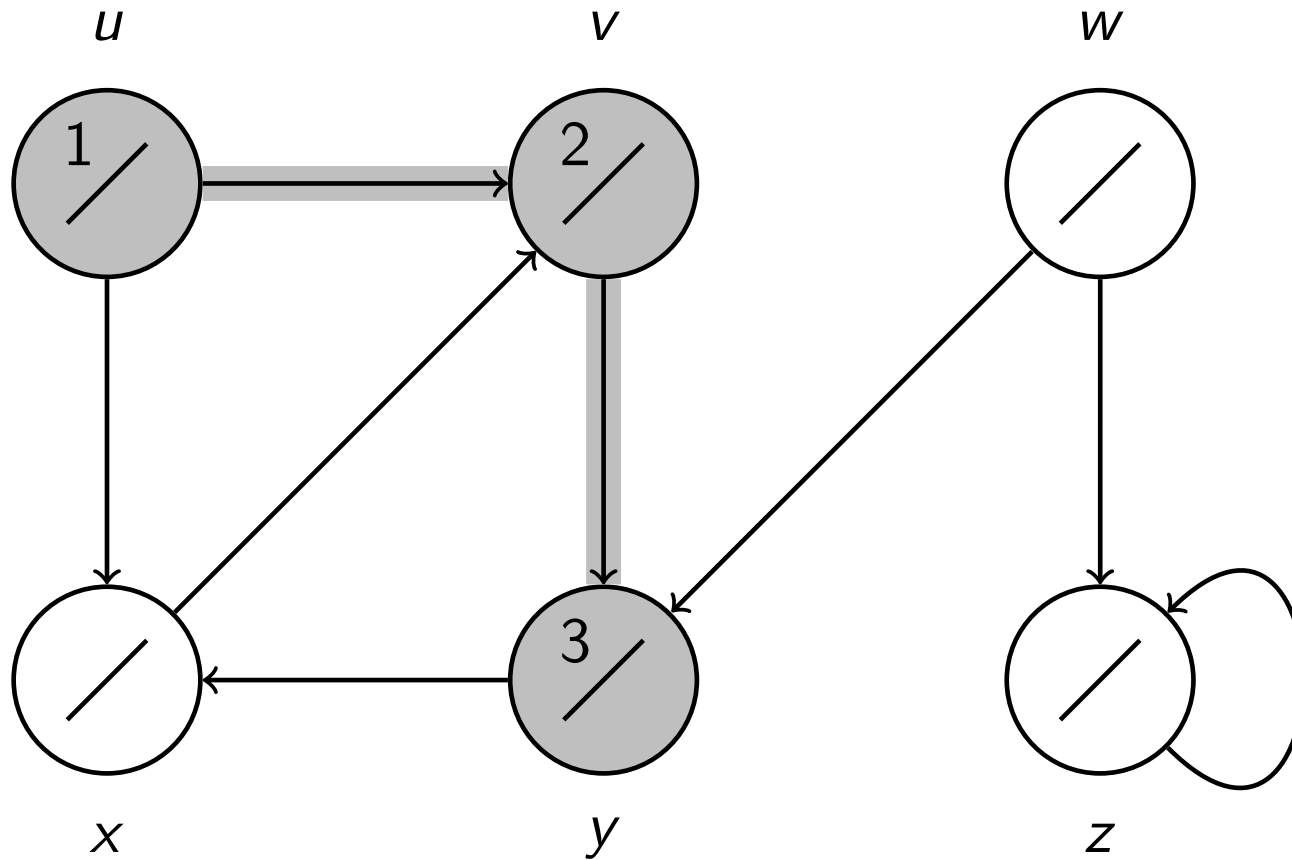
Example



Graphs I

22.3 Depth-first search

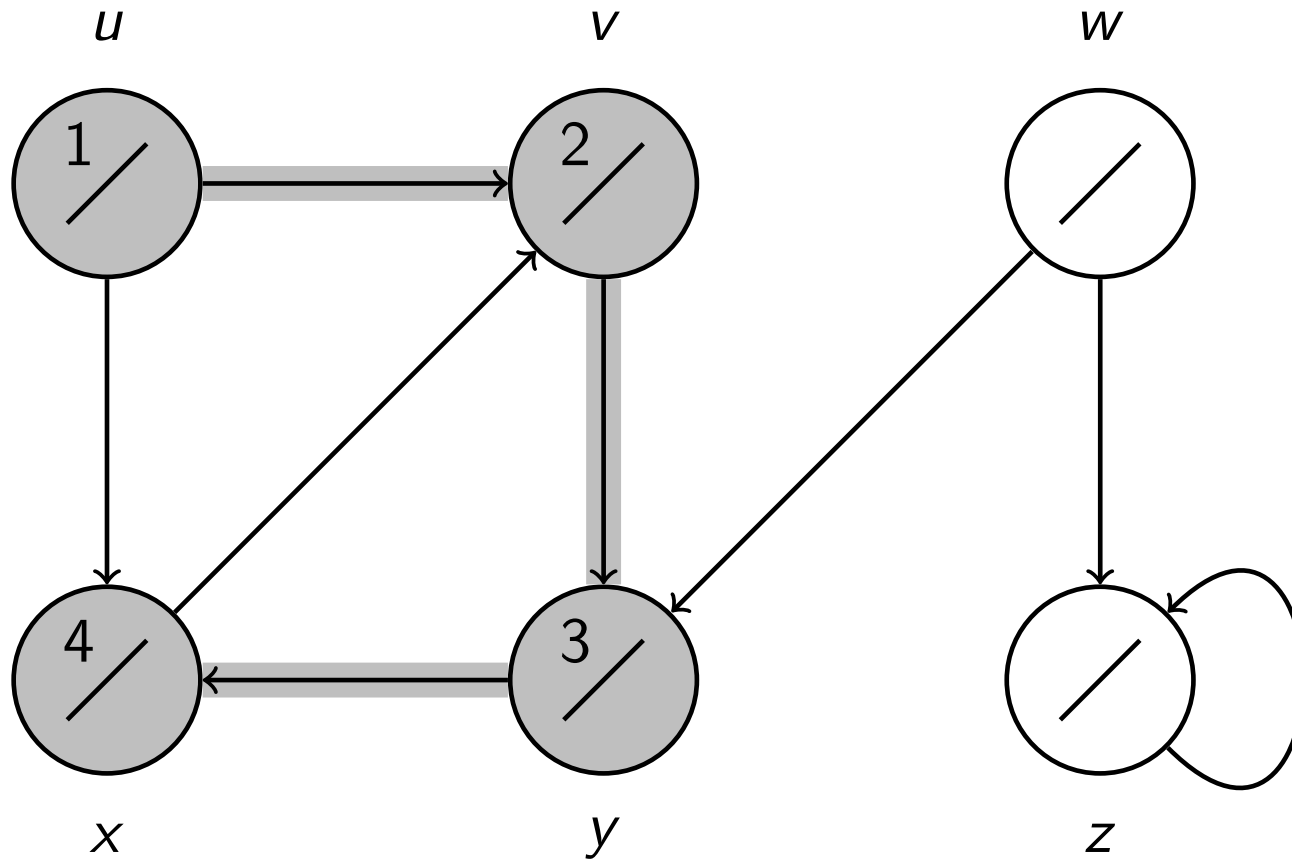
Example



Graphs I

22.3 Depth-first search

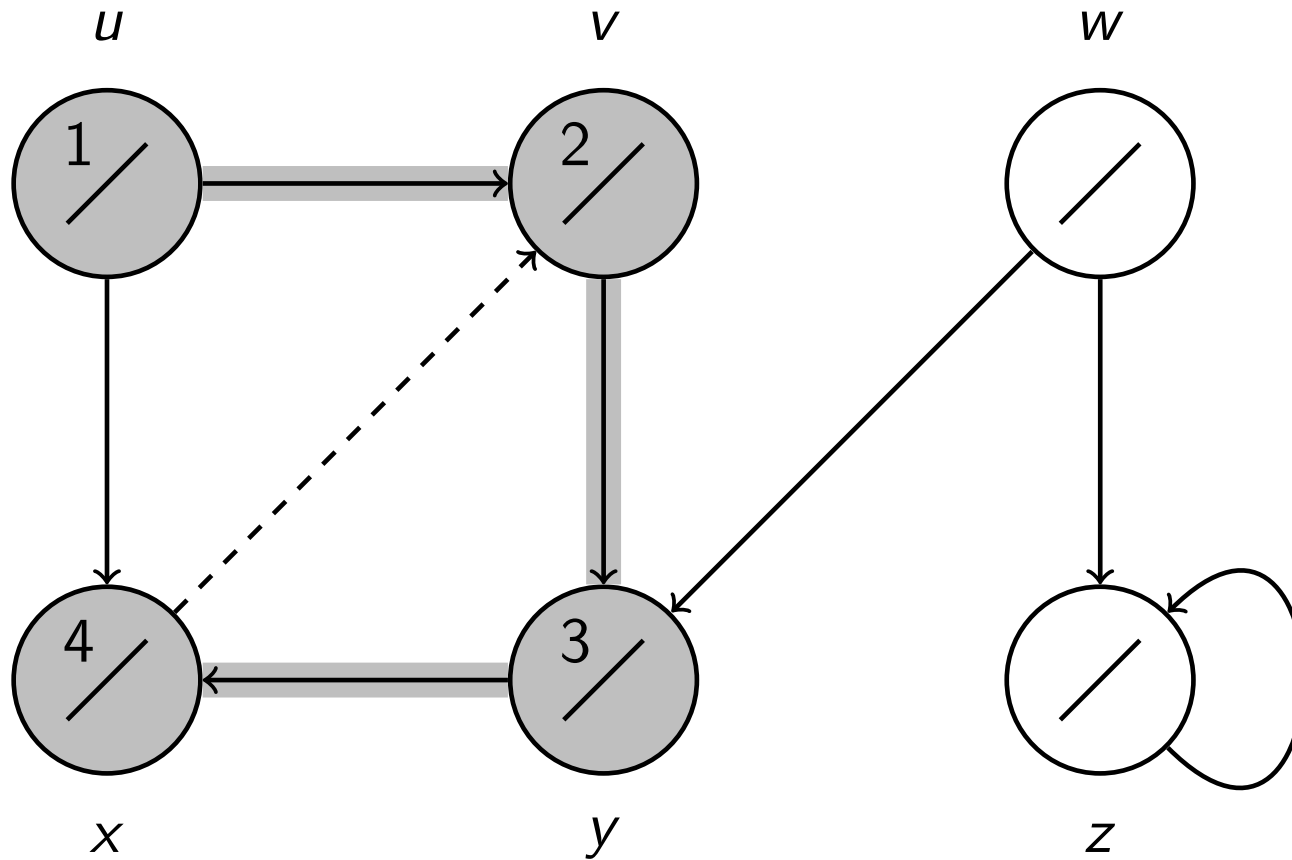
Example



Graphs I

22.3 Depth-first search

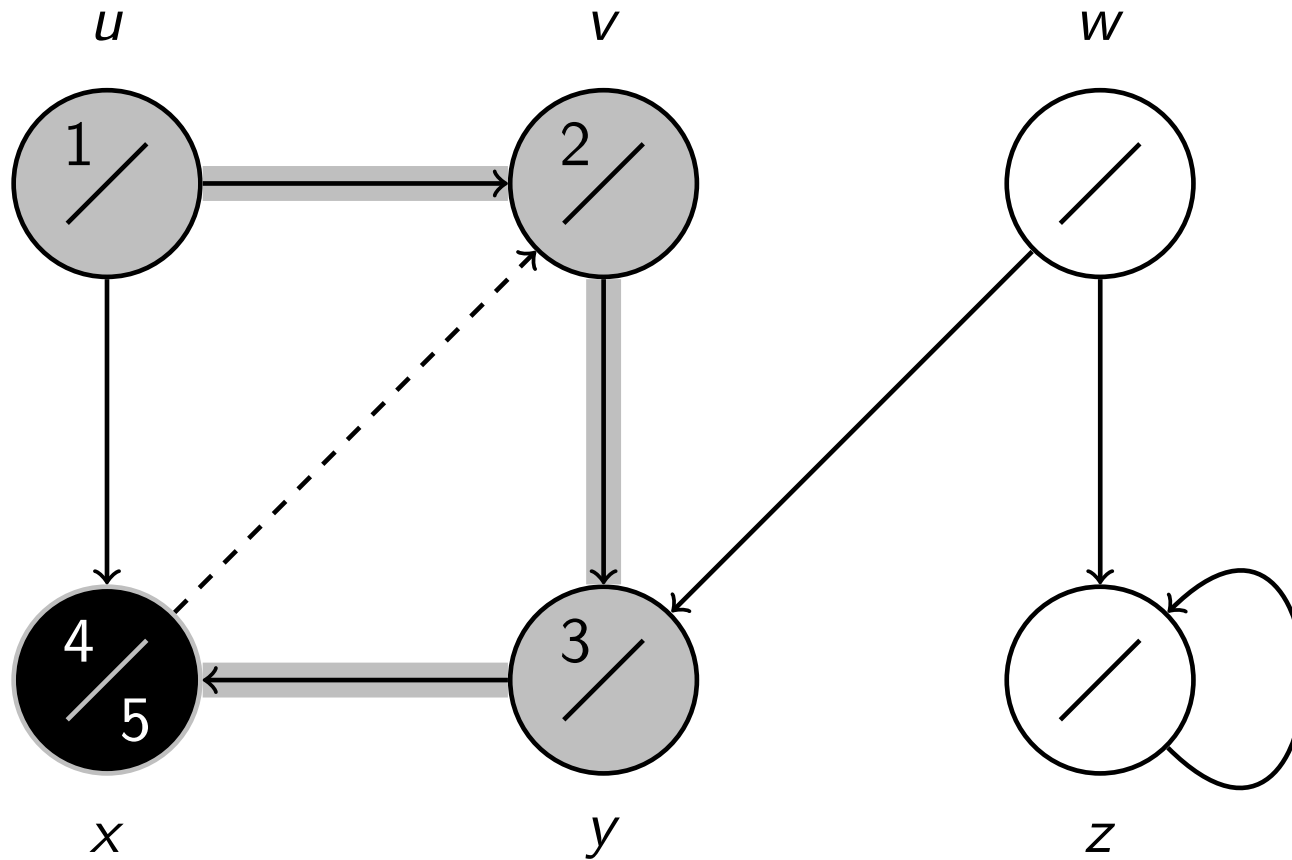
Example



Graphs I

22.3 Depth-first search

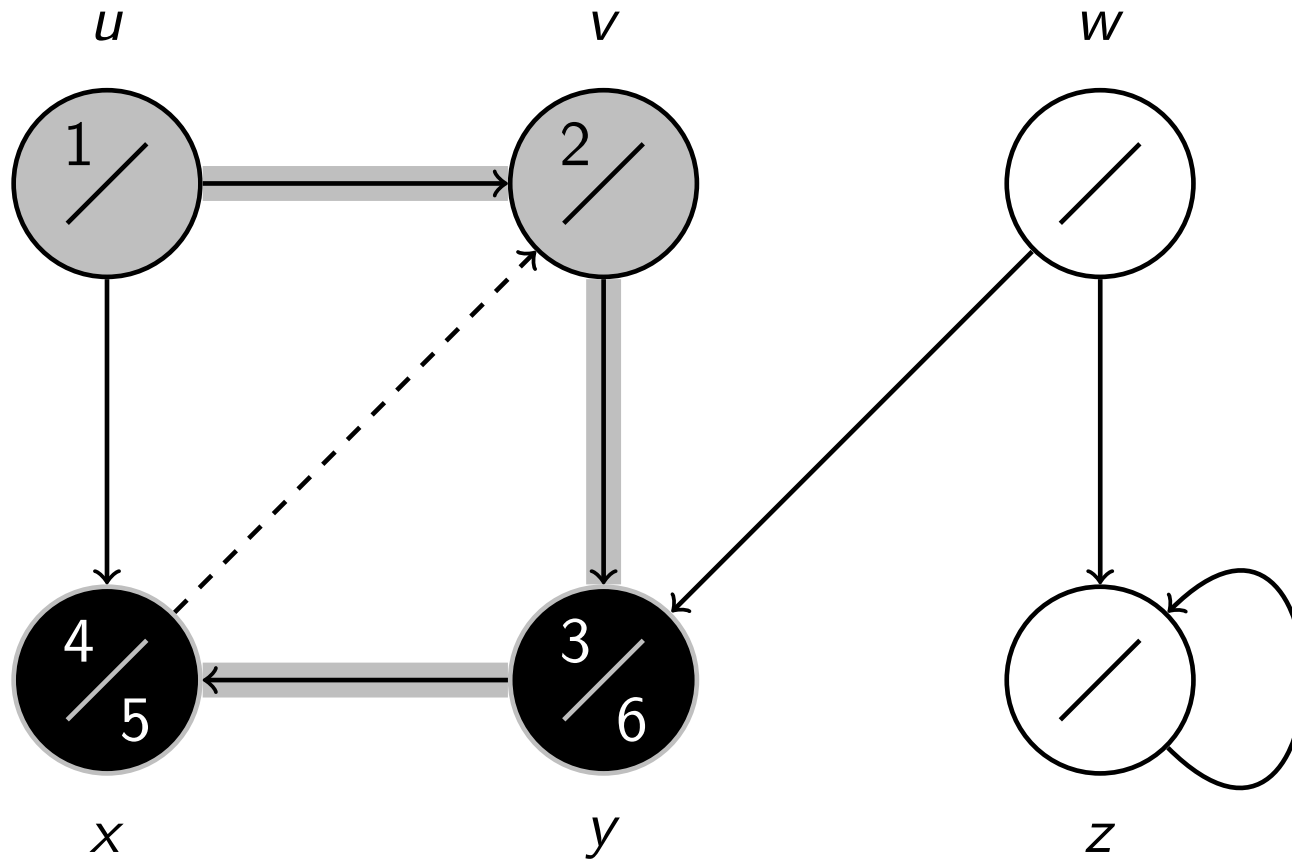
Example



Graphs I

22.3 Depth-first search

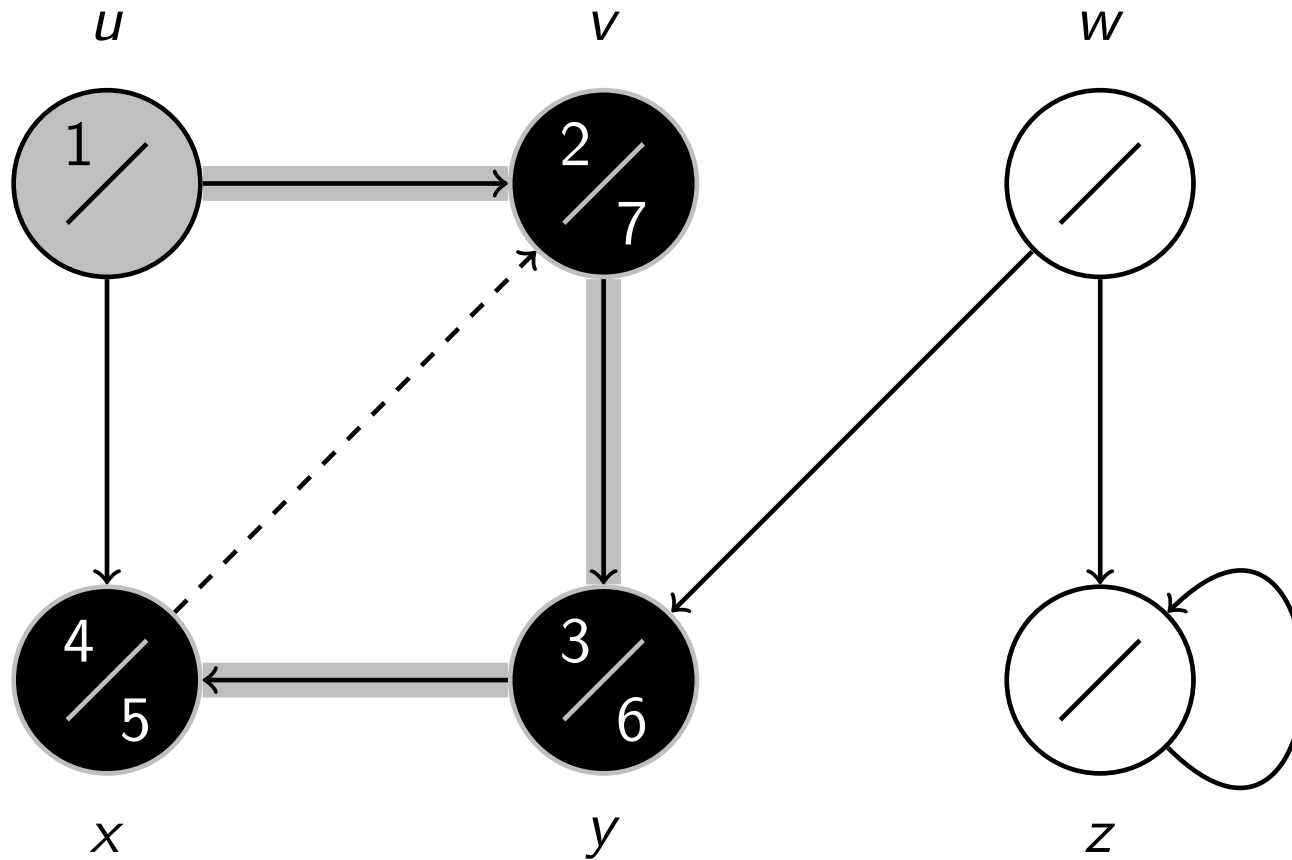
Example



Graphs I

22.3 Depth-first search

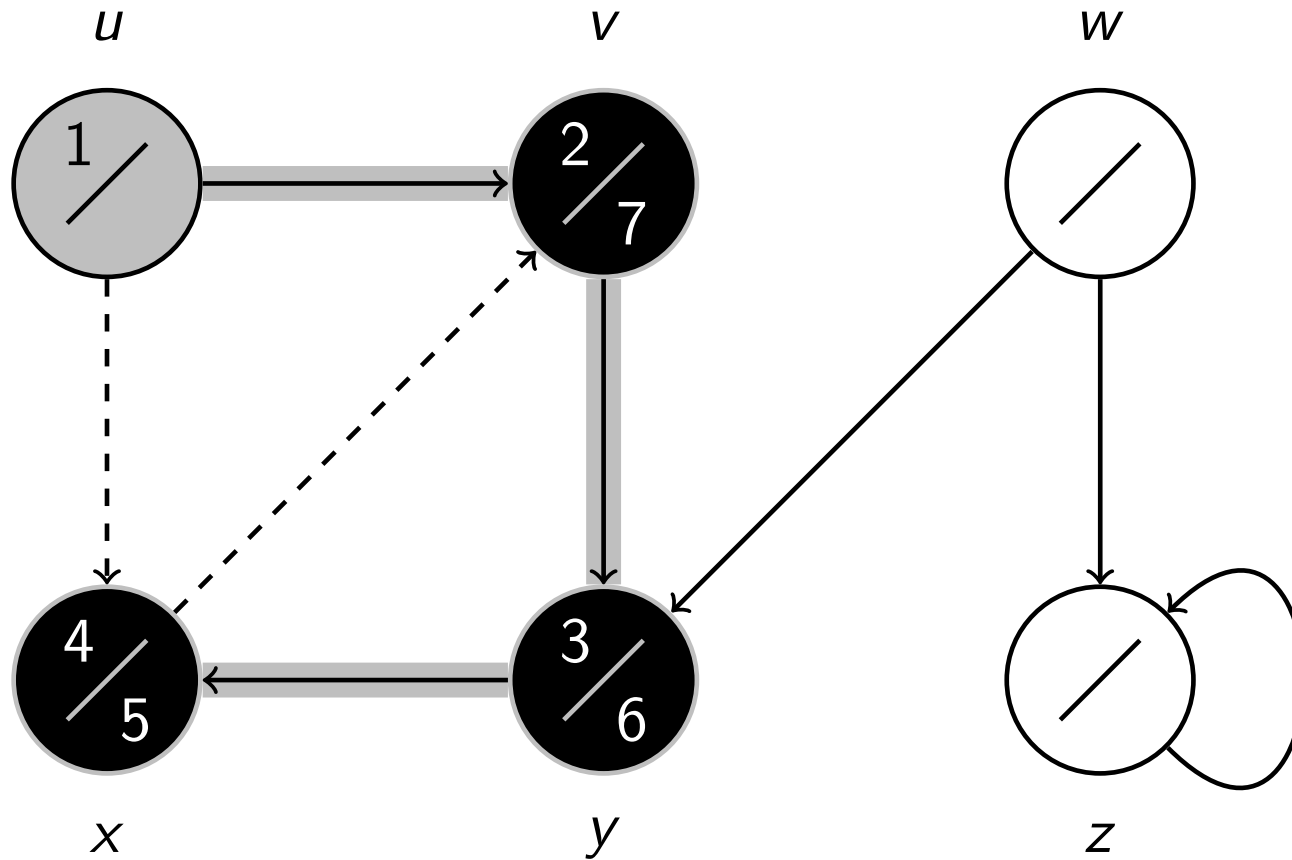
Example



Graphs I

22.3 Depth-first search

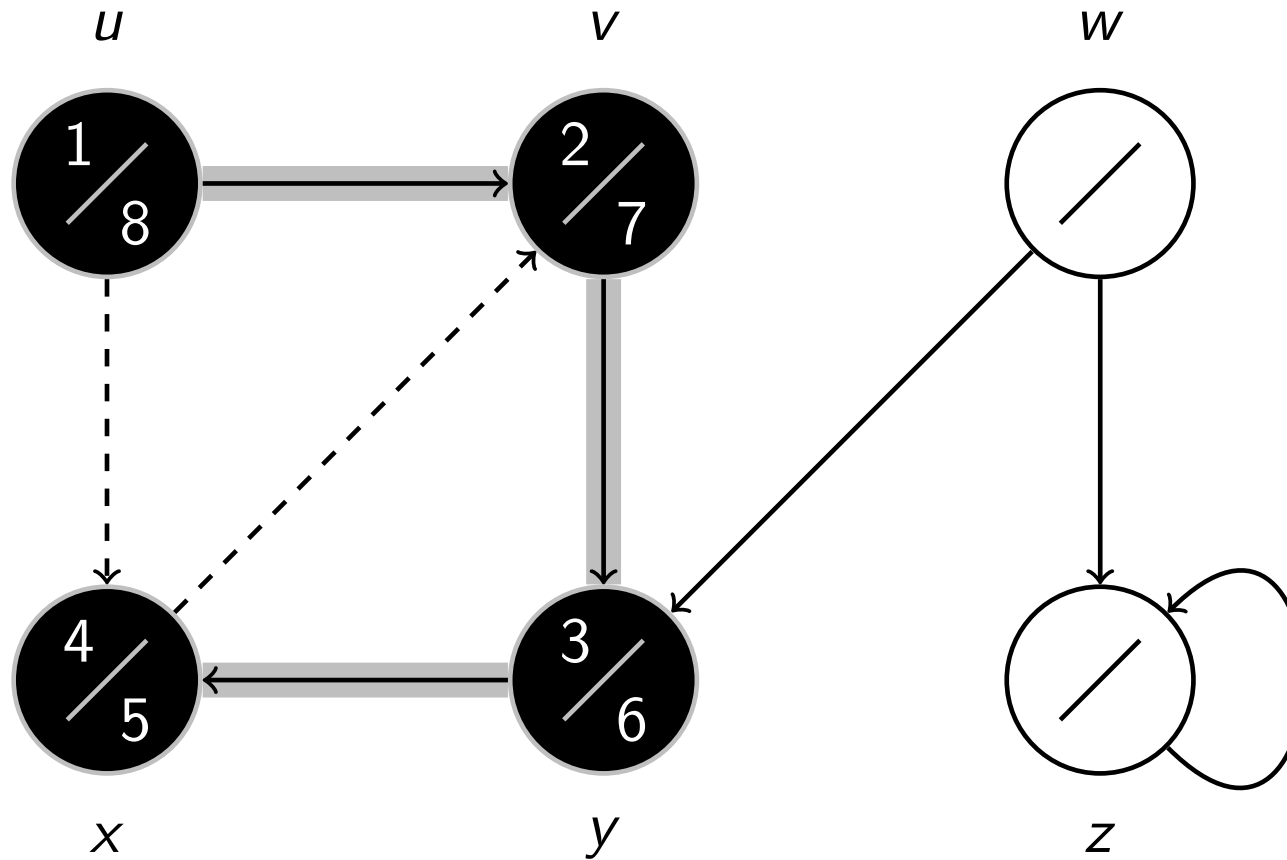
Example



Graphs I

22.3 Depth-first search

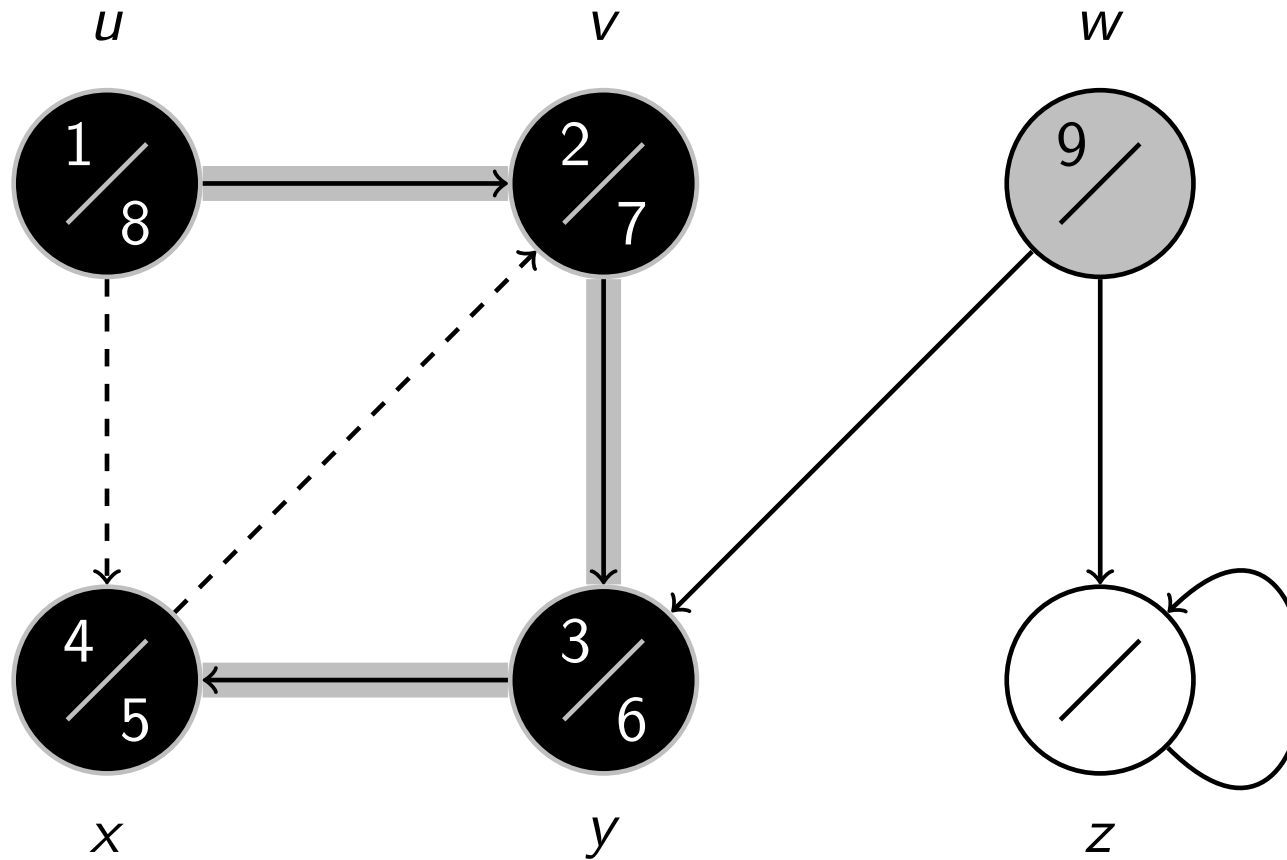
Example



Graphs I

22.3 Depth-first search

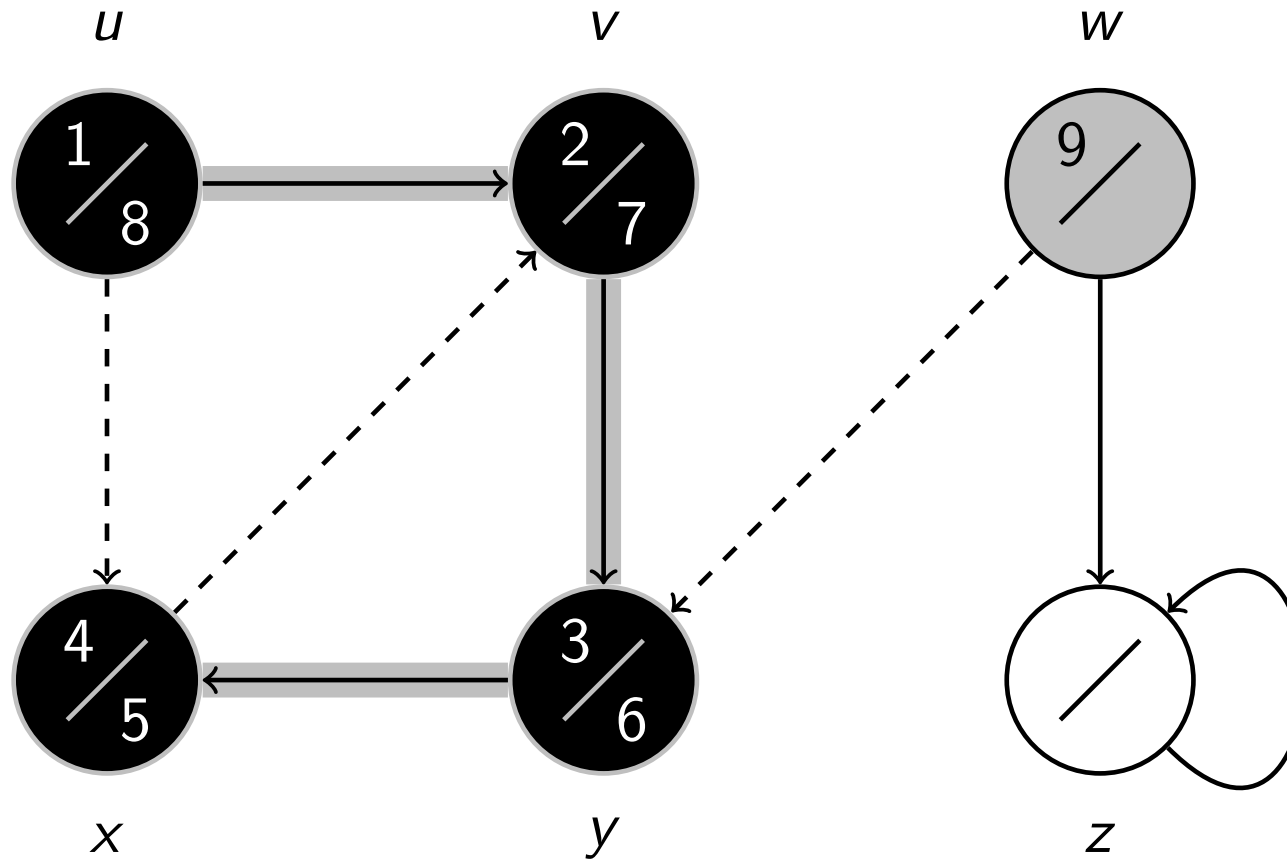
Example



Graphs I

22.3 Depth-first search

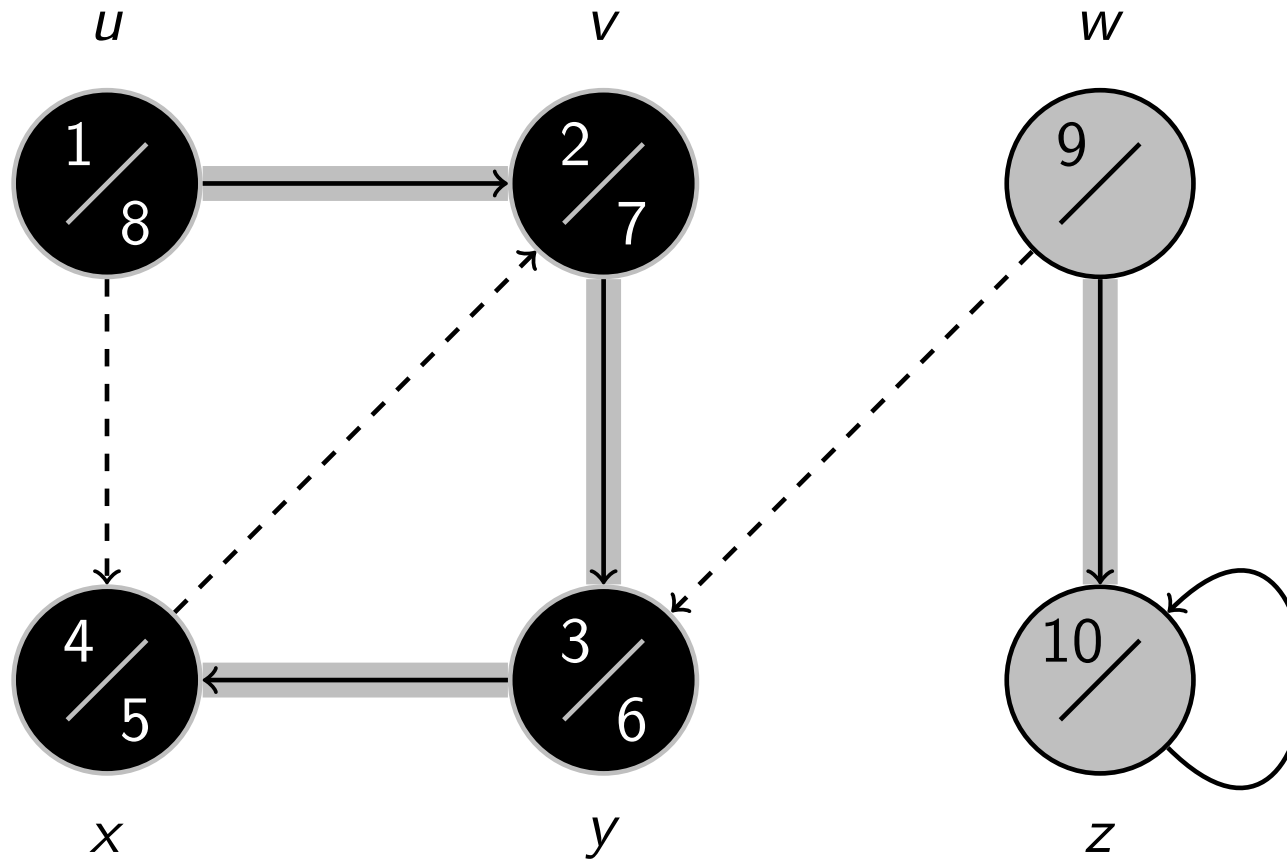
Example



Graphs I

22.3 Depth-first search

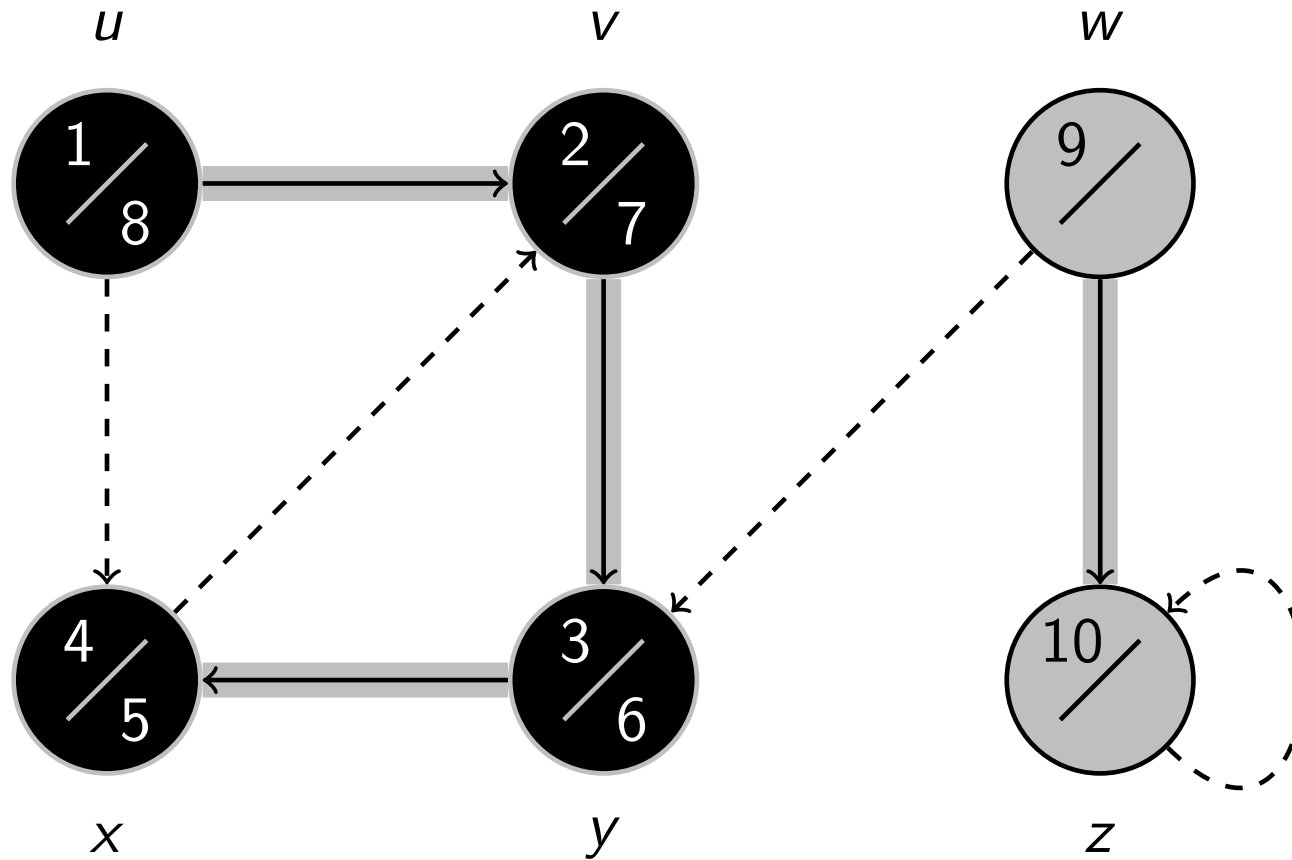
Example



Graphs I

22.3 Depth-first search

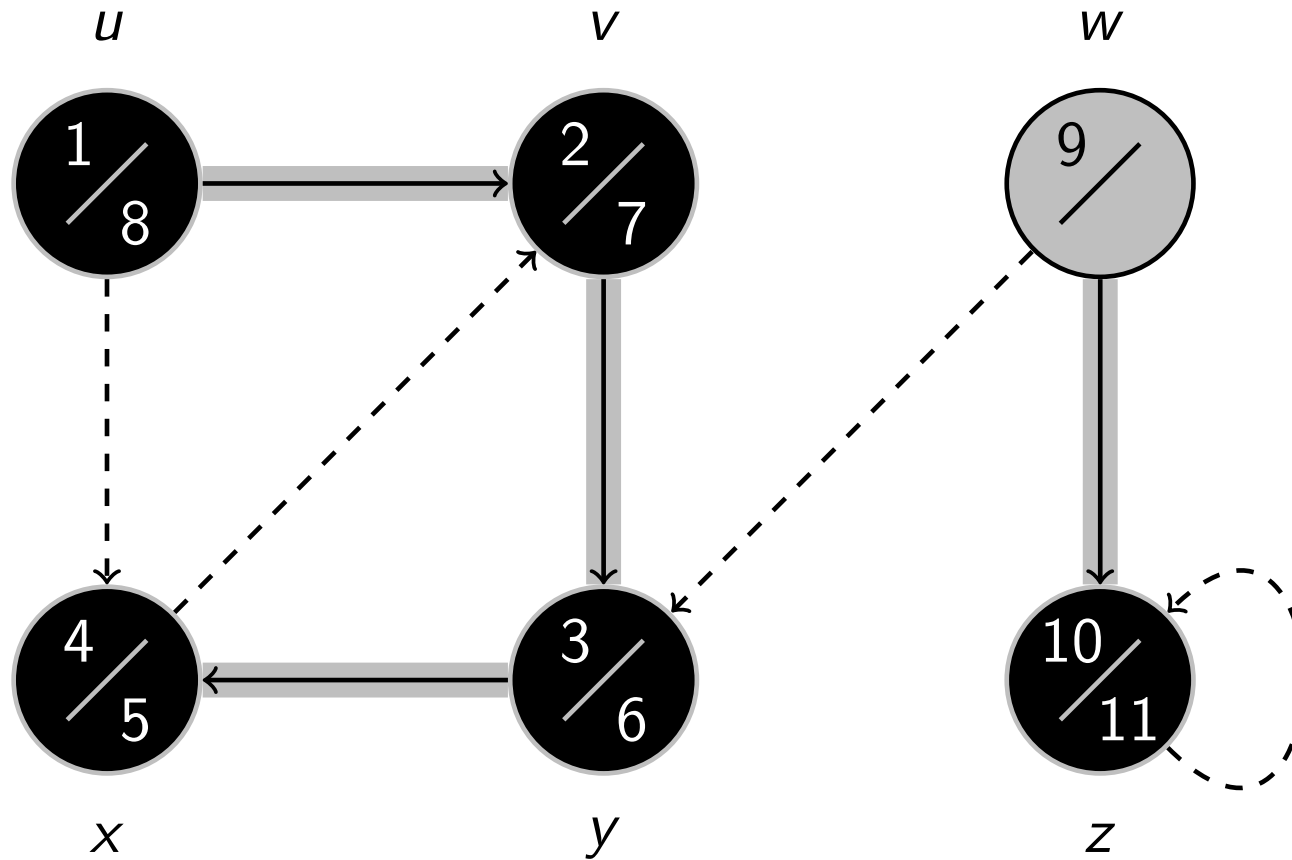
Example



Graphs I

22.3 Depth-first search

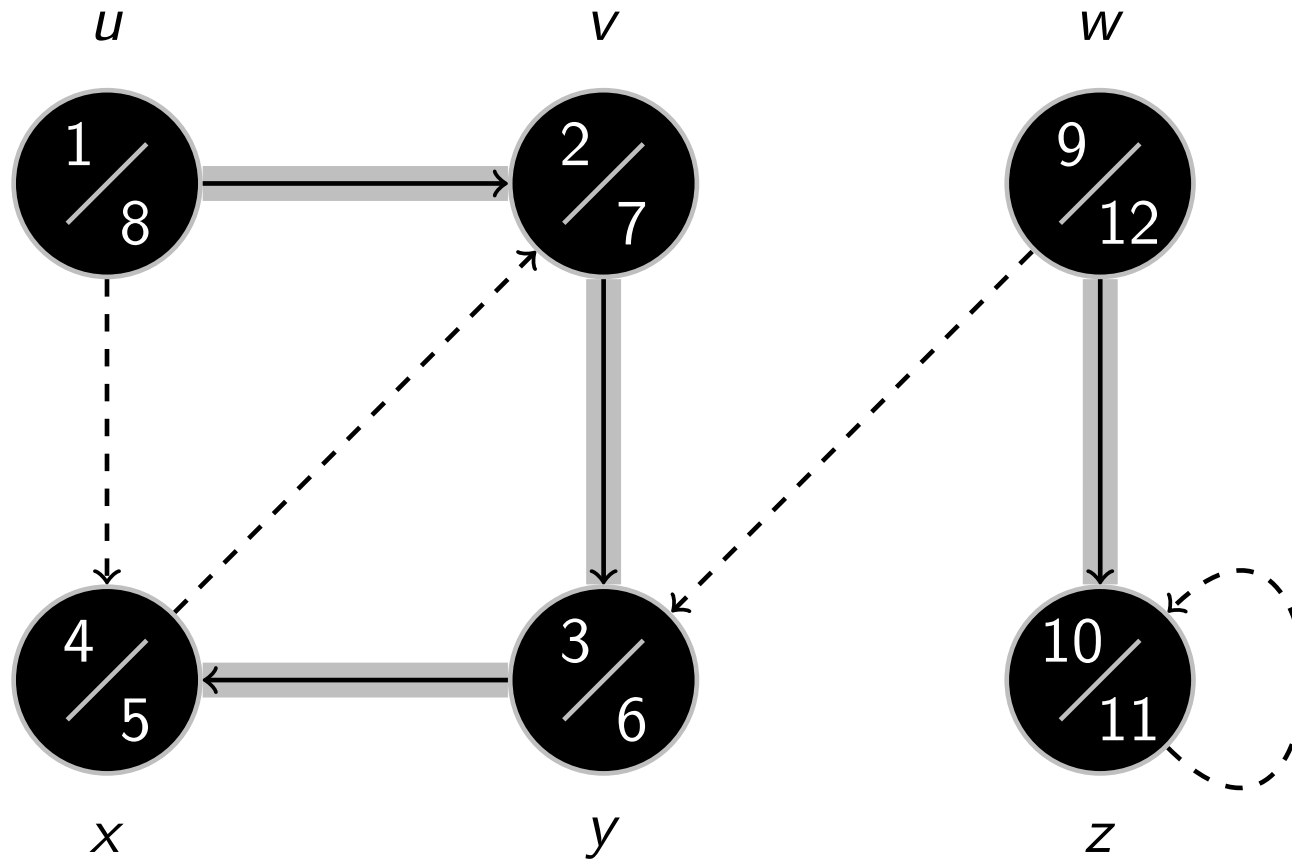
Example



Graphs I

22.3 Depth-first search

Example



Graphs I

22.3 Depth-first search

Theorem (Parenthesis theorem)

For any two vertices u and v , exactly one of the following holds:

- 1. The intervals $[u.d, u.f]$ and $[v.d, v.f]$ are disjoint and neither u nor v is a descendant of the other.*
- 2. $[u.d, u.f]$ is contained in $[v.d, v.f]$ and u is a descendant of v .*
- 3. $[v.d, v.f]$ is contained in $[u.d, u.f]$ and v is a descendant of u .*

Corollary (Nesting of descendant intervals)

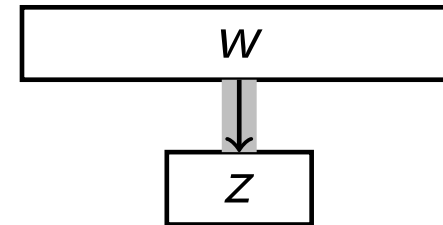
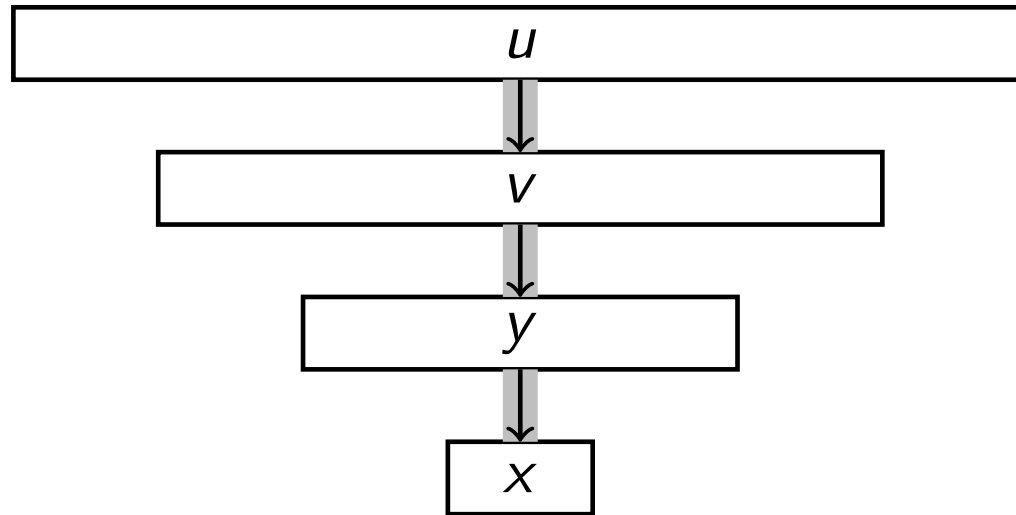
Vertex v is a proper descendant of vertex u if and only if $u.d < v.d < v.f < u.f$.

Theorem (White-path theorem)

Vertex v is a descendant of vertex u if and only if at time $u.d$, there is a path from u to v consisting entirely of white vertices.

Graphs I

22.3 Depth-first search



1	2	3	4	5	6	7	8	9	10	11	12
(	(	(	(	)	)	)	)	(	(	)	)

Graphs I

22.3 Depth-first search

Classification of edges

- Edge (u, v) is a **tree edge** if vertex v was first discovered by exploring edge (u, v) .
- Edge (u, v) is a **back edge** if vertex v is an ancestor of vertex u .
- Edge (u, v) is a **forward edge** if it is not a tree edge and vertex v is a descendant of vertex u .
- Edge (u, v) is a **cross edge** if it is not a tree edge, a back edge, or a forward edge.

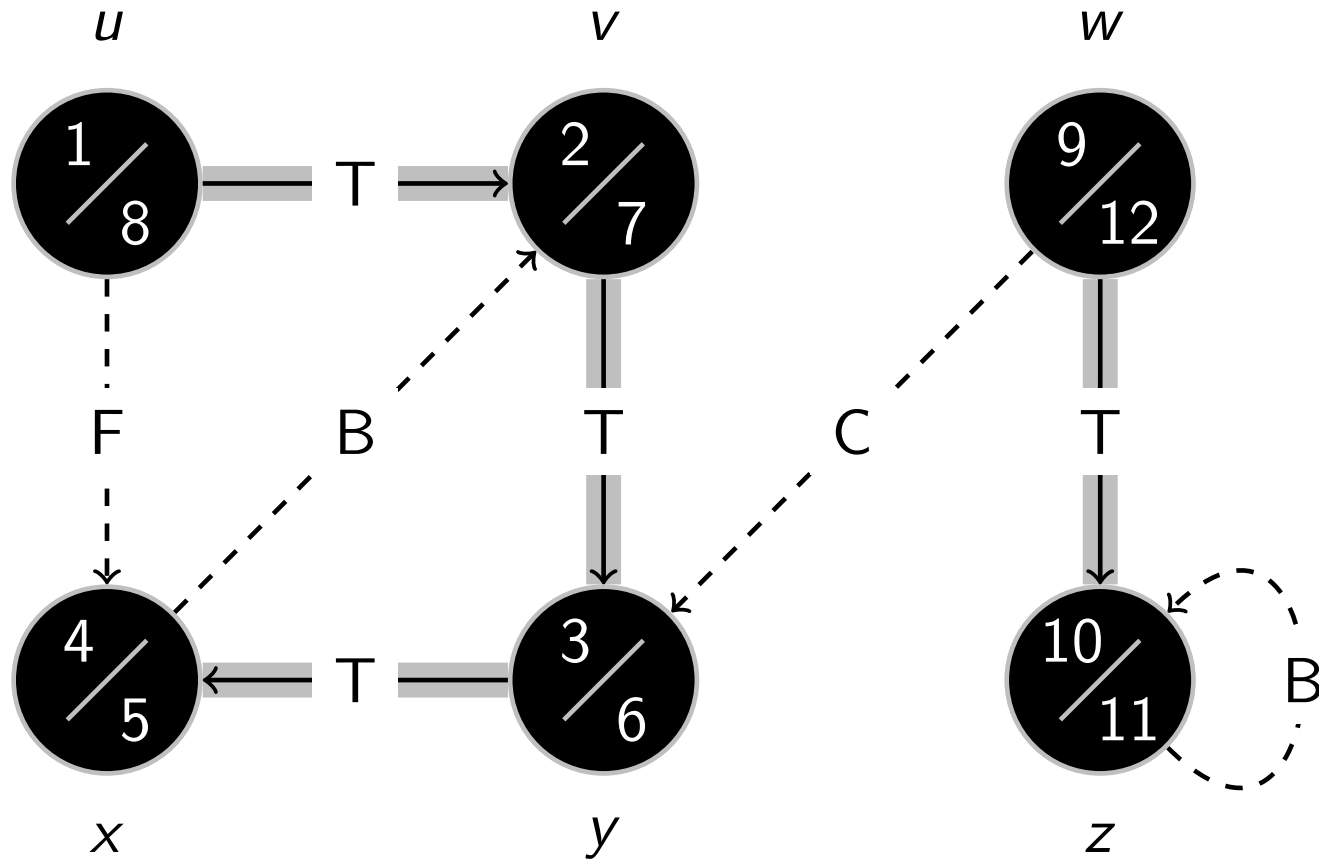
Theorem

In a depth-first search of an undirected graph, every edge is either a tree edge or a back edge.

Graphs I

22.3 Depth-first search

Example



Graphs I

22.3 Depth-first search

Exercise 22.3-7

Rewrite the procedure DFS, using a stack to eliminate recursion.

Solution to Exercise 22.3-7

```
DFS( $G$ )  
  for each vertex  $u \in G.V$  do  
     $u.color = \text{WHITE}$   
   $time = 0$   
  for each vertex  $u \in G.V$  do  
    if  $u.color = \text{WHITE}$  then  
      DFS-VISIT( $G, u$ )
```

Graphs I

22.3 Depth-first search

Solution to Exercise 22.3-7 (Cont'd)

DFS-VISIT(G, u)

$time = time + 1; u.d = time; u.color = \text{GRAY}$

$S = \emptyset$

PUSH(S, u)

while $S \neq \emptyset$ **do**

$u = \text{POP}(S)$

for each $v \in G.Adj[u]$ **do**

if $v.color = \text{WHITE}$ **then**

$time = time + 1; v.d = time; v.color = \text{GRAY}$

PUSH(S, v)

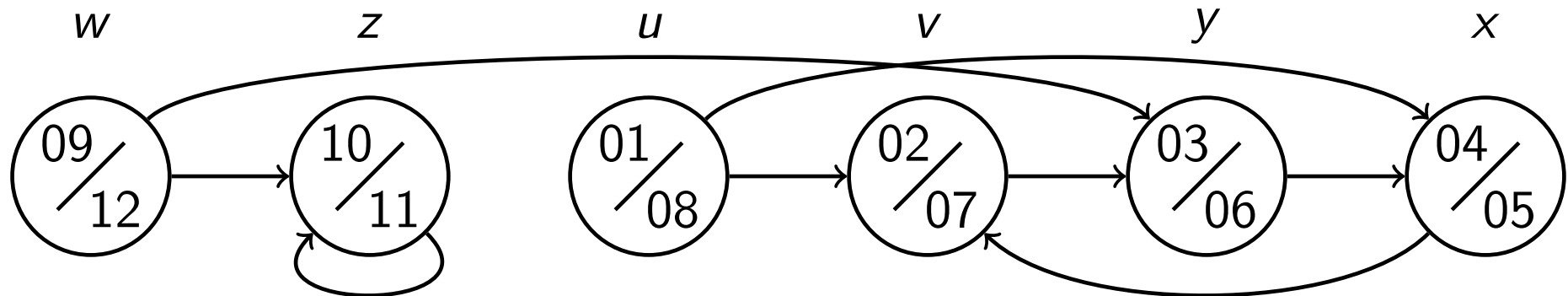
$time = time + 1; u.f = time; u.color = \text{BLACK}$

Graphs I

22.4 Topological sort

- A **directed acyclic graph** is a directed graph with no cycles.
- A **topological sort** of a directed acyclic graph $G = (V, E)$ is a linear ordering of V such that if $(u, v) \in E$, then u appears before v in the ordering.

Example



Graphs I

22.4 Topological sort

TOPOLOGICAL-SORT(G)

call DFS(G) to compute finishing times $v.f$ for all $v \in G.V$
output vertices in order of decreasing finishing times

Correctness

- It suffices to show that, if $(u, v) \in E$, then $v.f < u.f$.
- Consider any edge (u, v) being explored by DFS(G).
 - v cannot be gray, since then v would be an ancestor of u and (u, v) would be a back edge.
 - If v is white, v becomes a descendant of u and, by the parenthesis theorem, $v.f < u.f$.
 - If v is black, v has already been finished, and $v.f$ has already been set. We are still exploring from u and, when we assign a finishing time to u , we will have $v.f < u.f$.

Graphs I

22.4 Topological sort

Lemma

A directed graph G is acyclic if and only if a depth-first search of G yields no back edges.

Proof.

Suppose there is a back edge (u, v) . Then vertex v is an ancestor of vertex u . Thus, G contains a path from vertex v to vertex u , and the back edge (u, v) completes a cycle.

Conversely, suppose that G contains a cycle c . Let v be the first vertex discovered in c , and let (u, v) be the preceding edge in c . At time $v.d$, the vertices of c form a path of white vertices from v to u . By the white-path theorem, vertex u is a descendant of vertex v . Therefore, (u, v) is a back edge.

Graphs I

22.4 Topological sort

Exercise 22.4-5

Another way to perform topological sorting on a directed acyclic graph $G = (V, E)$ is to repeatedly find a vertex of in-degree 0, output it, and remove it and all of its outgoing edges from the graph. Explain how to implement this idea so that it runs in time $O(|V| + |E|)$. What happens to this algorithm if G has cycles?

Graphs I

22.4 Topological sort

Solution to Exercise 22.4-5

```
TOPOLOGICAL-SORT( $G$ )  
  for each vertex  $u \in G.V$  do  
     $u.indegree = 0$   
  
   $time = 0$   
  for each vertex  $u \in G.V$  do  
    for each  $v \in G.Adj[u]$  do  
       $v.indegree = v.indegree + 1$   
  
   $Q = \emptyset$   
  for each vertex  $u \in G.V$  do  
    if  $u.indegree = 0$  then  
      ENQUEUE( $Q, u$ )
```

Graphs I

22.4 Topological sort

Solution to Exercise 22.4-5 (Cont'd)

```
while  $Q \neq \emptyset$  do  
     $u = \text{DEQUEUE}(Q)$   
    output  $u$   
    for each  $v \in G.\text{Adj}[u]$  do  
         $v.\text{indegree} = v.\text{indegree} - 1$   
        if  $v.\text{indegree} = 0$  then  
             $\text{ENQUEUE}(Q, v)$   
for each vertex  $u \in G.V$  do  
    if  $u.\text{indegree} \neq 0$  then  
        report that there is a cycle
```

Lecture 9

Graphs II

Graphs II

Contents

- Breadth-first search CLRS 22.2
- Dijkstra's algorithm CLRS 24.3
- Growing a minimum spanning tree CLRS 23.1
- The algorithms of (Kruskal and) Prim CLRS 23.2

Reading

- T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein.
Introduction to Algorithms. The MIT Press, 3rd edition, 2009

Graphs II

22.2 Breadth-first search

Overview

- The strategy used by **breadth-first search** is to expand the frontier between discovered and undiscovered vertices uniformly across the breadth of the frontier.
- Breadth-first search computes the distance (smallest number of edges) $v.d$ from a distinguished **source** vertex s to every vertex v that is reachable from s .
- Breadth-first search can **color** vertices during the search to indicate their state.
- Breadth-first search can also compute the predecessor $v.\pi$ of each vertex v .
- The input graph G may be directed or undirected.

Graphs II

22.2 Breadth-first search

```
BFS( $G, s$ )  
  for each vertex  $u \in G.V - \{s\}$  do  
     $u.color = \text{WHITE}$   
     $u.d = \infty$   
   $s.color = \text{GRAY}$   
   $s.d = 0$   
   $Q = \emptyset$   
  ENQUEUE( $Q, s$ )  
  while  $Q \neq \emptyset$  do  
     $u = \text{DEQUEUE}(Q)$   
    for each  $v \in G.Adj[u]$  do  
      if  $v.color = \text{WHITE}$  then  
         $v.color = \text{GRAY}$   
         $v.d = u.d + 1$   
        ENQUEUE( $Q, v$ )  
     $u.color = \text{BLACK}$ 
```

Graphs II

22.2 Breadth-first search

- All vertices start out white and may later become gray and then black.
- Gray and black vertices have been discovered.
- Gray vertices may have some adjacent white vertices; they represent the frontier between discovered and undiscovered vertices.

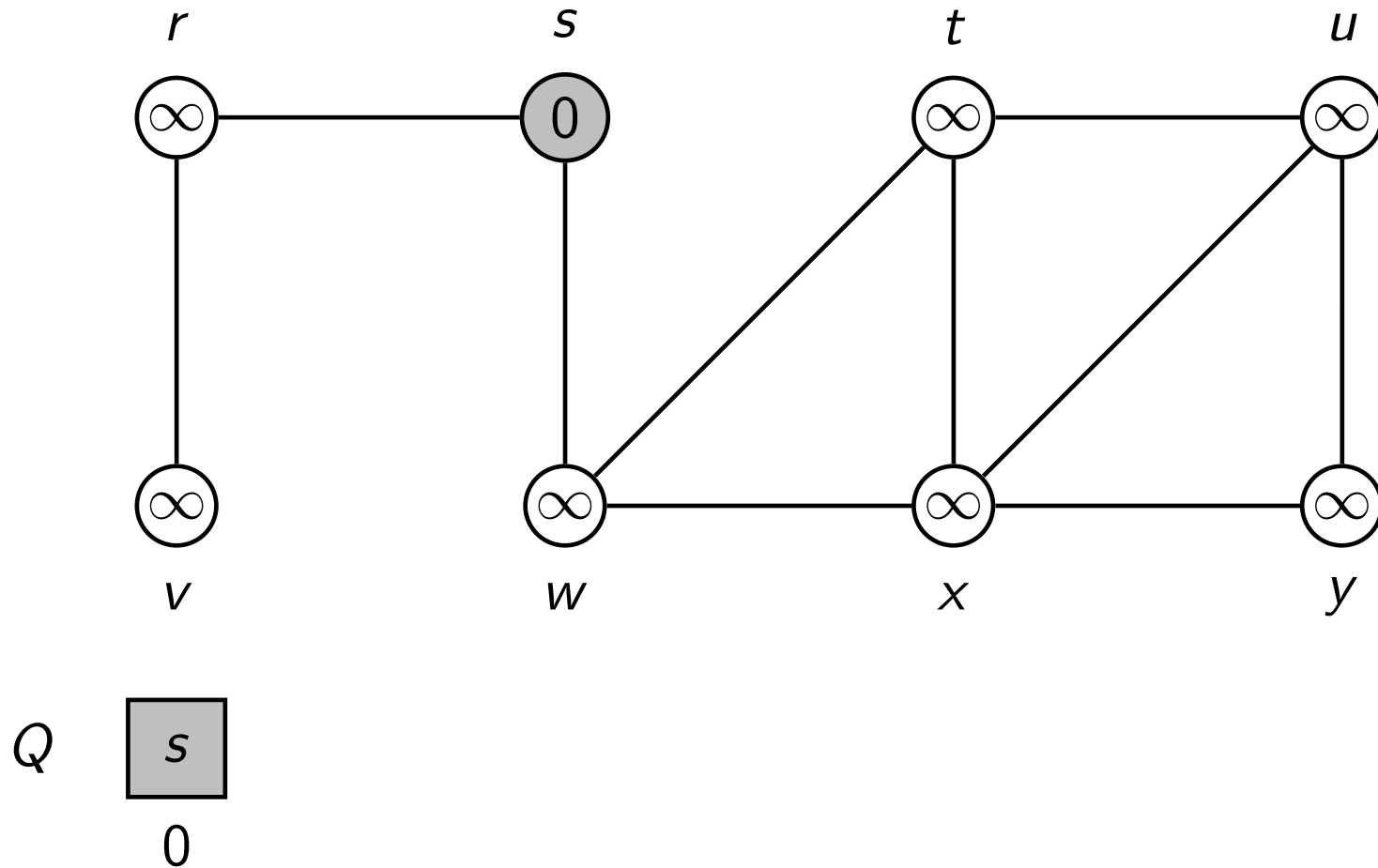
Analysis

- Breadth-first search takes $\Theta(|V| + |E|)$ time.
 - $O(|V|)$ because each vertex is enqueued at most once.
 - $O(|E|)$ because each vertex is dequeued at most once and each edge (u, v) is examined only once when u is dequeued.
- Breadth-first search takes time linear in the size of the adjacency-list representation of G .

Graphs II

22.2 Breadth-first search

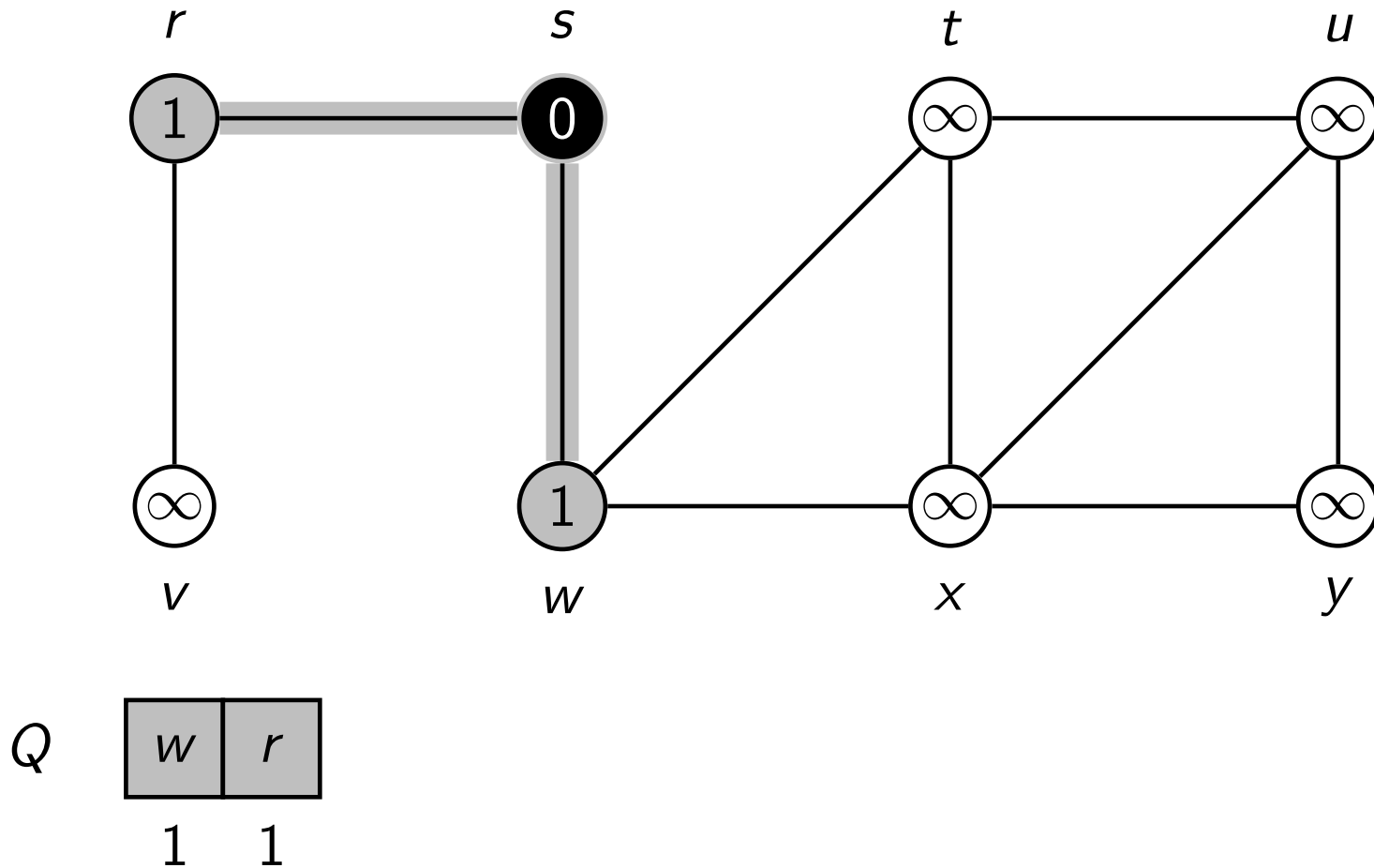
Example



Graphs II

22.2 Breadth-first search

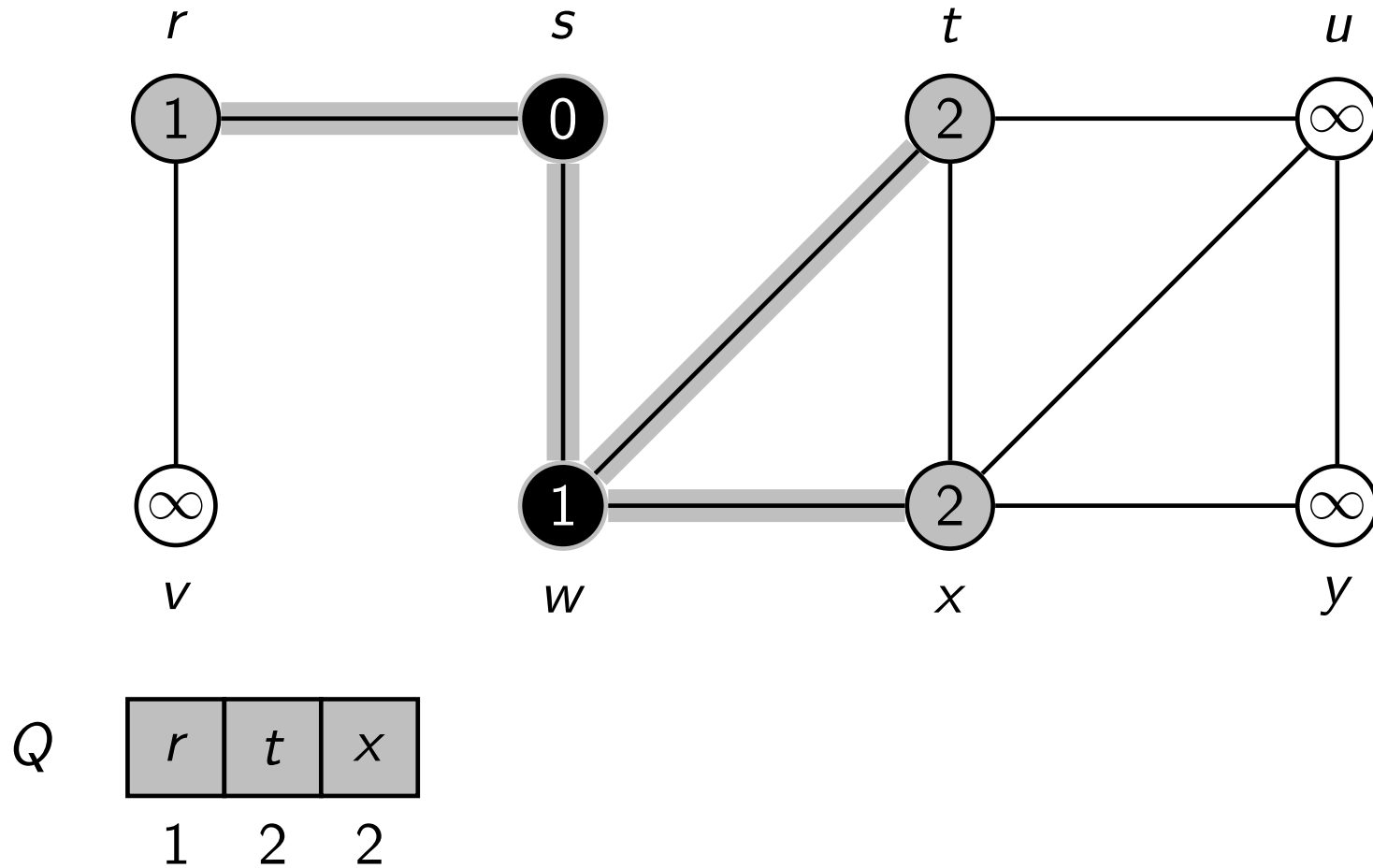
Example



Graphs II

22.2 Breadth-first search

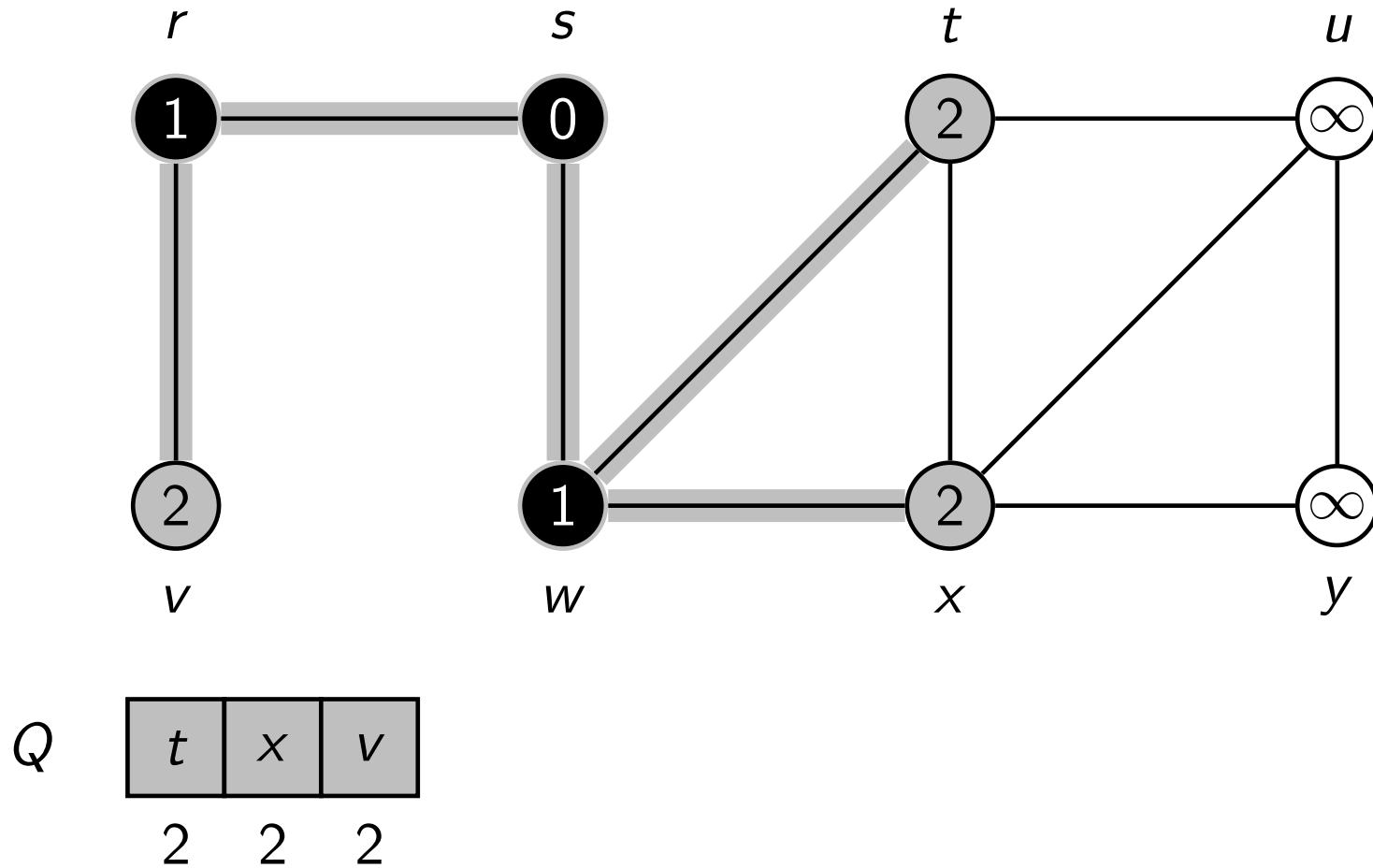
Example



Graphs II

22.2 Breadth-first search

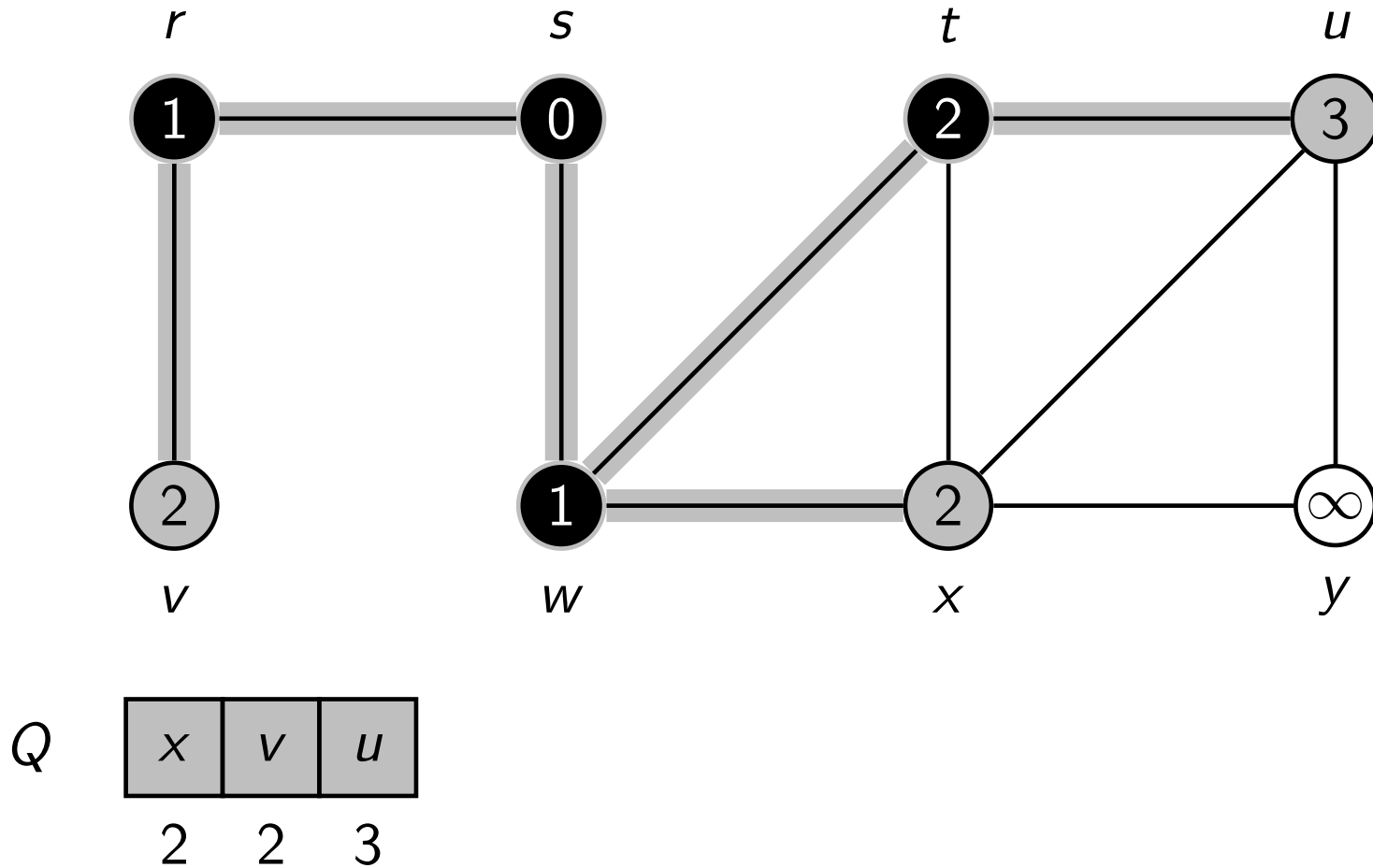
Example



Graphs II

22.2 Breadth-first search

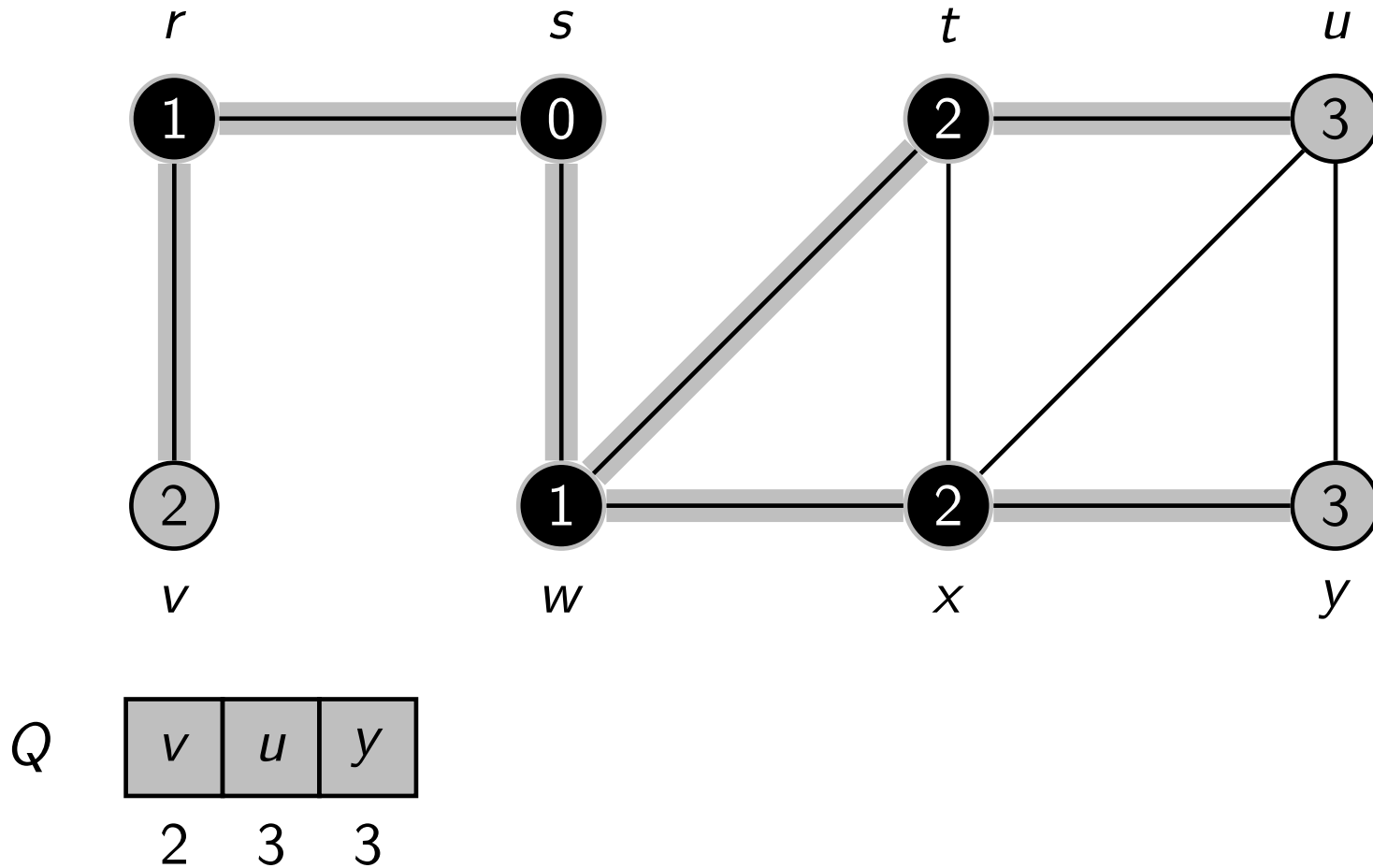
Example



Graphs II

22.2 Breadth-first search

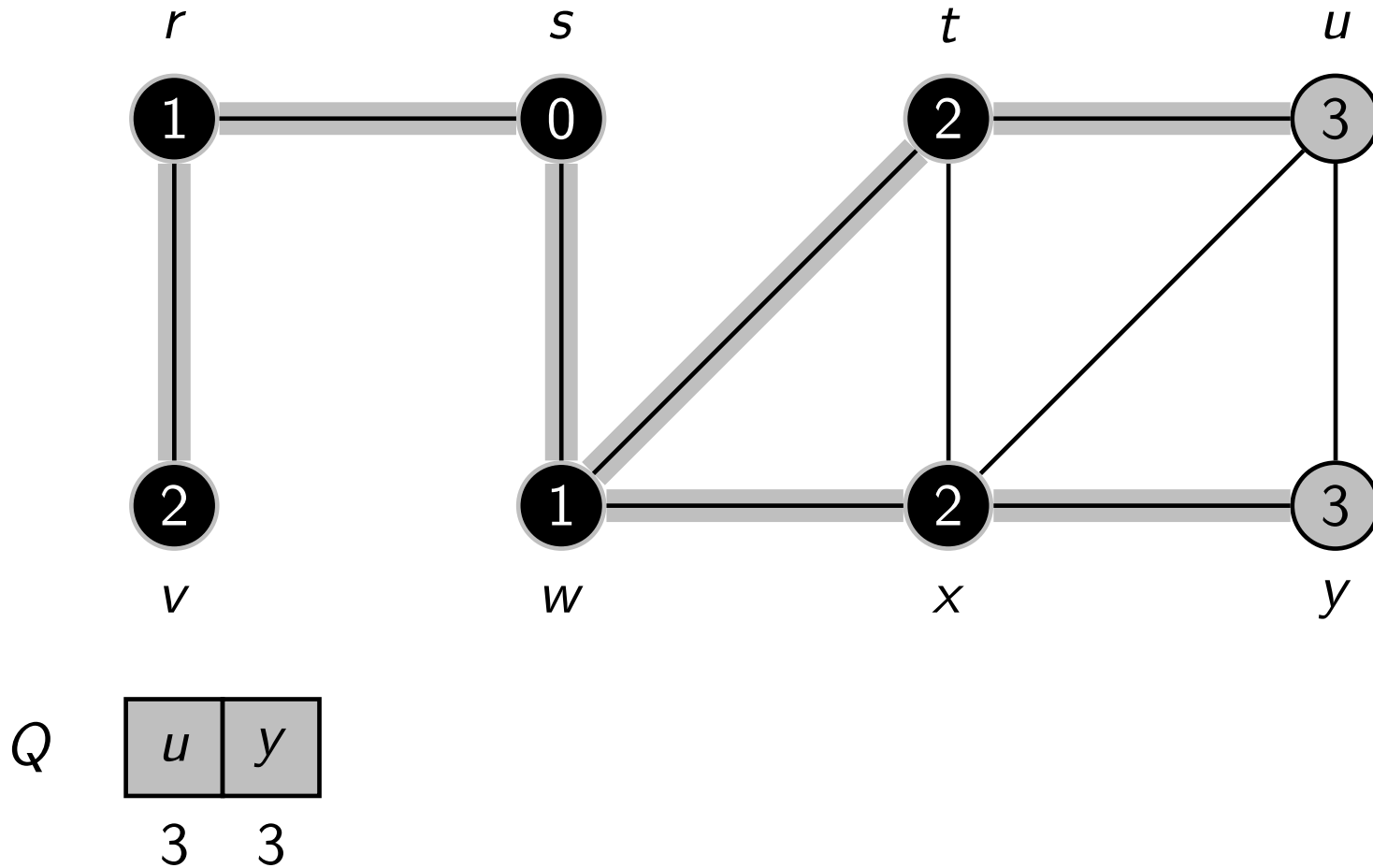
Example



Graphs II

22.2 Breadth-first search

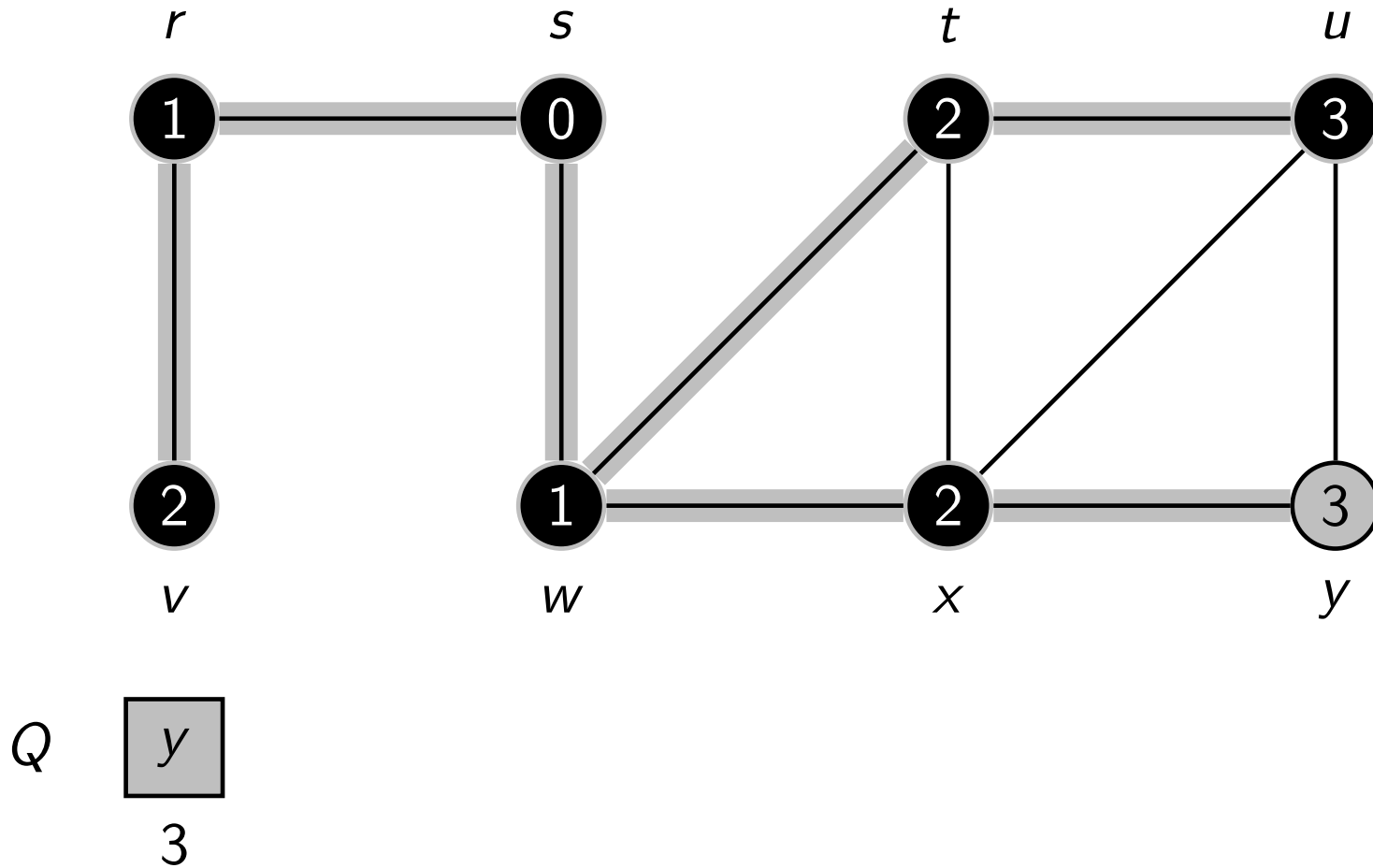
Example



Graphs II

22.2 Breadth-first search

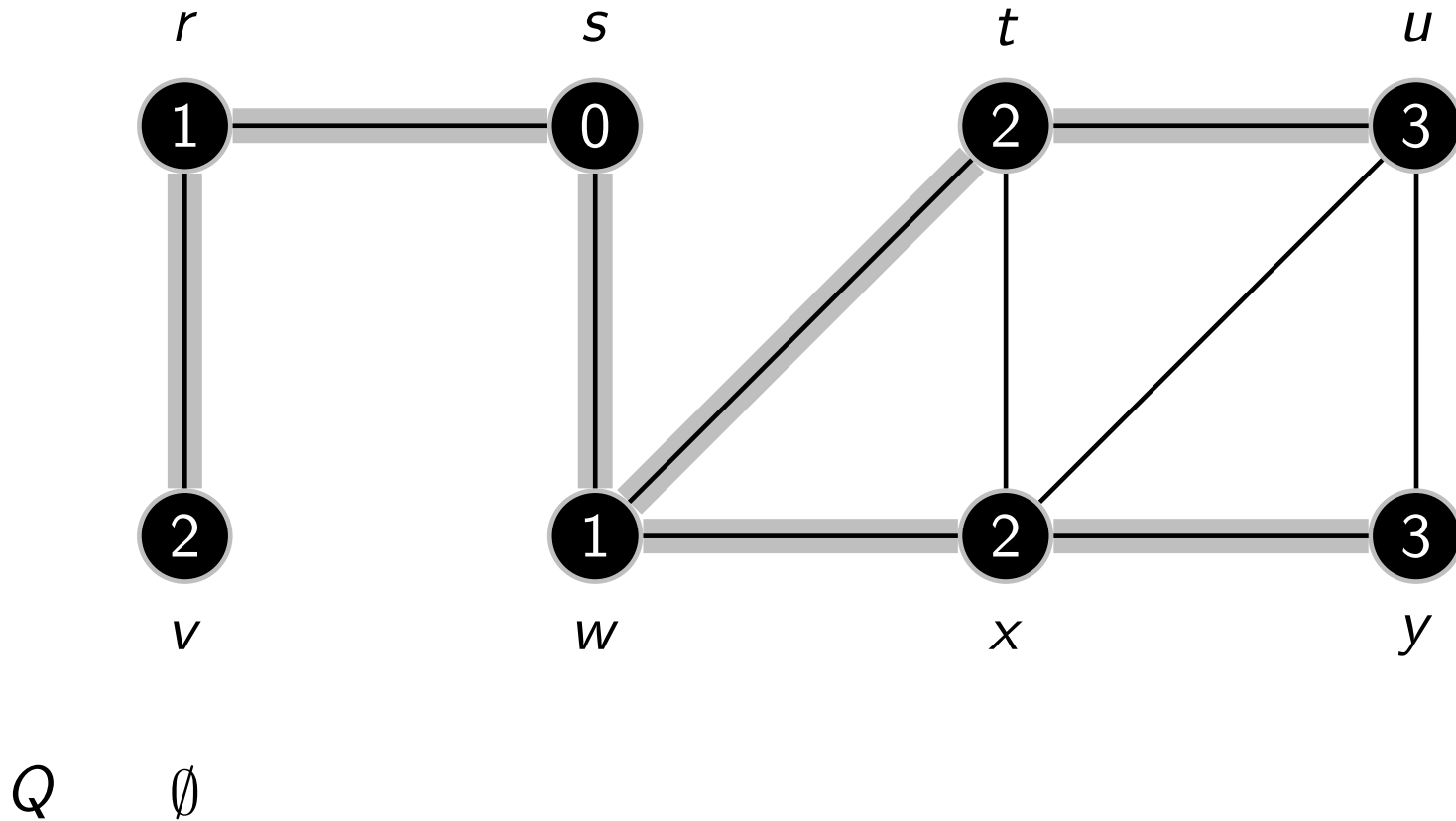
Example



Graphs II

22.2 Breadth-first search

Example



Graphs II

22.2 Breadth-first search

Exercise 22.2-4

What is the running time of BFS if we represent its input graph by an adjacency matrix and modify the algorithm to handle this form of input?

Solution to Exercise 22.2-4

Since the procedure scans the adjacencies of each vertex at most once, the total time spent in scanning adjacencies is $O(|V|^2)$ and the total running time of the BFS procedure is $O(|V| + |V|^2) = O(|V|^2)$. Thus, BFS runs in time linear in the size of the adjacency-matrix representation of G .

Graphs II

24.3 Dijkstra's algorithm

Overview

- Given a directed graph $G = (E, V)$ with weight function $w : E \rightarrow \mathbb{R}$, find a **shortest path** (a path of smallest total weight) from a given source vertex $s \in V$ to each vertex $v \in V$.
- BFS is a single-source shortest-paths algorithm that works on unweighted graphs (where each edge has unit weight).

24.3 Dijkstra's algorithm

The graph shows nodes 0, 5, 8, and 9 as highlighted. The edges and their labels are as follows:

- Node 0 (labeled s) has an incoming edge from node 8 (labeled 10) and an outgoing edge to node 5 (labeled 5).
- Node 5 (labeled y) has incoming edges from node 8 (labeled 2) and node 9 (labeled 9), and an outgoing edge to node 7 (labeled 2).
- Node 8 (labeled t) has an incoming edge from node 5 (labeled 3) and an outgoing edge to node 9 (labeled 1).
- Node 9 (labeled x) has incoming edges from node 7 (labeled 4) and node 6 (labeled 6), and an outgoing edge to node 0 (labeled 7).
- Node 7 (labeled z) has incoming edges from node 5 (labeled 2) and node 9 (labeled 4), and an outgoing edge to node 5 (labeled 7).

Thick gray lines highlight the edges (0, 5), (5, 8), (8, 9), and (9, 7).

Graphs II

24.3 Dijkstra's algorithm

Lemma

Any subpath of a shortest path is a shortest path.

Proof.

- Suppose $p = u \rightsquigarrow x \rightsquigarrow y \rightsquigarrow v$ is a shortest path from u to v .
- Then $\delta(u, v) = w(p) = w(u \rightsquigarrow x) + w(x \rightsquigarrow y) + w(y \rightsquigarrow v)$.
- Now suppose there exists a shorter path $x \rightsquigarrow y$.
- Then $w(x \rightsquigarrow y) < w(x \rightsquigarrow y)$.
- Construct $p' = u \rightsquigarrow x \rightsquigarrow y \rightsquigarrow v$.
- Then
$$\begin{aligned} w(p') &= w(u \rightsquigarrow x) + w(x \rightsquigarrow y) + w(y \rightsquigarrow v) \\ &< w(u \rightsquigarrow x) + w(x \rightsquigarrow y) + w(y \rightsquigarrow v) \\ &= w(p) \end{aligned}$$
- Contradicts the assumption that p is a shortest path.

Graphs II

24.3 Dijkstra's algorithm

Negative-weight edges

- If the graph $G = (V, E)$ contains no negative-weight cycles reachable from the source $s \in V$, then for all $v \in V$, the shortest-path weight $\delta(s, v)$ remains well defined, even if it has a negative value.
- If the graph contains a negative-weight cycle reachable from the source $s \in V$, shortest-path weights are not well-defined.
 - No path from s to a vertex on the cycle can be a shortest path.
 - We can always find a path with lower weight by following the proposed “shortest” path and then traversing the negative-weight cycle.
 - If there is a negative-weight cycle on some path from s to v , we define $\delta(s, v) = -\infty$.

Graphs II

24.3 Dijkstra's algorithm

Cycles

- Shortest paths cannot contain cycles.
 - Cannot contain negative-weight cycles.
Shortest-path weights are not well-defined.
 - Cannot contain positive-weight cycles.
We can get a shorter path by omitting the cycle.
 - Can contain zero-weight cycles.
We can still get a shortest path by omitting the cycle.
- Assume that shortest paths are **simple** (they have no cycles).
- Any acyclic path in a graph $G = (V, E)$ contains at most $|V|$ distinct vertices and at most $|V| - 1$ edges.

Graphs II

24.3 Dijkstra's algorithm

Representing shortest paths

- Maintain for each vertex $v \in V$ a **predecessor** $v.\pi$ that is either another vertex or NIL.
- The chain of predecessors originating at a vertex v runs backwards along a shortest path from s to v .

```
PRINT-PATH( $G, s, v$ )  
  if  $v = s$  then  
    print  $s$   
  else if  $v.\pi = \text{NIL}$  then  
    print “no path from”  $s$  “to”  $v$  “exists”  
  else  
    PRINT-PATH( $G, s, v.\pi$ )
```


Graphs II

24.3 Dijkstra's algorithm

Relaxation

- For each vertex $v \in V$, the **shortest-path estimate** $v.d$ is an upper bound on the weight of a shortest path from source s to v .

```
INITIALIZE-SINGLE-SOURCE( $G, s$ )
```

```
  for each vertex  $v \in G.V$  do
```

```
     $v.d = \infty$ 
```

```
     $v.\pi = \text{NIL}$ 
```

```
   $s.d = 0$ 
```

- After initialization, we have $v.\pi = \text{NIL}$ for all $v \in V$, $s.d = 0$, and $v.d = \infty$ for all $v \in V - \{s\}$.

Graphs II

24.3 Dijkstra's algorithm

Relaxation

- The process of **relaxing** an edge (u, v) consists of testing whether we can improve the shortest path estimate $v.d$ by going through u and taking (u, v) .

```
RELAX( $u, v, w$ )  
  if  $v.d > u.d + w(u, v)$  then  
     $v.d = u.d + w(u, v)$   
     $v.\pi = u$ 
```

- After initialization, shortest-path algorithms repeatedly relax edges.

Graphs II

24.3 Dijkstra's algorithm

Properties of shortest paths and relaxation

Lemma (Triangle inequality)

For any edge $(u, v) \in E$, we have $\delta(s, v) \leq \delta(s, u) + w(u, v)$.

Lemma (Upper-bound property)

We always have $v.d \geq \delta(s, v)$ for all vertices $v \in V$, and once $v.d = \delta(s, v)$, it never changes.

Lemma (No-path property)

If $\delta(s, v) = \infty$, then we always have $v.d = \infty$.

Graphs II

24.3 Dijkstra's algorithm

Properties of shortest paths and relaxation

Lemma (Convergence property)

If $s \rightsquigarrow u \rightarrow v$ is a shortest path, and if $u.d = \delta(s, u)$ before relaxing (u, v) , then $v.d = \delta(s, v)$ at all times afterward.

Lemma (Path-relaxation property)

If we relax the edges of a shortest path $s = v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k$ in the order $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$, even intermixed with other relaxations, then $v_k.d = \delta(s, v_k)$.

Graphs II

24.3 Dijkstra's algorithm

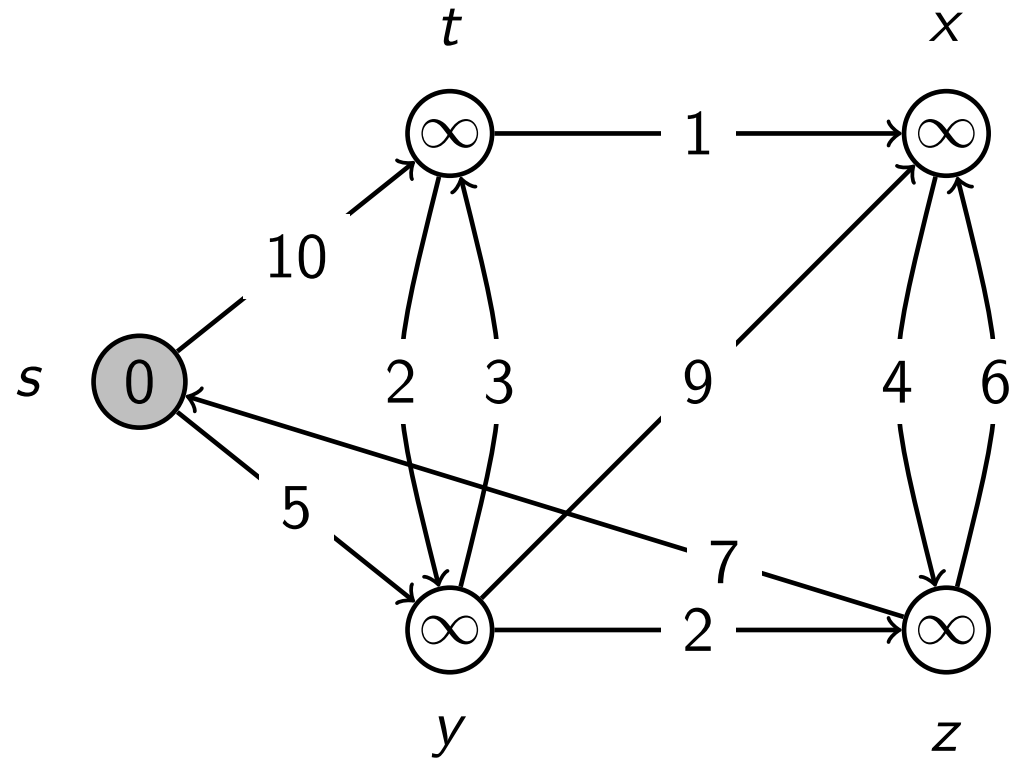
- Assume all edge weight are non-negative.
- Maintain a set S of vertices whose final shortest-path weights are determined.
- Maintain a set $V - S$ of vertices in a min-priority queue Q .

```
DIJKSTRA( $G, w, s$ )  
  INITIALIZE-SINGLE-SOURCE( $G, s$ )  
   $S = \emptyset$   
   $Q = G.V$   
  while  $Q \neq \emptyset$  do  
     $u = \text{EXTRACT-MIN}(Q)$   
     $S = S \cup \{u\}$   
    for each  $v \in G.Adj[u]$  do  
      RELAX( $u, v, w$ )
```

Graphs II

24.3 Dijkstra's algorithm

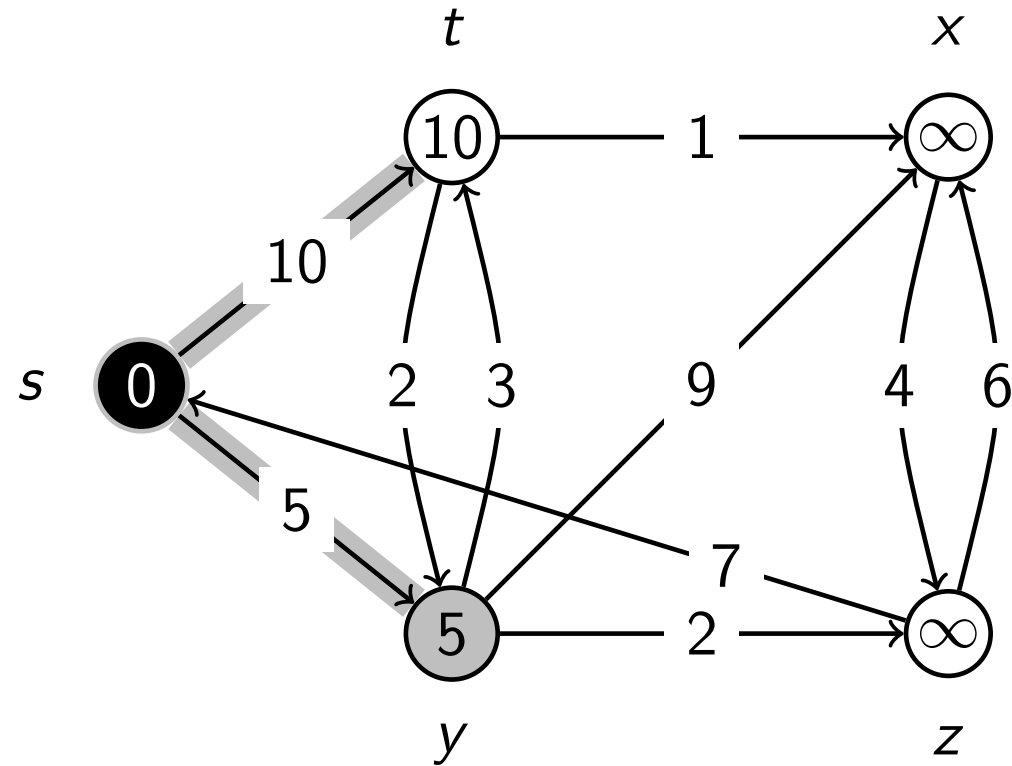
Example



Graphs II

24.3 Dijkstra's algorithm

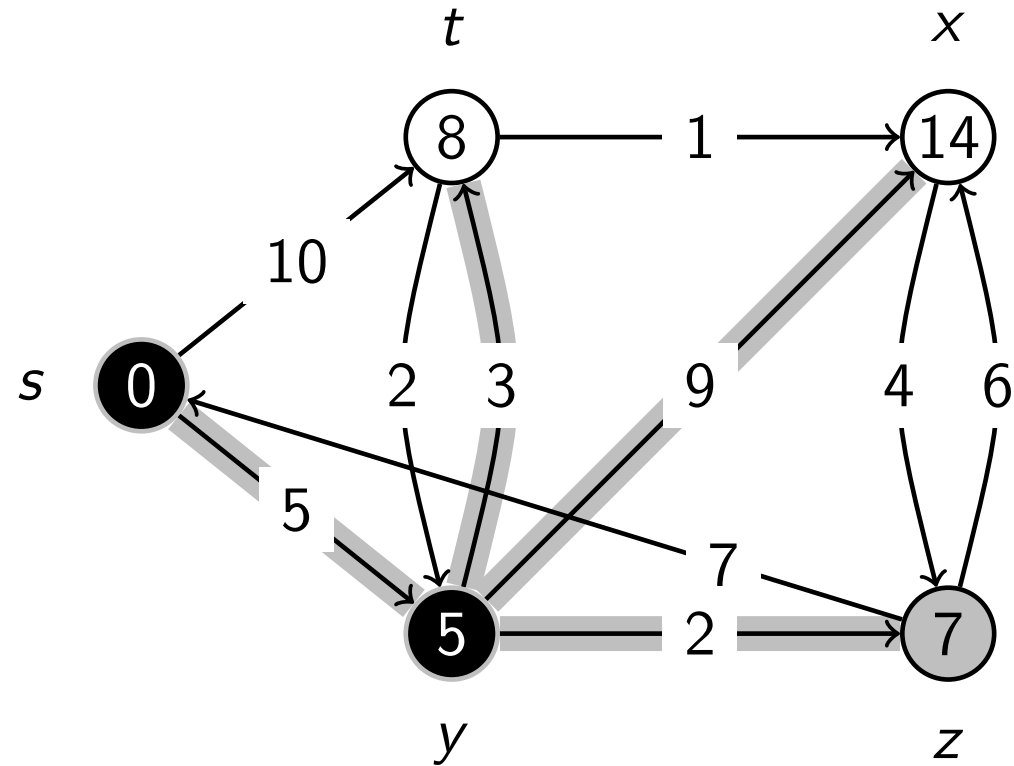
Example



Graphs II

24.3 Dijkstra's algorithm

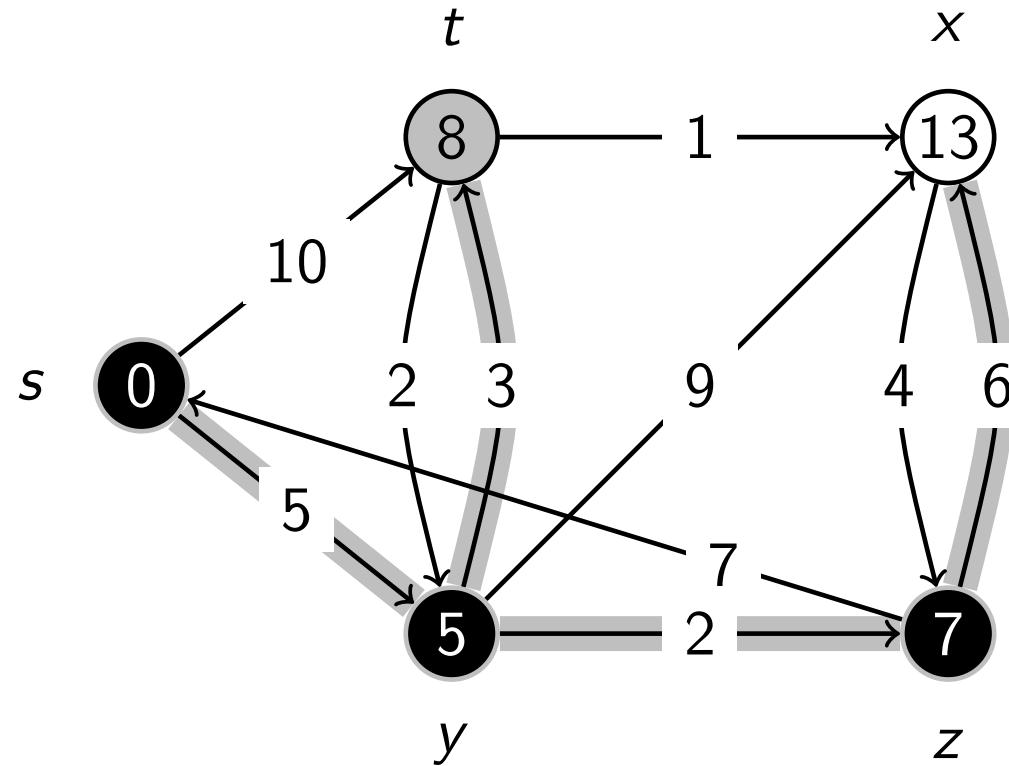
Example



Graphs II

24.3 Dijkstra's algorithm

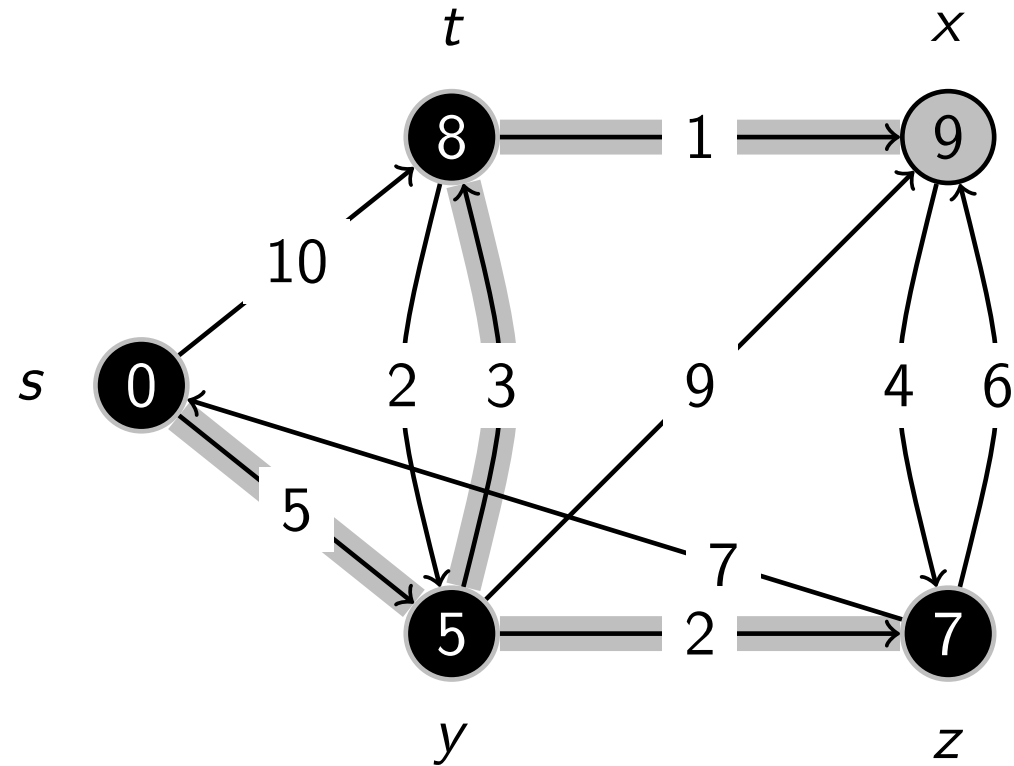
Example



Graphs II

24.3 Dijkstra's algorithm

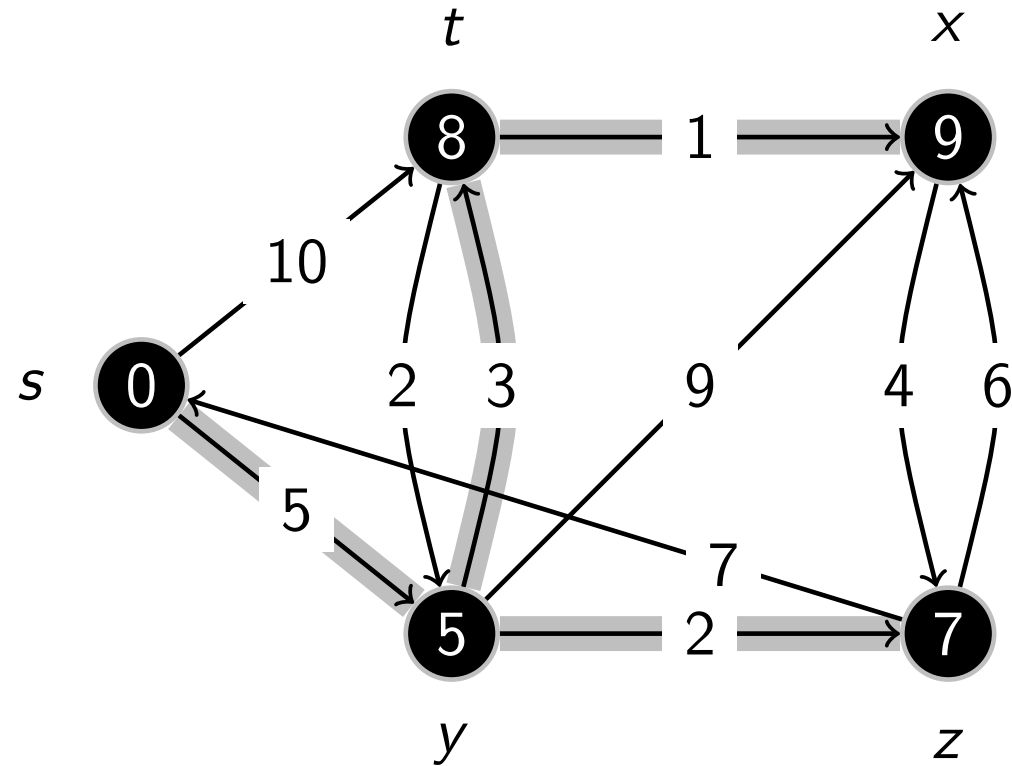
Example



Graphs II

24.3 Dijkstra's algorithm

Example



Graphs II

24.3 Dijkstra's algorithm

Correctness

- Dijkstra's algorithm always chooses the “lightest” or “closest” vertex in $V - S$ to add to set S .
- Each time it adds a vertex u to set S , we have $u.d = \delta(s, u)$.

Theorem (CLRS Theorem 24.6)

Dijkstra's algorithm, run on a directed graph $G = (V, E)$ with non-negative weight function $w : E \rightarrow \mathbb{R}$ and source vertex $s \in V$, terminates with $u.d = \delta(s, u)$ for all vertices $u \in V$.

Graphs II

24.3 Dijkstra's algorithm

Analysis

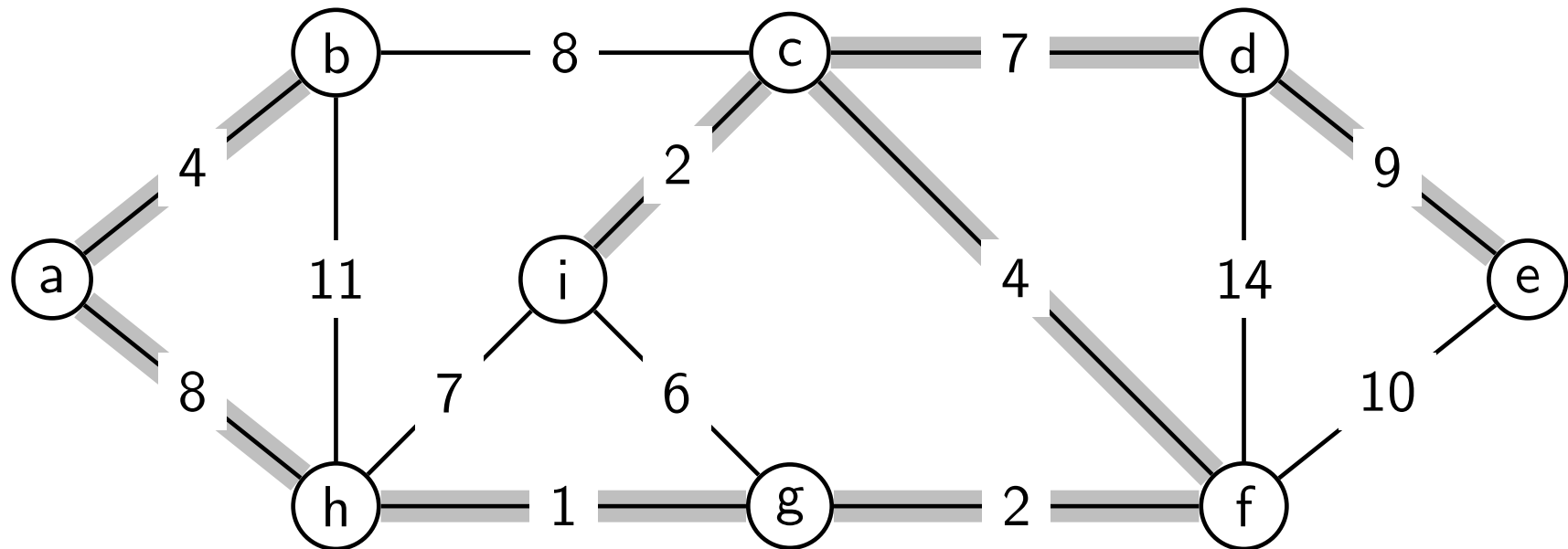
- Dijkstra's algorithm takes $O((|V| + |E|) \log |V|)$ time, by implementing the min-priority queue with a min-heap.
 - $O(|V|)$ time to build the min-heap.
 - $|V|$ min-heap EXTRACT-MIN operations, each taking $O(\log |V|)$ time.
 - At most $|E|$ min-heap DECREASE-KEY operations, each taking $O(\log |V|)$ time.

Graphs II

23.1 Growing a minimum spanning tree

- Given a connected, undirected graph $G = (E, V)$ with weight function $w : E \rightarrow \mathbb{R}$, find a **minimum spanning tree** (a spanning tree of smallest total weight) for G .

Example

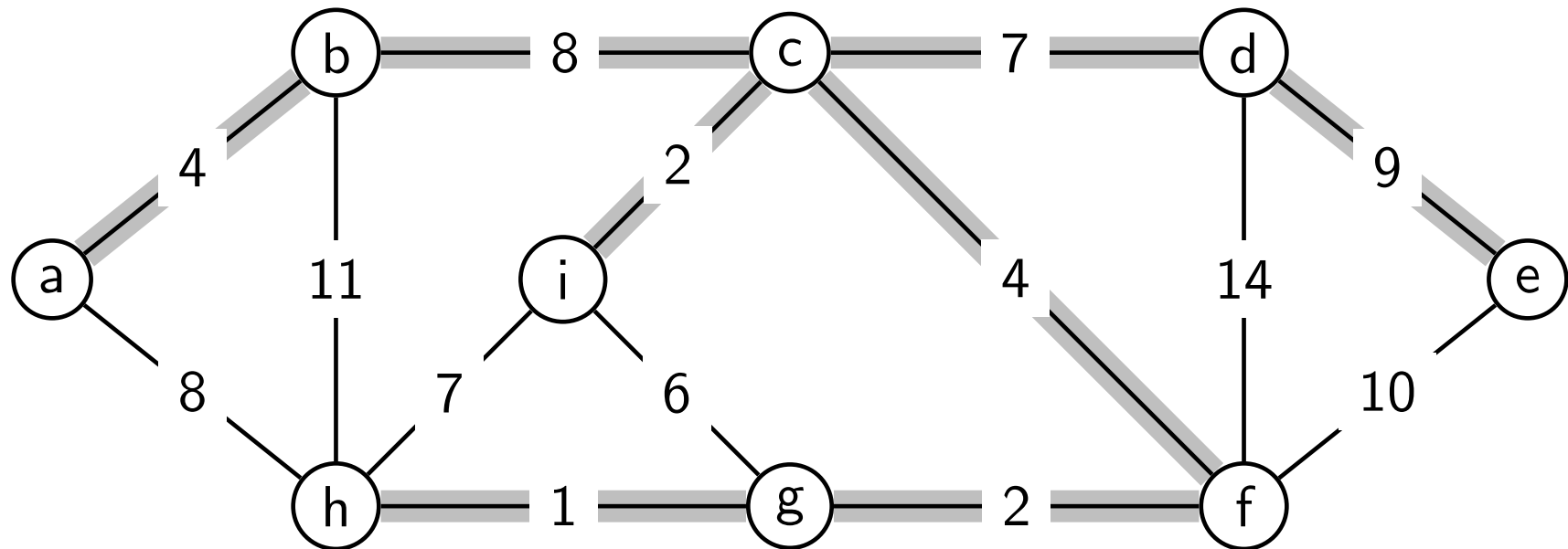


Graphs II

23.1 Growing a minimum spanning tree

- Given a connected, undirected graph $G = (E, V)$ with weight function $w : E \rightarrow \mathbb{R}$, find a **minimum spanning tree** (a spanning tree of smallest total weight) for G .

Example



Graphs II

23.1 Growing a minimum spanning tree

- Maintain the loop invariant that, prior to each iteration, A is a subset of some minimum spanning tree.
- An edge (u, v) is **safe** for A if $A \cup \{(u, v)\}$ is also a subset of a minimum spanning tree.

GENERIC-MST(G, w)

$A = \emptyset$

while A does not form a spanning tree **do**

 find an edge (u, v) that is safe for A

$A = A \cup \{(u, v)\}$

return A

Graphs II

23.1 Growing a minimum spanning tree

Correctness

- A **cut** $(S, V - S)$ of an undirected graph $G = (V, E)$ is a partition of V .
- An edge $(u, v) \in E$ **crosses** the cut $(S, V - S)$ if one endpoint is in S and the other is in $V - S$.
- A cut **respects** a set A of edges if no edge in A crosses the cut.
- An edge is a **light edge** crossing a cut if its weight is the minimum of any edge crossing the cut.

Theorem (CLRS Theorem 23.1)

Let A be a subset of some minimum spanning tree for $G = (V, E)$, let $(S, V - S)$ be a cut of G that respects A , and let (u, v) be a light edge crossing $(S, V - S)$. Then, edge (u, v) is safe for A .

Graphs II

23.2 The algorithms of (Kruskal and) Prim

Prim's algorithm

- Special case of the generic minimum-spanning-tree method.
- Much like Dijkstra's algorithm.
- The edges in the set A always form a tree, that starts from an arbitrary root vertex r and grows until the tree A spans all the vertices in V .
- Each step adds to the tree A a light edge that connects A to an isolated vertex, and the edge is safe for A .
- When the algorithm terminates, the edges in A form a minimum spanning tree.
- Need a fast way to find a light edge to add to the tree formed by the edges in A .

Graphs II

23.2 The algorithms of (Kruskal and) Prim

Prim's algorithm

- Use a min-priority queue Q to keep all vertices that are not in the tree.
- For each vertex $v \in V$, the attribute $v.key$ is the minimum weight of any edge connecting v to a vertex not in the tree.
- Set $v.key = \infty$ if there is no such edge.

Graphs II

23.2 The algorithms of (Kruskal and) Prim

Prim's algorithm

```
MST-PRIM( $G, w, r$ )  
  for each vertex  $u \in G.V$  do  
     $u.key = \infty$   
     $u.\pi = \text{NIL}$   
   $r.key = 0$   
   $Q = G.V$   
  while  $Q \neq \emptyset$  do  
     $u = \text{EXTRACT-MIN}(Q)$   
    for each  $v \in G.Adj[u]$  do  
      if  $v \in Q$  and  $w(u, v) < v.key$  then  
         $v.\pi = u$   
         $v.key = w(u, v)$ 
```

Graphs II

23.2 The algorithms of (Kruskal and) Prim

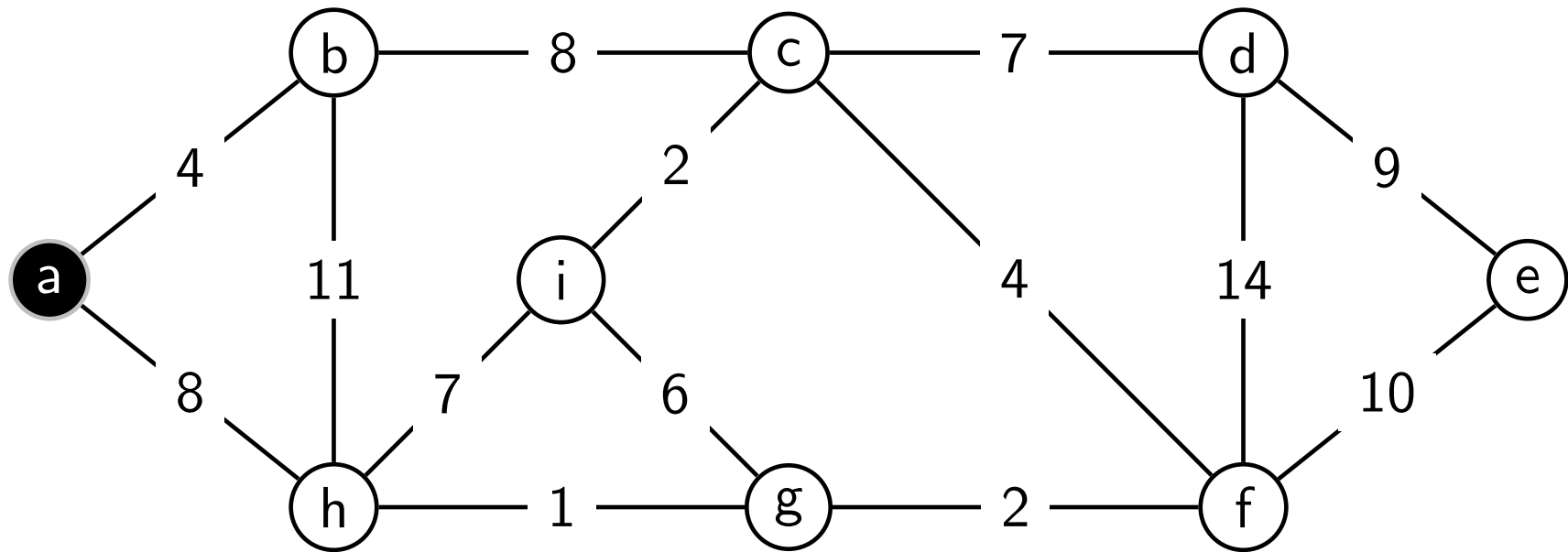
Prim's algorithm

```
MST-PRIM( $G, w, r$ )  
  for each vertex  $u \in G.V$  do  
     $u.key = \infty$   
     $u.\pi = \text{NIL}$   
    INSERT( $Q, u$ )  
  DECREASE-KEY( $Q, r, 0$ )  
  while  $Q \neq \emptyset$  do  
     $u = \text{EXTRACT-MIN}(Q)$   
    for each  $v \in G.Adj[u]$  do  
      if  $v \in Q$  and  $w(u, v) < v.key$  then  
         $v.\pi = u$   
        DECREASE-KEY( $Q, v, w(u, v)$ )
```

Graphs II

23.2 The algorithms of (Kruskal and) Prim

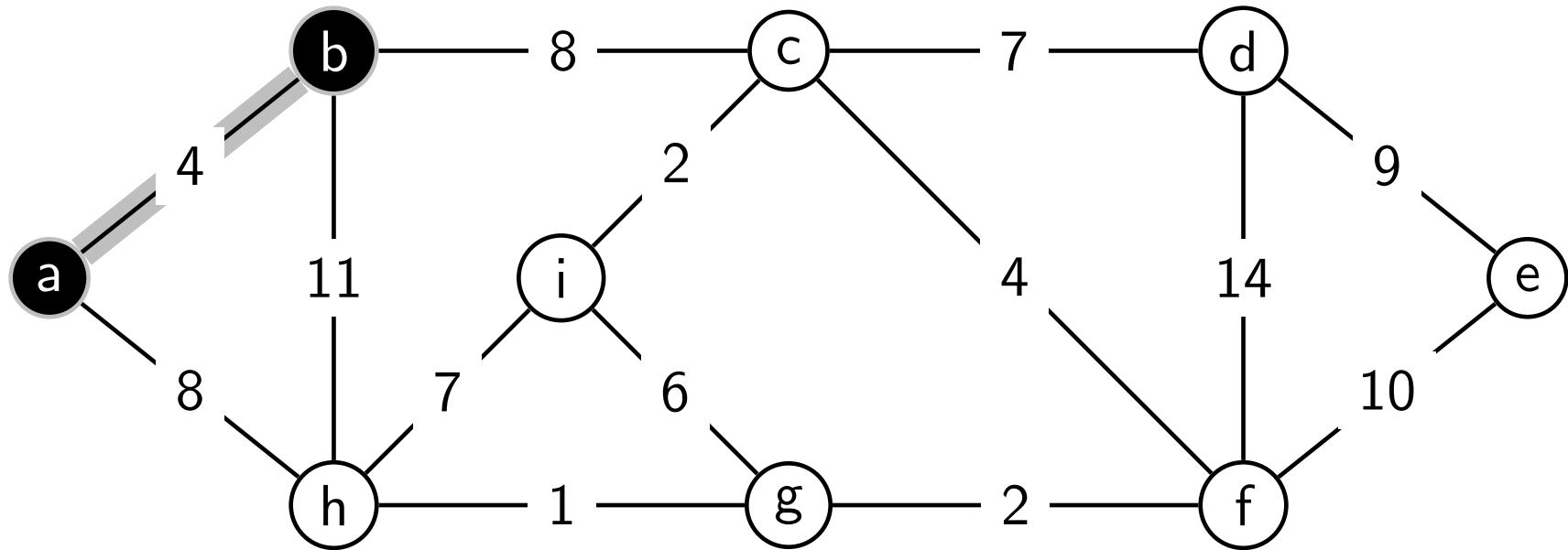
Example



Graphs II

23.2 The algorithms of (Kruskal and) Prim

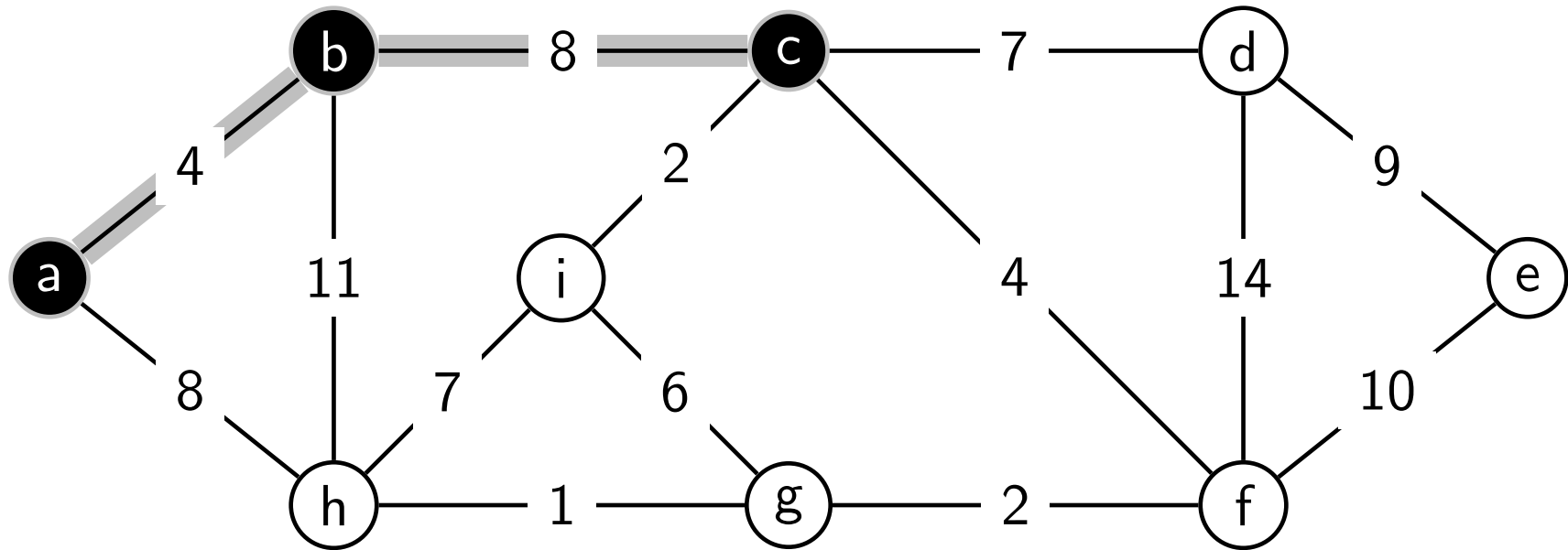
Example



Graphs II

23.2 The algorithms of (Kruskal and) Prim

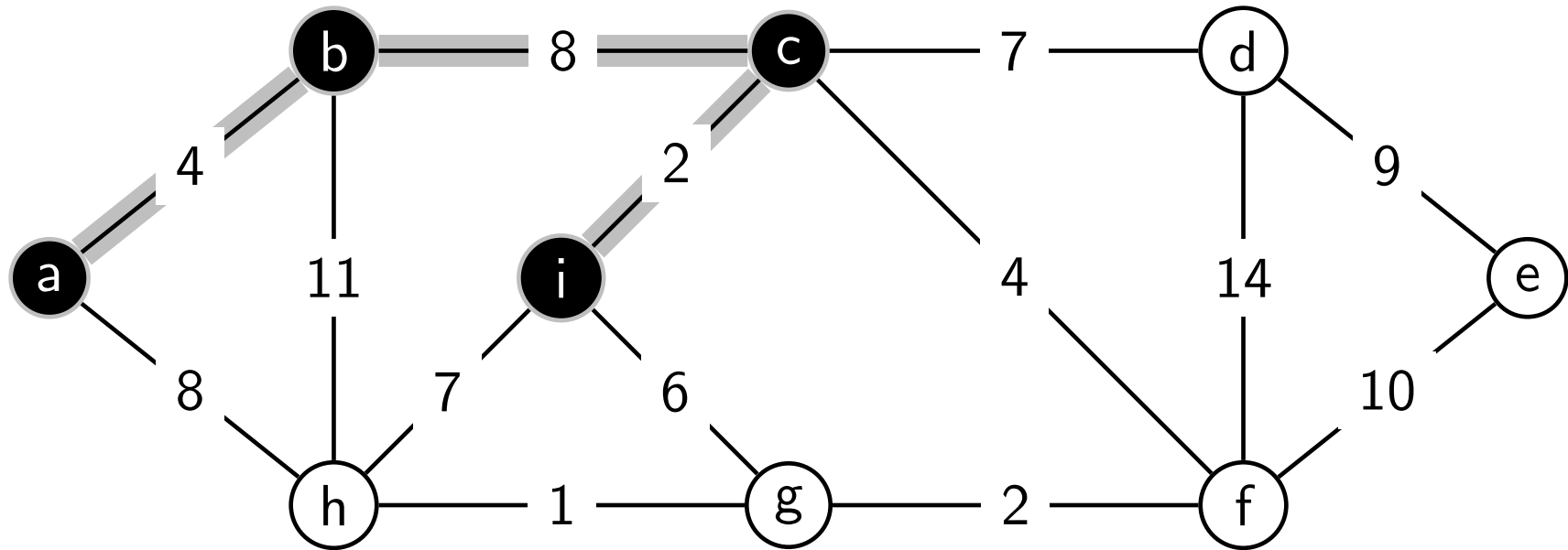
Example



Graphs II

23.2 The algorithms of (Kruskal and) Prim

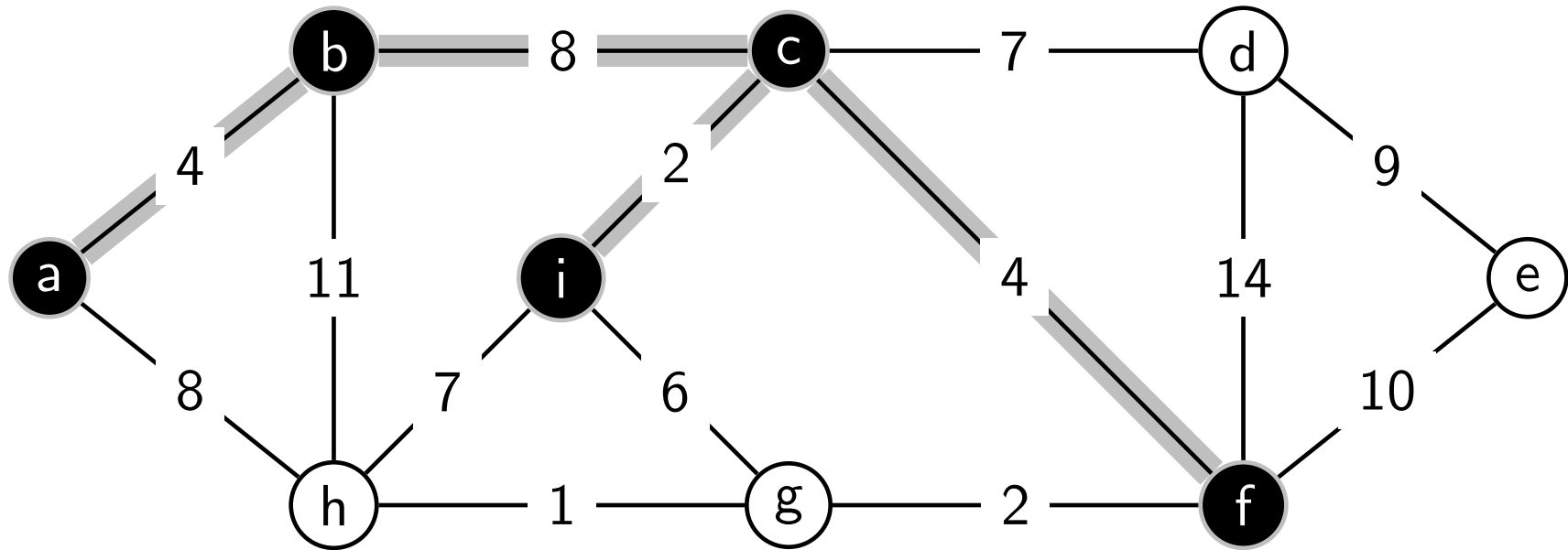
Example



Graphs II

23.2 The algorithms of (Kruskal and) Prim

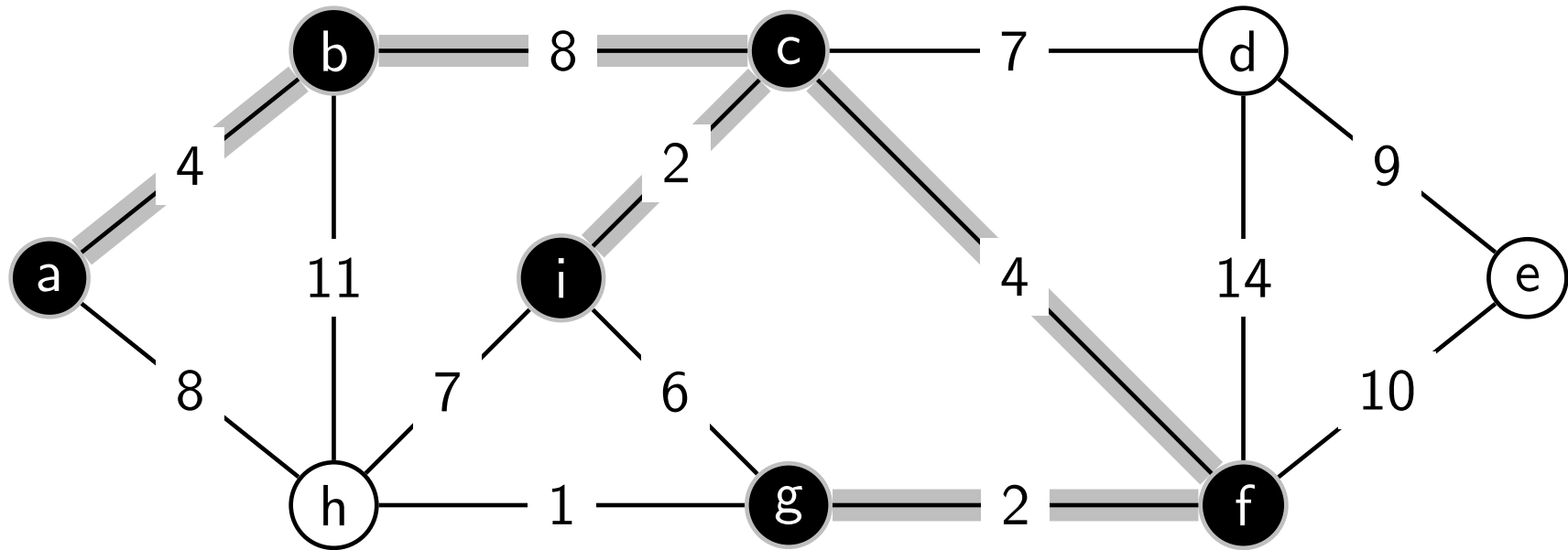
Example



Graphs II

23.2 The algorithms of (Kruskal and) Prim

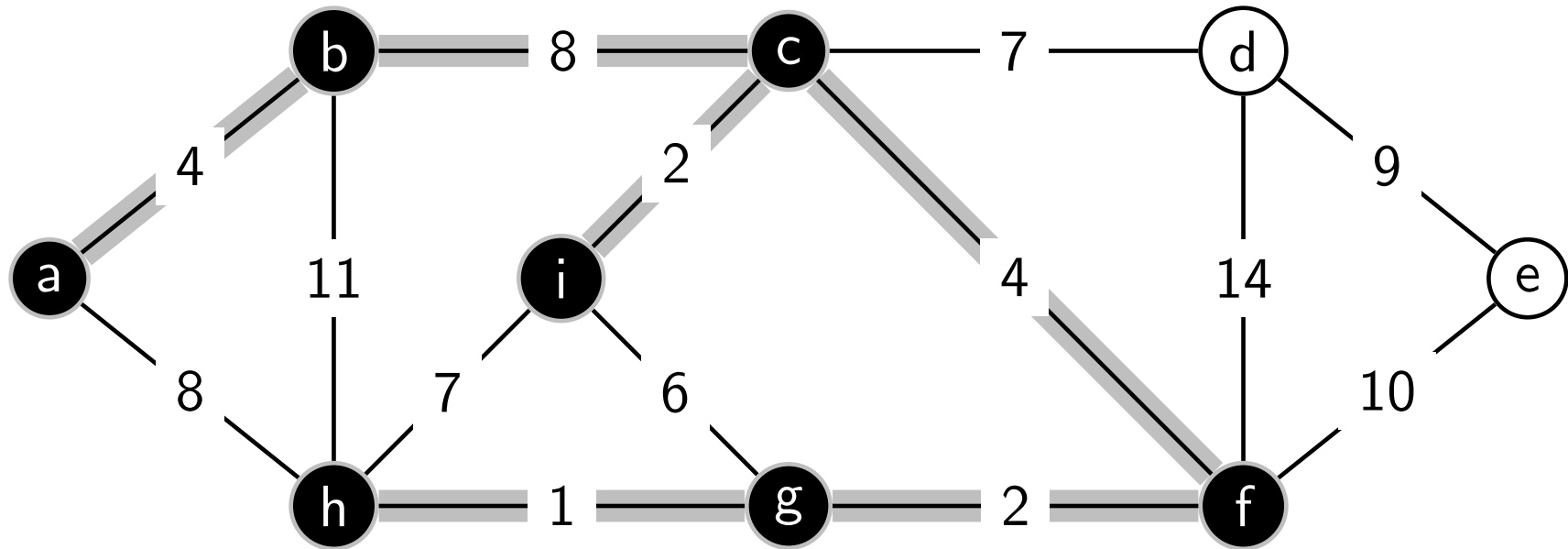
Example



Graphs II

23.2 The algorithms of (Kruskal and) Prim

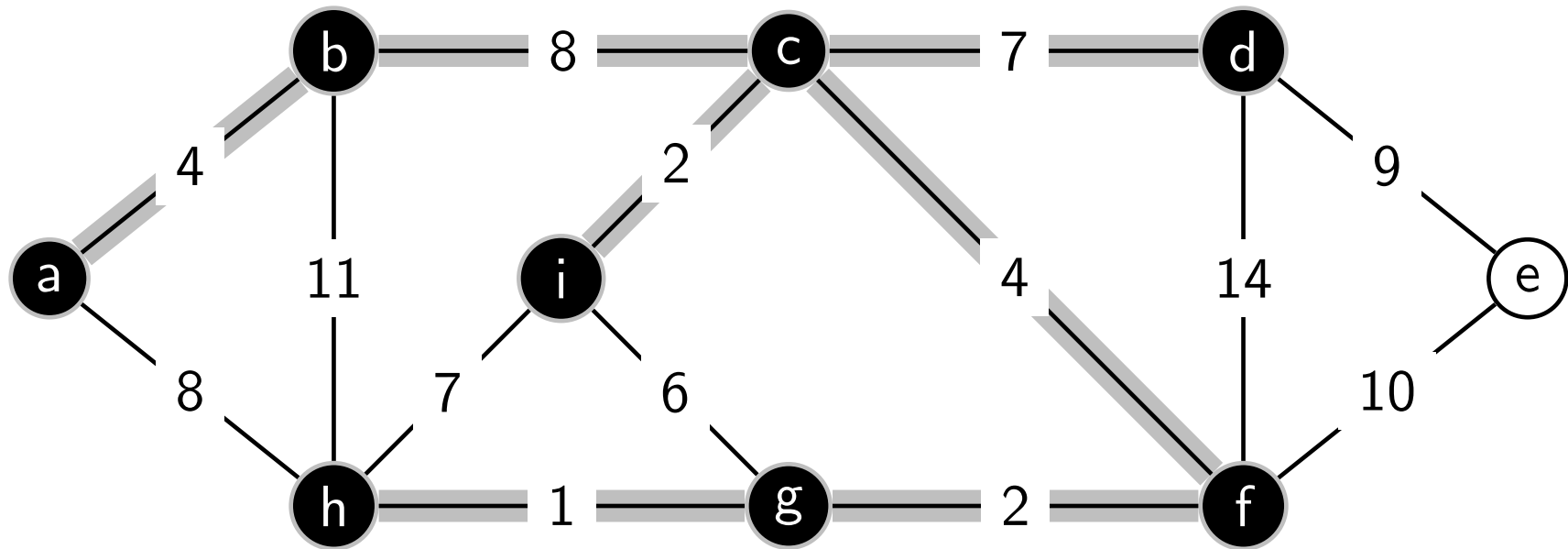
Example



Graphs II

23.2 The algorithms of (Kruskal and) Prim

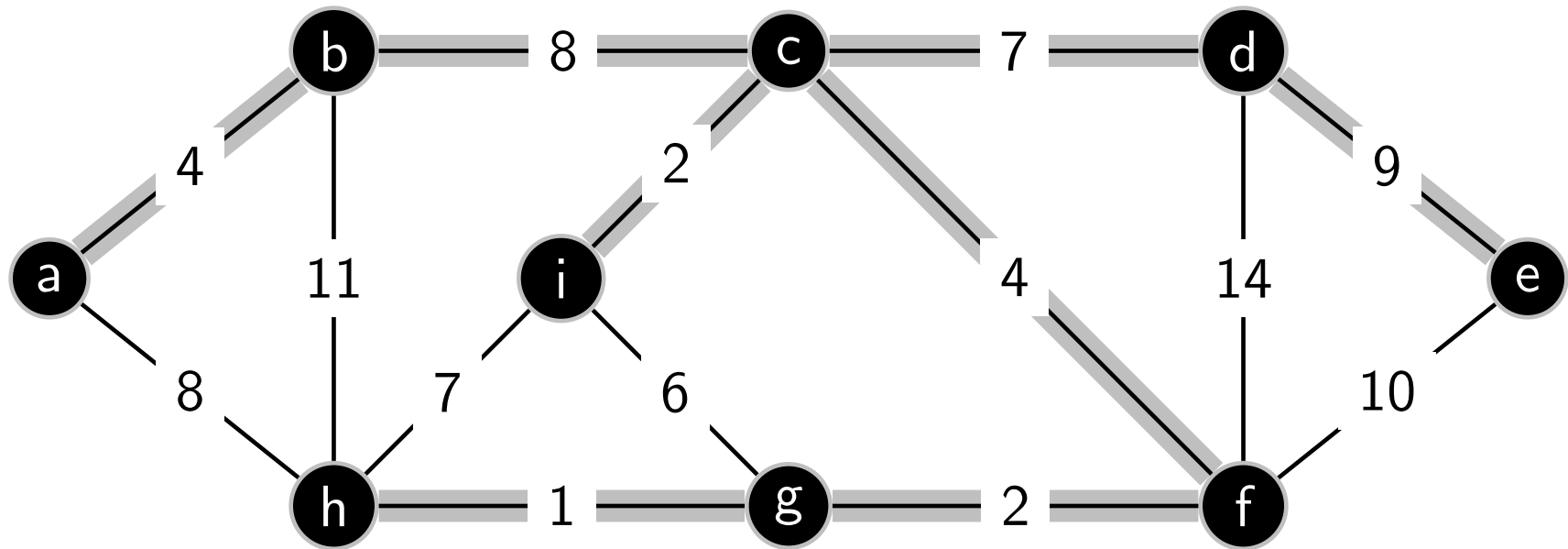
Example



Graphs II

23.2 The algorithms of (Kruskal and) Prim

Example



Graphs II

23.2 The algorithms of (Kruskal and) Prim

Prim's algorithm

- Implement the test for membership in Q by keeping a bit for each vertex $v \in V$ that tells whether or not $v \in Q$.

Analysis

- Prim's algorithm takes $O(|E| \log |V|)$ time, by implementing the min-priority queue with a min-heap.
 - $O(|V|)$ time to build the min-heap.
 - $|V|$ min-heap EXTRACT-MIN operations, each taking $O(\log |V|)$ time.
 - $|E|$ min-heap DECREASE-KEY operations, each taking $O(\log |V|)$ time.
 - $|V| + |E| = O(|E|)$ because the graph is connected.

Graphs II

23.2 The algorithms of (Kruskal and) Prim

Exercise 23.2-2

Suppose that we represent the graph $G = (V, E)$ as an adjacency matrix. Give a simple implementation of Prim's algorithm for this case that runs in $O(|V|^2)$ time.

Graphs II

23.2 The algorithms of (Kruskal and) Prim

Solution to Exercise 23.2-2

MST-PRIM(G, r)

for each vertex $u \in G.V$ **do**

$u.key = \infty$

$u.pred = \text{NIL}$

$r.key = 0$

$Q = G.V$

while $Q \neq \emptyset$ **do**

$u = \text{EXTRACT-MIN}(Q)$

for each vertex $v \in G.V$ **do**

if $A[u, v] \neq 0$ **and** $v \in Q$ **and** $A[u, v] < v.key$ **then**

$v.pred = u$

$v.key = A[u, v]$

Lecture 10

Exhaustive Search and Generation I

Exhaustive Search and Generation I

Contents

- The backtracking technique N 5.1
- The n -queens problem N 5.2
- The sum-of-subsets problem N 5.4
- Graph coloring N 5.5
- The hamiltonian circuits problem N 5.6
- The 0-1 knapsack problem N 5.7
- Exercise: All subsets
- Exercise: All permutations

Reading

- R. E. Neapolitan. *Foundations of Algorithms*. Jones & Bartlett Learning, 5th edition, 2015

Exhaustive Search and Generation I

The backtracking technique

- Backtracking is a technique is used to solve problems in which a **sequence** of objects is chosen from a specified **set** so that the sequence satisfies some **criterion**.
- The backtracking technique is a modified depth-first search of a **state space tree**, checking whether each node is promising, and, if it is non-promising, going back to the parent of the node and proceeding with the search on the next child.
 - A node is **non-promising** if when visiting the node we determine that it cannot possibly lead to a solution. Otherwise, it is **promising**.
 - Going back to the parent of the node is called **pruning** the state space tree.
 - The subtree consisting of the visited nodes is called the **pruned state space tree**.

Exhaustive Search and Generation I

The backtracking technique

- A backtracking algorithm is identical to a depth-first search, except that the children of a node are visited only when the node is promising and there is not a solution at the node.
- In some backtracking algorithms a solution can be found before reaching a leaf in the state space tree.
- A general algorithm for backtracking is as follows.

```
BACKTRACK( $u$ )  
  if PROMISING( $u$ ) then  
    if SOLUTION( $u$ ) then  
      write the solution  
    else  
      for each child  $v$  of  $u$  do  
        BACKTRACK( $v$ )
```

Exhaustive Search and Generation I

The backtracking technique

- You may have observed that there is some inefficiency in the general algorithm for backtracking.
- We check whether a partial solution is promising after passing it to the procedure. (This means that non-promising partial solutions are unnecessarily placed on the recursion stack.)
- We could avoid this by checking whether a partial solution is promising before passing it.

```
BACKTRACK( $u$ )  
  for each child  $v$  of  $u$  do  
    if PROMISING( $v$ ) then  
      if SOLUTION( $v$ ) then  
        write the solution  
      else  
        BACKTRACK( $v$ )
```

Exhaustive Search and Generation I

The n -queens problem

Backtracking is a technique is used to solve problems in which a sequence of objects is chosen from a specified set so that the sequence satisfies some criterion.

- The goal in the n -queens problem is to position n queens on an $n \times n$ chessboard so that no two queens threaten each other; that is, no two queens may be in the same row, column, or diagonal.
- The **sequence** in this problem is the n positions in which the queens are placed.
- The **set** for each choice is the n^2 possible positions on the chessboard.
- The **criterion** is that no two queens can threaten each other.

Exhaustive Search and Generation I

The n -queens problem

	1	2	3	4
1				
2				
3				
4				

Exhaustive Search and Generation I

The n -queens problem

	1	2	3	4
1				
2				
3				
4				

Exhaustive Search and Generation I

The n -queens problem

	1	2	3	4
1		X	X	X
2	X	X		
3	X		X	
4	X			X

Exhaustive Search and Generation I

The n -queens problem

	1	2	3	4
1		X	X	X
2	X	X		
3	X		X	
4	X			X

Exhaustive Search and Generation I

The n -queens problem

	1	2	3	4
1		X	X	X
2	X	X		X
3	X	X	X	X
4	X		X	X

Exhaustive Search and Generation I

The n -queens problem

	1	2	3	4
1		X	X	X
2	X	X		
3	X		X	
4	X			X

Exhaustive Search and Generation I

The n -queens problem

	1	2	3	4
1		X	X	X
2	X	X	X	
3	X		X	X
4	X	X		X

Exhaustive Search and Generation I

The n -queens problem

	1	2	3	4
1		X	X	X
2	X	X	X	
3	X		X	X
4	X	X		X

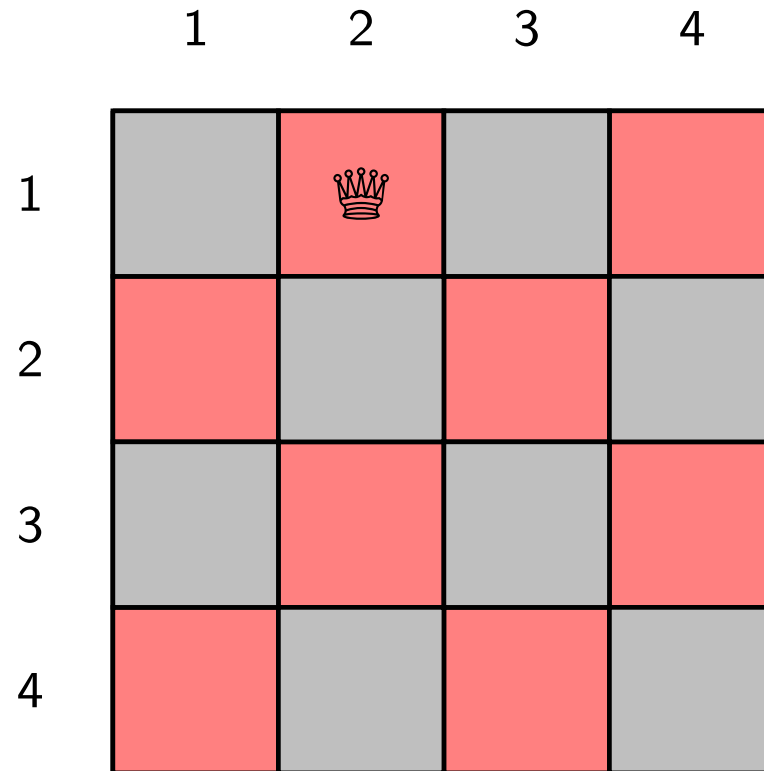
Exhaustive Search and Generation I

The n -queens problem

	1	2	3	4
1		X	X	X
2	X	X	X	
3	X		X	X
4	X	X	X	X

Exhaustive Search and Generation I

The n -queens problem



Exhaustive Search and Generation I

The n -queens problem

	1	2	3	4
1	X		X	X
2	X	X	X	
3		X		X
4		X		

Exhaustive Search and Generation I

The n -queens problem

	1	2	3	4
1	X		X	X
2	X	X	X	
3		X		X
4		X		

Exhaustive Search and Generation I

The n -queens problem

	1	2	3	4
1	X		X	X
2	X	X	X	
3		X	X	X
4		X		X

Exhaustive Search and Generation I

The n -queens problem

	1	2	3	4
1	X		X	X
2	X	X	X	
3		X	X	X
4		X		X

Exhaustive Search and Generation I

The n -queens problem

	1	2	3	4
1	X		X	X
2	X	X	X	
3		X	X	X
4	X	X		X

Exhaustive Search and Generation I

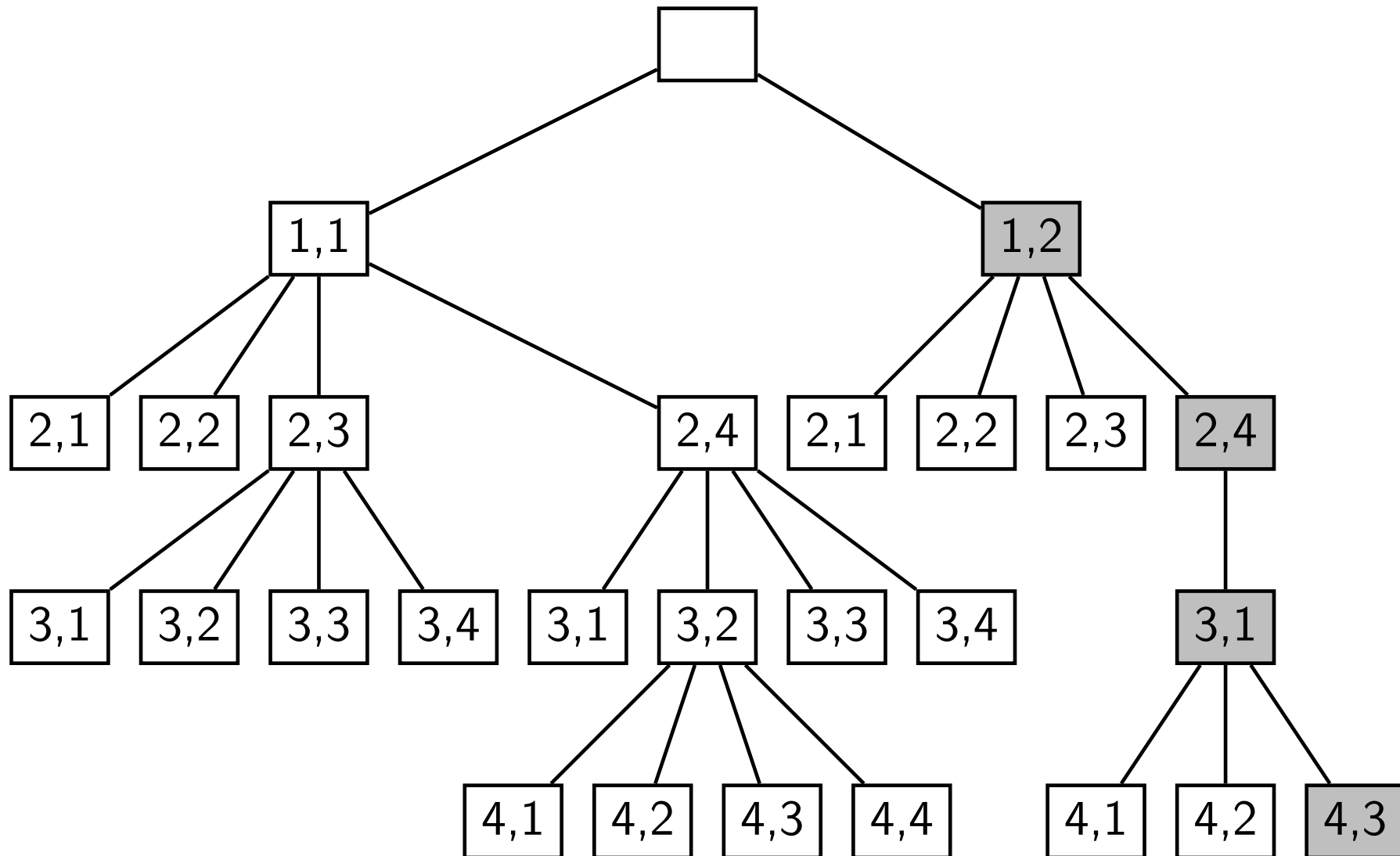
The n -queens problem

	1	2	3	4
1	X		X	X
2	X	X	X	
3		X	X	X
4	X	X		X

Exhaustive Search and Generation I

The n -queens problem

- A portion of the pruned state space tree for $n = 4$.



Exhaustive Search and Generation I

The n -queens problem

- Let $col[i]$ be the column where the queen in the i th row is placed.

N-QUEENS(0)

```
N-QUEENS( $i$ )  
  if PROMISING( $i$ ) then  
    if  $i = n$  then  
      write  $col[1], \dots, col[n]$   
    else  
      for  $j = 1$  to  $n$  do  
         $col[i + 1] = j$   
        N-QUEENS( $i + 1$ )
```

Exhaustive Search and Generation I

The n -queens problem

PROMISING(i)

for $j = 1$ **to** $i - 1$ **do**

if $col[i] = col[j]$ or $abs(col[i] - col[j]) = i - j$ **then**

return FALSE

return TRUE

	1	2	3	4	5	6	7	8
1								
2								♔
3	♔							
4								
5								
6				♔				
7								
8								

Exhaustive Search and Generation I

The sum-of-subsets problem

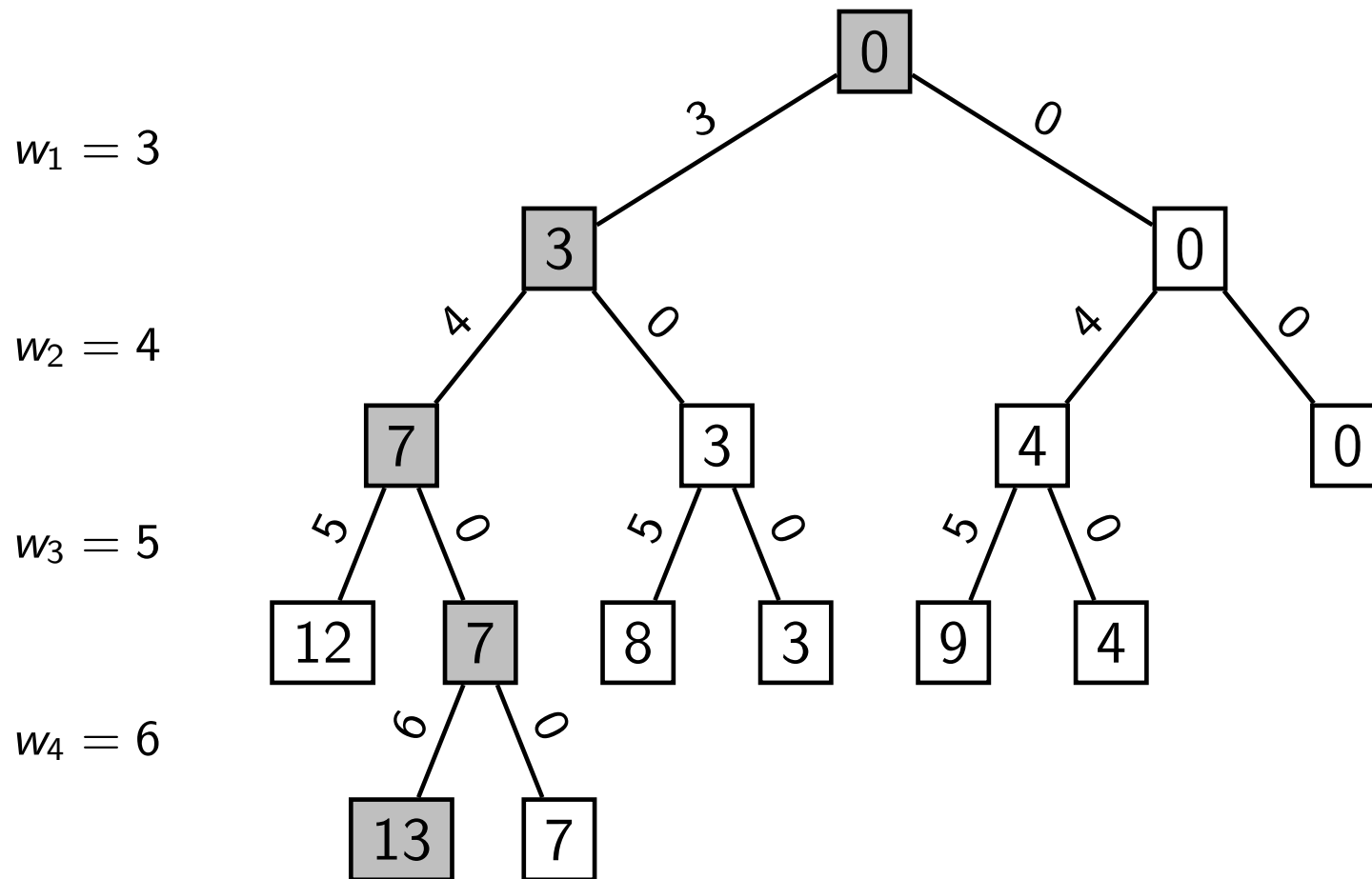
Backtracking is a technique is used to solve problems in which a sequence of objects is chosen from a specified set so that the sequence satisfies some criterion.

- Given n positive integers (weights) w_i and a positive integer W , the goal in the sum-of-subsets problem is to find a subset of the integers that sum to W .
- The **sequence** in this problem is the n integers.
- The **set** for each choice is to include or to exclude the integer.
- The **criterion** is that the chosen integers sum to W .

Exhaustive Search and Generation I

The sum-of-subsets problem

- The pruned state space tree for $n = 4$, $W = 13$, and $w_1 = 3$, $w_2 = 4$, $w_3 = 5$, $w_4 = 6$.



Exhaustive Search and Generation I

The sum-of-subsets problem

- Let $include[i]$ be true if and only if $w[i]$ is to be included.

```
SUM-OF-SUBSETS(0, 0)
```

```
SUM-OF-SUBSETS( $i, w$ )  
  if PROMISING( $i, w$ ) then  
    if  $w = W$  then  
      write  $include[1], \dots, include[i]$   
    else  
       $include[i + 1] = \text{TRUE}$   
      SUM-OF-SUBSETS( $i + 1, w + w[i + 1]$ )  
       $include[i + 1] = \text{FALSE}$   
      SUM-OF-SUBSETS( $i + 1, w$ )
```

```
PROMISING( $i, w$ )  
  return  $w \leq W$ 
```

Exhaustive Search and Generation I

Graph coloring

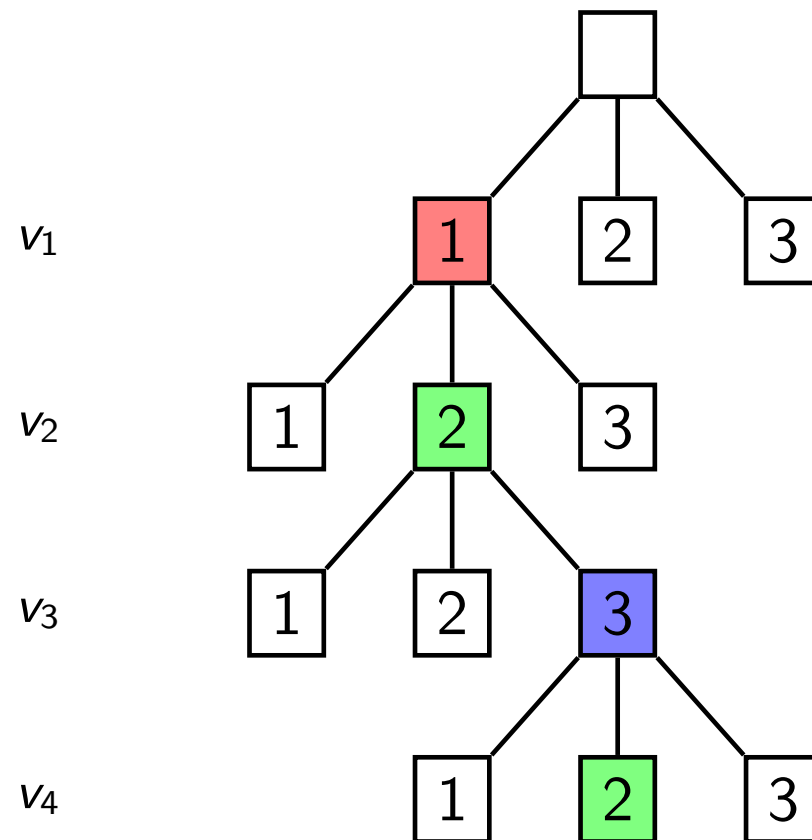
Backtracking is a technique is used to solve problems in which a sequence of objects is chosen from a specified set so that the sequence satisfies some criterion.

- The goal in the graph coloring problem is to color an undirected graph using at most m different colors, so that no two adjacent vertices are the same color.
- The **sequence** in this problem is the vertices of the graph.
- The **set** for each choice is the m possible colors.
- The **criterion** is that no two adjacent vertices are the same color.

Exhaustive Search and Generation I

Graph coloring

- A portion of the pruned state space tree for $m = 3$,
 $V = \{v_1, v_2, v_3, v_4\}$, and
 $E = \{(v_1, v_2), (v_1, v_3), (v_1, v_4), (v_2, v_3), (v_3, v_4)\}$.



Exhaustive Search and Generation I

Graph coloring

- A node is nonpromising if a vertex that is adjacent to the vertex being colored at the node has already been colored the color that is being used at the node.
- Let $color[i]$ be the color assigned to the i th vertex.

M-COLORING(0)

```
M-COLORING( $i$ )  
  if PROMISING( $i$ ) then  
    if  $i = n$  then  
      write  $color[1], \dots, color[n]$   
    else  
      for  $c = 1$  to  $m$  do  
         $color[i + 1] = c$   
        M-COLORING( $i + 1$ )
```


Exhaustive Search and Generation I

Graph coloring

- The undirected graph is represented by an adjacency matrix $G.Adj[1..n, 1..n]$.

```
PROMISING(i)  
  for j = 1 to i - 1 do  
    if  $G.Adj[i, j]$  and  $color[i] = color[j]$  then  
      return FALSE  
  return TRUE
```

Exhaustive Search and Generation I

Graph coloring

Exercise CLRS 34-3.a

Give an efficient algorithm to determine a 2-coloring of a graph, if one exists.

Solution

```
2-COLORING( $G$ )  
  for each vertex  $u \in G.V$  do  
     $u.color = \text{WHITE}$   
  for each vertex  $u \in G.V$  do  
    if  $u.color = \text{WHITE}$  then  
       $u.color = \text{BLUE}$   
      if not 2-COLORING( $G, u$ ) then  
        return FALSE  
  return TRUE
```

Exhaustive Search and Generation I

Graph coloring

```
2-COLORING( $G, u$ )
   $Q = \emptyset$ 
  ENQUEUE( $Q, u$ )
  while  $Q \neq \emptyset$  do
     $u = \text{DEQUEUE}(Q)$ 
    for each vertex  $v \in G.\text{Adj}[u]$  do
      if  $v.\text{color} = \text{WHITE}$  then
        if  $u.\text{color} = \text{BLUE}$  then
           $v.\text{color} = \text{RED}$ 
        else
           $v.\text{color} = \text{BLUE}$ 
          ENQUEUE( $Q, v$ )
      else
        if  $u.\text{color} = v.\text{color}$  then
          return FALSE
  return TRUE
```

Exhaustive Search and Generation I

The hamiltonian circuits problem

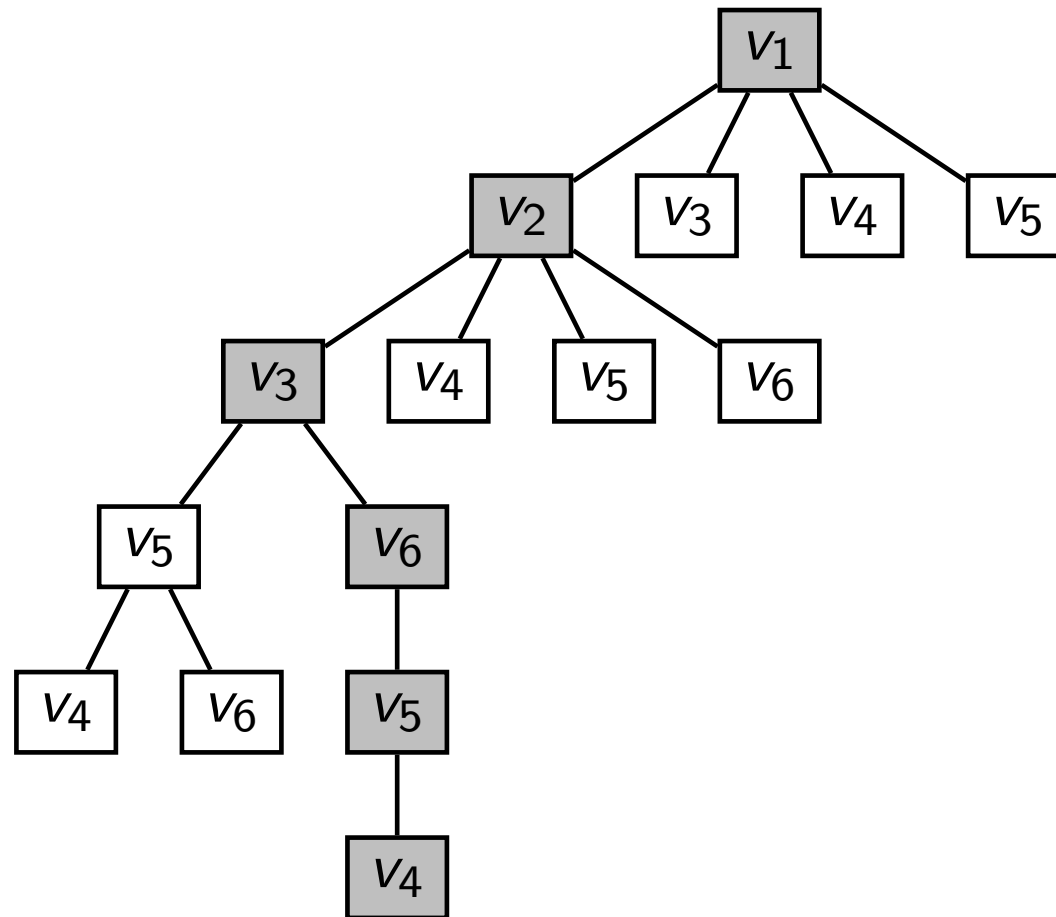
Backtracking is a technique is used to solve problems in which a sequence of objects is chosen from a specified set so that the sequence satisfies some criterion.

- The goal in the hamiltonian circuits problem is to find a hamiltonian circuit (a path that starts at a given vertex, visits each vertex in the graph exactly once, and ends at the starting vertex) in a connected, undirected graph.
- The **sequence** in this problem is the vertices of the graph.
- The **set** for each choice is the vertices of the graph.
- The **criterion** is that the chosen vertices form a simple path or cycle in the graph.

Exhaustive Search and Generation I

The hamiltonian circuits problem

- A portion of the pruned state space tree for $V = \{v_1, \dots, v_6\}$ and $E = \{(v_1, v_2), (v_1, v_4), (v_1, v_5), (v_2, v_3), (v_2, v_4), (v_2, v_5), (v_2, v_6), (v_3, v_5), (v_3, v_6), (v_4, v_5), (v_5, v_6)\}$.



Exhaustive Search and Generation I

The hamiltonian circuits problem

- The undirected graph is represented by an adjacency matrix $G.Adj[1..n, 1..n]$.
- A hamiltonian circuit is represented by a vector $I[1..n]$ of vertex indices.

```
HAMILTONIAN-CIRCUITS(0)
```

```
HAMILTONIAN-CIRCUITS( $i$ )
```

```
  if PROMISING( $i$ ) then
```

```
    if  $i = n$  then
```

```
      write  $I[1], \dots, I[n]$ 
```

```
    else
```

```
      for  $j = 1$  to  $n$  do
```

```
         $I[i + 1] = j$ 
```

```
        HAMILTONIAN-CIRCUITS( $i + 1$ )
```

Exhaustive Search and Generation I

The hamiltonian circuits problem

PROMISING(i)

if $i = n$ and not $G.Adj[I[n - 1], I[0]]$ **then**

return FALSE

else if $i > 1$ and not $G.Adj[I[i - 1], I[i]]$ **then**

return FALSE

else

for $j = 1$ **to** $i - 1$ **do**

if $I[i] = I[j]$ **then**

return FALSE

return TRUE

Exhaustive Search and Generation I

The 0-1 knapsack problem

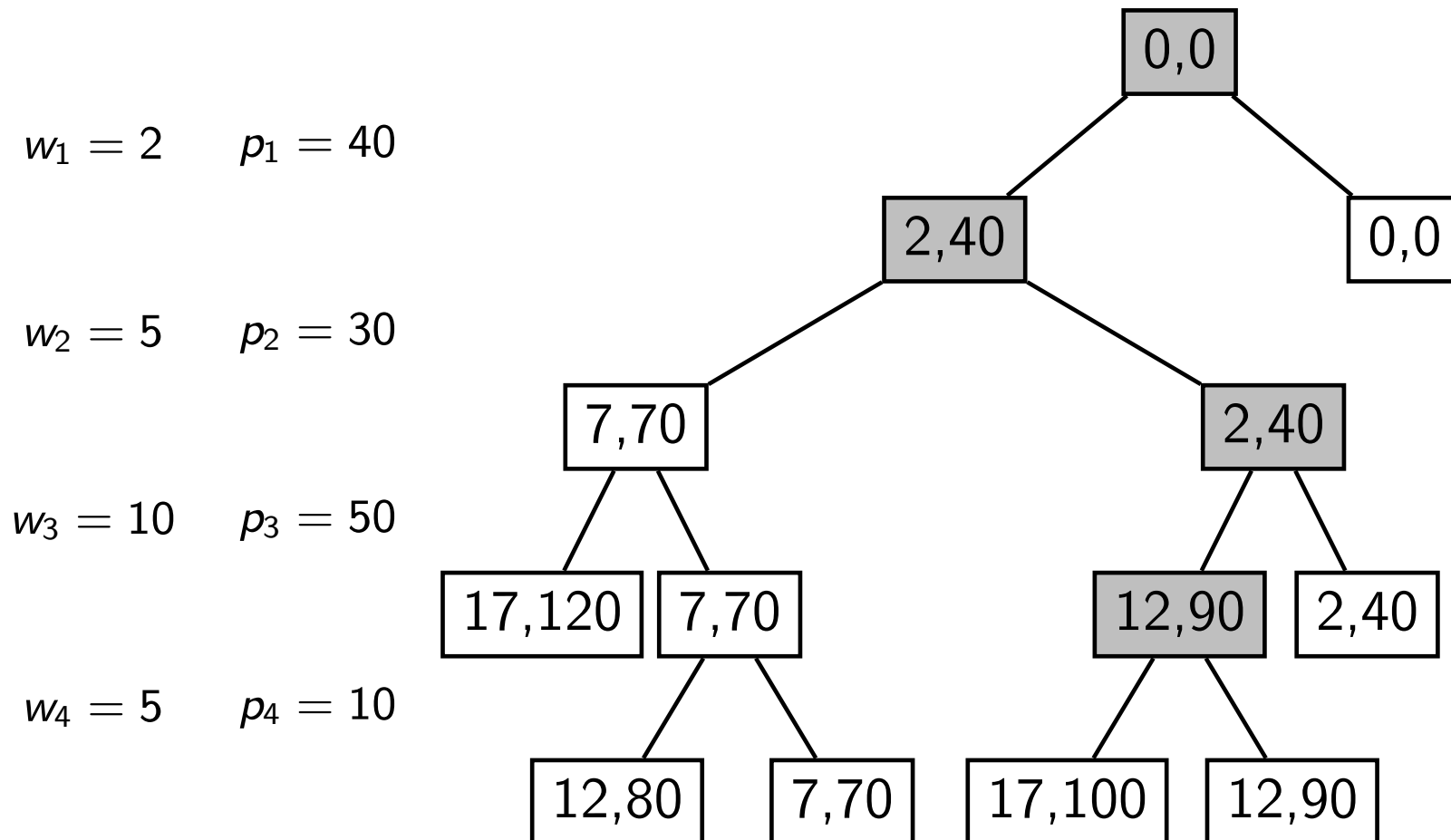
Backtracking is a technique is used to solve problems in which a sequence of objects is chosen from a specified set so that the sequence satisfies some criterion.

- Given a set of n items, each of which has a positive integer weight w_i and a positive integer profit p_i , and given a positive integer W , the goal in the 0-1 knapsack problem is to find a subset of the items that maximizes the total profit under the constraint that the total weight cannot exceed W .
- The **sequence** in this problem is the n items.
- The **set** for each choice is to include or to exclude the item.
- The **criterion** is that the total weight of the chosen items does not exceed W .

Exhaustive Search and Generation I

The 0-1 knapsack problem

- A portion of the pruned state space tree for $n = 4$, $W = 16$, $w_1 = 2$, $w_2 = 5$, $w_3 = 10$, $w_4 = 5$, and $p_1 = 40$, $p_2 = 30$, $p_3 = 50$, $p_4 = 10$.



Exhaustive Search and Generation I

The 0-1 knapsack problem

- Let $include[i]$ be true if and only if $w[i]$ is to be included.

```
0-1-KNAPSACK( $i, w, p$ )  
  if  $w \leq W$  and  $p > max\_p$  then  
     $max\_p = p$   
     $best\_i = i$   
     $best\_include = include$   
  if PROMISING( $i, w, p$ ) then  
     $include[i + 1] = \text{TRUE}$   
    0-1-KNAPSACK( $i + 1, w + w[i + 1], p + p[i + 1]$ )  
     $include[i + 1] = \text{FALSE}$   
    0-1-KNAPSACK( $i + 1, w, p$ )
```

Exhaustive Search and Generation I

The 0-1 knapsack problem

```
best_i = 0  
max_p = 0  
0-1-KNAPSACK(0, 0, 0)  
write best_include[1], ..., best_include[best_i]
```

- A node is non-promising if when visiting the node we determine that it cannot possibly lead to a solution.
- In optimization problems, a node is non-promising if when visiting the node we determine that no expansion to the children can possibly lead to a solution.

```
PROMISING(i, w, p)  
  return  $w < W$ 
```

Exhaustive Search and Generation I

Exercise: Generating all subsets

Exercise

Show that the number of subsets, including the empty set, of a set containing n items is 2^n .

Solution

For $0 \leq k \leq n$, the number of subsets of size k is the number of combinations of n objects taken k at a time, which is $\binom{n}{k}$. This means that the total number of subsets is

$$\sum_{k=0}^n \binom{n}{k} = \sum_{k=0}^n \binom{n}{k} 1^k 1^{n-k} = (1 + 1)^n = 2^n$$

The second-to-last equality is by the Binomial theorem:

$$(x + y)^n = \sum_{k=0}^n \binom{n}{k} x^k y^{n-k}$$

Exhaustive Search and Generation I

Exercise: All subsets

Backtracking is a technique is used to solve problems in which a sequence of objects is chosen from a specified set so that the sequence satisfies some criterion.

- The goal in the all subsets problem is to write all the subsets of the set of n integers $1, \dots, n$.
- The **sequence** in this problem is the n integers $1, \dots, n$.
- The **set** for each choice is to include or to exclude the integer.
- The **criterion** is none.

Exhaustive Search and Generation I

Exercise: All subsets

- Let $include[i]$ be true if and only if i is to be included.

```
ALL-SUBSETS(0)
```

```
ALL-SUBSETS( $i$ )
```

```
  if PROMISING( $i$ ) then
```

```
    if  $i = n$  then
```

```
      write  $include[1], \dots, include[n]$ 
```

```
    else
```

```
       $include[i + 1] = \text{TRUE}$ 
```

```
      ALL-SUBSETS( $i + 1$ )
```

```
       $include[i + 1] = \text{FALSE}$ 
```

```
      ALL-SUBSETS( $i + 1$ )
```

```
PROMISING( $i$ )
```

```
  return TRUE
```

Exhaustive Search and Generation I

Exercise: All permutations

Backtracking is a technique is used to solve problems in which a sequence of objects is chosen from a specified set so that the sequence satisfies some criterion.

- The goal in the all permutations problem is to write all the permutations of n integers $1, \dots, n$.
- The **sequence** in this problem is the n integers $1, \dots, n$.
- The **set** for each choice is the n integers $1, \dots, n$.
- The **criterion** is that the chosen integers are not repeated.

Exhaustive Search and Generation I

Exercise: All permutations

- Let $P[1], \dots, P[n]$ be a permutation of $1, \dots, n$.

$P = \emptyset$

ALL-PERMUTATIONS(0)

ALL-PERMUTATIONS(i)

if PROMISING(i) **then**

if $i = n$ **then**

 write $P[1], \dots, P[n]$

else

for $j = 1$ **to** n **do**

 PUSH(P, j)

 ALL-PERMUTATIONS($i + 1$)

 POP(P)

Exhaustive Search and Generation I

Exercise: All permutations

```
PROMISING(i)  
  for  $j = 1$  to  $i - 1$  do  
    if  $P[i] = P[j]$  then  
      return FALSE  
  return TRUE
```

Lecture 11

Exhaustive Search and Generation II

Exhaustive Search and Generation II

Contents

- Branch-and-bound N 6
- The sum-of-subsets problem N 5.4
- The 0-1 knapsack problem N 5.7, N 6.1
- The traveling salesman problem N 6.2

Reading

- R. E. Neapolitan. *Foundations of Algorithms*. Jones & Bartlett Learning, 5th edition, 2015

Exhaustive Search and Generation II

Branch-and-bound

- The branch-and-bound technique is an extension of the backtracking technique **for optimization problems**.
- A branch-and-bound algorithm computes a number (bound) at a node to determine whether the node is promising.
- The number is a bound on the value of the solution that could be obtained by expanding beyond the node.
- If that bound is no better than the value of the best solution found so far, the node is **non-promising**. Otherwise, it is **promising**.
- Better is smaller or larger, depending on the optimal value for the problem being a minimum or a maximum.

Exhaustive Search and Generation II

Branch-and-bound

- A general algorithm for branch-and-bound is as follows.

```
BRANCH_AND_BOUND( $u$ ,  $best$ )  
  if PROMISING( $u$ ) then  
    for each child  $v$  of  $u$  do  
      if VALUE( $v$ ) is better than  $best$  then  
         $best = \text{VALUE}(v)$   
      if BOUND( $v$ ) is better than  $best$  then  
        BRANCH_AND_BOUND( $v$ ,  $best$ )
```

Exhaustive Search and Generation II

The sum-of-subsets problem

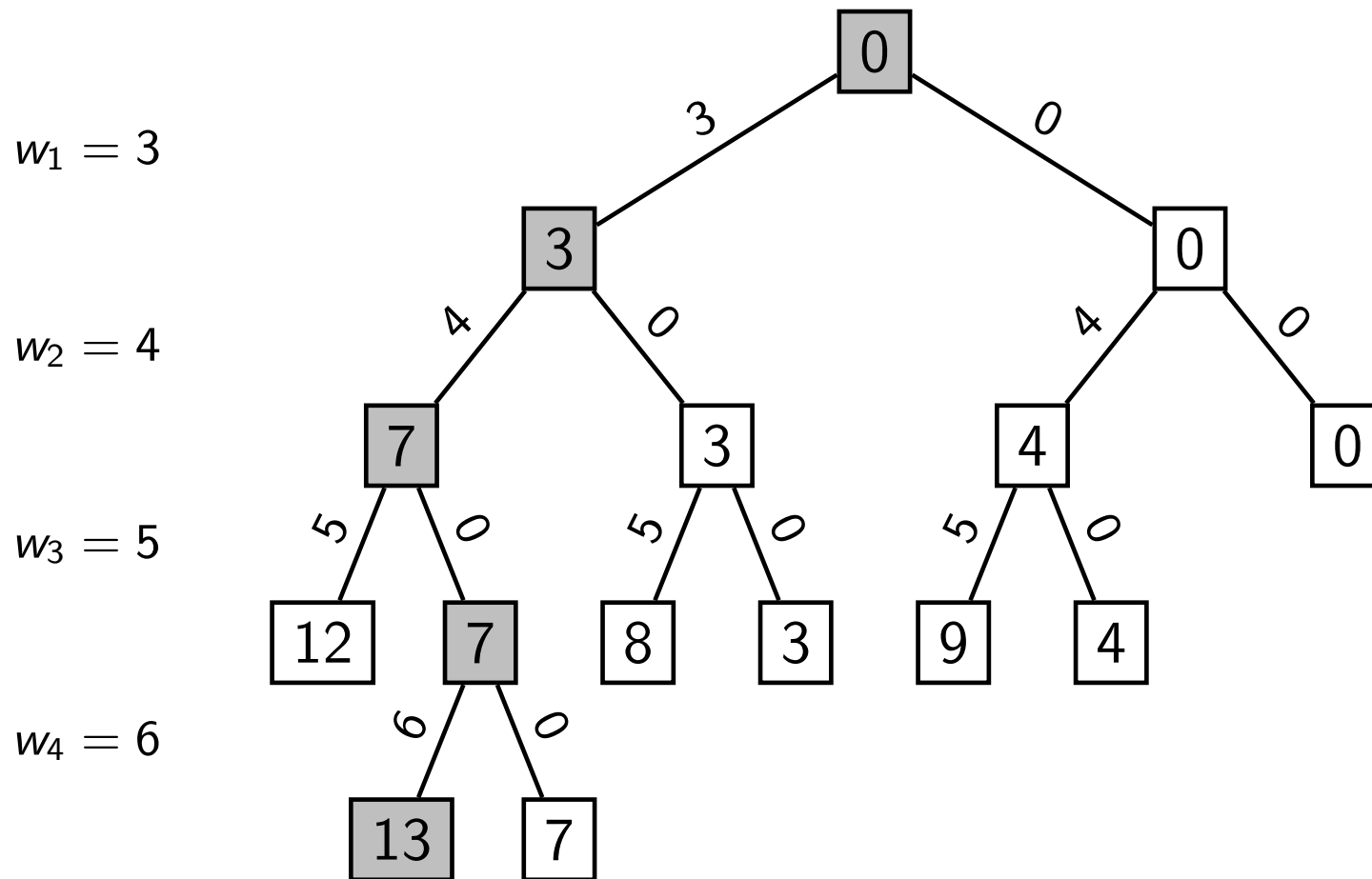
Backtracking is a technique is used to solve problems in which a sequence of objects is chosen from a specified set so that the sequence satisfies some criterion.

- Given n positive integers (weights) w_i and a positive integer W , the goal in the sum-of-subsets problem is to find a subset of the integers that sum to W .
- The **sequence** in this problem is the n integers.
- The **set** for each choice is to include or to exclude the integer.
- The **criterion** is that the chosen integers sum to W .

Exhaustive Search and Generation II

The sum-of-subsets problem

- The pruned state space tree for $n = 4$, $W = 13$, and $w_1 = 3$, $w_2 = 4$, $w_3 = 5$, $w_4 = 6$.



Exhaustive Search and Generation II

The sum-of-subsets problem

- Let $include[i]$ be true if and only if $w[i]$ is to be included.

```
SUM-OF-SUBSETS(0, 0)
```

```
SUM-OF-SUBSETS( $i, w$ )  
  if PROMISING( $i, w$ ) then  
    if  $w = W$  then  
      write  $include[1], \dots, include[i]$   
    else  
       $include[i + 1] = \text{TRUE}$   
      SUM-OF-SUBSETS( $i + 1, w + w[i + 1]$ )  
       $include[i + 1] = \text{FALSE}$   
      SUM-OF-SUBSETS( $i + 1, w$ )
```

```
PROMISING( $i, w$ )  
  return  $w \leq W$ 
```


Exhaustive Search and Generation II

The sum-of-subsets problem

- If we sort the weights in non-decreasing order before doing the search, there is an obvious sign telling us that a node is non-promising.
- Let w be the sum of the weights that have been included up to a node at the i th level.
- If w_{i+1} would bring the value of w above W , then so would any other weight following it.
- Therefore, unless $w = W$ (which means there is a solution at the node), a node at the i th level is non-promising if $w + w_{i+1} > W$.

Exhaustive Search and Generation II

The sum-of-subsets problem

- Let $include[i]$ be true if and only if $w[i]$ is to be included, let t be the total weight of the remaining weights, and let, initially, $t = \sum_{j=1}^n w[j]$.

SUM-OF-SUBSETS(0, 0, t)

SUM-OF-SUBSETS(i, w, t)

if PROMISING(i, w, t) **then**

if $w = W$ **then**

 write $include[1], \dots, include[i]$

else

$include[i + 1] = \text{TRUE}$

 SUM-OF-SUBSETS($i + 1, w + w[i + 1], t - w[i + 1]$)

$include[i + 1] = \text{FALSE}$

 SUM-OF-SUBSETS($i + 1, w, t - w[i + 1]$)

Exhaustive Search and Generation II

The sum-of-subsets problem

PROMISING(i, w, t)

return $w + t \geq W$ and ($w = W$ or $w + w[i + 1] \leq W$)

Exhaustive Search and Generation II

The 0-1 knapsack problem

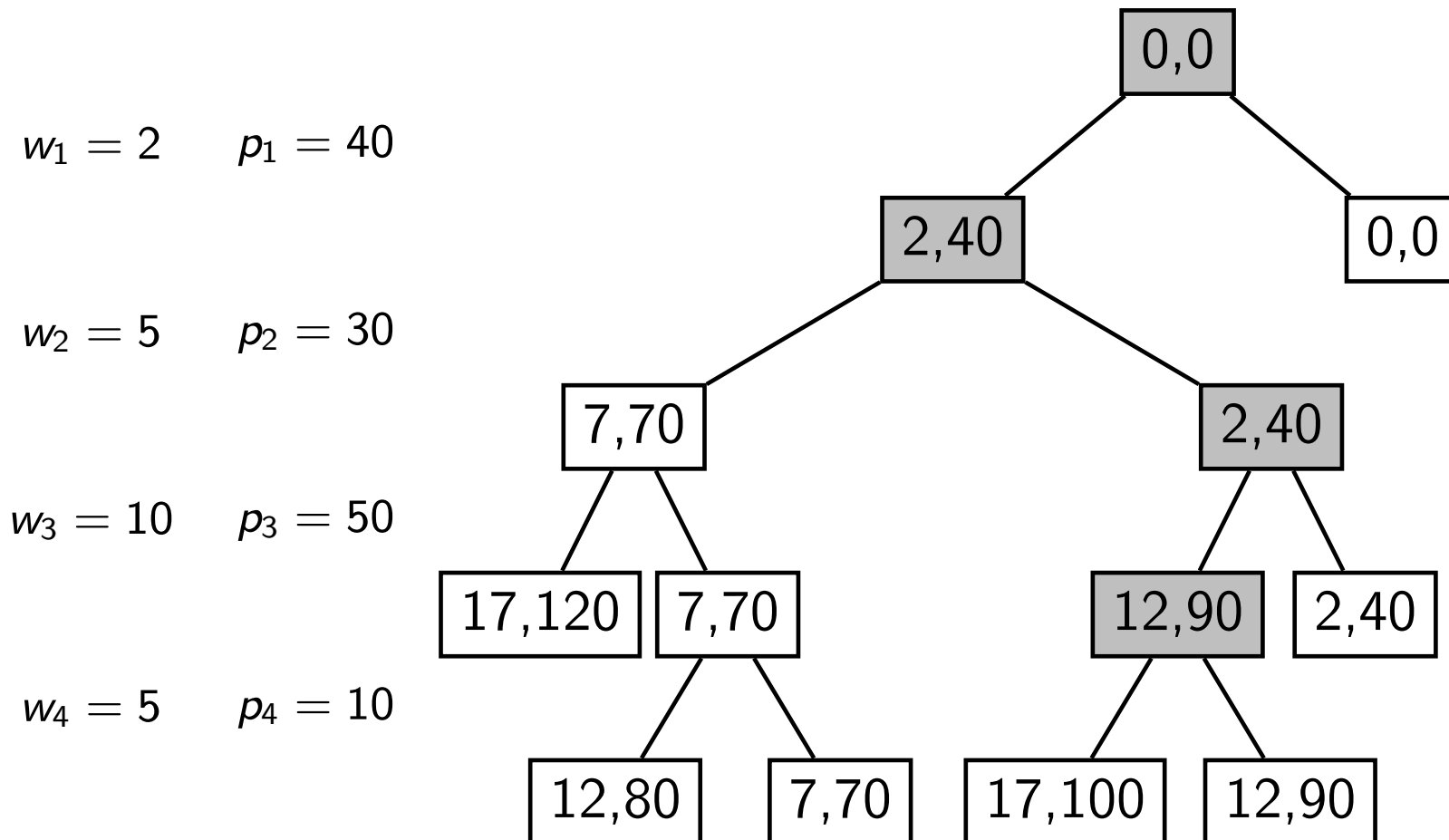
Backtracking is a technique is used to solve problems in which a sequence of objects is chosen from a specified set so that the sequence satisfies some criterion.

- Given a set of n items, each of which has a positive integer weight w_i and a positive integer profit p_i , and given a positive integer W , the goal in the 0-1 knapsack problem is to find a subset of the items that maximizes the total profit under the constraint that the total weight cannot exceed W .
- The **sequence** in this problem is the n items.
- The **set** for each choice is to include or to exclude the item.
- The **criterion** is that the total weight of the chosen items does not exceed W .

Exhaustive Search and Generation II

The 0-1 knapsack problem

- A portion of the pruned state space tree for $n = 4$, $W = 16$, $w_1 = 2$, $w_2 = 5$, $w_3 = 10$, $w_4 = 5$, and $p_1 = 40$, $p_2 = 30$, $p_3 = 50$, $p_4 = 10$.



Exhaustive Search and Generation II

The 0-1 knapsack problem

- Let $include[i]$ be true if and only if $w[i]$ is to be included.

```
0-1-KNAPSACK( $i, w, p$ )
```

```
  if  $w \leq W$  and  $p > max\_p$  then
```

```
     $max\_p = p$ 
```

```
     $best\_i = i$ 
```

```
     $best\_include = include$ 
```

```
  if PROMISING( $i, w, p$ ) then
```

```
     $include[i + 1] = \text{TRUE}$ 
```

```
    0-1-KNAPSACK( $i + 1, w + w[i + 1], p + p[i + 1]$ )
```

```
     $include[i + 1] = \text{FALSE}$ 
```

```
    0-1-KNAPSACK( $i + 1, w, p$ )
```

Exhaustive Search and Generation II

The 0-1 knapsack problem

```
best_i = 0  
max_p = 0  
0-1-KNAPSACK(0, 0, 0)  
write best_include[1], ..., best_include[best_i]
```

- A node is non-promising if when visiting the node we determine that it cannot possibly lead to a solution.
- In optimization problems, a node is non-promising if when visiting the node we determine that no expansion to the children can possibly lead to a solution.

```
PROMISING(i, w, p)  
  return  $w < W$ 
```

Exhaustive Search and Generation II

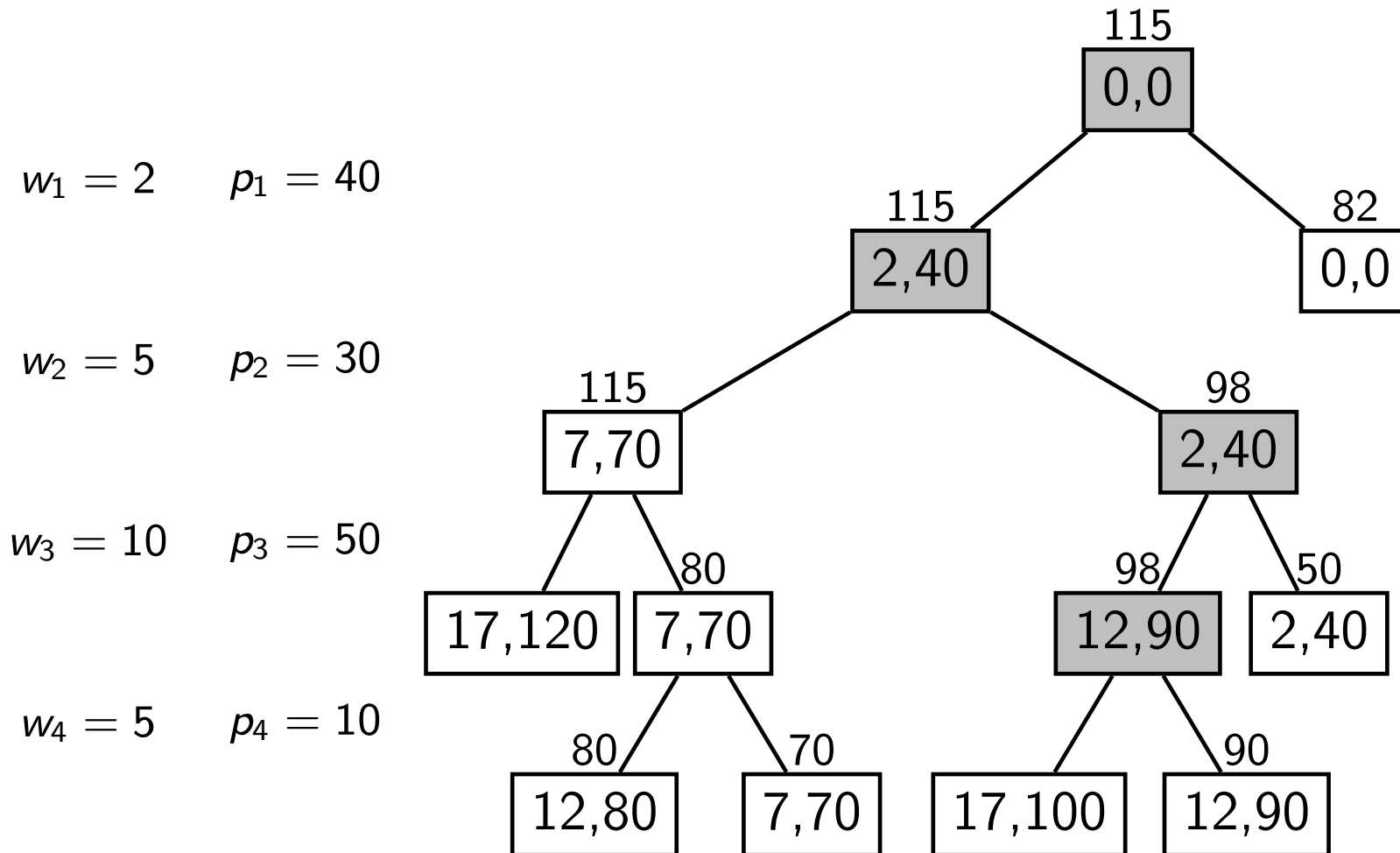
The 0-1 knapsack problem

- An obvious sign that a node is non-promising is that the total weight of the items included up to the node is $w \geq W$.
- If we sort the items in non-increasing order of profit-to-weight ratio before doing the search, there is a less obvious sign telling us that a node is non-promising.
- Let w be the sum of the weights that have been included up to a node at the i th level, and let *bound* be the total profit of all the items up to, but not including, the item that would bring the total weight above W , plus the fraction of that item allowed by the remaining weight.
- If we are able to get only a fraction of this last weight, *bound* is still an upper bound on the profit we could achieve by expanding beyond the node.
- Therefore, a node at the i th level is non-promising if *bound* $\leq$ *max_p*.

Exhaustive Search and Generation II

The 0-1 knapsack problem

- A portion of the pruned state space tree for $n = 4$, $W = 16$, $w_1 = 2$, $w_2 = 5$, $w_3 = 10$, $w_4 = 5$, and $p_1 = 40$, $p_2 = 30$, $p_3 = 50$, $p_4 = 10$.



Exhaustive Search and Generation II

The 0-1 knapsack problem

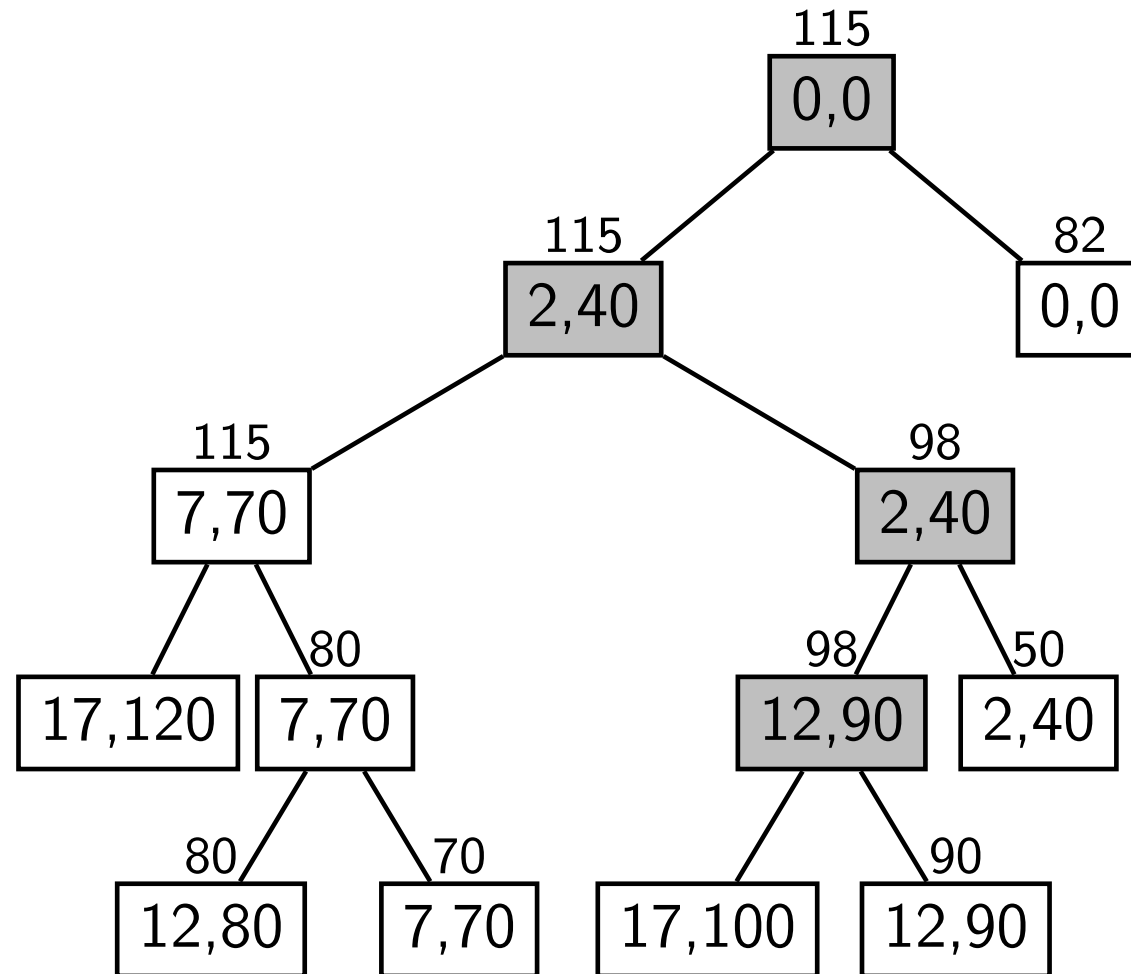
- $115 = 0 + 40 + 30 + (16 - 7) 50/10$

$$w_1 = 2 \quad p_1 = 40$$

$$w_2 = 5 \quad p_2 = 30$$

$$w_3 = 10 \quad p_3 = 50$$

$$w_4 = 5 \quad p_4 = 10$$



Exhaustive Search and Generation II

The 0-1 knapsack problem

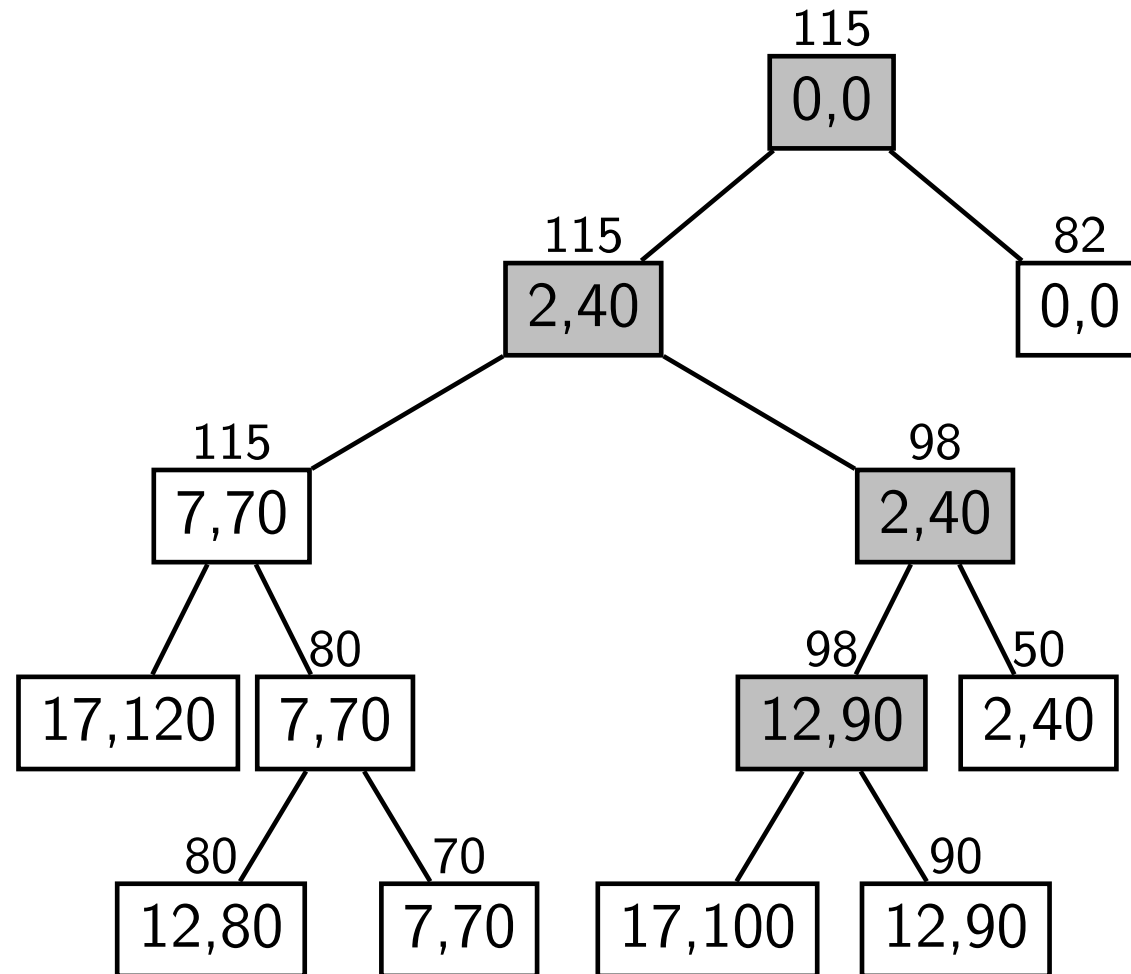
- $115 = 40 + 30 + (16 - 7) 50/10$

$$w_1 = 2 \quad p_1 = 40$$

$$w_2 = 5 \quad p_2 = 30$$

$$w_3 = 10 \quad p_3 = 50$$

$$w_4 = 5 \quad p_4 = 10$$



Exhaustive Search and Generation II

The 0-1 knapsack problem

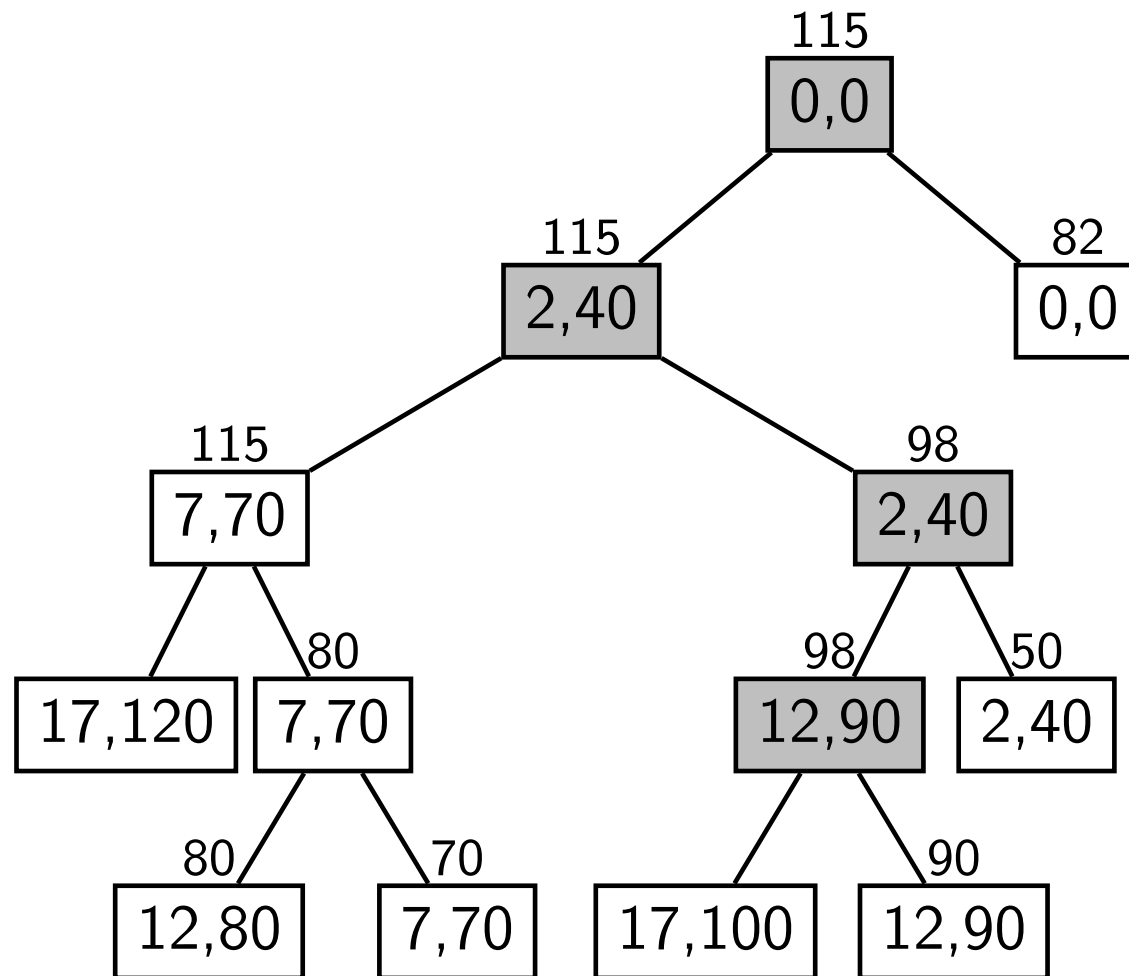
- $82 = 0 + 30 + 50 + (16 - 15) 10/5$

$$w_1 = 2 \quad p_1 = 40$$

$$w_2 = 5 \quad p_2 = 30$$

$$w_3 = 10 \quad p_3 = 50$$

$$w_4 = 5 \quad p_4 = 10$$



Exhaustive Search and Generation II

The 0-1 knapsack problem

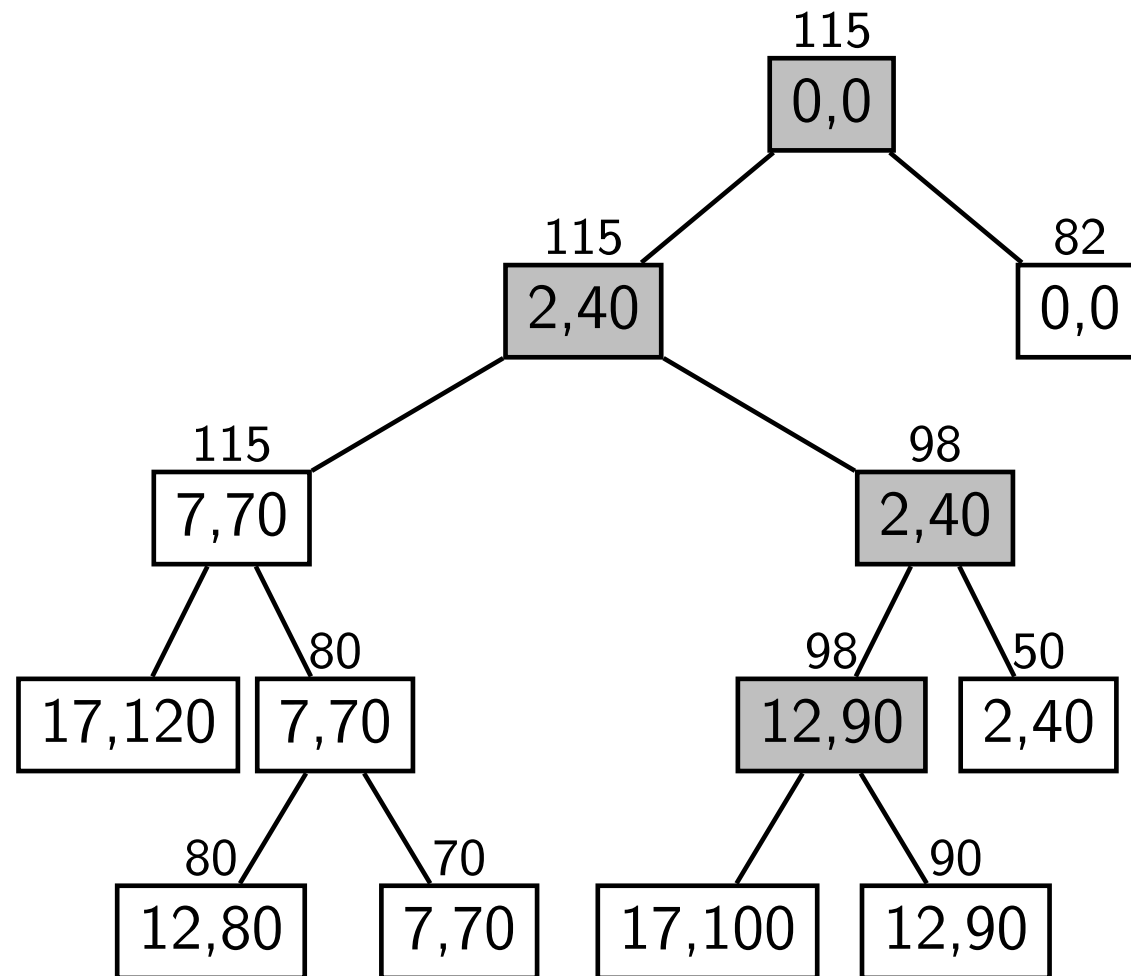
- $115 = 70 + (16 - 7) 50/10$

$$w_1 = 2 \quad p_1 = 40$$

$$w_2 = 5 \quad p_2 = 30$$

$$w_3 = 10 \quad p_3 = 50$$

$$w_4 = 5 \quad p_4 = 10$$



Exhaustive Search and Generation II

The 0-1 knapsack problem

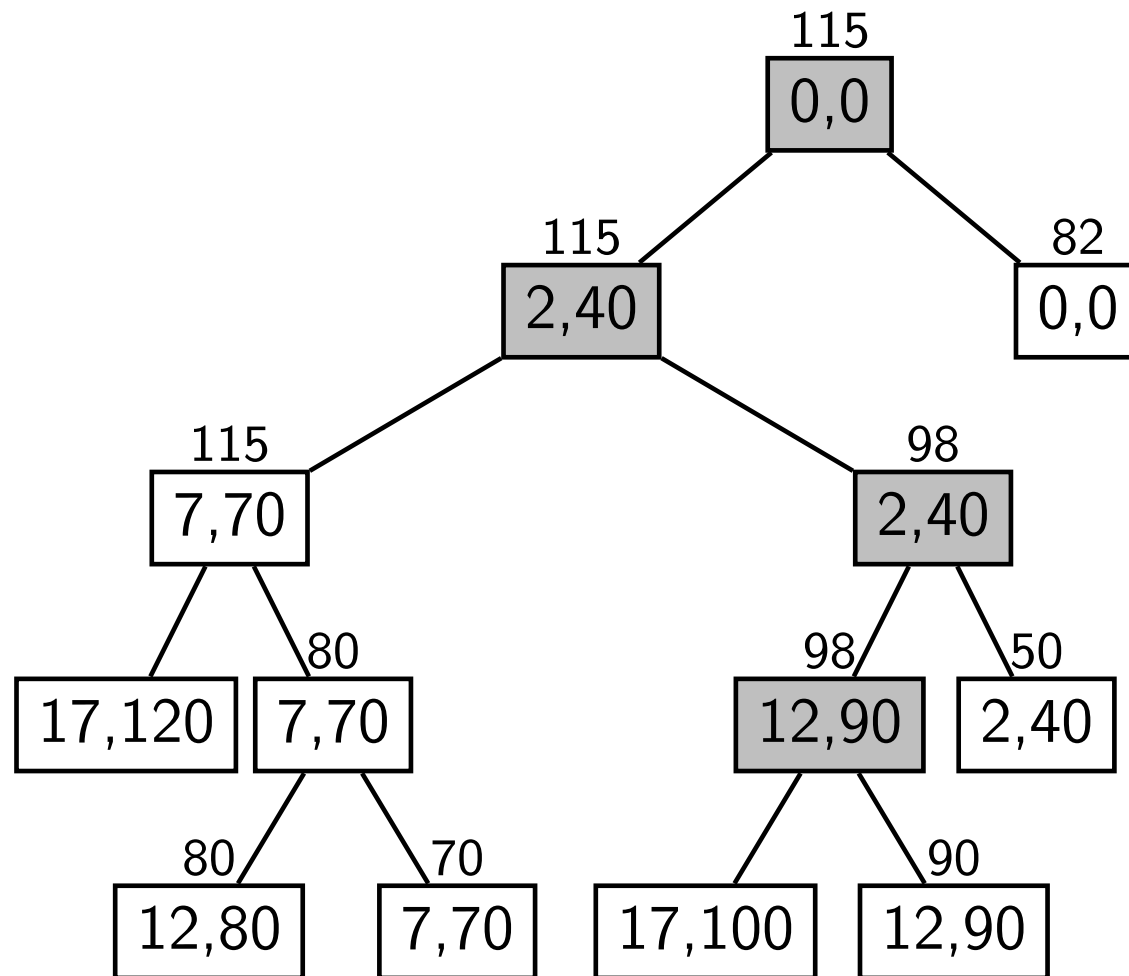
- $98 = 40 + 50 + (16 - 12) 10/5$

$$w_1 = 2 \quad p_1 = 40$$

$$w_2 = 5 \quad p_2 = 30$$

$$w_3 = 10 \quad p_3 = 50$$

$$w_4 = 5 \quad p_4 = 10$$



Exhaustive Search and Generation II

The 0-1 knapsack problem

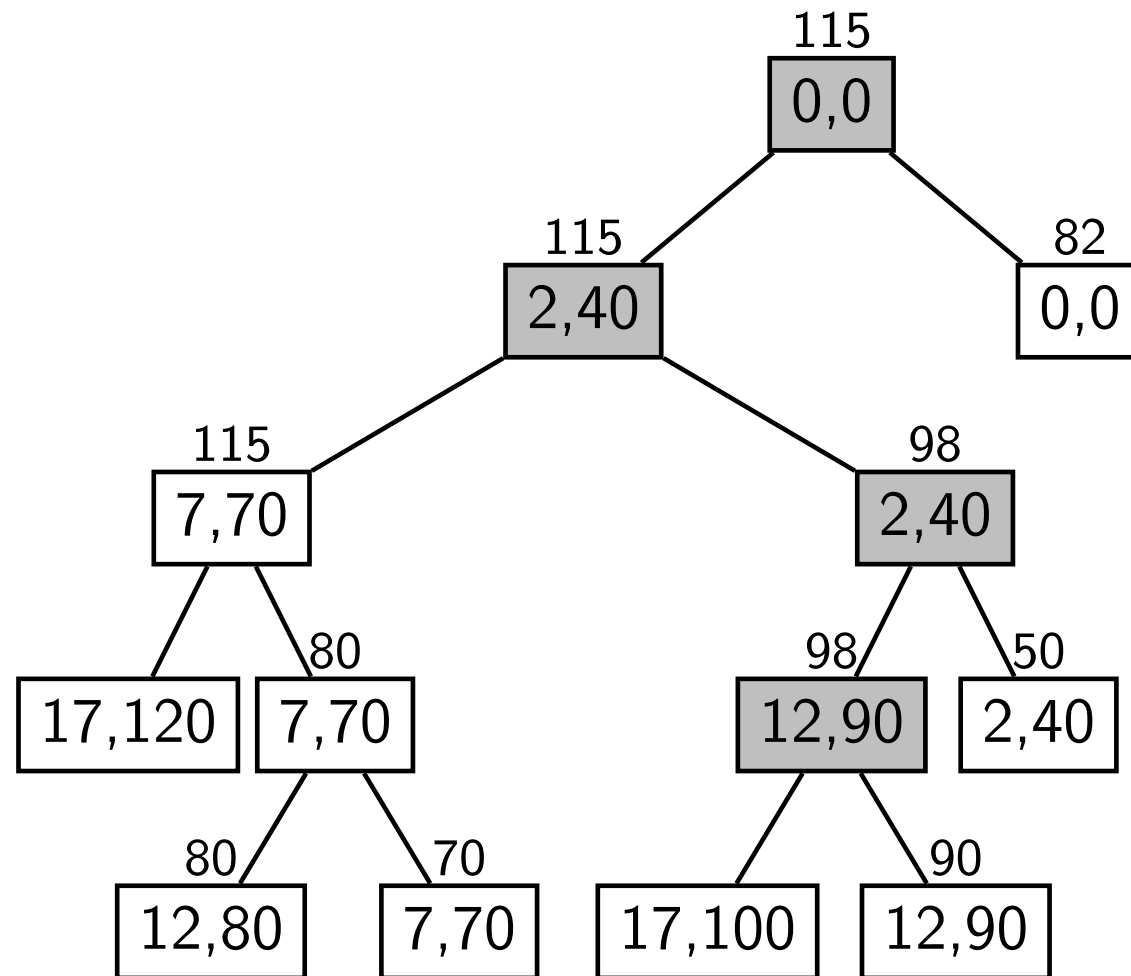
- $80 = 70 + 10$

$$w_1 = 2 \quad p_1 = 40$$

$$w_2 = 5 \quad p_2 = 30$$

$$w_3 = 10 \quad p_3 = 50$$

$$w_4 = 5 \quad p_4 = 10$$



Exhaustive Search and Generation II

The 0-1 knapsack problem

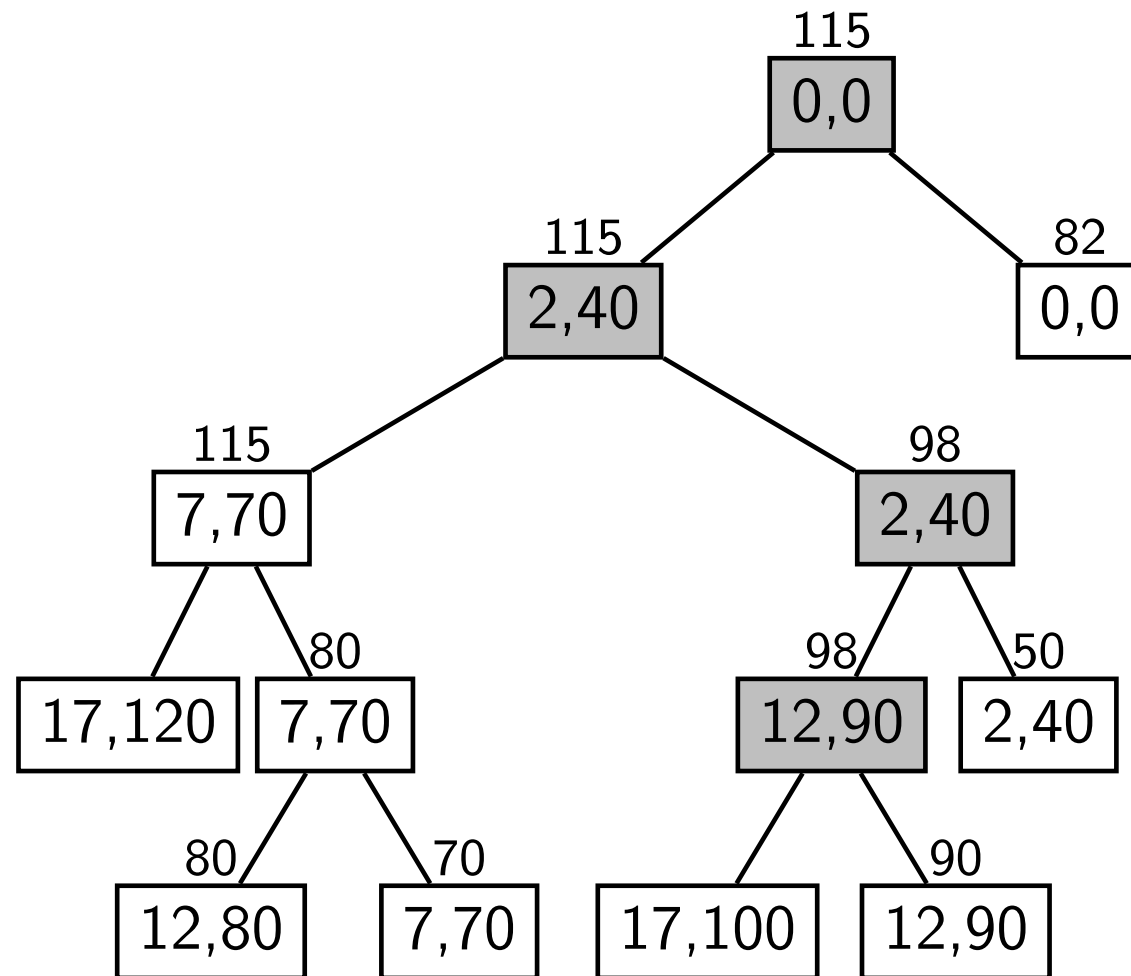
- $98 = 90 + (16 - 12) 10/5$

$$w_1 = 2 \quad p_1 = 40$$

$$w_2 = 5 \quad p_2 = 30$$

$$w_3 = 10 \quad p_3 = 50$$

$$w_4 = 5 \quad p_4 = 10$$



Exhaustive Search and Generation II

The 0-1 knapsack problem

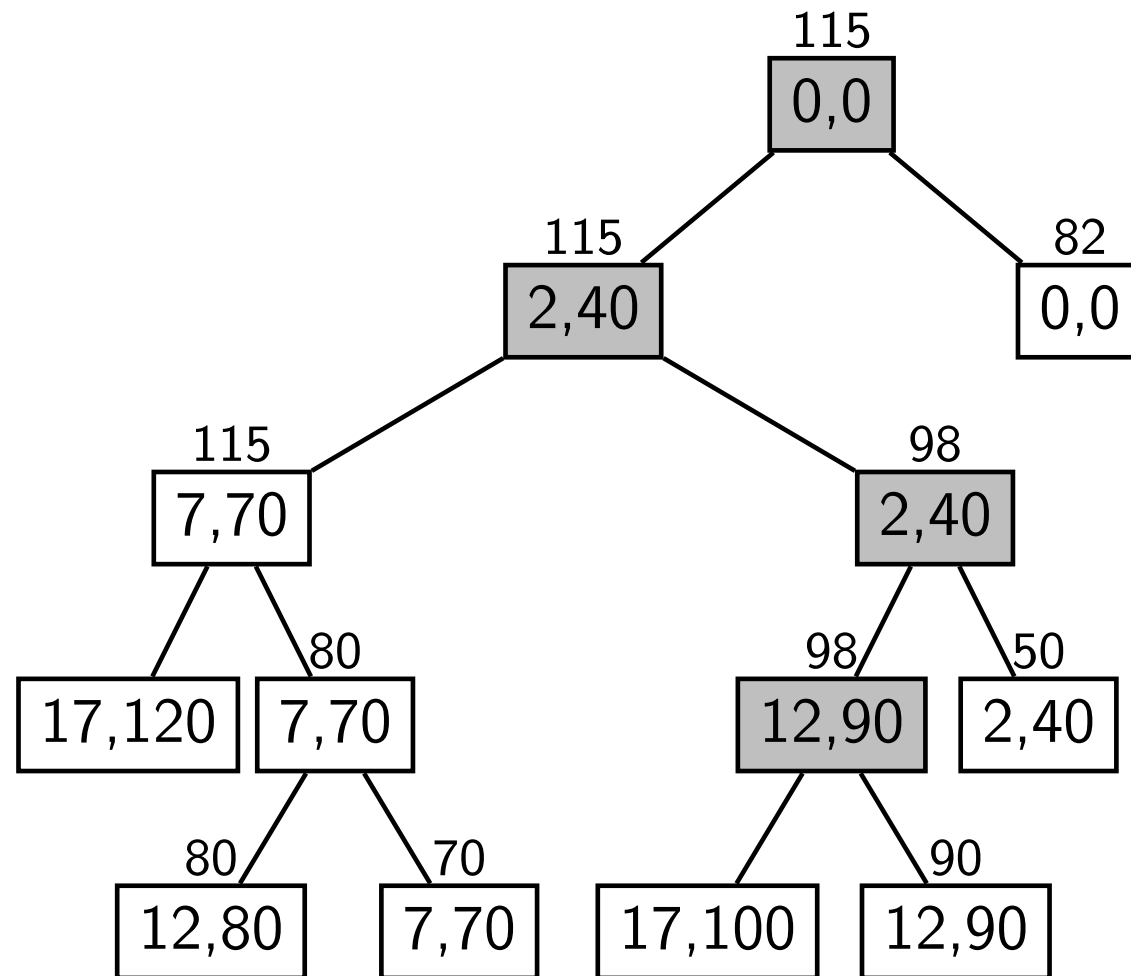
- $50 = 40 + 10$

$$w_1 = 2 \quad p_1 = 40$$

$$w_2 = 5 \quad p_2 = 30$$

$$w_3 = 10 \quad p_3 = 50$$

$$w_4 = 5 \quad p_4 = 10$$



Exhaustive Search and Generation II

The 0-1 knapsack problem

```
PROMISING( $i, w, p$ )  
  if  $w \geq W$  then  
    return FALSE  
  else  
     $j = i + 1$   
     $bound = p$   
     $t = w$   
    while  $j \leq n$  and  $t + w[j] \leq W$  do  
       $t = t + w[j]$   
       $bound = bound + p[j]$   
       $j = j + 1$   
    if  $j \leq n$  then  
       $bound = bound + (W - t) p[j] / w[j]$   
    return  $bound > max\_p$ 
```

Exhaustive Search and Generation II

The hamiltonian circuits problem

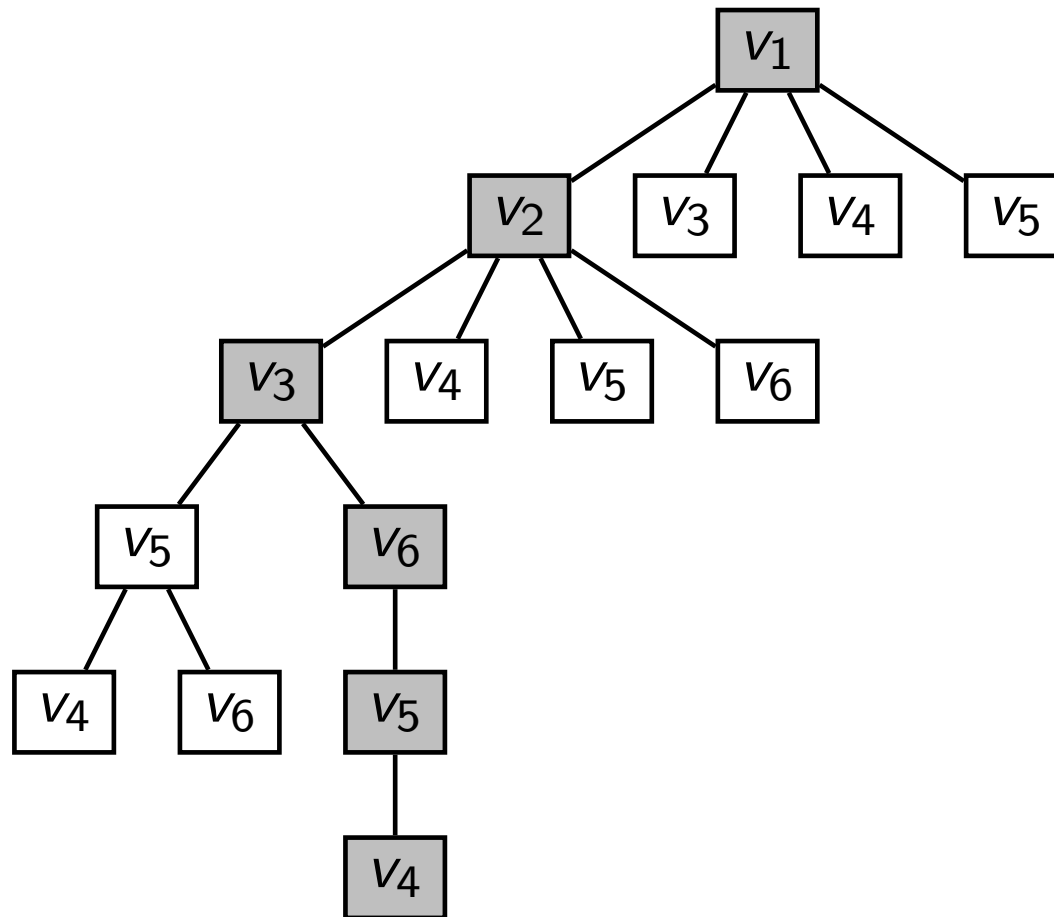
Backtracking is a technique is used to solve problems in which a sequence of objects is chosen from a specified set so that the sequence satisfies some criterion.

- The goal in the hamiltonian circuits problem is to find a hamiltonian circuit (a path that starts at a given vertex, visits each vertex in the graph exactly once, and ends at the starting vertex) in a connected, undirected graph.
- The **sequence** in this problem is the vertices of the graph.
- The **set** for each choice is the vertices of the graph.
- The **criterion** is that the chosen vertices form a simple path or cycle in the graph.

Exhaustive Search and Generation II

The hamiltonian circuits problem

- A portion of the pruned state space tree for $V = \{v_1, \dots, v_6\}$ and $E = \{(v_1, v_2), (v_1, v_4), (v_1, v_5), (v_2, v_3), (v_2, v_4), (v_2, v_5), (v_2, v_6), (v_3, v_5), (v_3, v_6), (v_4, v_5), (v_5, v_6)\}$.



Exhaustive Search and Generation II

The hamiltonian circuits problem

- The undirected graph is represented by an adjacency matrix $G.Adj[1..n, 1..n]$.
- A hamiltonian circuit is represented by a vector $I[1..n]$ of vertex indices.

```
HAMILTONIAN-CIRCUITS(0)
```

```
HAMILTONIAN-CIRCUITS( $i$ )
```

```
  if PROMISING( $i$ ) then
```

```
    if  $i = n$  then
```

```
      write  $I[1], \dots, I[n]$ 
```

```
    else
```

```
      for  $j = 1$  to  $n$  do
```

```
         $I[i + 1] = j$ 
```

```
        HAMILTONIAN-CIRCUITS( $i + 1$ )
```

Exhaustive Search and Generation II

The hamiltonian circuits problem

PROMISING(i)

if $i = n$ and not $G.Adj[I[n - 1], I[0]]$ **then**

return FALSE

else if $i > 1$ and not $G.Adj[I[i - 1], I[i]]$ **then**

return FALSE

else

for $j = 1$ **to** $i - 1$ **do**

if $I[i] = I[j]$ **then**

return FALSE

return TRUE

Exhaustive Search and Generation II

The traveling salesman problem

Backtracking is a technique is used to solve problems in which a sequence of objects is chosen from a specified set so that the sequence satisfies some criterion.

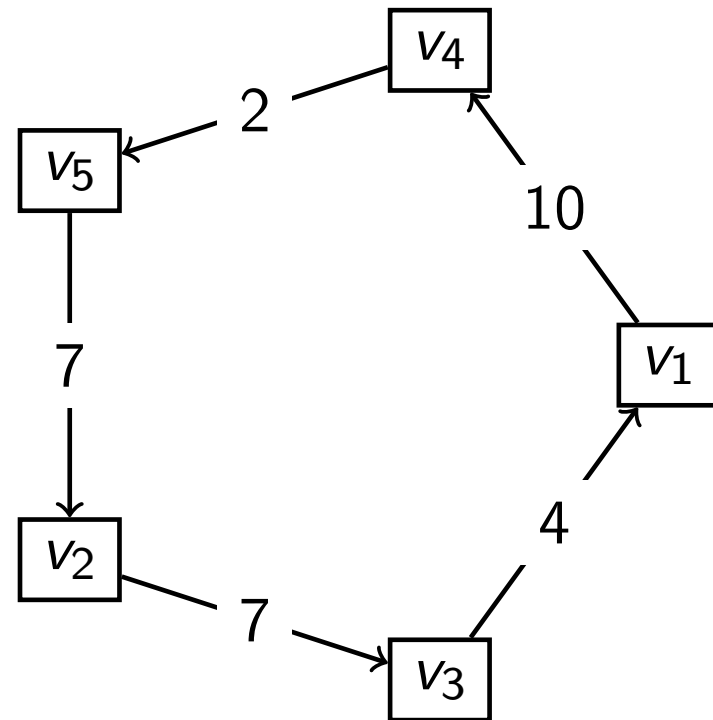
- The goal in the traveling salesman problem is to find a hamiltonian circuit (a path that starts at a given vertex, visits each vertex in the graph exactly once, and ends at the starting vertex) of smallest weight in a connected, **directed** graph.
- The **sequence** in this problem is the vertices of the graph.
- The **set** for each choice is the vertices of the graph.
- The **criterion** is that the chosen vertices form a simple path or cycle in the graph.

Exhaustive Search and Generation II

The traveling salesman problem

Example (Adjacency matrix representation of a graph that has an edge from every vertex to every other vertex, and an optimal tour for the graph.)

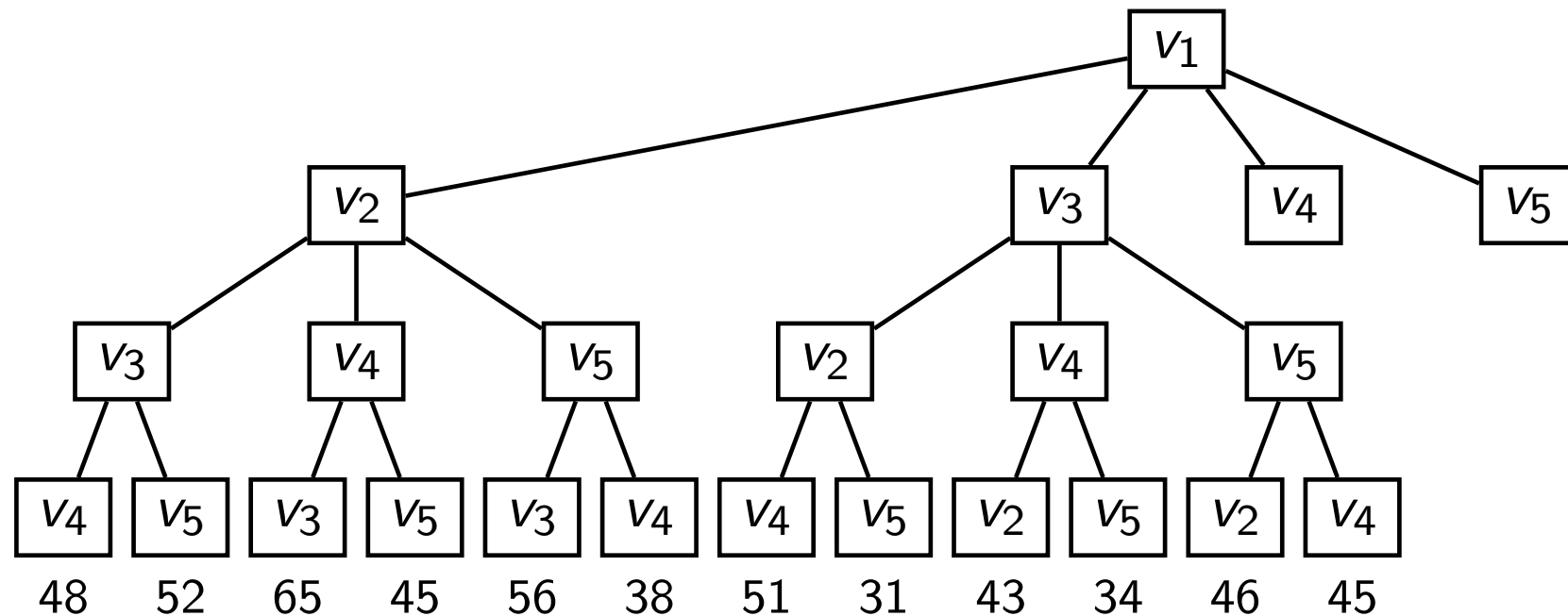
$$\begin{pmatrix} 0 & 14 & 4 & 10 & 20 \\ 14 & 0 & 7 & 8 & 7 \\ 4 & 5 & 0 & 7 & 16 \\ 11 & 7 & 9 & 0 & 2 \\ 18 & 7 & 17 & 4 & 0 \end{pmatrix}$$



Exhaustive Search and Generation II

The traveling salesman problem

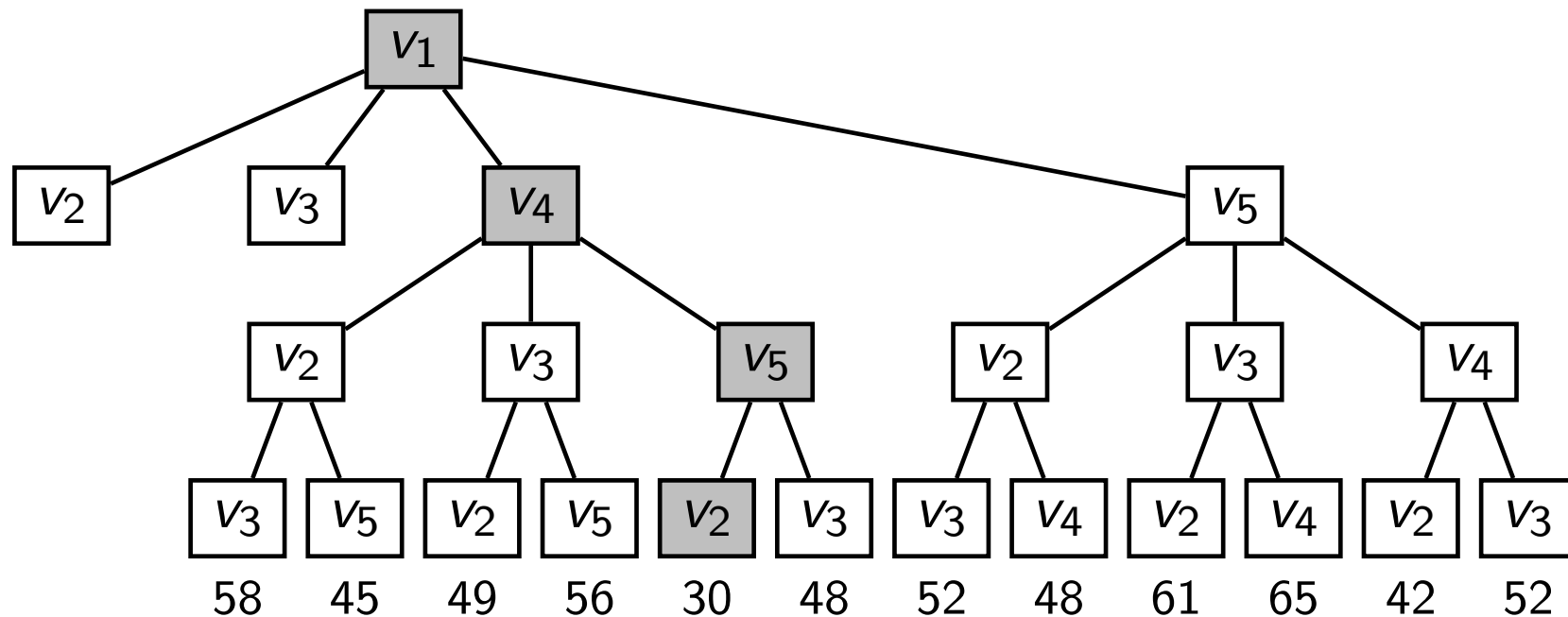
- A portion of the pruned state space tree for the previous graph.



Exhaustive Search and Generation II

The traveling salesman problem

- A portion of the pruned state space tree for the previous graph.



Exhaustive Search and Generation II

The traveling salesman problem

- In optimization problems, we need to be able to determine a bound for each node.
- Because of the objective in the 0-1 knapsack problem (to maximize profit while keeping the total weight from exceeding W), we computed an **upper bound** on the amount of profit that could be obtained by expanding beyond a given node, and we called a node promising if its bound was greater than the current maximum profit.
- In the traveling salesman problem, we need to determine a **lower bound** on the length of any tour that can be obtained by expanding beyond a given node, and we call the node promising if its bound is less than the current minimum tour length.

Exhaustive Search and Generation II

The traveling salesman problem

- The undirected graph is represented by a **weighted** adjacency matrix $G.Adj[1..n, 1..n]$.
- A tour is represented by a vector $I[1..n]$ of vertex indices.

```
TRAVELING-SALESMAN(0, 0)
```

```
TRAVELING-SALESMAN( $i, w$ )
```

```
  if  $i = n$  and  $w < min\_w$  then
```

```
     $w = min\_w$ 
```

```
     $best\_I = I$ 
```

```
  if PROMISING( $i, w$ ) then
```

```
    for  $j = 1$  to  $n$  do
```

```
       $I[i + 1] = j$ 
```

```
      TRAVELING-SALESMAN( $i + 1, w + G.Adj[I[i], j]$ )
```

Exhaustive Search and Generation II

The traveling salesman problem

```
min_w =  $\infty$   
best_l = 0, 0, ..., 0  
TRAVELING-SALESMAN(0, 0)  
write best_l[1], ..., best_l[n]
```

- A node is non-promising if when visiting the node we determine that it cannot possibly lead to a solution.
- In optimization problems, a node is non-promising if when visiting the node we determine that no expansion to the children can possibly lead to a solution.

```
PROMISING(i, w)  
  return w + bound < min_w
```

- A bound can be obtained by adding the length of the shortest edge going out of each of the remaining vertices.

Exhaustive Search and Generation II

The traveling salesman problem

PROMISING(i, w)

if $i = n$ and $G.Adj[I[n - 1], I[0]] = 0$ **then**

return FALSE

else if $i > 1$ and $G.Adj[I[i - 1], I[i]] = 0$ **then**

return FALSE

else

for $j = 1$ **to** $i - 1$ **do**

if $I[i] = I[j]$ **then**

return FALSE

$bound = 0$

for j in $\{1, \dots, n\} - \{I[1], \dots, I[i]\}$ **do**

$bound = bound + \min_{k=1}^n \{ G.Adj[j, k] : G.Adj[j, k] \neq 0 \}$

return $w + bound < min_w$